Scientific Computing

Aditya Dendukuri

UC Santa Barbara

Contents

Preface **7**

1 Fundamentals **8**

1.1 Vectors, Coordinates, and Linear Maps . . . 8
1.1.1 Span, Independence, and Basis . . . 8
1.1.2 Matrices as Linear Maps . . . 10
1.1.3 Matrix Products . . . 11
1.1.4 Transpose and Symmetry . . . 11
1.1.5 Inverses . . . 12
1.1.6 Exercises . . . 13
1.2 Inner Products, Geometry, and Norms . . . 14
1.2.1 Inner Products and Euclidean Geometry . . . 14
1.2.2 Projection as Best Approximation . . . 16
1.2.3 Norms as Choices of Measurement . . . 18
1.3 Operator Norms and Matrix Geometry . . . 21
1.3.1 Induced Norms as Worst-Case Amplification . . . 22
1.3.2 Dual Norms and Sharp Linear Bounds . . . 24
1.3.3 The Frobenius Geometry of Matrices . . . 25
1.3.4 Condition Number as Distortion . . . 26
1.4 Floating Point Arithmetic . . . 28
1.4.1 Floating Point Representation . . . 28
1.4.2 IEEE 754 Special Values . . . 30
1.4.3 Arithmetic Pitfalls . . . 31
1.5 Quantifying Approximation Error . . . 32
1.5.1 Absolute and Relative Errors . . . 32
1.5.2 Error Analysis Frameworks . . . 34
1.6 Cost Models and Sparse Storage . . . 34
1.6.1 Asymptotic Notation and Operation Counts . . . 35
1.6.2 Sparse Matrix Storage . . . 36
1.6.3 Exercises . . . 37

2 Direct Methods for Linear Systems **39**
2.1 Linearity and Fundamental Subspaces . . . 39
2.1.1 Linear Systems and Geometric Interpretations . . . 39
2.1.2 Invertibility and the Four Fundamental Subspaces . . . 40
2.1.3 Exercises . . . 42
2.2 Conditioning and Numerical Stability . . . 43
2.2.1 Residual, Forward Error, and Backward Error . . . 43
2.2.2 Sensitivity to Perturbations in the Data . . . 44
2.2.3 Backward Error and Backward Stability . . . 45
2.3 LU Factorization . . . 47
2.3.1 Gaussian Elimination as Triangular Factorization . . . 47
2.3.2 Triangular Solves and Computational Complexity . . . 50
2.3.3 Partial Pivoting and the Growth Factor . . . 50
2.3.4 Iterative Refinement . . . 52
2.3.5 Exercises . . . 53
2.4 Symmetric Positive Definite Matrices . . . 55
2.4.1 Positive Quadratic Energy . . . 55
2.4.2 Cholesky Factorization . . . 56
2.4.3 The Cholesky Algorithm . . . 57
2.4.4 Cost and Stability . . . 58
2.4.5 Sparse Cholesky and the Elimination Graph . . . 59
2.4.6 Exercises . . . 60
2.5 Reusing Factorizations after Structured Changes . . . 61
2.5.1 Rank-One Changes and the Sherman–Morrison Formula . . . 61
2.5.2 Several Changes and the Woodbury Formula . . . 63
2.5.3 Tridiagonal Systems . . . 65
2.5.4 Block-Tridiagonal Systems . . . 66
2.5.5 The Symmetric Positive Definite Case . . . 67
2.5.6 Finding the Blocks and Reusing a Prefix . . . 68
2.5.7 Exercises . . . 69

3 Orthogonality and Least Squares **70**
3.1 Orthogonal Transformations and the QR Factorization . . . 70
3.1.1 Orthogonality, Isometry, and Invariance . . . 70
3.1.2 The QR Factorization . . . 71
3.1.3 Householder Triangularization . . . 72
3.1.4 Gram–Schmidt Orthogonalization . . . 73
3.1.5 Orthogonal Projectors . . . 73
3.2 Least Squares as Projection . . . 75
3.2.1 The Geometric Problem . . . 75
3.2.2 Rank-Deficient Problems . . . 77
3.2.3 Exercises . . . 78

4 Eigenvalues and the SVD **79**
4.1 Eigenvalues, Invariant Subspaces, and Their Sensitivity . . . 79
4.1.1 Invariant Directions and Change of Basis . . . 80
4.1.2 Multiplicity and Schur Form . . . 81
4.1.3 Sensitivity of Eigenvalues . . . 84
4.1.4 Exercises . . . 86
4.2 Vector Iterations and the QR Algorithm . . . 87
4.2.1 Power Iteration and Shifted Inverse Iteration . . . 87
4.2.2 The Practical QR Algorithm . . . 89
4.2.3 The Jacobi Eigenvalue Algorithm . . . 90
4.2.4 Shifted Systems from One Hessenberg Reduction . . . 92
4.2.5 Exercises . . . 93
4.3 The Singular Value Decomposition . . . 94
4.3.1 The SVD Theorem and Subspace Characterization . . . 95
4.3.2 Norms and Condition Numbers . . . 96
4.3.3 Low-Rank Matrix Approximation . . . 97
4.3.4 Least Squares and Regularization . . . 99
4.3.5 Exercises . . . 101
4.4 Krylov Subspaces and the Arnoldi Process . . . 102
4.4.1 From Power Iteration to Krylov Subspaces . . . 102
4.4.2 The Arnoldi Process . . . 103
4.4.3 Ritz Pairs and Breakdown . . . 104
4.4.4 Exercises . . . 105
4.5 The Lanczos Algorithm . . . 106
4.5.1 Symmetry Gives a Three-Term Recurrence . . . 106
4.5.2 Ritz Values as Optimal Estimates . . . 107
4.5.3 Loss of Orthogonality and Ghost Eigenvalues . . . 109
4.5.4 Exercises . . . 110

5 Approximation and Integration **111**
5.1 Polynomial and Spline Interpolation . . . 111
5.1.1 Existence, Uniqueness, and the Vandermonde Matrix . . . 111
5.1.2 The Lagrange Form . . . 112
5.1.3 The Barycentric Interpolation Formula . . . 113
5.1.4 The Interpolation Error Formula . . . 114
5.1.5 Runge's Phenomenon and Chebyshev Nodes . . . 116
5.1.6 Piecewise Polynomials: Cubic Splines . . . 116
5.1.7 Exercises . . . 119
5.2 Chebyshev Polynomials and Minimax Approximation . . . 119
5.2.1 Two Failures of Polynomial Interpolation . . . 120
5.2.2 Chebyshev Polynomials and Fundamental Identities . . . 120
5.2.3 Roots, Extrema, and the Nodal Error Polynomial . . . 121
5.2.4 The Minimax Property and Equioscillation . . . 122
5.2.5 Chebyshev Series and Spectral Accuracy . . . 124
5.2.6 Exercises . . . 125

5.3 Numerical Quadrature . . . 126
5.3.1 Degree of Exactness and Interpolatory Quadrature . . . 127
5.3.2 Newton–Cotes Formulas and Composite Rules . . . 127
5.3.3 Peano Kernel Error Representation . . . 129
5.3.4 Gauss–Legendre Quadrature . . . 130
5.3.5 Richardson Extrapolation and Romberg Integration . . . 132
5.3.6 Periodic Functions and Spectral Convergence . . . 132
5.3.7 Exercises . . . 133
5.4 Fourier Analysis as Change of Basis . . . 135
5.4.1 The Discrete Fourier Transform . . . 135
5.4.2 Circulant Matrices and the Convolution Theorem . . . 137
5.4.3 The Fast Fourier Transform . . . 138
5.4.4 Spectral Differentiation . . . 139
5.4.5 Exercises . . . 140

6 Sparse Systems and Iterative Methods 141
6.1 Linear Operators and the Poisson Equation . . . 141
6.1.1 Differential Operators and Discretization . . . 141
6.1.2 Operator Norms and Conditioning . . . 144
6.1.3 Exercises . . . 144
6.2 Iterative Methods for Linear Systems . . . 145
6.2.1 Matrix Splittings and Stationary Iterations . . . 145
6.2.2 Convergence Theory and the Spectral Radius . . . 146
6.2.3 Krylov Subspaces and Matrix-Free Solvers . . . 147
6.3 Jacobi, Gauss-Seidel, and SOR Methods . . . 148
6.3.1 The Jacobi Method . . . 148
6.3.2 The Gauss–Seidel Method . . . 149
6.3.3 Convergence Theorems . . . 150
6.3.4 Successive Over-Relaxation (SOR) . . . 151
6.4 The Conjugate Gradient Method . . . 152
6.4.1 Quadratic Optimization and Steepest Descent . . . 153
6.4.2 Conjugate Directions and Subspace Decoupling . . . 154
6.4.3 The Conjugate Gradient Algorithm . . . 155
6.4.4 Convergence Rate and Preconditioning . . . 157
6.5 The Generalized Minimal Residual (GMRES) Method . . . 159
6.5.1 Reduction to Hessenberg Least Squares . . . 159
6.5.2 Solution via Givens Rotations . . . 160
6.5.3 Restarted GMRES and Practical Considerations . . . 161
6.5.4 Exercises . . . 162
6.6 Multigrid Methods . . . 163
6.6.1 Oscillating Error and the Smoothing Property . . . 163
6.6.2 The Two-Grid Correction Scheme . . . 167
6.6.3 The Multigrid V-Cycle . . . 169
6.7 Randomized Methods in Linear Algebra . . . 172
6.7.1 Trace and Diagonal Estimation . . . 172

6.7.2 Randomized Low-Rank Approximation . . . 173
6.7.3 Random Sampling in Gaussian Elimination . . . 174
6.7.4 Exercises . . . 175

7 Ordinary Differential Equations 175
7.1 Numerical Methods for Ordinary Differential Equations . . . 175
7.1.1 Initial Value Problems and Well-Posedness . . . 176
7.1.2 Consistency, Stability, and the Discrete Gronwall Lemma . . . 177
7.1.3 Euler Methods and Stability Functions . . . 178
7.1.4 Runge–Kutta Methods . . . 179
7.1.5 Symplectic Integrators and Geometric Mechanics . . . 180
7.1.6 Exercises . . . 182

8 Applications and Research Bridges 183
8.1 Principal Component Analysis (PCA) . . . 183
8.1.1 Definitions and Preprocessing . . . 183
8.1.2 PCA via SVD . . . 184
8.1.3 Variance and Dimensionality Reduction . . . 185
8.1.4 Implementation Details . . . 185
8.1.5 Exercises . . . 186
8.2 Spectral Graph Theory, Diffusion Maps, and Graph Laplacians . . . 186
8.2.1 Graph Matrices and the Laplacian . . . 186
8.2.2 Spectral Clustering . . . 188
8.2.3 Diffusion Maps and Nonlinear Manifolds . . . 189
8.2.4 Permutation Invariance and Graph Spectra . . . 191
8.2.5 Bandwidth Selection via Affinity Scaling . . . 192
8.2.6 Exercises . . . 192
8.3 Matrix Exponentials and Krylov Subspace Actions . . . 193
8.3.1 Definition, Properties, and the Sparse Bottleneck . . . 194
8.3.2 Krylov Subspace Approximation of the Exponential Action . . . 194
8.3.3 The Continuous-Adjoint Sensitivity Formula . . . 196
8.3.4 Shared Krylov Spaces and Quadrature . . . 197
8.3.5 Generator Recovery via the Matrix Logarithm . . . 198
8.3.6 Exercises . . . 198
8.4 Markov Chains, Continuous Generators, and the CME . . . 199
8.4.1 Discrete-Time Markov Chains . . . 199
8.4.2 Continuous-Time Markov Chains and the CME . . . 201
8.4.3 Assembling the Generator and Simulating Trajectories . . . 202
8.4.4 Finite State Projection (FSP) . . . 203
8.4.5 Truncation Strategy: Probability Flux versus Probability Mass . . . 205
8.4.6 Exercises . . . 205
8.5 Koopman Theory, EDMD, and Continuous Generators . . . 207
8.5.1 Koopman Operator Theory . . . 207
8.5.2 Dynamic Mode Decomposition (DMD) and EDMD . . . 208
8.5.3 CME Koopman Generator and Polynomial Closure . . . 209

8.5.4 Continuous Rate Recovery and Logarithmic Noise Explosion 210
8.5.5 Exercises 211

9 Appendix: Notation and Software **213**
9.1 Mathematical Notation and Proof Techniques 213
9.1.1 Logic and Set Notation 213
9.1.2 Proof Techniques 214
9.1.3 Notation Conventions 214
9.2 Python and NumPy Primer 215
9.2.1 Arrays and Basic Operations 215
9.2.2 Linear Algebra Operations 216
9.2.3 Key Idioms 216

References **218**

Preface

These notes started as a study guide for CS 111 at UC Santa Barbara and turned into a habit. Whenever I ran into an idea I couldn't explain from scratch, I wrote it down until I could. Feynman's test (if you can't explain something simply, you don't really understand it) turns out to be a reliable way to find gaps, and research kept turning up new ones in eigenvalue algorithms, iterative solvers, spectral graph methods, and Koopman theory. What you're reading is a working reference I wrote as a graduate student, mostly for my own use.

Computers were first built for numerical work. A "computer" was originally a person, someone paid to evaluate functions by hand. During the Second World War the U.S. Army employed rooms full of them to produce artillery firing tables, and the backlog was large enough to justify building ENIAC (1945). Among the machine's first jobs were calculations for thermonuclear weapons design, work in which John von Neumann was closely involved. His 1947 paper with Goldstine laid the foundations of numerical stability analysis. In 1950, Jule Charney's group used ENIAC to run the first computer weather forecast, a calculation Lewis Fry Richardson had once estimated would take 64,000 people working in concert. Later, the Apollo Guidance Computer navigated to the Moon with a few kilobytes of working memory, and the Cooley–Tukey FFT (1965) cut a core signal-processing computation from $O(n^2)$ to $O(n \log n)$.

Most of what follows is linear algebra. Nearly every numerical method works by reducing a nonlinear problem to a sequence of linear ones, because linear problems are the ones we understand best. We know when solutions exist, how sensitive they are to perturbations, and how to compute them reliably. Newton's method, finite element discretization, PCA, and gradient-based optimization all come down to repeated linearization. Once linear algebra feels familiar, most of the other methods in these notes are easier to follow.

For a more rigorous treatment, start with Principles of Scientific Computing by Bindel and Goodman and G. W. Stewart's Afternotes on Numerical Analysis. For a broader tour of the literature, the CS 6241 reading list at Cornell is a great place to look. Much of the perspective here comes from **CS 210A (Matrix Analysis)**, taught by Professor Shivkumar Chandrasekaran in the ECE department at UC Santa Barbara. If you get the chance to take it, do.

Aditya Dendukuri
UC Santa Barbara, 2025

1 Fundamentals

1.1 Vectors, Coordinates, and Linear Maps

Scientific computing begins with representation. A physical state, a polynomial, a sampled signal, and an image are different objects, yet each can be represented by a finite list of numbers. The list is useful only after we say what its entries mean. That distinction between an object and its coordinates will matter throughout these notes.

Throughout, $\mathbb{R}$, $\mathbb{Z}$, and $\mathbb{C}$ denote the real numbers, integers, and complex numbers. Most examples use real arithmetic. Complex numbers appear naturally when we study oscillations and eigenvalues.

Def 1.1 **(Scalar)** An element of the number system under discussion is a **scalar**. In these notes that number system is usually $\mathbb{R}$ and occasionally $\mathbb{C}$.

Def 1.2 **(Vector)** A **vector** is an object that can be added to another vector and multiplied by a scalar, with the usual arithmetic laws. Once a basis has been chosen, a vector in an n-dimensional space is represented by a column of n coordinates.

Remark 1.1 A column of numbers therefore represents a vector only after we know what its coordinates refer to. The coordinates $(3, 2)^T$ might describe a point, a velocity, the coefficients of $3 + 2t$, or two temperature readings. Computation manipulates the coordinates, while interpretation tells us what vector they represent.

Def 1.3 **($\mathbb{R}^n$)** The vector space $\mathbb{R}^n$ consists of all ordered n-tuples of real numbers. We write vectors as columns unless stated otherwise:

$$\boldsymbol{x} = \begin{pmatrix} x_1 \\ \vdots \\ x_n \end{pmatrix} \in \mathbb{R}^n.$$

Vectors are added entry by entry and multiplied by scalars entry by entry. The zero vector is denoted by $\mathbf{0}$.

Remark 1.2 NumPy normally stores a vector as an array of shape `(n,)`, which is neither a row nor a column. A shape `(n,1)` array is a genuine two-dimensional column. This implementation detail affects transposes and broadcasting, so shapes should be checked rather than guessed.

1.1.1 Span, Independence, and Basis

Def 1.4 **(Linear Combination, Span, and Subspace)** Given vectors $\boldsymbol{v}_1, \ldots, \boldsymbol{v}_k$, a **linear combination** is any vector of the form

$$c_1\boldsymbol{v}_1 + \cdots + c_k\boldsymbol{v}_k.$$

Their **span** is the set of all such combinations. A nonempty set $S \subseteq \mathbb{R}^n$ is a **subspace** if every linear combination of vectors in S remains in S.

Example The vectors $(1,0,0)^T$ and $(0,1,0)^T$ span the coordinate plane in $\mathbb{R}^3$ on which the third coordinate is zero. A plane not passing through the origin is not a subspace: it does not contain $\mathbf{0}$ and is not closed under scalar multiplication.

Def 1.5 **(Linear Independence and Basis)** Vectors $\boldsymbol{v}_1, \ldots, \boldsymbol{v}_k$ are **linearly independent** if

$$c_1\boldsymbol{v}_1 + \cdots + c_k\boldsymbol{v}_k = \mathbf{0}$$

forces every coefficient c_i to be zero. A **basis** of a vector space is a linearly independent spanning set.

Thm 1.1 **(Exchange, Extension, and Dimension)** Let V be spanned by m vectors.

1. Every linearly independent list in V contains at most m vectors.
2. Any linearly independent list can be extended to a basis by adding vectors from a spanning list.
3. Any two bases of V contain the same number of vectors. This common number is the **dimension** of V.

Remark 1.3 The proof is left as an exercise below because the replacement argument is best learned by working through it. This theorem justifies phrases used later such as "extend a basis of the null space to a basis of the domain."

Thm 1.2 **(A Basis Gives Unique Coordinates)** Let $\boldsymbol{v}_1, \ldots, \boldsymbol{v}_n$ be a basis of a vector space V. Every $\boldsymbol{x} \in V$ has exactly one representation

$$\boldsymbol{x} = x_1\boldsymbol{v}_1 + \cdots + x_n\boldsymbol{v}_n.$$

The scalars $x_1, \ldots, x_n$ are the coordinates of $\boldsymbol{x}$ in this basis.

Proof Because the basis spans V, at least one representation exists. Suppose $\boldsymbol{x}$ admits two representations:

$$\boldsymbol{x} = \sum_{i=1}^{n} x_i\boldsymbol{v}_i = \sum_{i=1}^{n} y_i\boldsymbol{v}_i.$$

Subtracting the second sum from the first gives

$$\mathbf{0} = \sum_{i=1}^{n} x_i\boldsymbol{v}_i - \sum_{i=1}^{n} y_i\boldsymbol{v}_i = \sum_{i=1}^{n} (x_i - y_i)\boldsymbol{v}_i.$$

By linear independence of the basis vectors $\boldsymbol{v}_1, \ldots, \boldsymbol{v}_n$, every coefficient in this linear combination must vanish:

$$x_i - y_i = 0 \quad \Longrightarrow \quad x_i = y_i, \qquad i = 1, \ldots, n.$$

The coordinates are therefore unique. □

Remark 1.4 Coordinates change when the basis changes, but the vector does not. For example, the polynomial $p(t) = 3 + 2t$ has coordinates $(3, 2)^T$ in the basis $1, t$ and coordinates $(5, 2)^T$ in the basis $1, t - 1$. Confusing a vector with one coordinate list is a common source of conceptual errors in change-of-basis and eigenvalue arguments.

1.1.2 Matrices as Linear Maps

Def 1.6 **(Linear Map)** A map $T : V \to W$ is **linear** if, for all vectors $\boldsymbol{x}, \boldsymbol{y} \in V$ and scalars α, β,

$$T(\alpha \boldsymbol{x} + \beta \boldsymbol{y}) = \alpha T(\boldsymbol{x}) + \beta T(\boldsymbol{y}).$$

Linearity says that the response to a combination of inputs is the same combination of the individual responses.

Def 1.7 **(Matrix)** A **matrix** $\mathbf{A} \in \mathbb{R}^{m \times n}$ is a rectangular array of scalars with m rows and n columns. It represents a linear map from $\mathbb{R}^n$ to $\mathbb{R}^m$ once bases have been selected in the domain and range.

Thm 1.3 **(The Columns Determine the Map)** Let $\boldsymbol{e}_1, \ldots, \boldsymbol{e}_n$ be the standard basis of $\mathbb{R}^n$, and let $T : \mathbb{R}^n \to \mathbb{R}^m$ be linear. If the jth column of $\mathbf{A}$ is $\boldsymbol{a}_j = T(\boldsymbol{e}_j)$, then

$$T(\boldsymbol{x}) = \mathbf{A}\boldsymbol{x} = \sum_{j=1}^{n} x_j \boldsymbol{a}_j.$$

Thus a linear map is completely determined by what it does to a basis.

Proof Every vector has the standard-coordinate expansion $\boldsymbol{x} = \sum_{j=1}^{n} x_j \boldsymbol{e}_j$. Linearity gives

$$T(\boldsymbol{x}) = T\left(\sum_{j=1}^{n} x_j \boldsymbol{e}_j\right) = \sum_{j=1}^{n} x_j T(\boldsymbol{e}_j) = \sum_{j=1}^{n} x_j \boldsymbol{a}_j,$$

which is precisely the definition of the matrix-vector product $\mathbf{A}\boldsymbol{x}$. □

Remark 1.5 This column view is more informative than memorizing the row-by-column rule. The set of all possible outputs $\mathbf{A}\boldsymbol{x}$ is the span of the columns. A zero output means that one linear combination of the columns vanishes. Existence and uniqueness for linear systems will grow directly from these two observations.

Example For

$$\mathbf{A} = \begin{pmatrix} 1 & 2 \\ 3 & -1 \end{pmatrix}, \qquad \boldsymbol{x} = \begin{pmatrix} 4 \\ 5 \end{pmatrix},$$

the product is best read as

$$\mathbf{A}\boldsymbol{x} = 4\begin{pmatrix} 1 \\ 3 \end{pmatrix} + 5\begin{pmatrix} 2 \\ -1 \end{pmatrix} = \begin{pmatrix} 14 \\ 7 \end{pmatrix}.$$

The same calculation can be read row by row: each output coordinate is a weighted sum of the input coordinates.

1.1.3 Matrix Products

Composition of linear maps defines the matrix product. If $\mathbf{B} \in \mathbb{R}^{n\times p}$ represents a map from $\mathbb{R}^p$ to $\mathbb{R}^n$ and $\mathbf{A} \in \mathbb{R}^{m\times n}$ represents a map from $\mathbb{R}^n$ to $\mathbb{R}^m$, then the composition sends $\boldsymbol{e}_j$ first to the jth column $\boldsymbol{b}_j$ of $\mathbf{B}$ and then to $\mathbf{A}\boldsymbol{b}_j$. By Thm 1.3, the matrix of the composition therefore has columns $\mathbf{A}\boldsymbol{b}_1, \ldots, \mathbf{A}\boldsymbol{b}_p$.

Def 1.8 **(Matrix Product)** For $\mathbf{A} \in \mathbb{R}^{m\times n}$ and $\mathbf{B} \in \mathbb{R}^{n\times p}$, the product $\mathbf{AB} \in \mathbb{R}^{m\times p}$ is the matrix with entries

$$(\mathbf{AB})_{ij} = \sum_{k=1}^{n} a_{ik} b_{kj}.$$

Equivalently, the jth column of $\mathbf{AB}$ is $\mathbf{A}\boldsymbol{b}_j$. The product is defined only when the number of columns of $\mathbf{A}$ equals the number of rows of $\mathbf{B}$.

Because the product represents composition, it is associative and distributes over addition. It is not commutative. For

$$\mathbf{A} = \begin{pmatrix} 1 & 0 \\ 0 & 2 \end{pmatrix}, \qquad \mathbf{B} = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}, \qquad \mathbf{AB} = \begin{pmatrix} 0 & 1 \\ 2 & 0 \end{pmatrix} \neq \begin{pmatrix} 0 & 2 \\ 1 & 0 \end{pmatrix} = \mathbf{BA}.$$

Scaling the coordinates and then exchanging them is a different map from exchanging them and then scaling.

1.1.4 Transpose and Symmetry

Def 1.9 **(Matrix Transpose)** The **transpose** of $\mathbf{A} \in \mathbb{R}^{m\times n}$ is the matrix $\mathbf{A}^T \in \mathbb{R}^{n\times m}$ defined by

$$(\mathbf{A}^T)_{ij} = a_{ji}.$$

It turns columns into rows and rows into columns.

Def 1.10 **(Symmetric and Skew-Symmetric Matrices)** A square matrix $\mathbf{A}$ is **symmetric** if $\mathbf{A}^T = \mathbf{A}$ and **skew-symmetric** if $\mathbf{A}^T = -\mathbf{A}$.

Remark 1.6 Every square matrix splits uniquely into symmetric and skew-symmetric parts:

$$\mathbf{A} = \frac{\mathbf{A} + \mathbf{A}^T}{2} + \frac{\mathbf{A} - \mathbf{A}^T}{2}.$$

Symmetric matrices will be especially important because they possess real eigenvalues and orthogonal eigenvectors.

Thm 1.4 **(Properties of the Transpose)** For conformable matrices and any scalar α,

$$(\mathbf{A}^T)^T = \mathbf{A}, \qquad (\mathbf{A} + \mathbf{B})^T = \mathbf{A}^T + \mathbf{B}^T,$$
$$(\alpha\mathbf{A})^T = \alpha\mathbf{A}^T, \qquad (\mathbf{AB})^T = \mathbf{B}^T\mathbf{A}^T.$$

Proof The first three identities follow entry by entry. For the product,

$$((\mathbf{AB})^T)_{ij} = (\mathbf{AB})_{ji} = \sum_k a_{jk}b_{ki} = \sum_k (\mathbf{B}^T)_{ik}(\mathbf{A}^T)_{kj} = (\mathbf{B}^T\mathbf{A}^T)_{ij}.$$

□

Cor 1.5 **(Reversal Law for Multiple Factors)** Repeated use of the product rule gives

$$(\mathbf{A}_1\mathbf{A}_2\cdots\mathbf{A}_k)^T = \mathbf{A}_k^T\cdots\mathbf{A}_2^T\mathbf{A}_1^T.$$

Cor 1.6 **(Outer Product is Symmetric of Rank One)** For every nonzero $\boldsymbol{x} \in \mathbb{R}^n$, the outer product $\boldsymbol{x}\boldsymbol{x}^T$ is symmetric and has rank one. Moreover,

$$\boldsymbol{x}^T\boldsymbol{x} = \sum_{i=1}^n x_i^2 > 0.$$

1.1.5 Inverses

A linear map from $\mathbb{R}^n$ to itself can be undone exactly when its matrix has an inverse. The inverse is a useful theoretical object, but, as the remark below explains, it is rarely computed.

Def 1.11 **(Matrix Inverse)** A square matrix $\mathbf{A} \in \mathbb{R}^{n\times n}$ is **invertible**, or nonsingular, if there is a matrix $\mathbf{A}^{-1}$ with

$$\mathbf{A}\mathbf{A}^{-1} = \mathbf{A}^{-1}\mathbf{A} = \mathbf{I}.$$

The inverse is unique. If $\mathbf{B}$ and $\mathbf{C}$ both satisfy the definition, then $\mathbf{B} = \mathbf{B}(\mathbf{AC}) = (\mathbf{BA})\mathbf{C} = \mathbf{C}$.

Thm 1.7 **(Properties of the Inverse)** Let $\mathbf{A}, \mathbf{B} \in \mathbb{R}^{n\times n}$ be invertible and let $\alpha \neq 0$. Then $\mathbf{A}^{-1}$, $\mathbf{AB}$, $\alpha\mathbf{A}$, and $\mathbf{A}^T$ are invertible, with

$$(\mathbf{A}^{-1})^{-1} = \mathbf{A}, \qquad (\mathbf{AB})^{-1} = \mathbf{B}^{-1}\mathbf{A}^{-1}, \qquad (\alpha\mathbf{A})^{-1} = \alpha^{-1}\mathbf{A}^{-1}, \qquad (\mathbf{A}^T)^{-1} = (\mathbf{A}^{-1})^T.$$

Proof Each claim follows by multiplying the proposed inverse with the matrix and using uniqueness. The definition of $\mathbf{A}^{-1}$ shows that $\mathbf{A}$ is an inverse of $\mathbf{A}^{-1}$. For the product,

$$(\mathbf{AB})(\mathbf{B}^{-1}\mathbf{A}^{-1}) = \mathbf{A}(\mathbf{B}\mathbf{B}^{-1})\mathbf{A}^{-1} = \mathbf{A}\mathbf{A}^{-1} = \mathbf{I},$$

and the product in the other order is $\mathbf{I}$ by the same calculation. The scalar case is $(\alpha\mathbf{A})(\alpha^{-1}\mathbf{A}^{-1}) = \mathbf{I}$. For the transpose, transposing $\mathbf{A}\mathbf{A}^{-1} = \mathbf{I}$ and $\mathbf{A}^{-1}\mathbf{A} = \mathbf{I}$ with Thm 1.4 gives $(\mathbf{A}^{-1})^T\mathbf{A}^T = \mathbf{A}^T(\mathbf{A}^{-1})^T = \mathbf{I}$. □

Remark 1.7 **(Factor instead of inverting)** Scientific codes almost never form $\mathbf{A}^{-1}$. To solve $\mathbf{A}\boldsymbol{x} = \boldsymbol{b}$, they factor $\mathbf{A}$ into triangular or orthogonal factors and solve with each factor in turn, which costs less and is more accurate than forming the inverse and multiplying. The reversal law $(\mathbf{AB})^{-1} = \mathbf{B}^{-1}\mathbf{A}^{-1}$ describes this procedure. To solve $\mathbf{AB}\boldsymbol{x} = \boldsymbol{b}$, we first solve $\mathbf{A}\boldsymbol{y} = \boldsymbol{b}$ and then $\mathbf{B}\boldsymbol{x} = \boldsymbol{y}$.

1.1.6 Exercises

Ex 1.1

1. **Check.** Decide which of the following subsets of $\mathbb{R}^3$ are subspaces: the plane $x_1 + x_2 = 0$, the plane $x_1 + x_2 = 1$, the set of vectors with at least one zero entry, and the span of $(1, 1, 0)^T$ and $(0, 1, 1)^T$. For every negative answer, identify a subspace axiom that fails.

2. **Derive.** Prove that a list containing the zero vector is dependent. Prove that two nonzero vectors are dependent exactly when one is a scalar multiple of the other. Then explain why three vectors in $\mathbb{R}^2$ cannot be independent.

3. **Derive.** Prove Thm. 1.1. Begin with a spanning list $\boldsymbol{w}_1, \ldots, \boldsymbol{w}_m$ and a nonzero vector $\boldsymbol{v}_1$. Express $\boldsymbol{v}_1$ in the spanning list and replace one $\boldsymbol{w}_j$ having a nonzero coefficient by $\boldsymbol{v}_1$. Show that the new list still spans. Repeat with $\boldsymbol{v}_2, \ldots, \boldsymbol{v}_k$, using independence to show that a replaceable old vector always remains. Apply the result in both directions to two bases.

4. **Explore.** Let $p(t) = a + bt + ct^2$. Find its coordinates in the bases $1, t, t^2$ and $1, t-1, (t-1)^2$. Construct the matrix that converts the second coordinate vector into the first, and interpret each column.

5. **Derive.** Suppose $T : \mathbb{R}^n \to \mathbb{R}^m$ is linear and $T(\boldsymbol{e}_j) = \mathbf{0}$ for every standard basis vector. Prove that T is the zero map. Why is checking only one non-basis collection insufficient?

6. **Compute.** Write a matrix as both a collection of columns and a collection of rows. Use the two views to explain, without entrywise multiplication, what $\mathbf{A}\boldsymbol{x}$ and $\boldsymbol{y}^T\mathbf{A}$ mean.

7. **Check.** For $\mathbf{A} = \begin{pmatrix} 2 & 1 \\ 0 & 3 \end{pmatrix}$ and $\mathbf{B} = \begin{pmatrix} 1 & -1 \\ 2 & 4 \end{pmatrix}$, compute $\mathbf{AB}$ and $\mathbf{BA}$ by entries and by columns. Find a matrix other than a multiple of $\mathbf{I}$ that commutes with $\mathbf{A}$.

8. **Derive.** A square matrix is **involutory** if $\mathbf{A}^2 = \mathbf{I}$. Prove that $\mathbf{A}$ is involutory exactly when $\mathbf{A}^{-1} = \mathbf{A}$, and give an involutory 2×2 matrix other than $\pm\mathbf{I}$. Interpret your example geometrically.

9. **Derive.** Prove Cor. 1.6. In particular, show that every column of $\boldsymbol{x}\boldsymbol{x}^T$ is a multiple of $\boldsymbol{x}$, and identify one nonzero column.

1.2 Inner Products, Geometry, and Norms

Words such as "small error," "closest approximation," and "stable computation" have no mathematical content until a measurement has been chosen. That choice determines which perturbations matter, which approximation is best, and how strongly a matrix can amplify an error. We build Euclidean geometry from the inner product, derive orthogonal projection as a best approximation, and then distinguish inner-product geometry from the more general concept of a vector norm.

1.2.1 Inner Products and Euclidean Geometry

Def 1.12 **(Real Inner Product)** An **inner product** on $\mathbb{R}^n$ is a function $\langle \cdot, \cdot \rangle : \mathbb{R}^n \times \mathbb{R}^n \to \mathbb{R}$ satisfying, for all vectors $\boldsymbol{x}, \boldsymbol{y}, \boldsymbol{z}$ and scalars α, β,

1. **Symmetry:** $\langle \boldsymbol{x}, \boldsymbol{y} \rangle = \langle \boldsymbol{y}, \boldsymbol{x} \rangle$.
2. **Linearity:** $\langle \alpha\boldsymbol{x} + \beta\boldsymbol{y}, \boldsymbol{z} \rangle = \alpha\langle \boldsymbol{x}, \boldsymbol{z} \rangle + \beta\langle \boldsymbol{y}, \boldsymbol{z} \rangle$.
3. **Positive definiteness:** $\langle \boldsymbol{x}, \boldsymbol{x} \rangle \geq 0$, with equality if and only if $\boldsymbol{x} = \mathbf{0}$.

The standard Euclidean inner product is

$$\langle \boldsymbol{x}, \boldsymbol{y} \rangle_2 = \boldsymbol{x}^T\boldsymbol{y} = \sum_{i=1}^{n} x_i y_i.$$

The three axioms encode more than a formula. Symmetry says that the comparison has no preferred order. Linearity makes the comparison compatible with linear combinations. Positive definiteness makes self-comparison a genuine measure of size.

Def 1.13 **(Norm Induced by an Inner Product)** Every inner product defines a norm by

$$\|\boldsymbol{x}\| = \sqrt{\langle \boldsymbol{x}, \boldsymbol{x} \rangle}.$$

For the Euclidean inner product this gives

$$\|\boldsymbol{x}\|_2 = \sqrt{\boldsymbol{x}^T \boldsymbol{x}}.$$

The square root is chosen so that scaling a vector by α scales its length by $|\alpha|$. The triangle inequality is less immediate and follows from the Cauchy–Schwarz inequality.

Thm 1.8 **(Cauchy–Schwarz Inequality)** For every real inner product and all vectors $\boldsymbol{x}, \boldsymbol{y}$,

$$|\langle \boldsymbol{x}, \boldsymbol{y} \rangle| \leq \|\boldsymbol{x}\| \, \|\boldsymbol{y}\|.$$

Equality holds if and only if $\boldsymbol{x}$ and $\boldsymbol{y}$ are linearly dependent.

Proof The claim is immediate when $\boldsymbol{y} = \boldsymbol{0}$. Suppose that $\boldsymbol{y} \neq \boldsymbol{0}$ and choose

$$\alpha = \frac{\langle \boldsymbol{x}, \boldsymbol{y} \rangle}{\langle \boldsymbol{y}, \boldsymbol{y} \rangle}.$$

Positive definiteness gives

$$\begin{aligned} 0 &\leq \|\boldsymbol{x} - \alpha \boldsymbol{y}\|^2 \\ &= \langle \boldsymbol{x} - \alpha \boldsymbol{y}, \boldsymbol{x} - \alpha \boldsymbol{y} \rangle \\ &= \|\boldsymbol{x}\|^2 - 2\alpha \langle \boldsymbol{x}, \boldsymbol{y} \rangle + \alpha^2 \|\boldsymbol{y}\|^2 \\ &= \|\boldsymbol{x}\|^2 - \frac{|\langle \boldsymbol{x}, \boldsymbol{y} \rangle|^2}{\|\boldsymbol{y}\|^2}. \end{aligned}$$

Multiplying by $\|\boldsymbol{y}\|^2$ and taking square roots proves the inequality. Equality holds precisely when $\|\boldsymbol{x} - \alpha \boldsymbol{y}\| = 0$, which is equivalent to $\boldsymbol{x} = \alpha \boldsymbol{y}$. □

Cor 1.9 **(Triangle Inequality for an Inner-Product Norm)** For all vectors $\boldsymbol{x}$ and $\boldsymbol{y}$,

$$\|\boldsymbol{x} + \boldsymbol{y}\| \leq \|\boldsymbol{x}\| + \|\boldsymbol{y}\|.$$

Proof Expanding the squared norm and applying Cauchy–Schwarz gives

$$\begin{aligned} \|\boldsymbol{x} + \boldsymbol{y}\|^2 &= \|\boldsymbol{x}\|^2 + 2\langle \boldsymbol{x}, \boldsymbol{y} \rangle + \|\boldsymbol{y}\|^2 \\ &\leq \|\boldsymbol{x}\|^2 + 2\|\boldsymbol{x}\|\|\boldsymbol{y}\| + \|\boldsymbol{y}\|^2 \\ &= (\|\boldsymbol{x}\| + \|\boldsymbol{y}\|)^2. \end{aligned}$$

Taking non-negative square roots proves the result. □

Def 1.14 **(Angle and Orthogonality)** For nonzero vectors $\boldsymbol{x}$ and $\boldsymbol{y}$, their angle $\theta \in [0, \pi]$ is defined by

$$\cos\theta = \frac{\langle \boldsymbol{x}, \boldsymbol{y} \rangle}{\|\boldsymbol{x}\| \, \|\boldsymbol{y}\|}.$$

Cauchy–Schwarz guarantees that the right-hand side lies in $[-1, 1]$. The vectors are **orthogonal**, written $\boldsymbol{x} \perp \boldsymbol{y}$, when $\langle \boldsymbol{x}, \boldsymbol{y} \rangle = 0$.

Thm 1.10 **(Pythagorean Theorem and Its Converse)** For vectors $\boldsymbol{x}$ and $\boldsymbol{y}$ in a real inner-product space,

$$\boldsymbol{x} \perp \boldsymbol{y} \quad \Longleftrightarrow \quad \|\boldsymbol{x} + \boldsymbol{y}\|^2 = \|\boldsymbol{x}\|^2 + \|\boldsymbol{y}\|^2.$$

Proof The inner-product expansion gives

$$\|\boldsymbol{x} + \boldsymbol{y}\|^2 = \|\boldsymbol{x}\|^2 + 2\langle \boldsymbol{x}, \boldsymbol{y} \rangle + \|\boldsymbol{y}\|^2.$$

The Pythagorean identity therefore holds exactly when $\langle \boldsymbol{x}, \boldsymbol{y} \rangle = 0$. □

1.2.2 Projection as Best Approximation

The coefficient used in the proof of Cauchy–Schwarz has a geometric meaning. It selects the point on the line through $\boldsymbol{y}$ that is closest to $\boldsymbol{x}$.

Def 1.15 **(Projection onto a Nonzero Vector)** For $\boldsymbol{y} \neq \boldsymbol{0}$, the orthogonal projection of $\boldsymbol{x}$ onto the line spanned by $\boldsymbol{y}$ is

$$\operatorname{proj}_{\boldsymbol{y}}(\boldsymbol{x}) = \frac{\langle \boldsymbol{x}, \boldsymbol{y} \rangle}{\langle \boldsymbol{y}, \boldsymbol{y} \rangle} \boldsymbol{y}.$$

The residual $\boldsymbol{x} - \operatorname{proj}_{\boldsymbol{y}}(\boldsymbol{x})$ is orthogonal to $\boldsymbol{y}$.

Thm 1.11 **(Closest Point on a Line)** Let $\boldsymbol{y} \neq \boldsymbol{0}$ and let

$$\alpha = \frac{\langle \boldsymbol{x}, \boldsymbol{y} \rangle}{\langle \boldsymbol{y}, \boldsymbol{y} \rangle}.$$

Then $\alpha \boldsymbol{y}$ is the unique closest point to $\boldsymbol{x}$ among all scalar multiples of $\boldsymbol{y}$:

$$\|\boldsymbol{x} - \alpha \boldsymbol{y}\| \leq \|\boldsymbol{x} - \beta \boldsymbol{y}\| \qquad \text{for every } \beta \in \mathbb{R}.$$

Proof The residual $\boldsymbol{x} - \alpha \boldsymbol{y}$ is orthogonal to $\boldsymbol{y}$. For any scalar β,

$$\boldsymbol{x} - \beta \boldsymbol{y} = (\boldsymbol{x} - \alpha \boldsymbol{y}) + (\alpha - \beta) \boldsymbol{y}.$$

The two terms on the right are orthogonal. The Pythagorean theorem gives

$$\|\boldsymbol{x} - \beta\boldsymbol{y}\|^2 = \|\boldsymbol{x} - \alpha\boldsymbol{y}\|^2 + (\alpha - \beta)^2\|\boldsymbol{y}\|^2.$$

The second term is non-negative and vanishes only when $\beta = \alpha$. □

Def 1.16 **(Orthogonal Complement)** For a subspace $S \subseteq \mathbb{R}^n$, its orthogonal complement is

$$S^\perp = \{\boldsymbol{z} \in \mathbb{R}^n : \langle \boldsymbol{z}, \boldsymbol{s}\rangle = 0 \text{ for every } \boldsymbol{s} \in S\}.$$

Thm 1.12 **(Projection onto a Subspace with an Orthonormal Basis)** Let the columns of $\mathbf{Q} \in \mathbb{R}^{n\times k}$ be an orthonormal basis for a subspace S. Then

$$\mathbf{P} = \mathbf{Q}\mathbf{Q}^T$$

is the orthogonal projector onto S. Every vector has the unique decomposition

$$\boldsymbol{x} = \mathbf{P}\boldsymbol{x} + (\mathbf{I} - \mathbf{P})\boldsymbol{x}, \qquad \mathbf{P}\boldsymbol{x} \in S, \qquad (\mathbf{I} - \mathbf{P})\boldsymbol{x} \in S^\perp.$$

Moreover, $\mathbf{P}\boldsymbol{x}$ is the unique point in S closest to $\boldsymbol{x}$.

Proof The vector $\mathbf{P}\boldsymbol{x} = \mathbf{Q}(\mathbf{Q}^T\boldsymbol{x})$ is a linear combination of the columns of $\mathbf{Q}$, so it lies in S. For any vector $\boldsymbol{s} = \mathbf{Q}\boldsymbol{c} \in S$,

$$\begin{aligned}\langle(\mathbf{I} - \mathbf{P})\boldsymbol{x}, \boldsymbol{s}\rangle &= \boldsymbol{c}^T\mathbf{Q}^T(\mathbf{I} - \mathbf{Q}\mathbf{Q}^T)\boldsymbol{x} \\ &= \boldsymbol{c}^T(\mathbf{Q}^T - \mathbf{Q}^T\mathbf{Q}\mathbf{Q}^T)\boldsymbol{x} \\ &= \boldsymbol{c}^T(\mathbf{Q}^T - \mathbf{I}_k\mathbf{Q}^T)\boldsymbol{x} = 0.\end{aligned}$$

Thus $(\mathbf{I} - \mathbf{P})\boldsymbol{x} \in S^\perp$. For any $\boldsymbol{s} \in S$,

$$\boldsymbol{x} - \boldsymbol{s} = (\boldsymbol{x} - \mathbf{P}\boldsymbol{x}) + (\mathbf{P}\boldsymbol{x} - \boldsymbol{s})$$

is an orthogonal decomposition because $\mathbf{P}\boldsymbol{x} - \boldsymbol{s} \in S$ and $\boldsymbol{x} - \mathbf{P}\boldsymbol{x} \in S^\perp$. By the Pythagorean theorem,

$$\|\boldsymbol{x} - \boldsymbol{s}\|^2 = \|\boldsymbol{x} - \mathbf{P}\boldsymbol{x}\|^2 + \|\mathbf{P}\boldsymbol{x} - \boldsymbol{s}\|^2.$$

Since $\|\mathbf{P}\boldsymbol{x} - \boldsymbol{s}\|^2 \geq 0$ with equality if and only if $\boldsymbol{s} = \mathbf{P}\boldsymbol{x}$, this proves that $\mathbf{P}\boldsymbol{x}$ is the unique closest point. □

Remark 1.8 For an orthonormal basis, the projection coefficients are simply the inner products $\mathbf{Q}^T\boldsymbol{x}$. If the basis columns are stored in a general full-rank matrix $\mathbf{V}$, the closest vector $\mathbf{V}\boldsymbol{c}$ instead satisfies

$$\mathbf{V}^T\mathbf{V}\boldsymbol{c} = \mathbf{V}^T\boldsymbol{x}.$$

The matrix $\mathbf{V}^T\mathbf{V}$ records all pairwise inner products between the basis vectors. Orthogonality reduces this matrix to the identity. This is the first reason orthogonal bases simplify computation.

1.2.3 Norms as Choices of Measurement

An inner product supplies both length and angle. A norm keeps only the notion of length. This weaker structure is enough for error bounds and convergence, even when no meaningful angle is available.

Def 1.17 **(Vector Norm)** A function $\|\cdot\| : \mathbb{R}^n \to \mathbb{R}$ is a **vector norm** if, for all vectors $\boldsymbol{x}, \boldsymbol{y}$ and scalars α,

1. $\|\boldsymbol{x}\| \geq 0$, with equality if and only if $\boldsymbol{x} = \boldsymbol{0}$.
2. $\|\alpha\boldsymbol{x}\| = |\alpha|\|\boldsymbol{x}\|$.
3. $\|\boldsymbol{x} + \boldsymbol{y}\| \leq \|\boldsymbol{x}\| + \|\boldsymbol{y}\|$.

Def 1.18 **(The Standard 1-, 2-, and Infinity Norms)** For $\boldsymbol{x} \in \mathbb{R}^n$,

$$\|\boldsymbol{x}\|_1 = \sum_{i=1}^{n} |x_i|, \qquad \|\boldsymbol{x}\|_2 = \left(\sum_{i=1}^{n} x_i^2\right)^{1/2}, \qquad \|\boldsymbol{x}\|_\infty = \max_{1\leq i\leq n} |x_i|.$$

More generally, for $1 \leq p < \infty$,

$$\|\boldsymbol{x}\|_p = \left(\sum_{i=1}^{n} |x_i|^p\right)^{1/p}.$$

Each norm answers a different question. The 1-norm measures total magnitude across coordinates. The 2-norm measures Euclidean energy. The infinity norm measures the largest coordinate error. A tolerance stated without a norm is therefore incomplete.

Example Let $\boldsymbol{x} = (1, 1, 1, 1)^T$ and $\boldsymbol{y} = (4, 0, 0, 0)^T$. Both vectors satisfy $\|\boldsymbol{x}\|_1 = \|\boldsymbol{y}\|_1 = 4$, but

$$\|\boldsymbol{x}\|_2 = 2, \qquad \|\boldsymbol{y}\|_2 = 4, \qquad \|\boldsymbol{x}\|_\infty = 1, \qquad \|\boldsymbol{y}\|_\infty = 4.$$

The vectors have the same total magnitude, but one distributes it evenly and the other concentrates it in one coordinate. No single norm measures every interpretation of size.

Thm 1.13 **(Sharp Comparisons of the Standard Norms)** For every $\boldsymbol{x} \in \mathbb{R}^n$,

$$\|\boldsymbol{x}\|_\infty \leq \|\boldsymbol{x}\|_2 \leq \|\boldsymbol{x}\|_1 \leq \sqrt{n}\,\|\boldsymbol{x}\|_2 \leq n\|\boldsymbol{x}\|_\infty.$$

The factors $\sqrt{n}$ and n cannot be replaced by smaller constants that work for every vector.

Proof Let $|x_k| = \max_{1\leq i\leq n} |x_i| = \|\boldsymbol{x}\|_\infty$. Then

$$\|\boldsymbol{x}\|_\infty^2 = |x_k|^2 \leq \sum_{i=1}^{n} x_i^2 = \|\boldsymbol{x}\|_2^2 \quad \Longrightarrow \quad \|\boldsymbol{x}\|_\infty \leq \|\boldsymbol{x}\|_2.$$

Squaring $\|\boldsymbol{x}\|_1$ gives

$$\|\boldsymbol{x}\|_1^2 = \left(\sum_{i=1}^n |x_i|\right)^2 = \sum_{i=1}^n x_i^2 + 2\sum_{1\le i<j\le n} |x_i x_j| \ge \sum_{i=1}^n x_i^2 = \|\boldsymbol{x}\|_2^2,$$

which yields $\|\boldsymbol{x}\|_2 \le \|\boldsymbol{x}\|_1$. Applying Cauchy–Schwarz to $(|x_1|,\dots,|x_n|)^T$ and $(1,\dots,1)^T$ gives

$$\|\boldsymbol{x}\|_1 = \sum_{i=1}^n |x_i|\cdot 1 \le \left(\sum_{i=1}^n |x_i|^2\right)^{1/2}\left(\sum_{i=1}^n 1^2\right)^{1/2} = \sqrt{n}\,\|\boldsymbol{x}\|_2.$$

Finally, using $|x_i| \le \|\boldsymbol{x}\|_\infty$ for each $i = 1,\dots,n$,

$$\|\boldsymbol{x}\|_1 = \sum_{i=1}^n |x_i| \le \sum_{i=1}^n \|\boldsymbol{x}\|_\infty = n\|\boldsymbol{x}\|_\infty.$$

The coordinate vector $\boldsymbol{e}_1$ attains $\|\boldsymbol{e}_1\|_\infty = \|\boldsymbol{e}_1\|_2 = \|\boldsymbol{e}_1\|_1 = 1$, and the all-ones vector $\mathbf{1}$ satisfies $\|\mathbf{1}\|_1 = n = \sqrt{n}\|\mathbf{1}\|_2 = n\|\mathbf{1}\|_\infty$, showing that all constants are sharp. □

Thm 1.14 **(Equivalence of Norms in Fixed Finite Dimension)** For any two norms $\|\cdot\|_\alpha$ and $\|\cdot\|_\beta$ on $\mathbb{R}^n$, there are constants $c, C > 0$ such that

$$c\|\boldsymbol{x}\|_\beta \le \|\boldsymbol{x}\|_\alpha \le C\|\boldsymbol{x}\|_\beta \qquad \text{for every } \boldsymbol{x} \in \mathbb{R}^n.$$

Consequently, the two norms define the same convergent sequences on a fixed $\mathbb{R}^n$.

Proof Let $\boldsymbol{e}_1,\dots,\boldsymbol{e}_n$ be the coordinate vectors. The triangle inequality and Cauchy–Schwarz give

$$\|\boldsymbol{x}\|_\alpha = \left\|\sum_{i=1}^n x_i\boldsymbol{e}_i\right\|_\alpha \le \sum_{i=1}^n |x_i|\|\boldsymbol{e}_i\|_\alpha \le \left(\sum_{i=1}^n \|\boldsymbol{e}_i\|_\alpha^2\right)^{1/2}\|\boldsymbol{x}\|_2.$$

This bound and the reverse triangle inequality $|\|\boldsymbol{x}\|_\alpha - \|\boldsymbol{y}\|_\alpha| \le \|\boldsymbol{x}-\boldsymbol{y}\|_\alpha$ show that $\|\cdot\|_\alpha$ is continuous with respect to the Euclidean norm. On the compact Euclidean unit sphere $\mathbb{S}^{n-1} = \{\boldsymbol{x} \in \mathbb{R}^n : \|\boldsymbol{x}\|_2 = 1\}$, the function $\|\cdot\|_\alpha$ attains a minimum m and a maximum M. Positive definiteness implies $m > 0$. For any nonzero $\boldsymbol{x}$, evaluating at $\boldsymbol{x}/\|\boldsymbol{x}\|_2 \in \mathbb{S}^{n-1}$ gives

$$m\|\boldsymbol{x}\|_2 \le \|\boldsymbol{x}\|_\alpha \le M\|\boldsymbol{x}\|_2.$$

Applying this comparison to both α and β yields

$$\frac{m_\alpha}{M_\beta}\|\boldsymbol{x}\|_\beta \le \|\boldsymbol{x}\|_\alpha \le \frac{M_\alpha}{m_\beta}\|\boldsymbol{x}\|_\beta,$$

proving the equivalence. □

Remark 1.9 **(Equivalent does not mean interchangeable)** Norm equivalence says that convergence in one norm implies convergence in another when the dimension is fixed. It does not say that the numerical values or bounds are equally useful. The comparison constants may grow with n. A bound containing $\sqrt{n}$ may be harmless for ten unknowns and useless for ten million.
The 2-norm is induced by an inner product. The 1- and infinity norms are not. The difference is visible in the geometry of their unit balls and in a simple algebraic identity.

Thm 1.15 **(Parallelogram Identity)** Every norm induced by an inner product satisfies

$$\|\boldsymbol{x}+\boldsymbol{y}\|^2+\|\boldsymbol{x}-\boldsymbol{y}\|^2=2\|\boldsymbol{x}\|^2+2\|\boldsymbol{y}\|^2.$$

Proof Expanding the squared norms via the inner product:

$$\begin{aligned}\|\boldsymbol{x}+\boldsymbol{y}\|^2&=\langle\boldsymbol{x}+\boldsymbol{y},\boldsymbol{x}+\boldsymbol{y}\rangle=\|\boldsymbol{x}\|^2+2\langle\boldsymbol{x},\boldsymbol{y}\rangle+\|\boldsymbol{y}\|^2,\\ \|\boldsymbol{x}-\boldsymbol{y}\|^2&=\langle\boldsymbol{x}-\boldsymbol{y},\boldsymbol{x}-\boldsymbol{y}\rangle=\|\boldsymbol{x}\|^2-2\langle\boldsymbol{x},\boldsymbol{y}\rangle+\|\boldsymbol{y}\|^2.\end{aligned}$$

Adding the two equations cancels the inner-product cross terms:

$$\begin{aligned}\|\boldsymbol{x}+\boldsymbol{y}\|^2+\|\boldsymbol{x}-\boldsymbol{y}\|^2&=\big(\|\boldsymbol{x}\|^2+2\langle\boldsymbol{x},\boldsymbol{y}\rangle+\|\boldsymbol{y}\|^2\big)+\big(\|\boldsymbol{x}\|^2-2\langle\boldsymbol{x},\boldsymbol{y}\rangle+\|\boldsymbol{y}\|^2\big)\\ &=2\|\boldsymbol{x}\|^2+2\|\boldsymbol{y}\|^2.\end{aligned}$$

□

Example Take $\boldsymbol{x}=(1,0)^T$ and $\boldsymbol{y}=(0,1)^T$. For the 1-norm, the left side of the parallelogram identity equals $2^2+2^2=8$, while the right side equals $2+2=4$. Thus the 1-norm cannot come from any inner product. The same test shows that the infinity norm is not induced by an inner product.

Def 1.19 **(Weighted Inner Product)** Let $\mathbf{W}\in\mathbb{R}^{n\times n}$ be symmetric and satisfy $\boldsymbol{x}^T\mathbf{W}\boldsymbol{x}>0$ for every nonzero $\boldsymbol{x}$. Then

$$\langle\boldsymbol{x},\boldsymbol{y}\rangle_{\mathbf{W}}=\boldsymbol{x}^T\mathbf{W}\boldsymbol{y},\qquad\|\boldsymbol{x}\|_{\mathbf{W}}=\sqrt{\boldsymbol{x}^T\mathbf{W}\boldsymbol{x}}$$

define an inner product and its induced norm.
A weighted inner product changes the geometry in which lengths and angles are measured. Directions with large quadratic weight are expensive, and directions with small weight are cheap. In a finite-element method the weight may represent physical energy. In statistics it may account for unequal uncertainty. In optimization it may encode local curvature. The chapter on positive definite matrices develops this geometry further.

Ex 1.2

1. **Derive. Cauchy–Schwarz as a one-variable minimization.** For fixed $\boldsymbol{x}$ and nonzero $\boldsymbol{y}$, minimize $\|\boldsymbol{x}-t\boldsymbol{y}\|_2^2$ over $t\in\mathbb{R}$. Derive the minimizing coefficient, the projection formula, and Cauchy–Schwarz from the non-negativity of the minimum.

2. **Derive. Projection and uniqueness.** Let S have orthonormal basis matrix $\mathbf{Q}$. Prove directly that $\mathbf{P} = \mathbf{Q}\mathbf{Q}^T$ satisfies $\mathbf{P}^T = \mathbf{P}$ and $\mathbf{P}^2 = \mathbf{P}$. Then prove that these two identities characterize orthogonal projectors.

3. **Explore. Orthogonal versus oblique projection.** Let

$$\mathbf{P} = \begin{pmatrix} 1 & -2 \\ 0 & 0 \end{pmatrix}.$$

Verify that $\mathbf{P}^2 = \mathbf{P}$ but $\mathbf{P}^T \neq \mathbf{P}$. Find its range and null space. Give a vector for which $\mathbf{P}\boldsymbol{x}$ is not the closest point in the range of $\mathbf{P}$.

4. **Derive. Sharp norm constants.** Prove $\|\boldsymbol{x}\|_2 \leq \sqrt{n}\|\boldsymbol{x}\|_\infty$ and $\|\boldsymbol{x}\|_1 \leq \sqrt{n}\|\boldsymbol{x}\|_2$. For each inequality, find all vectors that attain equality up to scaling and sign changes.

5. **Explore. A norm without angles.** Use the parallelogram identity to prove that neither $\|\cdot\|_1$ nor $\|\cdot\|_\infty$ on $\mathbb{R}^2$ can be induced by an inner product. Explain which step of the projection argument fails when only a general norm is available.

6. **Derive. Weighted geometry.** Let

$$\mathbf{W} = \begin{pmatrix} 4 & 0 \\ 0 & 1 \end{pmatrix}.$$

Verify the inner-product axioms for $\langle \boldsymbol{x}, \boldsymbol{y} \rangle_{\mathbf{W}}$. Sketch the unit ball $\|\boldsymbol{x}\|_{\mathbf{W}} = 1$, find the $\mathbf{W}$-orthogonal complement of $(1, 1)^T$, and compute the $\mathbf{W}$-orthogonal projection of $(1, 0)^T$ onto the line spanned by $(1, 1)^T$.

7. **Explore. The norm is part of the question.** Two approximations have error vectors $\boldsymbol{e}_1 = (10^{-3}, \ldots, 10^{-3})^T$ and $\boldsymbol{e}_2 = (10^{-2}, 0, \ldots, 0)^T$ in $\mathbb{R}^{1000}$. Compare them in the 1-, 2-, and infinity norms. State which approximation is preferable under each measurement and explain why no ranking is meaningful until the application chooses a norm.

1.3 Operator Norms and Matrix Geometry

A vector norm measures the size of an input or an error. A matrix norm measures what the matrix does to such vectors. This point matters because the entries of a matrix alone do not determine its effect on data. A matrix with modest entries can strongly amplify a carefully aligned vector, while a matrix with large entries may act harmlessly on the particular subspace being used.

The induced norm turns this question into an optimization problem, asking for the largest amplification over all nonzero inputs. We define induced norms, derive their fundamental inequalities, and interpret the condition number through this geometric distortion.

1.3.1 Induced Norms as Worst-Case Amplification

Def 1.20 **(Induced Operator Norm)** Let $\|\cdot\|_{\text{in}}$ be a norm on $\mathbb{R}^n$ and let $\|\cdot\|_{\text{out}}$ be a norm on $\mathbb{R}^m$. For $\mathbf{A} \in \mathbb{R}^{m\times n}$, the induced operator norm is

$$\|\mathbf{A}\| = \sup_{\boldsymbol{x}\neq\mathbf{0}} \frac{\|\mathbf{A}\boldsymbol{x}\|_{\text{out}}}{\|\boldsymbol{x}\|_{\text{in}}} = \max_{\|\boldsymbol{x}\|_{\text{in}}=1} \|\mathbf{A}\boldsymbol{x}\|_{\text{out}}.$$

When the same p-norm is used in the domain and range, we write $\|\mathbf{A}\|_p$.
The quotient is unchanged when $\boldsymbol{x}$ is rescaled, so only directions matter. Restricting to the unit sphere removes the irrelevant magnitude. In finite dimensions the unit sphere is compact and the objective is continuous, so the maximum is attained.

Thm 1.16 **(Fundamental Properties of Induced Norms)** For compatible matrices and vectors,

1. **Consistency:** $\|\mathbf{A}\boldsymbol{x}\| \leq \|\mathbf{A}\|\,\|\boldsymbol{x}\|$.
2. **Submultiplicativity:** $\|\mathbf{AB}\| \leq \|\mathbf{A}\|\,\|\mathbf{B}\|$.
3. **Identity:** $\|\mathbf{I}\| = 1$.
4. **Norm axioms:** the induced norm is positive definite, absolutely homogeneous, and satisfies the triangle inequality.

Proof For $\boldsymbol{x} \neq \mathbf{0}$, the definition gives

$$\frac{\|\mathbf{A}\boldsymbol{x}\|}{\|\boldsymbol{x}\|} \leq \|\mathbf{A}\|,$$

which proves consistency. Applying consistency to $\mathbf{A}$ and then to $\mathbf{B}$ gives

$$\|\mathbf{AB}\boldsymbol{x}\| \leq \|\mathbf{A}\|\,\|\mathbf{B}\boldsymbol{x}\| \leq \|\mathbf{A}\|\,\|\mathbf{B}\|\,\|\boldsymbol{x}\|.$$

Taking the maximum over unit vectors proves submultiplicativity. The identity property follows from $\|\mathbf{I}\boldsymbol{x}\|/\|\boldsymbol{x}\| = 1$. Homogeneity follows by pulling $|\alpha|$ out of the numerator. For the triangle inequality,

$$\|(\mathbf{A}+\mathbf{B})\boldsymbol{x}\| \leq \|\mathbf{A}\boldsymbol{x}\| + \|\mathbf{B}\boldsymbol{x}\| \leq (\|\mathbf{A}\| + \|\mathbf{B}\|)\|\boldsymbol{x}\|.$$

Taking the maximum over unit vectors completes the proof. □

Example For the diagonal matrix

$$\mathbf{D} = \begin{pmatrix} 4 & 0 \\ 0 & 1/2 \end{pmatrix},$$

the Euclidean unit circle is sent to an ellipse with semiaxis lengths 4 and $1/2$. Hence $\|\mathbf{D}\|_2 = 4$. The norm records the greatest stretch, not the average stretch and not the

largest entry by convention. In this example those quantities happen to agree because the coordinate axes are the stretching directions.

Thm 1.17 **(The Induced 1- and Infinity Norms)** For $\mathbf{A} = (a_{ij}) \in \mathbb{R}^{m\times n}$,

$$\|\mathbf{A}\|_1 = \max_{1\le j\le n} \sum_{i=1}^{m} |a_{ij}|, \qquad\qquad \|\mathbf{A}\|_\infty = \max_{1\le i\le m} \sum_{j=1}^{n} |a_{ij}|.$$

Thus the induced 1-norm is the largest absolute column sum, while the induced infinity norm is the largest absolute row sum.

Proof Let $C = \max_j \sum_i |a_{ij}|$. For every vector $\boldsymbol{x}$,

$$\begin{aligned}\|\mathbf{A}\boldsymbol{x}\|_1 &\le \sum_{i=1}^{m}\sum_{j=1}^{n} |a_{ij}||x_j| \\ &= \sum_{j=1}^{n} |x_j| \sum_{i=1}^{m} |a_{ij}| \\ &\le C\|\boldsymbol{x}\|_1.\end{aligned}$$

If column j_* attains C, then $\|\boldsymbol{e}_{j_*}\|_1 = 1$ and $\|\mathbf{A}\boldsymbol{e}_{j_*}\|_1 = C$, so the bound is sharp.
Let $R = \max_i \sum_j |a_{ij}|$. If $\|\boldsymbol{x}\|_\infty = 1$, then

$$\|\mathbf{A}\boldsymbol{x}\|_\infty \le \max_i \sum_j |a_{ij}||x_j| \le R.$$

Choose a row i_* attaining R and take $x_j = \operatorname{sign}(a_{i_*j})$, with either sign allowed when an entry is zero. Then $\|\boldsymbol{x}\|_\infty = 1$ and the i_*-th component of $\mathbf{A}\boldsymbol{x}$ has magnitude R. □

Def 1.21 **(The Induced Euclidean Norm)** The induced Euclidean norm is

$$\|\mathbf{A}\|_2 = \max_{\|\boldsymbol{x}\|_2=1} \|\mathbf{A}\boldsymbol{x}\|_2.$$

Equivalently,

$$\|\mathbf{A}\|_2^2 = \max_{\|\boldsymbol{x}\|_2=1} \boldsymbol{x}^T\mathbf{A}^T\mathbf{A}\boldsymbol{x}.$$

The second expression follows by squaring the output norm. It already shows why $\mathbf{A}^T\mathbf{A}$ appears in questions about Euclidean amplification. Once the singular value decomposition is available, Thm 4.19 proves the computable formula

$$\|\mathbf{A}\|_2 = \sqrt{\lambda_{\max}(\mathbf{A}^T\mathbf{A})} = \sigma_{\max}(\mathbf{A}).$$

Thm 1.18 **(Orthogonal Transformations Have Unit Euclidean Norm)** If $\mathbf{Q}^T\mathbf{Q} = \mathbf{I}$, then

$$\|\mathbf{Q}\boldsymbol{x}\|_2 = \|\boldsymbol{x}\|_2 \qquad \text{and} \qquad \|\mathbf{Q}\|_2 = 1.$$

Proof For every vector $\boldsymbol{x}$,

$$\|\mathbf{Q}\boldsymbol{x}\|_2^2 = \boldsymbol{x}^T\mathbf{Q}^T\mathbf{Q}\boldsymbol{x} = \boldsymbol{x}^T\boldsymbol{x} = \|\boldsymbol{x}\|_2^2.$$

Every unit vector is therefore mapped to a unit vector, so the maximum amplification is one. □

1.3.2 Dual Norms and Sharp Linear Bounds

An inner product turns a vector $\boldsymbol{y}$ into the linear measurement $\boldsymbol{x} \mapsto \boldsymbol{y}^T\boldsymbol{x}$. The dual norm measures the largest value of this measurement on the unit ball.

Def 1.22 **(Dual Norm)** For a norm $\|\cdot\|$ on $\mathbb{R}^n$, its dual norm is

$$\|\boldsymbol{y}\|_* = \max_{\|\boldsymbol{x}\|=1} |\boldsymbol{y}^T\boldsymbol{x}|.$$

Consequently,

$$|\boldsymbol{y}^T\boldsymbol{x}| \le \|\boldsymbol{y}\|_*\|\boldsymbol{x}\|.$$

Thm 1.19 **(Duals of the Standard Norms)** The Euclidean norm is self-dual. The 1- and infinity norms are dual to one another:

$$\|\boldsymbol{y}\|_{2,*} = \|\boldsymbol{y}\|_2, \qquad \|\boldsymbol{y}\|_{1,*} = \|\boldsymbol{y}\|_\infty, \qquad \|\boldsymbol{y}\|_{\infty,*} = \|\boldsymbol{y}\|_1.$$

Proof Cauchy–Schwarz gives $|\boldsymbol{y}^T\boldsymbol{x}| \le \|\boldsymbol{y}\|_2\|\boldsymbol{x}\|_2$, with equality when $\boldsymbol{x}$ is a normalized copy of $\boldsymbol{y}$. For the 1-norm,

$$|\boldsymbol{y}^T\boldsymbol{x}| \le \sum_i |y_i||x_i| \le \|\boldsymbol{y}\|_\infty\|\boldsymbol{x}\|_1.$$

A signed coordinate vector supported where $|y_i|$ is largest attains equality. For the infinity norm,

$$|\boldsymbol{y}^T\boldsymbol{x}| \le \sum_i |y_i||x_i| \le \|\boldsymbol{y}\|_1\|\boldsymbol{x}\|_\infty.$$

Choosing $x_i = \operatorname{sign}(y_i)$ attains equality. □

Dual norms explain why the norm used for a residual determines the norm that appears in the corresponding linear functional. They also explain the row-sum and column-sum formulas above. The transpose interchanges inputs with linear measurements, giving the useful identity

$$\|\mathbf{A}\|_1 = \|\mathbf{A}^T\|_\infty.$$

1.3.3 The Frobenius Geometry of Matrices

Matrices themselves form a vector space. When a matrix is being treated as data rather than as an operator, the natural Euclidean measurement is entrywise.

Def 1.23 **(Frobenius Inner Product and Norm)** For matrices $\mathbf{A}, \mathbf{B} \in \mathbb{R}^{m\times n}$, the Frobenius inner product is

$$\langle \mathbf{A}, \mathbf{B}\rangle_F = \sum_{i=1}^{m}\sum_{j=1}^{n} a_{ij}b_{ij} = \operatorname{trace}(\mathbf{A}^T\mathbf{B}).$$

Its induced norm is

$$\|\mathbf{A}\|_F = \left(\sum_{i=1}^{m}\sum_{j=1}^{n} a_{ij}^2\right)^{1/2}.$$

Remark 1.10 **(Two different questions)** The Frobenius norm measures the collective size of the entries. The induced 2-norm measures the largest amplification of a vector. Both are useful, but they answer different questions. Calling both a "matrix norm" does not make them interchangeable.

Thm 1.20 **(Bounds Relating Frobenius and Operator Norms)** For $\mathbf{A} \in \mathbb{R}^{m\times n}$,

$$\|\mathbf{A}\|_2 \le \|\mathbf{A}\|_F \le \sqrt{n}\,\|\mathbf{A}\|_2.$$

For compatible matrices $\mathbf{A}$ and $\mathbf{B}$,

$$\|\mathbf{AB}\|_F \le \|\mathbf{A}\|_2\|\mathbf{B}\|_F.$$

Proof Let $\boldsymbol{a}_i^T$ denote row i of $\mathbf{A}$. For a Euclidean unit vector $\boldsymbol{x}$, Cauchy–Schwarz gives

$$\|\mathbf{A}\boldsymbol{x}\|_2^2 = \sum_i |\boldsymbol{a}_i^T\boldsymbol{x}|^2 \le \sum_i \|\boldsymbol{a}_i\|_2^2\|\boldsymbol{x}\|_2^2 = \|\mathbf{A}\|_F^2.$$

Taking the maximum over unit vectors proves $\|\mathbf{A}\|_2 \le \|\mathbf{A}\|_F$. Writing $\mathbf{A}$ by columns gives

$$\|\mathbf{A}\|_F^2 = \sum_{j=1}^{n} \|\mathbf{A}\boldsymbol{e}_j\|_2^2 \le \sum_{j=1}^{n} \|\mathbf{A}\|_2^2 = n\|\mathbf{A}\|_2^2.$$

Finally, applying consistency to every column $\boldsymbol{b}_j$ of $\mathbf{B}$:

$$\begin{aligned}\|\mathbf{AB}\|_F^2 &= \sum_{j=1}^{k} \|\mathbf{A}\boldsymbol{b}_j\|_2^2 \\ &\le \sum_{j=1}^{k} \|\mathbf{A}\|_2^2 \|\boldsymbol{b}_j\|_2^2 \\ &= \|\mathbf{A}\|_2^2 \sum_{j=1}^{k} \|\boldsymbol{b}_j\|_2^2 \\ &= \|\mathbf{A}\|_2^2 \|\mathbf{B}\|_F^2.\end{aligned}$$

Taking square roots yields $\|\mathbf{AB}\|_F \le \|\mathbf{A}\|_2 \|\mathbf{B}\|_F$. □

1.3.4 Condition Number as Distortion

An invertible matrix can stretch some directions and compress others. A solve with that matrix must undo both effects. The condition number measures the total distortion.

Def 1.24 **(Condition Number)** For a nonsingular square matrix $\mathbf{A}$ and an induced norm,

$$\kappa(\mathbf{A}) = \|\mathbf{A}\| \, \|\mathbf{A}^{-1}\|.$$

Thm 1.21 **(Two-Sided Stretching Bound)** For every nonzero vector $\boldsymbol{x}$,

$$\frac{1}{\|\mathbf{A}^{-1}\|} \le \frac{\|\mathbf{A}\boldsymbol{x}\|}{\|\boldsymbol{x}\|} \le \|\mathbf{A}\|.$$

The ratio between the largest possible upper stretch and the guaranteed lower stretch is $\kappa(\mathbf{A})$.

Proof The upper bound follows directly from the definition of the induced norm:

$$\frac{\|\mathbf{A}\boldsymbol{x}\|}{\|\boldsymbol{x}\|} \le \max_{\boldsymbol{z} \neq \mathbf{0}} \frac{\|\mathbf{A}\boldsymbol{z}\|}{\|\boldsymbol{z}\|} = \|\mathbf{A}\|.$$

For the lower bound, write $\boldsymbol{x} = \mathbf{A}^{-1}(\mathbf{A}\boldsymbol{x})$. Consistency applied to $\mathbf{A}^{-1}$ yields

$$\|\boldsymbol{x}\| = \|\mathbf{A}^{-1}(\mathbf{A}\boldsymbol{x})\| \le \|\mathbf{A}^{-1}\| \, \|\mathbf{A}\boldsymbol{x}\|.$$

Since $\boldsymbol{x} \neq \mathbf{0}$ and $\mathbf{A}$ is nonsingular, $\|\mathbf{A}\boldsymbol{x}\| > 0$. Dividing by $\|\mathbf{A}^{-1}\| \, \|\boldsymbol{x}\|$ gives

$$\frac{1}{\|\mathbf{A}^{-1}\|} \le \frac{\|\mathbf{A}\boldsymbol{x}\|}{\|\boldsymbol{x}\|}.$$

Combining the two inequalities establishes the bound. □

If $\kappa(\mathbf{A})$ is near one, the matrix treats every direction comparably. If it is large, at least one direction is compressed relative to another. Recovering a component in a strongly compressed direction requires amplification by $\mathbf{A}^{-1}$. Consequently, ill-conditioned linear systems can turn small data errors into large solution errors. Theorem 2.7 makes this statement quantitative.

Ex 1.3

1. **Compute. Three induced norms.** For

$$\mathbf{A} = \begin{pmatrix} 1 & -2 \\ 3 & 4 \end{pmatrix},$$

compute $\|\mathbf{A}\|_1$ and $\|\mathbf{A}\|_\infty$. Estimate $\|\mathbf{A}\|_2$ from below by testing at least four unit vectors and from above using $\|\mathbf{A}\|_F$.

2. **Derive. Submultiplicativity from amplification.** Prove $\|\mathbf{A}_1 \cdots \mathbf{A}_k\| \le \prod_{j=1}^{k} \|\mathbf{A}_j\|$. Explain why this bound may be conservative when the direction of greatest stretch for one factor is not mapped into the direction of greatest stretch for the next.

3. **Explore. Entries do not determine action.** Compare

$$\mathbf{A} = \begin{pmatrix} 1 & 1 \\ 1 & 1 \end{pmatrix}, \qquad \mathbf{B} = \begin{pmatrix} 1 & 1 \\ -1 & -1 \end{pmatrix}.$$

Both matrices have the same entry magnitudes. Find their ranges and null spaces, identify directions they annihilate, and explain why an entrywise description misses this structure.

4. **Derive. Duality and transpose.** Starting from the definition of a dual norm, prove $\|\mathbf{A}\|_1 = \|\mathbf{A}^T\|_\infty$. Then formulate the corresponding statement for an arbitrary norm and its dual.

5. **Derive. Frobenius geometry.** Verify the inner-product axioms for $\langle \mathbf{A}, \mathbf{B} \rangle_F$. Prove the matrix Cauchy–Schwarz inequality $|\langle \mathbf{A}, \mathbf{B} \rangle_F| \le \|\mathbf{A}\|_F \|\mathbf{B}\|_F$ by treating the entries as vectors.

6. **Explore. Conditioning as a thin ellipse.** Let $\mathbf{A} = \operatorname{diag}(1, \varepsilon)$ with $0 < \varepsilon < 1$. Describe the image of the Euclidean unit circle, compute $\|\mathbf{A}\|_2$, $\|\mathbf{A}^{-1}\|_2$, and $\kappa_2(\mathbf{A})$ directly from the definition, and explain which component of a solution is sensitive to perturbations.

7. **Explore. Changing units.** Suppose the two coordinates of $\boldsymbol{x}$ are measured in meters and millimeters. Introduce a diagonal scaling matrix that expresses both in meters. Derive the transformed matrix for the same linear map and compare its induced norm before and after scaling. Explain why a condition number is meaningful only after the input and output measurements have been specified.

1.4 Floating Point Arithmetic

Digital computers represent real numbers using a finite number of bits. The representable numbers form a finite, non-uniform grid. Numerical analysis begins with this mismatch. The mathematical algorithm acts on real numbers, and the computer executes a nearby algorithm on rounded numbers.

Floating-point arithmetic is usually accurate locally, since one elementary operation is rounded to a nearby representable number. Instability appears when an algorithm repeatedly amplifies these tiny local errors, or when it subtracts nearly equal quantities and exposes digits that were already contaminated by rounding.

1.4.1 Floating Point Representation

Most modern systems adhere to the IEEE 754 standard, which represents numbers in a scientific-like notation. Normalized numbers take the form $x = (-1)^s \times (1.f) \times 2^e$.

- **s:** Sign bit, determining whether the number is positive or negative.
- **f:** Fractional significand (mantissa). The leading 1 is implicit to maximize precision.
- **e:** Biased exponent, allowing numbers spanning many orders of magnitude to be represented.

The exponent controls scale and the significand controls precision. This means floating-point numbers have roughly fixed **relative** spacing, not fixed absolute spacing. There are many representable numbers near zero and far fewer, in absolute terms, near 10^{100}. The gap between adjacent numbers near x is proportional to $|x|$.

Remark 1.11 **(IEEE 754 Standards)**

- **Single (32-bit):** 1 sign, 8 exponent, 23 fraction bits.
- **Double (64-bit):** 1 sign, 11 exponent, 52 fraction bits.

Example **(16-bit float)** 1 sign, 5 exp (bias 15), 10 frac. Represent -3.25:

1. $s = 1$.
2. $3.25_{10} = 11.01_2 = 1.101_2 \times 2^1$.
3. $e = 1 + 15 = 16 = 10000_2$.
4. $f = 1010000000_2$.

Result: `1 10000 1010000000`.

Ex 1.4 **Check.**

1. Encode +5.5 in 16-bit format.

2. How many distinct normalized numbers for 5 exp bits, 10 frac bits?

Def 1.25 **(Machine Epsilon ($\varepsilon_{\text{mach}}$))** Machine epsilon is the distance from 1 to the next larger floating-point number. Equivalently, it is the smallest positive floating-point number ε such that $\text{fl}(1+\varepsilon) > 1$. It characterizes the spacing of representable numbers near 1 and is given by $\varepsilon_{\text{mach}} = 2^{1-p}$, where p is the number of bits in the significand.

- For double precision (64-bit), $\varepsilon_{\text{mach}} \approx 2.22 \times 10^{-16}$.

Remark 1.12 **(Spacing rule)** Near a normalized number x, the distance to the next representable number is approximately $\varepsilon_{\text{mach}}|x|$. Near 10^{16}, the gap is on the order of 1, whereas near 1, the gap is on the order of 10^{-16}. This explains why adding a small number to a sufficiently large number has no effect.

Thm 1.22 **(Absorption Criterion)** In round-to-nearest arithmetic, if $|y|$ is less than half the spacing between adjacent floating-point numbers near x, then

$$\text{fl}(x+y) = x.$$

Equivalently, small increments disappear when they are below the local resolution of the floating-point grid.

Ex 1.5 **Compute.**

1. Use Def 1.25 to predict whether `1.0 + 1e-16` and `1.0 + 2e-16` differ from `1.0`. Then test the prediction in Python.

2. Find $\varepsilon_{\text{mach}}$ empirically via loop.

3. Use Thm 1.22 to predict the smallest power of 10 for which `1e16 + 10**k` changes `1e16`.

4. Explain why the answer changes if `1e16` is replaced by `1.0`.

Def 1.26 **(Unit Roundoff (u))** Unit roundoff is the maximum relative error caused by rounding one real number to the nearest floating-point number:

$$\frac{|\text{fl}(x) - x|}{|x|} \leq u.$$

For round-to-nearest arithmetic, $u = \varepsilon_{\text{mach}}/2 = 2^{-p}$.

Remark 1.13 **(Machine epsilon vs. unit roundoff)** $\varepsilon_{\text{mach}}$ is a spacing near 1. The unit roundoff u is an error bound for rounding to nearest. Many texts use "machine epsilon" for both

quantities, so always check the convention. In these notes, elementary rounding errors are bounded by u.

Thm 1.23 **(Fundamental Axiom)** Suppose the exact result $a \circ b$ is finite, nonzero, and lies in the normalized floating-point range. In round-to-nearest arithmetic,

$$\mathrm{fl}(a \circ b) = (a \circ b)(1 + \delta), \qquad |\delta| \leq u,$$

for $\circ \in \{+, -, \times, \div\}$. Overflow and gradual underflow require separate absolute-error models.

Proof The real line near x is partitioned into intervals, each containing exactly one floating point number. In round-to-nearest mode, $\mathrm{fl}(x)$ is the closer endpoint, so:

$$|\mathrm{fl}(x) - x| \leq \frac{1}{2}\mathrm{gap} = \frac{1}{2}(\varepsilon_{\mathrm{mach}}|x|) = u|x|.$$

Dividing by $|x|$ gives the relative error bound $|\delta| \leq u$. For any correctly rounded basic operation whose exact result remains in the normalized range, rounding gives

$$\mathrm{fl}(a \circ b) = \mathrm{round}(a \circ b) = (a \circ b)(1 + \delta).$$

□

Remark 1.14 **(Accumulation of rounding errors)** A sequence of n benign operations often produces an error proportional to nu when $nu \ll 1$. The rule is a heuristic. Cancellation and ill-conditioning can amplify the perturbations well beyond nu. Operation order matters because every intermediate result is rounded.

Ex 1.6 **Check.**

1. Let $a = 1, b = -0.999\ldots$. Is error in $\mathrm{fl}(a + b)$ bounded by u?
2. Estimate worst-case relative error for 10^6 sequential additions.

1.4.2 IEEE 754 Special Values

- **Signed Zero:** $+0, -0$. $1/(+0) = \infty$, $1/(-0) = -\infty$.
- **Subnormals:** Fill the gap between 0 and the smallest normalized number. Reduced precision, allows gradual underflow.
- **Infinities:** $\pm\infty$. Result of overflow or $x/0$.
- **NaN:** Not a Number ($0/0$, $\infty - \infty$). Unequal to everything, including itself.

Remark 1.15 **(NaN propagation)** A NaN usually contaminates all later arithmetic. This behavior helps with debugging, because the NaN marks an invalid operation that happened earlier even if the failure is observed much later.

Ex 1.7 **Compute.**

1. Using NumPy scalars, compare `np.float64(1.0)/np.float64(0.0)` with division by `np.float64(-0.0)`. Explain the warnings and the signs of the results. (Python's built-in float division raises `ZeroDivisionError` instead.)
2. Evaluate `float('inf') - float('inf')` and `float('nan') == float('nan')`.

1.4.3 Arithmetic Pitfalls

Because floating-point numbers are discrete, the standard laws of algebra do not always hold. In particular, operations are neither associative nor distributive in general.

- **Non-Associativity:** The order of operations can change the result, i.e., $(a+b)+c \neq a+(b+c)$.
- **Non-Distributivity:** $a(b+c) \neq ab+ac$.
- **Absorption:** If $|a|$ is much larger than $|b|$, then $a+b$ may equal a exactly because b is smaller than the gap between representable numbers near a.

Example **(Absorption)** In double precision, `1e16 + 1.0 == 1e16` is true. The number 1 is smaller than the spacing between adjacent representable numbers near 10^{16}, so it is rounded away. Consequently `(1e16 + 1.0) - 1e16` returns `0.0`, not `1.0`.

Example **(Non-associativity)** Let $a = 10^{16}$, $b = -10^{16}$, and $c = 1$. Then

$$(a+b)+c = 1, \qquad a+(b+c) = 0$$

in double precision. The second expression first rounds $b+c$ back to -10^{16}, so the 1 is lost before the final addition.

Remark 1.16 **(Summation rule)** Add numbers of similar magnitude when possible. Summing from smallest to largest, pairwise summation, or compensated summation reduces the loss of small terms. Consequently, library reductions may give different answers than a hand-written loop and still be more accurate.

Ex 1.8 **Explore.**

1. Before running Python, use Thm 1.22 to predict `(1e16 + 1.0) - 1e16`.
2. `a=1e16, b=1`. Find `(a+b)-a` in Python. Explain result.
3. Find a, b, c where `a*(b+c) != a*b + a*c`.

4. Before running Python, predict the results of summing $[10^{16}, 1, -10^{16}]$ in the listed order and in the order $[10^{16}, -10^{16}, 1]$.

5. Compare naive summation, pairwise summation, and Kahan summation on a list containing one large number and many small numbers. Which method best preserves the small terms?

1.5 Quantifying Approximation Error

1.5.1 Absolute and Relative Errors

- **Absolute Error:** $|\hat{x} - x|$.
- **Relative Error:** $\varepsilon = \frac{|\hat{x}-x|}{|x|}$. This is the standard measure for precision in numerical computing.

Absolute error measures the size of the mistake in the original units. Relative error measures the size of the mistake compared to the quantity being computed. In numerical computing, relative error is usually more informative. An error of 10^{-6} is excellent for a number of size 10^3 and terrible for a number of size 10^{-9}.

Remark 1.17 **(Comparing floating-point numbers)** Avoid testing computed real numbers with exact equality. Use a tolerance such as

$$|\hat{x} - x| \leq \text{atol} + \text{rtol}|x|.$$

This principle underlies `np.isclose`. The absolute tolerance bounds errors near zero, whereas the relative tolerance scales with magnitude.

Ex 1.9 **Check.**

1. $\sqrt{2} \approx 1.414$. Errors?
2. Relative error of product $\hat{x}\hat{y}$? (Sum of individual relative errors).

Def 1.27 **(Catastrophic Cancellation) Catastrophic cancellation** occurs when two nearly equal approximations are subtracted. The leading digits cancel, so errors already present in the operands become large relative to the small difference.

- Relative error bound: $\varepsilon_{\text{rel}} \leq u\frac{|x|+|y|}{|x-y|}$.
- As $x \to y$, the amplification factor $(|x| + |y|)/|x - y|$ can become arbitrarily large.

Proof Let $\hat{x} = x(1+\varepsilon_x)$ and $\hat{y} = y(1+\varepsilon_y)$ with $|\varepsilon_x|, |\varepsilon_y| \leq u$. The error in the subtraction is:

$$\begin{aligned}(\hat{x} - \hat{y}) - (x - y) &= x(1+\varepsilon_x) - y(1+\varepsilon_y) - x + y \\ &= x\varepsilon_x - y\varepsilon_y.\end{aligned}$$

The relative error is bounded by:

$$\begin{aligned}\frac{|x\varepsilon_x - y\varepsilon_y|}{|x-y|} &\leq \frac{|x||\varepsilon_x| + |y||\varepsilon_y|}{|x-y|} \\ &\leq u\frac{|x|+|y|}{|x-y|}.\end{aligned}$$

As $x \to y$, the denominator $x - y \to 0$, causing the relative error to explode. □

Remark 1.18 **(Numerical stability principle)** Rounding error is an inevitable consequence of finite precision. However, catastrophic cancellation is an algorithmic flaw that can often be avoided by mathematically rearranging the problem.

Example **(Stabilizing Cancellation)** $f(x) = \sqrt{x+1} - \sqrt{x}$ for $x = 10^{16}$.

- **Direct:** In double precision, both square roots round to 10^8, so the computed difference is zero.
- **Stable:** $\frac{1}{\sqrt{x+1}+\sqrt{x}} \approx 5 \times 10^{-9}$.

Example **(Quadratic formula)** The roots of $ax^2 + bx + c = 0$ are

$$x_\pm = \frac{-b \pm \sqrt{b^2 - 4ac}}{2a}.$$

If $b^2 \gg 4ac$, one numerator subtracts nearly equal numbers. A stable strategy computes the root with no cancellation first, then uses $x_1 x_2 = c/a$ to recover the other root.

Remark 1.19 **(Use stable library functions)** Functions such as `log1p(x)`, `expm1(x)`, and `hypot(x, y)` implement stable rearrangements for common cancellation and overflow patterns. Prefer them over hand-expanded formulas when the inputs may be small, large, or nearly cancelling.

Ex 1.10 **Explore.** Stabilize:

1. Use Def 1.27 to explain why $(1 - \cos x)/x^2$ is unstable for small x, then derive a stable form.
2. Use Def 1.27 to identify which root of $x^2 - 10^8 x + 1 = 0$ is unstable in the quadratic formula, then compute it using $x_1 x_2 = c/a$.
3. Use the Taylor series of e^x to explain why $e^x - 1$ loses relative accuracy for small x. Compare with `np.expm1`.

1.5.2 Error Analysis Frameworks

A computed result can be judged in two ways. Forward error analysis bounds the distance between the computed output and the exact output. Backward error analysis instead seeks the smallest perturbation of the input for which the computed output is exact. An algorithm is **backward stable** if this perturbation is a modest multiple of the unit roundoff u for every input.

Backward error is often easier to bound than forward error, and it is directly comparable with the uncertainty in the data. If the input is known only to a relative accuracy larger than the backward error, the algorithm has introduced no error beyond what the data already contain. The condition number then converts a backward error into a forward error.

Def 1.28 **(Condition Number (κ))** The condition number measures the local sensitivity of a problem's output to perturbations in its input. For sufficiently small relative perturbations, the first-order relation has the form

$$\text{relative forward error} \lesssim \kappa \, \text{relative backward error},$$

with the precise definition of κ determined by the problem and the chosen norms. For the linear system $\mathbf{A}\boldsymbol{x} = \boldsymbol{b}$, Section 2.2 shows that this role is played by the matrix condition number of Def 1.24.

Remark 1.20 **(Stability vs. conditioning)** Conditioning is a property of the mathematical problem. Stability is a property of the algorithm. A stable algorithm can still return a poor answer on an ill-conditioned problem, because the problem itself amplifies small perturbations.

Ex 1.11 **Check.** A problem has condition number $\kappa = 10^8$, and an algorithm computes a solution with relative backward error 10^{-15}. Use Def 1.28 to estimate the largest relative forward error, and state how many correct significant digits the result can be expected to have.

1.6 Cost Models and Sparse Storage

Two algorithms that compute the same quantity can differ in cost by orders of magnitude. We measure cost by the number of floating-point operations an algorithm performs and by the memory it stores and reads. Most large matrices in these notes have only a few nonzero entries in each row, and for such matrices the storage format determines both the memory use and the cost of the basic operations.

We first define asymptotic notation and count the operations of the basic dense kernels. We then describe how a sparse matrix is stored, how a matrix–vector product is computed in that format, and how a sparse matrix is assembled from local contributions.

1.6.1 Asymptotic Notation and Operation Counts

Def 1.29 **(Asymptotic Notation)** Let f and g be positive functions of an integer n. We write $f(n) = O(g(n))$ if there are constants $C > 0$ and n_0 such that $f(n) \le Cg(n)$ for every $n \ge n_0$. We write $f(n) = \Theta(g(n))$ if both $f(n) = O(g(n))$ and $g(n) = O(f(n))$ hold.

Def 1.30 **(Flop)** A **flop** is one floating-point addition, subtraction, multiplication, division, or square root.

Thm 1.24 **(Costs of the Basic Dense Kernels)** Let $\boldsymbol{x}, \boldsymbol{y} \in \mathbb{R}^n$, $\mathbf{A} \in \mathbb{R}^{m\times n}$, $\mathbf{B} \in \mathbb{R}^{n\times p}$, and let $\mathbf{L} \in \mathbb{R}^{n\times n}$ be nonsingular and lower triangular. The standard algorithms require

1. $2n-1$ flops for the inner product $\boldsymbol{x}^T\boldsymbol{y}$,
2. $m(2n-1)$ flops for the product $\mathbf{A}\boldsymbol{x}$,
3. $mp(2n-1)$ flops for the product $\mathbf{AB}$,
4. n^2 flops for the solution of $\mathbf{L}\boldsymbol{z} = \boldsymbol{y}$ by forward substitution.

Proof The inner product forms n products and adds them with $n-1$ additions. The product $\mathbf{A}\boldsymbol{x}$ consists of m inner products of length n, and $\mathbf{AB}$ consists of mp such inner products, one for each entry. Forward substitution computes $z_i = (y_i - \sum_{j<i} \ell_{ij} z_j)/\ell_{ii}$, which uses $i-1$ multiplications, $i-1$ subtractions, and one division. Summing $2i-1$ over $i = 1, \dots, n$ gives n^2. □

The order of evaluation matters even when the result does not change. For $\mathbf{A}, \mathbf{B} \in \mathbb{R}^{n\times n}$ and $\boldsymbol{x} \in \mathbb{R}^n$, forming $(\mathbf{AB})\boldsymbol{x}$ costs about $2n^3$ flops, whereas $\mathbf{A}(\mathbf{B}\boldsymbol{x})$ costs about $4n^2$ flops. The factorizations of later sections have costs of the same kind. Dense LU factorization requires about $\frac{2}{3}n^3$ flops (Thm 2.13), Cholesky about $\frac{1}{3}n^3$, and Householder QR of an $m \times n$ matrix about $2mn^2 - \frac{2}{3}n^3$.

Example **(The dense barrier)** At a sustained rate of 10^{10} flops per second, a dense LU factorization with $n = 10^4$ requires about $\frac{2}{3} \cdot 10^{12}$ flops, or roughly one minute. With $n = 10^5$ it requires about $\frac{2}{3} \cdot 10^{15}$ flops, or roughly nineteen hours, and storing the matrix in double precision requires $8 \cdot 10^{10}$ bytes. A finite-difference discretization on a $300 \times 300 \times 300$ grid already has $n \approx 2.7 \times 10^7$ unknowns. Dense methods are therefore out of reach for such problems, and the matrix must be stored and used in a form that exploits its zero entries.

Remark 1.21 **(Memory traffic)** Flop counts ignore the cost of moving data between memory and the processor. A dense matrix–matrix product reuses each loaded entry $O(n)$ times, and blocked implementations run close to the peak arithmetic rate. A sparse matrix–vector product performs two flops for each stored nonzero but reads at least one value and one index for it. Its running time is therefore set by memory bandwidth rather than by the flop count, and reducing the number of stored entries reduces the running time directly.

1.6.2 Sparse Matrix Storage

A matrix is **sparse** when the number of its nonzero entries, written $\operatorname{nnz}(\mathbf{A})$, is much smaller than n^2. The finite-difference Poisson matrix on an $N \times N$ grid has $n = N^2$ unknowns and fewer than $5n$ nonzero entries. Stored densely, it would occupy n^2 words, and a matrix–vector product would cost $2n^2$ flops. A sparse format stores only the nonzero entries and their positions.

Def 1.31 **(Coordinate and Compressed Sparse Row Formats)** Let $\mathbf{A} \in \mathbb{R}^{n \times n}$ have $\operatorname{nnz}(\mathbf{A})$ nonzero entries.

1. The **coordinate** (COO) format stores three arrays of length $\operatorname{nnz}(\mathbf{A})$ that list each nonzero entry as a triplet (i, j, a_{ij}). The triplets may appear in any order, and repeated positions are understood to be summed.

2. The **compressed sparse row** (CSR) format stores the nonzero values row by row in an array `val`, their column indices in an array `col` of the same length, and an array `ptr` of length $n+1$. The entries of row i occupy positions $\texttt{ptr}_i, \dots, \texttt{ptr}_{i+1} - 1$ of `val` and `col`.

Example The matrix

$$\mathbf{A} = \begin{pmatrix} 4 & 0 & -1 & 0 \\ 0 & 3 & 0 & 0 \\ -1 & 0 & 5 & 2 \\ 0 & 0 & 2 & 6 \end{pmatrix}$$

has eight nonzero entries. Its CSR arrays are

$$\texttt{val} = (4, -1,\ 3,\ -1, 5, 2,\ 2, 6), \quad \texttt{col} = (1, 3,\ 2,\ 1, 3, 4,\ 3, 4), \quad \texttt{ptr} = (1, 3, 4, 7, 9).$$

Row 3 occupies positions $\texttt{ptr}_3 = 4$ through $\texttt{ptr}_4 - 1 = 6$.

Thm 1.25 **(Cost of a CSR Matrix–Vector Product)** For $\mathbf{A}$ stored in CSR format, the product $\boldsymbol{y} = \mathbf{A}\boldsymbol{x}$ computed by

$$y_i = \sum_{k=\texttt{ptr}_i}^{\texttt{ptr}_{i+1}-1} \texttt{val}_k \, x_{\texttt{col}_k}, \qquad i = 1, \dots, n,$$

requires at most $2\operatorname{nnz}(\mathbf{A})$ flops. The format stores $2\operatorname{nnz}(\mathbf{A}) + n + 1$ numbers.

Proof Each stored entry contributes one multiplication and at most one addition to the sum for its row, and each entry is visited once. The arrays `val` and `col` have length $\operatorname{nnz}(\mathbf{A})$, and `ptr` has length $n+1$. □

The CSR format is efficient for computing $\mathbf{A}\boldsymbol{x}$ but inconvenient for constructing a matrix, because the position of an entry in `val` depends on how many entries precede it. Matrices

are therefore usually assembled in COO format, where a new entry is simply appended, and converted to CSR once assembly is complete.

Thm 1.26 **(Conversion from COO to CSR)** A matrix given by t triplets can be converted to CSR format in $O(t+n)$ operations.

Proof We count the triplets in each row, which takes $O(t)$ operations. A running sum of these counts gives `ptr` in $O(n)$ operations. We then pass over the triplets a second time and place each one at the next free position of its row, which again takes $O(t)$ operations. Repeated positions within a row are merged by a final pass that accumulates entries with equal column index. The pass uses a work array of length n, initialized once, that records for each column the position of its entry in the row currently being processed, so no array is cleared between rows. Every step is linear in t or n. □

Example **(Assembling the Poisson matrix)** For the one-dimensional Poisson matrix $h^{-2}\,\mathrm{tridiag}(-1,2,-1)$, we append for each $i=1,\dots,n$ the triplet $(i,i,2h^{-2})$, together with $(i,i-1,-h^{-2})$ when $i>1$ and $(i,i+1,-h^{-2})$ when $i<n$. The result has $3n-2$ triplets and converts to CSR in $O(n)$ operations. Finite-element matrices are assembled in the same way by looping over elements, and each element contributes a small block of triplets that overlaps the contributions of its neighbors. The overlapping triplets are summed during the conversion.

Remark 1.22 **(Column storage and transposes)** The compressed sparse column (CSC) format stores $\mathbf{A}$ column by column and is the CSR format of $\mathbf{A}^T$. Column-oriented algorithms, such as sparse Cholesky factorization and column-oriented triangular solves, use CSC. A product $\mathbf{A}^T\boldsymbol{x}$ with a CSR matrix is computed by scattering $x_i\,\mathtt{val}_k$ into position $\mathtt{col}_k$ of the result. The scatter has the same flop count as the CSR product, but parallel threads that process different rows may then write to the same output entry.

1.6.3 Exercises

Ex 1.12

1. **Check.** Show that $3n^2+100n+5=\Theta(n^2)$, and arrange $5n^3+n$, $100n^2$, $n^2\log n$, and $2^{10}n^2$ in order of growth.

2. **Check.** Write the COO triplets and the CSR arrays of the 5×5 matrix $\mathrm{tridiag}(-1,2,-1)$. Then compute $\mathbf{A}\boldsymbol{x}$ for $\boldsymbol{x}=(1,2,3,4,5)^T$ directly from the CSR arrays.

3. **Derive.** Count the flops and the memory reads of the CSR product in Thm 1.25, assuming 8-byte values and 4-byte indices. Compute the ratio of flops to bytes read, and explain why this ratio makes the product limited by memory bandwidth.

4. **Derive.** Write the scatter algorithm for $\mathbf{A}^T\boldsymbol{x}$ from CSR arrays, and prove that it costs at most $2\,\mathrm{nnz}(\mathbf{A})$ flops.

5. **Compute.** Assemble the five-point Poisson matrix on an $N\times N$ grid in COO format, convert it to CSR, and compare the time of a sparse and a dense matrix–vector

product for $N = 50, 100, 200$. Report nnz, the storage of both formats, and the two times.

2 Direct Methods for Linear Systems

2.1 Linearity and Fundamental Subspaces

Linear systems recur throughout scientific computing. Methods for nonlinear equations, optimization, differential equations, and physical simulation routinely reduce a larger problem to a sequence of linear solves. Their reliable solution begins with the existence, uniqueness, and geometric structure encoded by the coefficient matrix.

2.1.1 Linear Systems and Geometric Interpretations

A matrix equation $\mathbf{A}\boldsymbol{x} = \boldsymbol{b}$ expresses the requirement that a data vector $\boldsymbol{b}$ be produced by a linear combination of the columns of $\mathbf{A}$, weighted by the unknown coordinates $\boldsymbol{x}$.

Def 2.1 **(Linear System)** A linear system of m equations in n unknowns is written in matrix-vector form as

$$\mathbf{A}\boldsymbol{x} = \boldsymbol{b},$$

where $\mathbf{A} \in \mathbb{R}^{m\times n}$ is the coefficient matrix, $\boldsymbol{x} \in \mathbb{R}^n$ is the unknown solution vector, and $\boldsymbol{b} \in \mathbb{R}^m$ is the right-hand side vector.

Remark 2.1 **(Row versus column perspectives)** There are two complementary geometric interpretations of $\mathbf{A}\boldsymbol{x} = \boldsymbol{b}$:

1. **Row view:** Each row $\boldsymbol{a}_i^T\boldsymbol{x} = b_i$ defines an affine hyperplane in $\mathbb{R}^n$. Solving the system means locating the mutual intersection of these m hyperplanes.
2. **Column view:** Partitioning $\mathbf{A} = [\boldsymbol{a}_1, \dots, \boldsymbol{a}_n]$ into its columns, the product evaluates to $\mathbf{A}\boldsymbol{x} = \sum_{j=1}^n x_j\boldsymbol{a}_j$. Solving the system means expressing the vector $\boldsymbol{b}$ as a linear combination of the column vectors of $\mathbf{A}$.

Thm 2.1 **(Superposition Principle)** Let $\mathbf{A} \in \mathbb{R}^{m\times n}$ define a linear operator. If $\mathbf{A}\boldsymbol{x}_1 = \boldsymbol{b}_1$ and $\mathbf{A}\boldsymbol{x}_2 = \boldsymbol{b}_2$, then for any scalars $\alpha, \beta \in \mathbb{R}$,

$$\mathbf{A}(\alpha\boldsymbol{x}_1 + \beta\boldsymbol{x}_2) = \alpha\boldsymbol{b}_1 + \beta\boldsymbol{b}_2.$$

Consequently, the solution to a linear system with a composite source term decomposes into a linear combination of individual response vectors.

Proof Using the distributivity and scalar associativity of matrix multiplication,

$$\mathbf{A}(\alpha\boldsymbol{x}_1 + \beta\boldsymbol{x}_2) = \mathbf{A}(\alpha\boldsymbol{x}_1) + \mathbf{A}(\beta\boldsymbol{x}_2) = \alpha(\mathbf{A}\boldsymbol{x}_1) + \beta(\mathbf{A}\boldsymbol{x}_2) = \alpha\boldsymbol{b}_1 + \beta\boldsymbol{b}_2.$$

□

2.1.2 Invertibility and the Four Fundamental Subspaces

The solvability and uniqueness of solutions are governed by four fundamental subspaces associated with $\mathbf{A}$.

Def 2.2 **(The Four Fundamental Subspaces)** Let $\mathbf{A} \in \mathbb{R}^{m\times n}$.

1. The **Column Space (Range)** $\mathcal{R}(\mathbf{A}) = \{\mathbf{A}\boldsymbol{x} : \boldsymbol{x} \in \mathbb{R}^n\} \subseteq \mathbb{R}^m$.
2. The **Null Space (Kernel)** $\mathcal{N}(\mathbf{A}) = \{\boldsymbol{x} \in \mathbb{R}^n : \mathbf{A}\boldsymbol{x} = \mathbf{0}\} \subseteq \mathbb{R}^n$.
3. The **Row Space** $\mathcal{R}(\mathbf{A}^T) = \{\mathbf{A}^T\boldsymbol{y} : \boldsymbol{y} \in \mathbb{R}^m\} \subseteq \mathbb{R}^n$.
4. The **Left Null Space** $\mathcal{N}(\mathbf{A}^T) = \{\boldsymbol{y} \in \mathbb{R}^m : \mathbf{A}^T\boldsymbol{y} = \mathbf{0}\} \subseteq \mathbb{R}^m$.

The rank $r = \operatorname{rank}(\mathbf{A})$ is the dimension of the column space $\mathcal{R}(\mathbf{A})$.

Thm 2.2 **(The Rank-Nullity Theorem)** Let $\mathbf{A} \in \mathbb{R}^{m\times n}$ with rank $r = \dim \mathcal{R}(\mathbf{A})$. Then

$$\operatorname{rank}(\mathbf{A}) + \dim \mathcal{N}(\mathbf{A}) = n.$$

Proof Let $k = \dim \mathcal{N}(\mathbf{A})$, and choose a basis $\{\boldsymbol{v}_1, \dots, \boldsymbol{v}_k\}$ for the null space. Extend this list to a basis $\{\boldsymbol{v}_1, \dots, \boldsymbol{v}_n\}$ of $\mathbb{R}^n$. For any $\boldsymbol{x} \in \mathbb{R}^n$, expand $\boldsymbol{x} = \sum_{i=1}^n c_i\boldsymbol{v}_i$. Linearity gives

$$\mathbf{A}\boldsymbol{x} = \sum_{i=1}^{k} c_i\mathbf{A}\boldsymbol{v}_i + \sum_{i=k+1}^{n} c_i\mathbf{A}\boldsymbol{v}_i = \mathbf{0} + \sum_{i=k+1}^{n} c_i\mathbf{A}\boldsymbol{v}_i = \sum_{i=k+1}^{n} c_i\mathbf{A}\boldsymbol{v}_i.$$

Thus the vectors $\{\mathbf{A}\boldsymbol{v}_{k+1}, \dots, \mathbf{A}\boldsymbol{v}_n\}$ span $\mathcal{R}(\mathbf{A})$. To establish linear independence, suppose $\sum_{i=k+1}^n d_i\mathbf{A}\boldsymbol{v}_i = \mathbf{0}$. Linearity yields

$$\mathbf{A}\left(\sum_{i=k+1}^{n} d_i\boldsymbol{v}_i\right) = \mathbf{0} \quad\Longrightarrow\quad \boldsymbol{w} = \sum_{i=k+1}^{n} d_i\boldsymbol{v}_i \in \mathcal{N}(\mathbf{A}).$$

Because $\{\boldsymbol{v}_1, \dots, \boldsymbol{v}_k\}$ is a basis for $\mathcal{N}(\mathbf{A})$, there exist coefficients $a_1, \dots, a_k$ such that $\boldsymbol{w} = \sum_{j=1}^k a_j\boldsymbol{v}_j$. Equating the two expressions gives

$$\sum_{j=1}^{k} a_j\boldsymbol{v}_j - \sum_{i=k+1}^{n} d_i\boldsymbol{v}_i = \mathbf{0}.$$

Since the full set $\{\boldsymbol{v}_1, \dots, \boldsymbol{v}_n\}$ is linearly independent, every coefficient vanishes:

$$a_1 = \cdots = a_k = d_{k+1} = \cdots = d_n = 0.$$

Therefore $\{\mathbf{A}\boldsymbol{v}_{k+1}, \dots, \mathbf{A}\boldsymbol{v}_n\}$ is a basis for $\mathcal{R}(\mathbf{A})$, containing exactly $n-k$ vectors. Hence

$$r = n - k = n - \dim \mathcal{N}(\mathbf{A}) \quad\Longrightarrow\quad \operatorname{rank}(\mathbf{A}) + \dim \mathcal{N}(\mathbf{A}) = n.$$

□

Thm 2.3 **(Fundamental Theorem of Linear Algebra and Orthogonal Decompositions)** For any matrix $\mathbf{A} \in \mathbb{R}^{m\times n}$:

1. The row space and null space are orthogonal complements in $\mathbb{R}^n$:

$$\mathcal{R}(\mathbf{A}^T) \perp \mathcal{N}(\mathbf{A}) \qquad \text{and} \qquad \mathbb{R}^n = \mathcal{R}(\mathbf{A}^T) \oplus \mathcal{N}(\mathbf{A}).$$

2. The column space and left null space are orthogonal complements in $\mathbb{R}^m$:

$$\mathcal{R}(\mathbf{A}) \perp \mathcal{N}(\mathbf{A}^T) \qquad \text{and} \qquad \mathbb{R}^m = \mathcal{R}(\mathbf{A}) \oplus \mathcal{N}(\mathbf{A}^T).$$

Consequently, $\dim \mathcal{R}(\mathbf{A}^T) = \dim \mathcal{R}(\mathbf{A}) = \operatorname{rank}(\mathbf{A})$.

Proof For any $\boldsymbol{x} \in \mathbb{R}^n$, the condition $\mathbf{A}\boldsymbol{x} = \mathbf{0}$ says that every row of $\mathbf{A}$ has zero inner product with $\boldsymbol{x}$. Since those rows span $\mathcal{R}(\mathbf{A}^T)$,

$$\boldsymbol{x} \in \mathcal{N}(\mathbf{A}) \quad \Longleftrightarrow \quad \boldsymbol{x} \in \mathcal{R}(\mathbf{A}^T)^{\perp}.$$

Taking orthogonal complements in finite-dimensional space gives

$$\mathcal{R}(\mathbf{A}^T) = \mathcal{N}(\mathbf{A})^{\perp}.$$

The identical argument with $\mathbf{A}^T$ gives $\mathcal{R}(\mathbf{A}) = \mathcal{N}(\mathbf{A}^T)^{\perp}$. Each space therefore decomposes into one of these subspaces and its orthogonal complement. Finally, rank–nullity gives

$$\dim \mathcal{R}(\mathbf{A}^T) = n - \dim \mathcal{N}(\mathbf{A}) = \operatorname{rank}(\mathbf{A}),$$

so row rank equals column rank. □

Thm 2.4 **(Solvability and the Left Null Space)** The linear system $\mathbf{A}\boldsymbol{x} = \boldsymbol{b}$ has a solution if and only if $\boldsymbol{b} \in \mathcal{R}(\mathbf{A})$. Equivalently, $\mathbf{A}\boldsymbol{x} = \boldsymbol{b}$ is solvable if and only if $\boldsymbol{b}$ is orthogonal to every vector in the left null space of $\mathbf{A}$:

$$\mathbf{A}\boldsymbol{x} = \boldsymbol{b} \text{ has a solution } \Longleftrightarrow \ \boldsymbol{y}^T\boldsymbol{b} = 0 \quad \text{for all } \boldsymbol{y} \text{ such that } \mathbf{A}^T\boldsymbol{y} = \mathbf{0}.$$

If a solution exists and $\boldsymbol{x}_p$ is one particular solution, the complete set of solutions is the affine space

$$\boldsymbol{x} = \boldsymbol{x}_p + \boldsymbol{z}, \qquad \boldsymbol{z} \in \mathcal{N}(\mathbf{A}).$$

Proof By Thm 2.3, $\mathbb{R}^m = \mathcal{R}(\mathbf{A}) \oplus \mathcal{N}(\mathbf{A}^T)$. Therefore $\boldsymbol{b} \in \mathcal{R}(\mathbf{A})$ if and only if $\boldsymbol{b} \perp \mathcal{N}(\mathbf{A}^T)$, which means

$$\langle \boldsymbol{y}, \boldsymbol{b} \rangle = \boldsymbol{y}^T\boldsymbol{b} = 0 \qquad \text{for all } \boldsymbol{y} \in \mathcal{N}(\mathbf{A}^T).$$

If $\mathbf{A}\boldsymbol{x}_p = \boldsymbol{b}$, then for any $\boldsymbol{z} \in \mathcal{N}(\mathbf{A})$,

$$\mathbf{A}(\boldsymbol{x}_p + \boldsymbol{z}) = \mathbf{A}\boldsymbol{x}_p + \mathbf{A}\boldsymbol{z} = \boldsymbol{b} + \mathbf{0} = \boldsymbol{b}.$$

Conversely, if $\mathbf{A}\boldsymbol{x} = \boldsymbol{b}$, then

$$\mathbf{A}(\boldsymbol{x} - \boldsymbol{x}_p) = \mathbf{A}\boldsymbol{x} - \mathbf{A}\boldsymbol{x}_p = \boldsymbol{b} - \boldsymbol{b} = \mathbf{0},$$

which implies $\boldsymbol{x} - \boldsymbol{x}_p \in \mathcal{N}(\mathbf{A})$, so $\boldsymbol{x} = \boldsymbol{x}_p + \boldsymbol{z}$ with $\boldsymbol{z} \in \mathcal{N}(\mathbf{A})$. □

Remark 2.2 This compatibility condition is often called the finite-dimensional Fredholm alternative. Each left-null vector describes one obstruction that a solvable right-hand side must avoid.

2.1.3 Exercises

Ex 2.1

1. **Compute. Dimensions and bases for the four fundamental subspaces.** Let

$$\mathbf{A} = \begin{pmatrix} 1 & 2 & 1 & 3 \\ 2 & 4 & 1 & 4 \\ 3 & 6 & 2 & 7 \end{pmatrix}.$$

   (a) Compute the row reduced echelon form of $\mathbf{A}$. Determine the rank $r = \operatorname{rank}(\mathbf{A})$.

   (b) Find bases for $\mathcal{R}(\mathbf{A})$, $\mathcal{N}(\mathbf{A})$, $\mathcal{R}(\mathbf{A}^T)$, and $\mathcal{N}(\mathbf{A}^T)$. After studying QR factorization, return and replace them by orthonormal bases.

   (c) Confirm that $\dim \mathcal{R}(\mathbf{A}) + \dim \mathcal{N}(\mathbf{A}^T) = 3$ and $\dim \mathcal{R}(\mathbf{A}^T) + \dim \mathcal{N}(\mathbf{A}) = 4$.

   (d) Verify numerically that every vector in your basis of $\mathcal{R}(\mathbf{A}^T)$ is orthogonal to every vector in your basis of $\mathcal{N}(\mathbf{A})$.

2. **Derive. Decomposition into row space and null space components.** Let $\mathbf{A} \in \mathbb{R}^{m \times n}$.

   (a) Prove that every vector $\boldsymbol{x} \in \mathbb{R}^n$ has a unique orthogonal decomposition $\boldsymbol{x} = \boldsymbol{x}_{\text{row}} + \boldsymbol{x}_{\text{null}}$ with $\boldsymbol{x}_{\text{row}} \in \mathcal{R}(\mathbf{A}^T)$ and $\boldsymbol{x}_{\text{null}} \in \mathcal{N}(\mathbf{A})$.

   (b) Show that $\mathbf{A}\boldsymbol{x} = \mathbf{A}\boldsymbol{x}_{\text{row}}$, demonstrating that the null-space component contributes nothing to the output.

   (c) Prove that among all solutions to a consistent underdetermined system $\mathbf{A}\boldsymbol{x} = \boldsymbol{b}$, the unique solution of minimal Euclidean norm $\|\boldsymbol{x}\|_2$ lies strictly in the row space: $\boldsymbol{x}_{\min} \in \mathcal{R}(\mathbf{A}^T)$.

3. **Derive. The Fredholm alternative and inconsistent systems.** Consider the 3×2 system

$$\mathbf{A} = \begin{pmatrix} 1 & 1 \\ 1 & 2 \\ 2 & 3 \end{pmatrix}, \qquad \boldsymbol{b} = \begin{pmatrix} 1 \\ 2 \\ b_3 \end{pmatrix}.$$

(a) Compute a basis for the left null space $\mathcal{N}(\mathbf{A}^T)$.

(b) Apply Thm 2.4 to find the exact value of b_3 that makes the system consistent.

(c) For $b_3 = 5$, find the orthogonal projection of $\boldsymbol{b}$ onto $\mathcal{R}(\mathbf{A})$ and compute the minimum residual norm $\min_{\boldsymbol{x}} \|\mathbf{A}\boldsymbol{x} - \boldsymbol{b}\|_2$.

2.2 Conditioning and Numerical Stability

Before constructing algorithms for $\mathbf{A}\boldsymbol{x} = \boldsymbol{b}$, we ask how accurately its solution can be computed at all. Two sources of error must be kept separate. The first is the sensitivity of the solution to perturbations in $\mathbf{A}$ and $\boldsymbol{b}$, which is a property of the problem and is measured by the condition number. The second is the rounding error committed by an algorithm, which is measured by the backward error.

We first relate the residual of an approximation to its error. We then bound the change in the solution caused by perturbations of the data, and finally compute the backward error of a given approximation from its residual. The factorizations in the following sections are judged by this backward error.

2.2.1 Residual, Forward Error, and Backward Error

Let $\mathbf{A}\boldsymbol{x} = \boldsymbol{b}$ be a nonsingular linear system with exact solution $\boldsymbol{x}^* = \mathbf{A}^{-1}\boldsymbol{b}$, and let $\hat{\boldsymbol{x}}$ denote a computed approximation. We assess the quality of $\hat{\boldsymbol{x}}$ through three interrelated quantities.

Def 2.3 **(Forward Error, Residual, and Backward Error)** For a computed approximation $\hat{\boldsymbol{x}}$:

1. The **forward error** is the vector difference $\Delta\boldsymbol{x} = \hat{\boldsymbol{x}} - \boldsymbol{x}^*$, with relative measure $\frac{\|\hat{\boldsymbol{x}} - \boldsymbol{x}^*\|}{\|\boldsymbol{x}^*\|}$.
2. The **residual** is the vector defect $\boldsymbol{r} = \boldsymbol{b} - \mathbf{A}\hat{\boldsymbol{x}}$, with relative measure $\frac{\|\boldsymbol{r}\|}{\|\mathbf{A}\|\|\hat{\boldsymbol{x}}\| + \|\boldsymbol{b}\|}$.
3. The **backward error** is the smallest perturbation $(\delta\mathbf{A}, \delta\boldsymbol{b})$ such that $\hat{\boldsymbol{x}}$ is the exact solution to the perturbed system $(\mathbf{A} + \delta\mathbf{A})\hat{\boldsymbol{x}} = \boldsymbol{b} + \delta\boldsymbol{b}$.

Thm 2.5 **(The Fundamental Residual-Error Bound)** Let $\mathbf{A} \in \mathbb{R}^{n\times n}$ be nonsingular, let $\boldsymbol{b} \neq \mathbf{0}$, and let $\boldsymbol{x}^*$ satisfy $\mathbf{A}\boldsymbol{x}^* = \boldsymbol{b}$. For any vector $\hat{\boldsymbol{x}}$ with residual $\boldsymbol{r} = \boldsymbol{b} - \mathbf{A}\hat{\boldsymbol{x}}$, the relative forward error satisfies

$$\frac{1}{\kappa(\mathbf{A})}\frac{\|\boldsymbol{r}\|}{\|\boldsymbol{b}\|} \leq \frac{\|\hat{\boldsymbol{x}} - \boldsymbol{x}^*\|}{\|\boldsymbol{x}^*\|} \leq \kappa(\mathbf{A})\frac{\|\boldsymbol{r}\|}{\|\boldsymbol{b}\|},$$

where $\kappa(\mathbf{A}) = \|\mathbf{A}\|\|\mathbf{A}^{-1}\|$ is the condition number of $\mathbf{A}$.

Proof Because $\boldsymbol{b} = \mathbf{A}\boldsymbol{x}^*$, the residual can be written in terms of the forward error as $\boldsymbol{r} = \boldsymbol{b} - \mathbf{A}\hat{\boldsymbol{x}} = \mathbf{A}(\boldsymbol{x}^* - \hat{\boldsymbol{x}})$. Multiplying both sides by $\mathbf{A}^{-1}$ yields $\boldsymbol{x}^* - \hat{\boldsymbol{x}} = \mathbf{A}^{-1}\boldsymbol{r}$. Taking vector norms and applying the submultiplicative property of induced matrix norms gives

$$\|\hat{\boldsymbol{x}} - \boldsymbol{x}^*\| \leq \|\mathbf{A}^{-1}\|\|\boldsymbol{r}\|.$$

Furthermore, from $\boldsymbol{b} = \mathbf{A}\boldsymbol{x}^*$, we obtain $\|\boldsymbol{b}\| \leq \|\mathbf{A}\|\|\boldsymbol{x}^*\|$, which rearranges to $\frac{1}{\|\boldsymbol{x}^*\|} \leq \frac{\|\mathbf{A}\|}{\|\boldsymbol{b}\|}$. Multiplying these two inequalities produces

$$\frac{\|\hat{\boldsymbol{x}} - \boldsymbol{x}^*\|}{\|\boldsymbol{x}^*\|} \leq (\|\mathbf{A}\|\|\mathbf{A}^{-1}\|)\frac{\|\boldsymbol{r}\|}{\|\boldsymbol{b}\|} = \kappa(\mathbf{A})\frac{\|\boldsymbol{r}\|}{\|\boldsymbol{b}\|}.$$

For the lower bound, $\boldsymbol{r} = \mathbf{A}(\boldsymbol{x}^* - \hat{\boldsymbol{x}})$ gives $\|\boldsymbol{r}\| \leq \|\mathbf{A}\|\|\boldsymbol{x}^* - \hat{\boldsymbol{x}}\|$, while $\boldsymbol{x}^* = \mathbf{A}^{-1}\boldsymbol{b}$ gives $\|\boldsymbol{x}^*\| \leq \|\mathbf{A}^{-1}\|\|\boldsymbol{b}\|$. Combining these inequalities yields

$$\frac{\|\hat{\boldsymbol{x}} - \boldsymbol{x}^*\|}{\|\boldsymbol{x}^*\|} \geq \frac{1}{\|\mathbf{A}\|\|\mathbf{A}^{-1}\|}\frac{\|\boldsymbol{r}\|}{\|\boldsymbol{b}\|}.$$

□

Example **(A small residual does not guarantee a small error)** Consider the diagonal system

$$\mathbf{A} = \begin{pmatrix} 1 & 0 \\ 0 & 10^{-8} \end{pmatrix}, \qquad \boldsymbol{b} = \begin{pmatrix} 1 \\ 10^{-8} \end{pmatrix}.$$

The exact solution is $\boldsymbol{x}^* = (1, 1)^T$. Suppose an algorithm produces the approximation $\hat{\boldsymbol{x}} = (1, 0)^T$. The residual evaluates to

$$\boldsymbol{r} = \boldsymbol{b} - \mathbf{A}\hat{\boldsymbol{x}} = \begin{pmatrix} 1 \\ 10^{-8} \end{pmatrix} - \begin{pmatrix} 1 \\ 0 \end{pmatrix} = \begin{pmatrix} 0 \\ 10^{-8} \end{pmatrix},$$

which satisfies $\|\boldsymbol{r}\|_2/\|\boldsymbol{b}\|_2 \approx 10^{-8}$. However, the forward error is $\hat{\boldsymbol{x}} - \boldsymbol{x}^* = (0, -1)^T$, yielding a relative forward error of $\|\hat{\boldsymbol{x}} - \boldsymbol{x}^*\|_2/\|\boldsymbol{x}^*\|_2 = 1/\sqrt{2} \approx 0.707$ (a seventy percent error). The matrix has condition number $\kappa_2(\mathbf{A}) = 10^8$, which amplifies the small residual into an $O(1)$ error in the solution.

2.2.2 Sensitivity to Perturbations in the Data

Theorem 2.5 measures the error of a given approximation through its residual. A related question asks how far the exact solution moves when the data $\mathbf{A}$ and $\boldsymbol{b}$ are perturbed. The answer is again governed by the condition number $\kappa(\mathbf{A}) = \|\mathbf{A}\|\|\mathbf{A}^{-1}\|$ of Def 1.24. Perturbing $\mathbf{A}$ requires one preliminary fact, namely that a sufficiently small perturbation of a nonsingular matrix remains nonsingular.

Lem 2.6 **(Perturbation of the Identity)** Let $\|\cdot\|$ be an induced matrix norm and let $\mathbf{M}$ be a square matrix with $\|\mathbf{M}\| < 1$. Then $\mathbf{I} + \mathbf{M}$ is nonsingular and

$$\|(\mathbf{I} + \mathbf{M})^{-1}\| \leq \frac{1}{1 - \|\mathbf{M}\|}.$$

Proof If $(\mathbf{I}+\mathbf{M})\boldsymbol{z}=\mathbf{0}$, then $\|\boldsymbol{z}\|=\|\mathbf{M}\boldsymbol{z}\|\le\|\mathbf{M}\|\|\boldsymbol{z}\|$, which forces $\boldsymbol{z}=\mathbf{0}$ because $\|\mathbf{M}\|<1$. Hence $\mathbf{I}+\mathbf{M}$ is nonsingular. For any vector $\boldsymbol{y}$, let $\boldsymbol{z}=(\mathbf{I}+\mathbf{M})^{-1}\boldsymbol{y}$. Then $\boldsymbol{z}=\boldsymbol{y}-\mathbf{M}\boldsymbol{z}$, so $\|\boldsymbol{z}\|\le\|\boldsymbol{y}\|+\|\mathbf{M}\|\|\boldsymbol{z}\|$. Rearranging gives $\|\boldsymbol{z}\|\le\|\boldsymbol{y}\|/(1-\|\mathbf{M}\|)$, and taking the maximum over unit vectors $\boldsymbol{y}$ proves the bound. □

Thm 2.7 **(Perturbation Bound for Linear Systems)** Let $\mathbf{A}$ be nonsingular, let $\boldsymbol{b}\neq\mathbf{0}$, and let $\mathbf{A}\boldsymbol{x}=\boldsymbol{b}$. Suppose the perturbations $\delta\mathbf{A}$ and $\delta\boldsymbol{b}$ satisfy $\kappa(\mathbf{A})\|\delta\mathbf{A}\|/\|\mathbf{A}\|<1$. Then $\mathbf{A}+\delta\mathbf{A}$ is nonsingular, and the solution of $(\mathbf{A}+\delta\mathbf{A})(\boldsymbol{x}+\delta\boldsymbol{x})=\boldsymbol{b}+\delta\boldsymbol{b}$ satisfies

$$\frac{\|\delta\boldsymbol{x}\|}{\|\boldsymbol{x}\|}\le\frac{\kappa(\mathbf{A})}{1-\kappa(\mathbf{A})\|\delta\mathbf{A}\|/\|\mathbf{A}\|}\left(\frac{\|\delta\mathbf{A}\|}{\|\mathbf{A}\|}+\frac{\|\delta\boldsymbol{b}\|}{\|\boldsymbol{b}\|}\right).$$

Proof Factor $\mathbf{A}+\delta\mathbf{A}=\mathbf{A}(\mathbf{I}+\mathbf{A}^{-1}\delta\mathbf{A})$. Since $\|\mathbf{A}^{-1}\delta\mathbf{A}\|\le\|\mathbf{A}^{-1}\|\|\delta\mathbf{A}\|=\kappa(\mathbf{A})\|\delta\mathbf{A}\|/\|\mathbf{A}\|<1$, Lem 2.6 shows that $\mathbf{A}+\delta\mathbf{A}$ is nonsingular. Subtracting $\mathbf{A}\boldsymbol{x}=\boldsymbol{b}$ from the perturbed equation gives

$$(\mathbf{A}+\delta\mathbf{A})\,\delta\boldsymbol{x}=\delta\boldsymbol{b}-\delta\mathbf{A}\,\boldsymbol{x},\qquad\text{so}\qquad\delta\boldsymbol{x}=(\mathbf{I}+\mathbf{A}^{-1}\delta\mathbf{A})^{-1}\mathbf{A}^{-1}(\delta\boldsymbol{b}-\delta\mathbf{A}\,\boldsymbol{x}).$$

Lemma 2.6 then gives

$$\|\delta\boldsymbol{x}\|\le\frac{\|\mathbf{A}^{-1}\|\big(\|\delta\boldsymbol{b}\|+\|\delta\mathbf{A}\|\|\boldsymbol{x}\|\big)}{1-\|\mathbf{A}^{-1}\|\|\delta\mathbf{A}\|}.$$

Dividing by $\|\boldsymbol{x}\|$ and using $\|\boldsymbol{b}\|\le\|\mathbf{A}\|\|\boldsymbol{x}\|$ to bound $\|\delta\boldsymbol{b}\|/\|\boldsymbol{x}\|$ by $\|\mathbf{A}\|\|\delta\boldsymbol{b}\|/\|\boldsymbol{b}\|$ proves the stated inequality. □

To first order, relative perturbations of size ε in $\mathbf{A}$ and in $\boldsymbol{b}$ change the solution by a relative amount of at most about $2\kappa(\mathbf{A})\varepsilon$. The hypothesis $\kappa(\mathbf{A})\|\delta\mathbf{A}\|/\|\mathbf{A}\|<1$ cannot be dropped, because a larger perturbation may make $\mathbf{A}+\delta\mathbf{A}$ singular. In the Euclidean norm, the smallest perturbation that does so has $\|\delta\mathbf{A}\|_2=\sigma_{\min}(\mathbf{A})=\|\mathbf{A}\|_2/\kappa_2(\mathbf{A})$ by Thm 4.21. The reciprocal $1/\kappa_2(\mathbf{A})$ is therefore the relative distance from $\mathbf{A}$ to the nearest singular matrix.

2.2.3 Backward Error and Backward Stability

The perturbation bound describes the problem and makes no reference to an algorithm. To apply it to a computed solution $\hat{\boldsymbol{x}}$, we need perturbations $\delta\mathbf{A}$ and $\delta\boldsymbol{b}$ for which $\hat{\boldsymbol{x}}$ is the exact solution. The smallest such perturbations can be computed from the residual.

Thm 2.8 **(Rigal–Gaches Theorem on Normwise Backward Error)** In the Euclidean vector norm and its induced matrix norm, let $\hat{\boldsymbol{x}}\neq\mathbf{0}$ have residual $\boldsymbol{r}=\boldsymbol{b}-\mathbf{A}\hat{\boldsymbol{x}}$. The smallest

relative perturbation size satisfying $(\mathbf{A}+\delta\mathbf{A})\hat{\boldsymbol{x}} = \boldsymbol{b}+\delta\boldsymbol{b}$ with $\|\delta\mathbf{A}\|_2 \le \eta\|\mathbf{A}\|_2$ and $\|\delta\boldsymbol{b}\|_2 \le \eta\|\boldsymbol{b}\|_2$ is

$$\eta(\hat{\boldsymbol{x}}) = \frac{\|\boldsymbol{r}\|_2}{\|\mathbf{A}\|_2\|\hat{\boldsymbol{x}}\|_2 + \|\boldsymbol{b}\|_2}.$$

An algorithm is **backward stable** if, for every nonsingular $\mathbf{A}$ and every $\boldsymbol{b}$, the computed solution satisfies $\eta(\hat{\boldsymbol{x}}) \le c_n u$, where u is the unit roundoff and c_n grows at most polynomially with n.

Rigal and Gaches proved this formula for the normwise backward error (Rigal and Gaches, 1967). The proof below constructs the minimizing perturbations explicitly.

Proof From $(\mathbf{A}+\delta\mathbf{A})\hat{\boldsymbol{x}} = \boldsymbol{b}+\delta\boldsymbol{b}$, rearranging gives $\boldsymbol{r} = \boldsymbol{b}-\mathbf{A}\hat{\boldsymbol{x}} = \delta\mathbf{A}\hat{\boldsymbol{x}} - \delta\boldsymbol{b}$. Taking norms and applying the triangle inequality yields

$$\|\boldsymbol{r}\|_2 \le \|\delta\mathbf{A}\|_2\|\hat{\boldsymbol{x}}\|_2 + \|\delta\boldsymbol{b}\|_2 \le \eta\|\mathbf{A}\|_2\|\hat{\boldsymbol{x}}\|_2 + \eta\|\boldsymbol{b}\|_2 = \eta(\|\mathbf{A}\|_2\|\hat{\boldsymbol{x}}\|_2 + \|\boldsymbol{b}\|_2),$$

which establishes that $\eta \ge \frac{\|\boldsymbol{r}\|_2}{\|\mathbf{A}\|_2\|\hat{\boldsymbol{x}}\|_2+\|\boldsymbol{b}\|_2}$. To prove that this lower bound is attainable, define

$$\delta\mathbf{A} = \frac{\|\mathbf{A}\|_2\|\hat{\boldsymbol{x}}\|_2}{\|\mathbf{A}\|_2\|\hat{\boldsymbol{x}}\|_2 + \|\boldsymbol{b}\|_2}\frac{\boldsymbol{r}\hat{\boldsymbol{x}}^T}{\|\hat{\boldsymbol{x}}\|_2^2}, \qquad \delta\boldsymbol{b} = -\frac{\|\boldsymbol{b}\|_2}{\|\mathbf{A}\|_2\|\hat{\boldsymbol{x}}\|_2 + \|\boldsymbol{b}\|_2}\boldsymbol{r}.$$

Direct substitution into the perturbed system gives

$$\begin{aligned}
(\mathbf{A}+\delta\mathbf{A})\hat{\boldsymbol{x}} &= \mathbf{A}\hat{\boldsymbol{x}} + \frac{\|\mathbf{A}\|_2\|\hat{\boldsymbol{x}}\|_2}{\|\mathbf{A}\|_2\|\hat{\boldsymbol{x}}\|_2 + \|\boldsymbol{b}\|_2}\frac{\boldsymbol{r}(\hat{\boldsymbol{x}}^T\hat{\boldsymbol{x}})}{\|\hat{\boldsymbol{x}}\|_2^2} \\
&= \mathbf{A}\hat{\boldsymbol{x}} + \frac{\|\mathbf{A}\|_2\|\hat{\boldsymbol{x}}\|_2}{\|\mathbf{A}\|_2\|\hat{\boldsymbol{x}}\|_2 + \|\boldsymbol{b}\|_2}\boldsymbol{r} \\
&= \boldsymbol{b} - \boldsymbol{r} + \frac{\|\mathbf{A}\|_2\|\hat{\boldsymbol{x}}\|_2}{\|\mathbf{A}\|_2\|\hat{\boldsymbol{x}}\|_2 + \|\boldsymbol{b}\|_2}\boldsymbol{r} \\
&= \boldsymbol{b} - \left(1 - \frac{\|\mathbf{A}\|_2\|\hat{\boldsymbol{x}}\|_2}{\|\mathbf{A}\|_2\|\hat{\boldsymbol{x}}\|_2 + \|\boldsymbol{b}\|_2}\right)\boldsymbol{r} \\
&= \boldsymbol{b} - \frac{\|\boldsymbol{b}\|_2}{\|\mathbf{A}\|_2\|\hat{\boldsymbol{x}}\|_2 + \|\boldsymbol{b}\|_2}\boldsymbol{r} = \boldsymbol{b}+\delta\boldsymbol{b}.
\end{aligned}$$

Furthermore, using $\|\boldsymbol{r}\hat{\boldsymbol{x}}^T\|_2 = \|\boldsymbol{r}\|_2\|\hat{\boldsymbol{x}}\|_2$:

$$\|\delta\mathbf{A}\|_2 = \frac{\|\mathbf{A}\|_2\|\boldsymbol{r}\|_2}{\|\mathbf{A}\|_2\|\hat{\boldsymbol{x}}\|_2 + \|\boldsymbol{b}\|_2} = \eta\|\mathbf{A}\|_2, \qquad \|\delta\boldsymbol{b}\|_2 = \frac{\|\boldsymbol{b}\|_2\|\boldsymbol{r}\|_2}{\|\mathbf{A}\|_2\|\hat{\boldsymbol{x}}\|_2 + \|\boldsymbol{b}\|_2} = \eta\|\boldsymbol{b}\|_2.$$

This confirms that the perturbations attain the minimal relative bound η. □

Cor 2.9 **(Forward Error of a Backward Stable Solution)** In the Euclidean norm, let $\hat{\boldsymbol{x}}$ have backward error $\eta = \eta(\hat{\boldsymbol{x}})$ as in Thm 2.8, and suppose $\kappa_2(\mathbf{A})\eta < 1$. Then

$$\frac{\|\hat{\boldsymbol{x}} - \boldsymbol{x}\|_2}{\|\boldsymbol{x}\|_2} \le \frac{2\kappa_2(\mathbf{A})\eta}{1 - \kappa_2(\mathbf{A})\eta}.$$

Proof By Thm 2.8, there are perturbations with $\|\delta\mathbf{A}\|_2 \le \eta\|\mathbf{A}\|_2$ and $\|\delta\boldsymbol{b}\|_2 \le \eta\|\boldsymbol{b}\|_2$ such that $(\mathbf{A}+\delta\mathbf{A})\hat{\boldsymbol{x}} = \boldsymbol{b}+\delta\boldsymbol{b}$. Applying Thm 2.7 with $\delta\boldsymbol{x} = \hat{\boldsymbol{x}} - \boldsymbol{x}$ gives the bound. □

For a backward stable algorithm, η is a modest multiple of u, and Cor 2.9 bounds the relative forward error by about $2\kappa_2(\mathbf{A})u$. In double precision, with $u \approx 1.1 \times 10^{-16}$, a system with $\kappa_2(\mathbf{A}) = 10^{10}$ can therefore be solved to about five correct digits by a stable method, and no method working with the rounded data can be expected to do better. A large forward error with a small backward error indicates an ill-conditioned problem, not an unstable algorithm.

Ex 2.2

1. **Compute.** Let $\mathbf{A} = \begin{pmatrix} 1 & 1 \\ 1 & 1+10^{-10} \end{pmatrix}$. Compute $\mathbf{A}^{-1}$ explicitly and evaluate $\kappa_\infty(\mathbf{A})$.
2. **Check.** Suppose $\mathbf{A}\boldsymbol{x} = \boldsymbol{b}$ is solved using a backward stable algorithm in arithmetic with unit roundoff $u \approx 10^{-16}$. If $\kappa_2(\mathbf{A}) = 10^{11}$, estimate the number of correct decimal digits in the computed solution.
3. **Derive.** Prove that for any two nonsingular matrices $\mathbf{A}$ and $\mathbf{B}$, $\kappa(\mathbf{AB}) \le \kappa(\mathbf{A})\kappa(\mathbf{B})$.
4. **Derive.** Show that for an orthogonal matrix $\mathbf{Q}$, $\kappa_2(\mathbf{QA}) = \kappa_2(\mathbf{A})$ for any square matrix $\mathbf{A}$.

2.3 LU Factorization

The solution of a square linear system $\mathbf{A}\boldsymbol{x} = \boldsymbol{b}$ is one of the central problems of computational science. Direct solvers transform the original system into a pair of triangular systems that can be evaluated sequentially. The most fundamental of these decompositions is the LU factorization, which formalizes Gaussian elimination as matrix multiplication.

The principle behind triangular factorization is the separation of elimination from substitution. Factoring the coefficient matrix $\mathbf{A}$ into a unit lower triangular factor $\mathbf{L}$ and an upper triangular factor $\mathbf{U}$ requires $O(n^3)$ operations. Once $\mathbf{L}$ and $\mathbf{U}$ are stored, the solution $\boldsymbol{x}$ for any right-hand side $\boldsymbol{b}$ is computed via forward and backward substitution in only $2n^2$ operations.

2.3.1 Gaussian Elimination as Triangular Factorization

A triangular system can be solved one unknown at a time. In an upper triangular system $\mathbf{U}\boldsymbol{x} = \boldsymbol{y}$, the final equation contains only x_n. Once x_n is known, the preceding equation contains only x_{n-1}, and backward substitution continues until all components are resolved.

Gaussian elimination systematically transforms a general system into this upper triangular form by subtracting multiples of each row from the rows beneath it.

Def 2.4 **(LU Factorization)** Let $\mathbf{A} \in \mathbb{R}^{n\times n}$. An LU factorization of $\mathbf{A}$ is a decomposition

$$\mathbf{A} = \mathbf{LU},$$

where $\mathbf{L} \in \mathbb{R}^{n\times n}$ is unit lower triangular ($\ell_{ii} = 1$ and $\ell_{ij} = 0$ for $j > i$) and $\mathbf{U} \in \mathbb{R}^{n\times n}$ is upper triangular ($u_{ij} = 0$ for $i > j$).

Lem 2.10 **(Algebra of Unit Lower Triangular Matrices)** The set of $n \times n$ unit lower triangular matrices forms a group under matrix multiplication. In particular:

1. The product of two unit lower triangular matrices is unit lower triangular.
2. The inverse of a unit lower triangular matrix is unit lower triangular.
3. The determinant of every unit lower triangular matrix equals one.

Proof Let $\mathbf{L}_1, \mathbf{L}_2$ be unit lower triangular matrices. Entry (i, j) of their product is

$$(\mathbf{L}_1\mathbf{L}_2)_{ij} = \sum_{k=1}^{n} (\mathbf{L}_1)_{ik}(\mathbf{L}_2)_{kj} = \sum_{k=j}^{i} (\mathbf{L}_1)_{ik}(\mathbf{L}_2)_{kj}.$$

If $i < j$, the summation range is empty, so $(\mathbf{L}_1\mathbf{L}_2)_{ij} = 0$. For the diagonal entries where $i = j$, the only surviving term is $k = i$, yielding $(\mathbf{L}_1\mathbf{L}_2)_{ii} = (\mathbf{L}_1)_{ii}(\mathbf{L}_2)_{ii} = 1 \cdot 1 = 1$. This proves that the product is unit lower triangular. To prove invertibility, let $\mathbf{L}$ be unit lower triangular. Because its determinant is the product of its diagonal entries, $\det(\mathbf{L}) = \prod_{i=1}^{n} 1 = 1 \neq 0$, so $\mathbf{L}^{-1}$ exists. Solving $\mathbf{LX} = \mathbf{I}$ column by column using forward substitution reveals that each column $\boldsymbol{x}_j$ has zeros above row j and $x_{jj} = 1$, which establishes that $\mathbf{L}^{-1}$ is unit lower triangular. □

Thm 2.11 **(Elimination Matrices and the Structure of L)** Let $\mathbf{A}^{(0)} = \mathbf{A}$. At step k of Gaussian elimination, assuming the pivot entry $a_{kk}^{(k-1)} \neq 0$, the subdiagonal entries in column k are annihilated by applying the elementary elimination matrix

$$\mathbf{M}_k = \mathbf{I} - \boldsymbol{\ell}_k \boldsymbol{e}_k^T,$$

where $\boldsymbol{\ell}_k = (0, \ldots, 0, \ell_{k+1,k}, \ldots, \ell_{n,k})^T$ contains the multipliers $\ell_{ik} = a_{ik}^{(k-1)}/a_{kk}^{(k-1)}$. After $n - 1$ elimination steps, the upper triangular factor is $\mathbf{U} = \mathbf{M}_{n-1} \ldots \mathbf{M}_1\mathbf{A}$. The lower triangular factor is the inverse of the accumulated elimination operations:

$$\mathbf{L} = \mathbf{M}_1^{-1}\mathbf{M}_2^{-1} \ldots \mathbf{M}_{n-1}^{-1} = \mathbf{I} + \sum_{k=1}^{n-1} \boldsymbol{\ell}_k \boldsymbol{e}_k^T.$$

Consequently, the entries of $\mathbf{L}$ strictly below the main diagonal are precisely the multipliers computed during Gaussian elimination.

Proof The inverse of an elementary elimination matrix $\mathbf{M}_k = \mathbf{I} - \boldsymbol{\ell}_k \boldsymbol{e}_k^T$ is $\mathbf{M}_k^{-1} = \mathbf{I} + \boldsymbol{\ell}_k \boldsymbol{e}_k^T$. This identity follows from the fact that the first k entries of $\boldsymbol{\ell}_k$ are zero, which implies $\boldsymbol{e}_k^T \boldsymbol{\ell}_k = 0$ and therefore

$$(\mathbf{I} - \boldsymbol{\ell}_k \boldsymbol{e}_k^T)(\mathbf{I} + \boldsymbol{\ell}_k \boldsymbol{e}_k^T) = \mathbf{I} - \boldsymbol{\ell}_k \boldsymbol{e}_k^T + \boldsymbol{\ell}_k \boldsymbol{e}_k^T - \boldsymbol{\ell}_k (\boldsymbol{e}_k^T \boldsymbol{\ell}_k) \boldsymbol{e}_k^T = \mathbf{I}.$$

Next, consider the product of two successive inverse matrices $\mathbf{M}_j^{-1} \mathbf{M}_k^{-1}$ with $j < k$. Because $\boldsymbol{e}_j^T \boldsymbol{\ell}_k = 0$, the cross term vanishes:

$$(\mathbf{I} + \boldsymbol{\ell}_j \boldsymbol{e}_j^T)(\mathbf{I} + \boldsymbol{\ell}_k \boldsymbol{e}_k^T) = \mathbf{I} + \boldsymbol{\ell}_j \boldsymbol{e}_j^T + \boldsymbol{\ell}_k \boldsymbol{e}_k^T + \boldsymbol{\ell}_j (\boldsymbol{e}_j^T \boldsymbol{\ell}_k) \boldsymbol{e}_k^T = \mathbf{I} + \boldsymbol{\ell}_j \boldsymbol{e}_j^T + \boldsymbol{\ell}_k \boldsymbol{e}_k^T.$$

By induction, the product $\mathbf{M}_1^{-1} \dots \mathbf{M}_{n-1}^{-1}$ simply places each column of multipliers into column k of the identity matrix without modifying any previously placed multipliers. Multiplying $\mathbf{U} = \mathbf{M}_{n-1} \dots \mathbf{M}_1 \mathbf{A}$ on the left by $\mathbf{L}$ yields $\mathbf{A} = \mathbf{L}\mathbf{U}$. □

Thm 2.12 **(Existence and Uniqueness of the LU Factorization)** A matrix $\mathbf{A} \in \mathbb{R}^{n \times n}$ has a unique LU factorization with unit lower triangular $\mathbf{L}$ and nonsingular upper triangular $\mathbf{U}$ if and only if every leading principal submatrix $\mathbf{A}_k = \mathbf{A}[1:k, 1:k]$ is nonsingular for each $k = 1, \dots, n$.

Proof We proceed by induction on the matrix dimension n. For $n = 1$, $\mathbf{A} = (a_{11})$. Because $\det(\mathbf{A}_1) = a_{11} \neq 0$, the unique factorization is $\mathbf{L} = (1)$ and $\mathbf{U} = (a_{11})$. Assume the result holds for matrices of dimension $n - 1$. Partition the $n \times n$ matrix $\mathbf{A}$ into blocks:

$$\mathbf{A} = \begin{pmatrix} \mathbf{A}_{n-1} & \boldsymbol{c} \\ \boldsymbol{d}^T & a_{nn} \end{pmatrix},$$

where $\mathbf{A}_{n-1} \in \mathbb{R}^{(n-1)\times(n-1)}$, $\boldsymbol{c}, \boldsymbol{d} \in \mathbb{R}^{n-1}$, and $a_{nn} \in \mathbb{R}$. By the induction hypothesis, $\mathbf{A}_{n-1}$ has a unique factorization $\mathbf{A}_{n-1} = \mathbf{L}_{n-1}\mathbf{U}_{n-1}$ with $\mathbf{L}_{n-1}$ unit lower triangular and $\mathbf{U}_{n-1}$ nonsingular. We seek block factors

$$\mathbf{L} = \begin{pmatrix} \mathbf{L}_{n-1} & \mathbf{0} \\ \boldsymbol{\ell}^T & 1 \end{pmatrix}, \qquad \mathbf{U} = \begin{pmatrix} \mathbf{U}_{n-1} & \boldsymbol{u} \\ \mathbf{0}^T & u_{nn} \end{pmatrix}.$$

Multiplying $\mathbf{L}$ and $\mathbf{U}$ and equating block by block gives:

$$\mathbf{L}_{n-1}\boldsymbol{u} = \boldsymbol{c}, \qquad \boldsymbol{\ell}^T \mathbf{U}_{n-1} = \boldsymbol{d}^T, \qquad \boldsymbol{\ell}^T \boldsymbol{u} + u_{nn} = a_{nn}.$$

Because $\mathbf{L}_{n-1}$ is unit lower triangular (hence invertible), $\boldsymbol{u} = \mathbf{L}_{n-1}^{-1}\boldsymbol{c}$ is uniquely determined. Because $\mathbf{U}_{n-1}$ is nonsingular, $\boldsymbol{\ell}^T = \boldsymbol{d}^T \mathbf{U}_{n-1}^{-1}$ is uniquely determined. Finally, the scalar diagonal entry is uniquely determined by $u_{nn} = a_{nn} - \boldsymbol{\ell}^T \boldsymbol{u}$. Taking determinants of the block decomposition gives

$$\det(\mathbf{A}) = \det(\mathbf{L}) \det(\mathbf{U}) = (1 \cdot \det(\mathbf{L}_{n-1}))(\det(\mathbf{U}_{n-1}) \cdot u_{nn}) = \det(\mathbf{A}_{n-1}) u_{nn}.$$

Because $\det(\mathbf{A}) \neq 0$ and $\det(\mathbf{A}_{n-1}) \neq 0$, we have $u_{nn} = \det(\mathbf{A}) / \det(\mathbf{A}_{n-1}) \neq 0$, which ensures that $\mathbf{U}$ is nonsingular. This completes the induction. □

2.3.2 Triangular Solves and Computational Complexity

Def 2.5 **(Forward and Backward Substitution)** Let $\mathbf{L} \in \mathbb{R}^{n\times n}$ be unit lower triangular and let $\mathbf{U} \in \mathbb{R}^{n\times n}$ be nonsingular upper triangular. Solving $\mathbf{A}\boldsymbol{x} = \boldsymbol{b}$ decouples into two triangular steps:

1. **Forward substitution ($\mathbf{L}\boldsymbol{y} = \boldsymbol{b}$):**

$$y_1 = b_1, \qquad y_i = b_i - \sum_{j=1}^{i-1} \ell_{ij} y_j \quad (i = 2, \ldots, n).$$

2. **Backward substitution ($\mathbf{U}\boldsymbol{x} = \boldsymbol{y}$):**

$$x_n = \frac{y_n}{u_{nn}}, \qquad x_i = \frac{1}{u_{ii}} \left(y_i - \sum_{j=i+1}^{n} u_{ij} x_j \right) \quad (i = n-1, \ldots, 1).$$

Thm 2.13 **(Computational Operation Count of LU Factorization)** For a general dense matrix $\mathbf{A} \in \mathbb{R}^{n\times n}$, computing the LU factorization requires asymptotically $\frac{2}{3}n^3$ floating-point operations. Evaluating the forward and backward triangular substitution steps requires $2n^2$ flops for each right-hand side vector $\boldsymbol{b}$.

Proof At step k of elimination, calculating the $n-k$ multipliers $\ell_{ik} = a_{ik}/a_{kk}$ requires $n-k$ divisions. Updating the active $(n-k)\times(n-k)$ submatrix via $a_{ij} \leftarrow a_{ij} - \ell_{ik}a_{kj}$ requires one multiplication and one subtraction for each entry, totaling $2(n-k)^2$ operations. Summing over all $n-1$ elimination steps gives

$$\sum_{k=1}^{n-1} \left[(n-k) + 2(n-k)^2 \right] = 2\sum_{m=1}^{n-1} m^2 + \sum_{m=1}^{n-1} m = 2\left(\frac{(n-1)n(2n-1)}{6} \right) + \frac{(n-1)n}{2} = \frac{2}{3}n^3 - \frac{1}{2}n^2 - \frac{1}{6}$$

For forward substitution with unit diagonal, row i requires $i-1$ multiplications and $i-1$ subtractions, totaling $\sum_{i=1}^{n} 2(i-1) = n^2 - n$ flops. For backward substitution, row i requires $n-i$ multiplications, $n-i$ subtractions, and one division, totaling $\sum_{i=1}^{n}(2(n-i)+1) = n^2$ flops. The combined solve phase requires $2n^2 - n \approx 2n^2$ flops. □

2.3.3 Partial Pivoting and the Growth Factor

In floating-point arithmetic, unpivoted Gaussian elimination fails if a pivot is zero, and it can be unstable when a pivot is small relative to the subdiagonal entries. Dividing by a small pivot creates large multipliers and can amplify rounding errors. Partial pivoting interchanges rows so that the pivot has maximal absolute value in its column. It keeps every multiplier at most one in magnitude and is usually stable, but it does not guarantee a small growth factor. The Wilkinson example below shows the exceptional worst case.

Def 2.6 **(PA = LU Factorization with Partial Pivoting)** Gaussian elimination with partial pivoting produces the decomposition

$$\mathbf{PA} = \mathbf{LU},$$

where $\mathbf{P} \in \mathbb{R}^{n\times n}$ is a permutation matrix recording row swaps, $\mathbf{L} \in \mathbb{R}^{n\times n}$ is unit lower triangular with all subdiagonal entries bounded by $|\ell_{ij}| \leq 1$, and $\mathbf{U} \in \mathbb{R}^{n\times n}$ is upper triangular.

Thm 2.14 **(Wilkinson Growth Factor Bound for Partial Pivoting)** Let $\mathbf{A} \in \mathbb{R}^{n\times n}$. The growth factor ρ of Gaussian elimination is defined by

$$\rho = \frac{\max_{i,j,k} |a_{ij}^{(k)}|}{\max_{i,j} |a_{ij}|},$$

where $a_{ij}^{(k)}$ denotes the entries of the active submatrix after step k. For Gaussian elimination with partial pivoting, the growth factor satisfies

$$\rho \leq 2^{n-1}.$$

Proof Let $M_k = \max_{i,j} |a_{ij}^{(k)}|$ denote the maximum absolute entry of the active matrix after k elimination steps, with $M_0 = \max_{i,j} |a_{ij}|$. At step k, partial pivoting guarantees that the pivot $|a_{kk}^{(k-1)}| = \max_{i\geq k} |a_{ik}^{(k-1)}|$, which ensures that all multipliers satisfy

$$|\ell_{ik}| = \left| \frac{a_{ik}^{(k-1)}}{a_{kk}^{(k-1)}} \right| \leq 1 \qquad \text{for all } i = k+1, \ldots, n.$$

The elimination update rule is $a_{ij}^{(k)} = a_{ij}^{(k-1)} - \ell_{ik} a_{kj}^{(k-1)}$. Applying the triangle inequality yields

$$|a_{ij}^{(k)}| \leq |a_{ij}^{(k-1)}| + |\ell_{ik}||a_{kj}^{(k-1)}| \leq M_{k-1} + (1)M_{k-1} = 2M_{k-1}.$$

Taking the maximum over all active indices gives $M_k \leq 2M_{k-1}$. By induction, $M_{n-1} \leq 2^{n-1} M_0$. Dividing by M_0 establishes $\rho \leq 2^{n-1}$. □

The growth factor matters because it controls the rounding errors committed during elimination. The next theorem expresses these errors as a perturbation of $\mathbf{A}$. It uses the unit roundoff u of Def 1.26 and the constant $\gamma_k = ku/(1-ku)$, defined whenever $ku < 1$. Absolute values and inequalities between matrices are taken entrywise.

Thm 2.15 **(Backward Error of Gaussian Elimination)** Let $\mathbf{A} \in \mathbb{R}^{n\times n}$ be nonsingular, and suppose Gaussian elimination with partial pivoting runs to completion in floating-point arithmetic. Let $\mathbf{P}$ be the computed permutation, let $\hat{\mathbf{L}}$ and $\hat{\mathbf{U}}$ be the computed factors,

and let $\hat{\boldsymbol{x}}$ be the solution of $\mathbf{A}\boldsymbol{x} = \boldsymbol{b}$ computed by forward and backward substitution. Then

$$(\mathbf{A} + \Delta\mathbf{A})\hat{\boldsymbol{x}} = \boldsymbol{b}, \qquad |\mathbf{P}\,\Delta\mathbf{A}| \le \gamma_{3n}|\hat{\mathbf{L}}||\hat{\mathbf{U}}|.$$

Proof We outline the argument, and Higham gives the complete proof (Higham, 2002). Row interchanges introduce no rounding error, so it suffices to analyze elimination without pivoting applied to $\mathbf{PA}$. Each entry of $\hat{\mathbf{L}}$ and $\hat{\mathbf{U}}$ is obtained from one entry of the identity $\mathbf{PA} = \mathbf{LU}$ by at most n multiplications and additions and possibly one division. Applying Thm 1.23 to each of these operations shows that $\hat{\mathbf{L}}\hat{\mathbf{U}} = \mathbf{PA} + \mathbf{E}$ with $|\mathbf{E}| \le \gamma_n|\hat{\mathbf{L}}||\hat{\mathbf{U}}|$. The same argument applied to the two triangular solves shows that the computed vectors satisfy

$$(\hat{\mathbf{L}} + \Delta\mathbf{L})\hat{\boldsymbol{y}} = \mathbf{P}\boldsymbol{b}, \qquad (\hat{\mathbf{U}} + \Delta\mathbf{U})\hat{\boldsymbol{x}} = \hat{\boldsymbol{y}}, \qquad |\Delta\mathbf{L}| \le \gamma_n|\hat{\mathbf{L}}|, \qquad |\Delta\mathbf{U}| \le \gamma_n|\hat{\mathbf{U}}|.$$

Multiplying the two perturbed factors gives $(\mathbf{PA}+\mathbf{E}+\hat{\mathbf{L}}\Delta\mathbf{U}+\Delta\mathbf{L}\hat{\mathbf{U}}+\Delta\mathbf{L}\Delta\mathbf{U})\hat{\boldsymbol{x}} = \mathbf{P}\boldsymbol{b}$. The four perturbation terms are bounded entrywise by $(3\gamma_n + \gamma_n^2)|\hat{\mathbf{L}}||\hat{\mathbf{U}}|$, and $3\gamma_n + \gamma_n^2 \le \gamma_{3n}$. □

Cor 2.16 **(Normwise Backward Error and the Growth Factor)** Under the hypotheses of Thm 2.15, let ρ be the growth factor of the computed elimination. Then

$$\frac{\|\Delta\mathbf{A}\|_\infty}{\|\mathbf{A}\|_\infty} \le n^2\gamma_{3n}\,\rho.$$

Proof Partial pivoting gives $|\hat{\ell}_{ik}| \le 1$, and the definition of ρ bounds every entry of $\hat{\mathbf{U}}$ by $\rho \max_{i,j}|a_{ij}|$. Each entry of $|\hat{\mathbf{L}}||\hat{\mathbf{U}}|$ is a sum of at most n products, so it is at most $n\rho\max_{i,j}|a_{ij}|$. The permutation $\mathbf{P}$ only reorders rows. A row of $|\Delta\mathbf{A}|$ therefore has absolute row sum at most $n \cdot \gamma_{3n} \cdot n\rho\max_{i,j}|a_{ij}|$, and $\max_{i,j}|a_{ij}| \le \|\mathbf{A}\|_\infty$. □

Corollary 2.16 shows that partial pivoting is backward stable whenever the growth factor is modest, and Cor 2.9 then bounds the forward error by the condition number. The worst-case bound $\rho \le 2^{n-1}$ of Thm 2.14 is attained by special matrices such as the Wilkinson matrix in the exercises below, but such growth is rarely observed. For random matrices, Trefethen and Schreiber measured average growth factors of about $n^{2/3}$ for partial pivoting (Trefethen and Schreiber, 1990). The factor n^2 in the corollary is also pessimistic, since the bound assumes that every rounding error takes its worst possible value and sign.

2.3.4 Iterative Refinement

Theorem 2.15 bounds the backward error of the computed solution, but the forward error can still be as large as $\kappa(\mathbf{A})$ times that backward error. Iterative refinement reuses the

computed factors to reduce the forward error. Each step costs one residual evaluation and one pair of triangular solves, which is small compared with the factorization.

Def 2.7 **(Iterative Refinement)** Let $\hat{\boldsymbol{x}}^{(0)}$ be computed from the factorization $\mathbf{PA} \approx \hat{\mathbf{L}}\hat{\mathbf{U}}$. For $k = 0, 1, 2, \ldots$:

1. Compute the residual $\boldsymbol{r}^{(k)} = \boldsymbol{b} - \mathbf{A}\hat{\boldsymbol{x}}^{(k)}$, if possible in higher precision.
2. Solve the correction equation $\mathbf{A}\boldsymbol{d}^{(k)} = \boldsymbol{r}^{(k)}$ with the existing factors.
3. Update $\hat{\boldsymbol{x}}^{(k+1)} = \hat{\boldsymbol{x}}^{(k)} + \boldsymbol{d}^{(k)}$.

Refinement improves the forward error when the factors are accurate enough to give a useful correction and the residual is computed more accurately than the factorization. The precise convergence conditions depend on $\kappa(\mathbf{A})$ and on the precisions used for the factorization, the residual, and the update (Higham, 2002).

2.3.5 Exercises

Ex 2.3

1. **Compute. Hand execution of unpivoted versus pivoted LU.** Consider the matrix

$$\mathbf{A} = \begin{pmatrix} 10^{-4} & 1 \\ 1 & 1 \end{pmatrix}, \qquad \boldsymbol{b} = \begin{pmatrix} 1 \\ 2 \end{pmatrix}.$$

   (a) Compute the unpivoted LU factorization $\mathbf{A} = \mathbf{LU}$ in exact arithmetic, and evaluate the multiplier ℓ_{21}.

   (b) Simulate standard 3-digit floating-point arithmetic with rounding. Show that unpivoted elimination computes the erroneous solution $\hat{\boldsymbol{x}} = (0, 1)^T$.

   (c) Compute the pivoted factorization $\mathbf{PA} = \mathbf{LU}$ using partial pivoting. Verify that in 3-digit arithmetic, the computed solution recovers $(1, 1)^T$ with negligible error.

2. **Explore. The Wilkinson counterexample and exponential growth.** Consider the $n \times n$ matrix

$$\mathbf{W}_n = \begin{pmatrix} 1 & 0 & 0 & \cdots & 0 & 1 \\ -1 & 1 & 0 & \cdots & 0 & 1 \\ -1 & -1 & 1 & \cdots & 0 & 1 \\ \vdots & \vdots & \vdots & \ddots & \vdots & \vdots \\ -1 & -1 & -1 & \cdots & 1 & 1 \\ -1 & -1 & -1 & \cdots & -1 & 1 \end{pmatrix}.$$

   (a) Show that partial pivoting performs no row swaps on $\mathbf{W}_n$ because every diagonal entry already has absolute value 1, matching the maximum in its column.

(b) Compute the entries of the final column after each elimination step $k = 1, 2, \ldots, n-1$.

(c) Prove that the final bottom-right entry grows as $u_{nn} = 2^{n-1}$, confirming that the bound in Thm 2.14 is tight.

3. **Derive. Banded matrices and fill-in.** Let $\mathbf{A} \in \mathbb{R}^{n \times n}$ be a tridiagonal matrix ($a_{ij} = 0$ for $|i - j| > 1$).

(a) Prove that without pivoting, the factors $\mathbf{L}$ and $\mathbf{U}$ retain the tridiagonal sparsity structure, meaning $\mathbf{L}$ has bandwidth 1 on the subdiagonal and $\mathbf{U}$ has bandwidth 1 on the superdiagonal.

(b) Calculate the flop count for unpivoted tridiagonal LU factorization and forward/backward substitution. Show that the total work is $O(n)$ instead of $O(n^3)$.

(c) Explain what happens to the upper bandwidth of $\mathbf{U}$ when partial pivoting performs row interchanges.

4. **Derive. Block LU decomposition and the Schur complement.** Partition a nonsingular matrix $\mathbf{A}$ into blocks

$$\mathbf{A} = \begin{pmatrix} \mathbf{A}_{11} & \mathbf{A}_{12} \\ \mathbf{A}_{21} & \mathbf{A}_{22} \end{pmatrix},$$

where $\mathbf{A}_{11} \in \mathbb{R}^{k \times k}$ is nonsingular.

(a) Show that block Gaussian elimination factors $\mathbf{A}$ as

$$\mathbf{A} = \begin{pmatrix} \mathbf{I} & \mathbf{0} \\ \mathbf{A}_{21}\mathbf{A}_{11}^{-1} & \mathbf{I} \end{pmatrix} \begin{pmatrix} \mathbf{A}_{11} & \mathbf{A}_{12} \\ \mathbf{0} & \mathbf{S} \end{pmatrix},$$

where $\mathbf{S} = \mathbf{A}_{22} - \mathbf{A}_{21}\mathbf{A}_{11}^{-1}\mathbf{A}_{12}$ is the Schur complement of $\mathbf{A}_{11}$ in $\mathbf{A}$.

(b) Prove that $\det(\mathbf{A}) = \det(\mathbf{A}_{11}) \det(\mathbf{S})$.

(c) Relate the Schur complement $\mathbf{S}$ to the active submatrix remaining after k steps of standard Gaussian elimination.

5. **Compute. Determinants and multiple right-hand sides.** Let $\mathbf{PA} = \mathbf{LU}$ be the computed LU factorization with partial pivoting.

(a) Prove that $\det(\mathbf{A}) = (-1)^s \prod_{i=1}^{n} u_{ii}$, where s is the number of row interchanges recorded by $\mathbf{P}$.

(b) Explain why computing the determinant via $\mathbf{PA} = \mathbf{LU}$ requires $O(n^3)$ flops, whereas evaluating the Leibniz expansion formula requires $O(n!)$ flops.

(c) For m distinct right-hand side vectors $\boldsymbol{b}_1, \ldots, \boldsymbol{b}_m$, analyze the computational cost of solving $\mathbf{A}\boldsymbol{x}_j = \boldsymbol{b}_j$ by reusing the single factorization $\mathbf{PA} = \mathbf{LU}$ versus computing m independent solves from scratch.

2.4 Symmetric Positive Definite Matrices

Symmetric positive definite matrices occur whenever a nonzero displacement must have positive energy. Springs, diffusion operators, covariance models, and strictly convex quadratic objectives all lead to this structure. For these matrices an algebraic condition, a geometry, and an efficient algorithm describe the same property.

2.4.1 Positive Quadratic Energy

Def 2.8 **(Symmetric Positive Definite Matrix)** A real matrix $\mathbf{A} \in \mathbb{R}^{n\times n}$ is **symmetric positive definite**, abbreviated SPD, if $\mathbf{A} = \mathbf{A}^T$ and

$$\boldsymbol{x}^T\mathbf{A}\boldsymbol{x} > 0 \qquad \text{for every } \boldsymbol{x} \neq \mathbf{0}.$$

It is **symmetric positive semidefinite** if the strict inequality is replaced by $\boldsymbol{x}^T\mathbf{A}\boldsymbol{x} \geq 0$.

Thm 2.17 **(First Consequences of Positive Definiteness)** If $\mathbf{A}$ is SPD, then every diagonal entry is positive, every principal submatrix is SPD, and $\mathbf{A}$ is nonsingular.

Proof Taking $\boldsymbol{x} = \boldsymbol{e}_i$ gives $a_{ii} = \boldsymbol{e}_i^T\mathbf{A}\boldsymbol{e}_i > 0$. A vector supported only on an index set I has quadratic energy equal to the quadratic form of the principal submatrix $\mathbf{A}[I, I]$, so that submatrix is positive definite. Finally, if $\mathbf{A}\boldsymbol{x} = \mathbf{0}$ for a nonzero $\boldsymbol{x}$, then $\boldsymbol{x}^T\mathbf{A}\boldsymbol{x} = 0$, a contradiction. Thus the null space is trivial. □

Def 2.9 **(The A-Inner Product and Energy Norm)** If $\mathbf{A}$ is SPD, then

$$\langle \boldsymbol{x}, \boldsymbol{y}\rangle_{\mathbf{A}} = \boldsymbol{x}^T\mathbf{A}\boldsymbol{y}, \qquad \|\boldsymbol{x}\|_{\mathbf{A}} = \sqrt{\boldsymbol{x}^T\mathbf{A}\boldsymbol{x}}$$

define an inner product and its induced norm. The scalar $\boldsymbol{x}^T\mathbf{A}\boldsymbol{x}$ is often called the **energy** of $\boldsymbol{x}$.

Proof Linearity follows from matrix multiplication, symmetry from $\mathbf{A}^T = \mathbf{A}$, and positive definiteness from the definition of SPD. The general inner-product results of Sec. 1.2 then give Cauchy–Schwarz and the triangle inequality for $\|\cdot\|_{\mathbf{A}}$. □

Example For

$$\mathbf{A} = \begin{pmatrix} 4 & 0 \\ 0 & 1 \end{pmatrix},$$

the unit-energy curve is $4x_1^2 + x_2^2 = 1$. A displacement in the first coordinate costs four times as much energy as the same displacement in the second. A norm chosen this way also expresses which errors matter in the model.

2.4.2 Cholesky Factorization

Thm 2.18 **(Cholesky Factorization)** Every SPD matrix $\mathbf{A} \in \mathbb{R}^{n\times n}$ has a unique factorization

$$\mathbf{A} = \mathbf{L}\mathbf{L}^T,$$

where $\mathbf{L}$ is lower triangular with positive diagonal entries.

Proof We proceed by induction on n. For $n = 1$, $\mathbf{A} = (\alpha)$ with $\alpha > 0$, so $\mathbf{L} = (\sqrt{\alpha})$ is the unique positive square root. For the induction step, partition $\mathbf{A} \in \mathbb{R}^{n\times n}$ as

$$\mathbf{A} = \begin{pmatrix} \alpha & \boldsymbol{r}^T \\ \boldsymbol{r} & \mathbf{B} \end{pmatrix}.$$

Since $\mathbf{A}$ is SPD, $\alpha = \boldsymbol{e}_1^T\mathbf{A}\boldsymbol{e}_1 > 0$. Set $\ell_{11} = \sqrt{\alpha}$ and $\boldsymbol{\ell} = \boldsymbol{r}/\ell_{11}$. Form the Schur complement

$$\mathbf{S} = \mathbf{B} - \boldsymbol{\ell}\boldsymbol{\ell}^T = \mathbf{B} - \frac{\boldsymbol{r}\boldsymbol{r}^T}{\alpha}.$$

For any nonzero $\boldsymbol{y} \in \mathbb{R}^{n-1}$, choose $\boldsymbol{x} = (-\boldsymbol{\ell}^T\boldsymbol{y}/\ell_{11}, \boldsymbol{y}^T)^T \in \mathbb{R}^n$. Expanding the quadratic form:

$$\begin{aligned} \boldsymbol{x}^T\mathbf{A}\boldsymbol{x} &= \begin{pmatrix} -\frac{\boldsymbol{\ell}^T\boldsymbol{y}}{\ell_{11}} \\ \boldsymbol{y} \end{pmatrix}^T \begin{pmatrix} \alpha & \boldsymbol{r}^T \\ \boldsymbol{r} & \mathbf{B} \end{pmatrix} \begin{pmatrix} -\frac{\boldsymbol{\ell}^T\boldsymbol{y}}{\ell_{11}} \\ \boldsymbol{y} \end{pmatrix} \\ &= \alpha\left(\frac{\boldsymbol{\ell}^T\boldsymbol{y}}{\ell_{11}}\right)^2 - 2\frac{(\boldsymbol{\ell}^T\boldsymbol{y})(\boldsymbol{r}^T\boldsymbol{y})}{\ell_{11}} + \boldsymbol{y}^T\mathbf{B}\boldsymbol{y} \\ &= (\boldsymbol{\ell}^T\boldsymbol{y})^2 - 2(\boldsymbol{\ell}^T\boldsymbol{y})^2 + \boldsymbol{y}^T\mathbf{B}\boldsymbol{y} \\ &= \boldsymbol{y}^T(\mathbf{B} - \boldsymbol{\ell}\boldsymbol{\ell}^T)\boldsymbol{y} = \boldsymbol{y}^T\mathbf{S}\boldsymbol{y}. \end{aligned}$$

Since $\boldsymbol{y} \neq \mathbf{0}$, the vector $\boldsymbol{x}$ is nonzero, so $\boldsymbol{y}^T\mathbf{S}\boldsymbol{y} = \boldsymbol{x}^T\mathbf{A}\boldsymbol{x} > 0$. Hence $\mathbf{S}$ is SPD. By induction, $\mathbf{S} = \mathbf{K}\mathbf{K}^T$ for a unique lower triangular $\mathbf{K}$ with positive diagonal entries. Block multiplication gives

$$\mathbf{A} = \begin{pmatrix} \ell_{11} & \mathbf{0}^T \\ \boldsymbol{\ell} & \mathbf{K} \end{pmatrix}\begin{pmatrix} \ell_{11} & \boldsymbol{\ell}^T \\ \mathbf{0} & \mathbf{K}^T \end{pmatrix} = \begin{pmatrix} \ell_{11}^2 & \ell_{11}\boldsymbol{\ell}^T \\ \ell_{11}\boldsymbol{\ell} & \boldsymbol{\ell}\boldsymbol{\ell}^T + \mathbf{K}\mathbf{K}^T \end{pmatrix} = \begin{pmatrix} \alpha & \boldsymbol{r}^T \\ \boldsymbol{r} & \mathbf{B} \end{pmatrix}.$$

The positivity of the diagonal requires $\ell_{11} = \sqrt{\alpha}$, which uniquely fixes $\boldsymbol{\ell} = \boldsymbol{r}/\ell_{11}$, and uniqueness of $\mathbf{K}$ makes the entire factor $\mathbf{L}$ unique. □

Thm 2.19 **(Equivalent Characterizations of Positive Definiteness)** For a real symmetric matrix $\mathbf{A}$, the following statements are equivalent.

1. $\boldsymbol{x}^T\mathbf{A}\boldsymbol{x} > 0$ for every nonzero $\boldsymbol{x}$.
2. $\mathbf{A} = \mathbf{L}\mathbf{L}^T$ for a lower triangular $\mathbf{L}$ with positive diagonal.

3. $\mathbf{A} = \mathbf{W}^T\mathbf{W}$ for a nonsingular matrix $\mathbf{W}$.

4. Every leading principal minor is positive.

Proof The implication from (1) to (2) is Thm. 2.18. Statement (2) implies (3) by taking $\mathbf{W} = \mathbf{L}^T$. If (3) holds, then

$$\boldsymbol{x}^T\mathbf{A}\boldsymbol{x} = \|\mathbf{W}\boldsymbol{x}\|_2^2 > 0$$

for every nonzero $\boldsymbol{x}$, so (3) implies (1).
If (2) holds, the leading k by k block satisfies $\mathbf{A}_k = \mathbf{L}_k\mathbf{L}_k^T$. Hence

$$\det(\mathbf{A}_k) = \det(\mathbf{L}_k)^2 = \prod_{j=1}^{k} \ell_{jj}^2 > 0,$$

which proves (4). Conversely, suppose every leading principal minor is positive. Apply the Cholesky formulas in order. At step k, the next squared pivot v_k satisfies

$$\det(\mathbf{A}_k) = \left(\prod_{j=1}^{k-1} \ell_{jj}^2\right) v_k.$$

The determinant on the left and the preceding factors are positive, so $v_k > 0$. Thus every square root exists and the algorithm produces (2). □

Remark 2.3 After the spectral theorem is available, one more characterization follows directly. A real symmetric matrix is SPD exactly when all its eigenvalues are positive. The Cholesky argument does not need this fact, so we postpone it and keep the present argument self-contained.

2.4.3 The Cholesky Algorithm

Thm 2.20 **(The Cholesky Algorithm)** The entries of the factor in $\mathbf{A} = \mathbf{L}\mathbf{L}^T$ are computed column by column:

$$\ell_{jj} = \sqrt{a_{jj} - \sum_{k=1}^{j-1} \ell_{jk}^2},$$
$$\ell_{ij} = \frac{a_{ij} - \sum_{k=1}^{j-1} \ell_{ik}\ell_{jk}}{\ell_{jj}}, \qquad i = j+1, \ldots, n.$$

For an SPD matrix every expression under a square root is positive. A nonpositive pivot signals that the input is not positive definite, apart from the effects of rounding.

Proof For $i \geq j$, equating entry (i, j) in $\mathbf{A}$ and $\mathbf{L}\mathbf{L}^T$ gives

$$a_{ij} = \sum_{k=1}^{j} \ell_{ik}\ell_{jk}.$$

When $i = j$, solve this identity for the positive diagonal entry. When $i > j$, solve it for ℓ_{ij}. The order of computation ensures that all earlier terms are already known. □

Example For

$$\mathbf{A} = \begin{pmatrix} 4 & 2 & -2 \\ 2 & 10 & 2 \\ -2 & 2 & 6 \end{pmatrix},$$

the algorithm gives

$$\begin{aligned} \ell_{11} &= 2, \quad \ell_{21} = 1, \quad \ell_{31} = -1, \\ \ell_{22} &= 3, \quad \ell_{32} = 1, \quad \ell_{33} = 2. \end{aligned}$$

Thus

$$\mathbf{L} = \begin{pmatrix} 2 & 0 & 0 \\ 1 & 3 & 0 \\ -1 & 1 & 2 \end{pmatrix}, \qquad \mathbf{L}\mathbf{L}^T = \mathbf{A}.$$

To solve $\mathbf{A}\boldsymbol{x} = \boldsymbol{b}$, first solve $\mathbf{L}\boldsymbol{y} = \boldsymbol{b}$ and then $\mathbf{L}^T\boldsymbol{x} = \boldsymbol{y}$.

2.4.4 Cost and Stability

Thm 2.21 **(Flop Count and Storage)** Dense Cholesky factorization requires $n^3/3 + O(n^2)$ floating-point operations. Only one triangular half of the matrix and factor need be stored. The two triangular solves for one right-hand side require $2n^2 + O(n)$ further operations.

Proof Column j updates $n - j + 1$ entries using inner products of length about j. Summing these costs gives

$$\sum_{j=1}^{n} 2(n-j)j = \frac{n^3}{3} + O(n^2).$$

The leading cost is half that of dense LU factorization. □

Remark 2.4 Cholesky needs no pivoting for an SPD matrix. Moreover, the identity

$$a_{ii} = \sum_{k=1}^{i} \ell_{ik}^2$$

implies $|\ell_{ik}| \le \sqrt{a_{ii}}$, so the factor cannot exhibit the element growth possible in general elimination. In standard floating-point arithmetic, and absent overflow or underflow, the computed factor is the exact Cholesky factor of a nearby symmetric matrix. This is the appropriate backward-stability statement. It does not say that an ill-conditioned system has an accurate forward solution.

Remark 2.5 Once singular values are available, Cor 4.20 gives

$$\kappa_2(\mathbf{A}) = \frac{\lambda_{\max}(\mathbf{A})}{\lambda_{\min}(\mathbf{A})}, \qquad \kappa_2(\mathbf{L}) = \sqrt{\kappa_2(\mathbf{A})}.$$

These are spectral interpretations of structure already visible in the energy and the factorization.

2.4.5 Sparse Cholesky and the Elimination Graph

For a sparse SPD matrix, the Cholesky factor can have many more nonzero entries than the matrix itself. These new entries are called **fill**. Fill determines the memory and the arithmetic of a sparse factorization, and it depends strongly on the order in which the unknowns are eliminated. A graph associated with the matrix predicts the fill before any arithmetic is done.

Def 2.10 **(Graph of a Symmetric Matrix)** The graph of a symmetric matrix $\mathbf{A} \in \mathbb{R}^{n\times n}$ has vertices $1, \dots, n$ and an edge $\{i, j\}$ for each pair $i \neq j$ with $a_{ij} \neq 0$. The neighbors of a vertex v are the vertices joined to v by an edge, and the degree of v is the number of its neighbors.

Thm 2.22 **(Elimination as a Graph Operation)** Let $\mathbf{A}$ be SPD with graph G, and partition $\mathbf{A}$ as in the proof of Thm 2.18, with first column $(\alpha, \boldsymbol{r}^T)^T$ and trailing block $\mathbf{B}$. The first column of $\mathbf{L}$ is nonzero exactly in the rows of the neighbors of vertex 1. Apart from exact numerical cancellation, the graph of the Schur complement $\mathbf{S} = \mathbf{B} - \boldsymbol{r}\boldsymbol{r}^T/\alpha$ is obtained from G by deleting vertex 1 and joining every pair of its neighbors by an edge.

Proof The first column of $\mathbf{L}$ is $(\sqrt{\alpha}, \boldsymbol{r}^T/\sqrt{\alpha})^T$, and $r_i \neq 0$ exactly when i is a neighbor of vertex 1. The entries of the Schur complement are $s_{ij} = b_{ij} - r_i r_j/\alpha$. The correction $r_i r_j/\alpha$ is nonzero exactly when both i and j are neighbors of vertex 1. For such a pair, s_{ij} is nonzero unless b_{ij} happens to equal $r_i r_j/\alpha$, and for every other pair $s_{ij} = b_{ij}$. □

Applying Thm 2.22 repeatedly describes the whole factorization. Eliminating a vertex turns its remaining neighbors into a clique, and the nonzero pattern of column k of $\mathbf{L}$ is the set of neighbors of vertex k at the moment it is eliminated. Reordering the unknowns by a permutation $\mathbf{P}$ and factoring $\mathbf{PAP}^T$ changes the elimination order and therefore the fill, without changing the solution.

Example **(An arrow matrix)** Let $\mathbf{A}$ be SPD with nonzero entries only on the diagonal and in the first row and column. Its graph is a star with center 1. If the center is eliminated first, its $n-1$ neighbors become a clique, the Schur complement is full, and $\mathbf{L}$ has about $n^2/2$ nonzero entries. If the center is eliminated last, every leaf has the center as its only neighbor, no edges are added, and $\mathbf{L}$ has $2n-1$ nonzero entries. The two orders give the same solution at costs of $O(n^3)$ and $O(n)$ flops.

Remark 2.6 **(Fill-reducing orderings)** Finding the ordering with the least fill is NP-complete (Yannakakis, 1981), so practical codes use heuristics. The **minimum degree** heuristic always eliminates a vertex of smallest current degree, which bounds the fill created at each step by the square of that degree. The degrees change after every elimination, and an efficient implementation stores the vertices in a bucket queue, with one doubly linked list of vertices for each degree. Moving a vertex to a new bucket then takes constant time, and the next vertex is found by scanning upward from the lowest nonempty bucket (Davis, 2006). **Nested dissection** instead removes a small set of vertices that separates the graph into two parts, orders the separator last, and recurses on the parts.

Nested dissection determines the cost of sparse Cholesky for the Poisson problem. On an $N \times N$ grid with $n = N^2$ unknowns, the natural row-by-row order gives a factor with bandwidth N, so $\mathbf{L}$ has about $n^{3/2}$ nonzero entries and the factorization costs about n^2 flops. Nested dissection with grid lines as separators reduces these to $O(n \log n)$ nonzero entries and $O(n^{3/2})$ flops (George, 1973). On an $N \times N \times N$ grid, separators are planes with $n^{2/3}$ vertices, and nested dissection leaves $O(n^{4/3})$ nonzero entries and $O(n^2)$ flops. The growth of the factor with n is the reason that large three-dimensional problems are solved by the iterative methods of Section 6.2.

2.4.6 Exercises

Ex 2.4

1. **Compute.** Compute the Cholesky factor of

$$\mathbf{A} = \begin{pmatrix} 9 & 3 & -6 \\ 3 & 26 & -7 \\ -6 & -7 & 9 \end{pmatrix}$$

   by hand, then use it to solve $\mathbf{A}\boldsymbol{x} = (6, 44, 2)^T$.

2. **Derive.** Prove directly from the quadratic form that every principal submatrix of an SPD matrix is SPD. Explain why the same statement for an arbitrary submatrix is meaningless.

3. **Derive.** Complete the block calculation in the proof of Thm. 2.18: verify exactly that $\boldsymbol{x}^T\mathbf{A}\boldsymbol{x} = \boldsymbol{y}^T\mathbf{S}\boldsymbol{y}$. Then prove the converse statement that if $\alpha > 0$ and $\mathbf{S}$ is SPD, the block matrix $\mathbf{A}$ is SPD.

4. **Derive.** Prove that the largest entry of an SPD matrix in absolute value lies on the main diagonal. Begin with a two by two principal submatrix and choose a useful vector, or use its positive determinant.

5. **Derive.** Let $\mathbf{X} \in \mathbb{R}^{m \times n}$ with $m \geq n$. Prove that $\mathbf{X}^T\mathbf{X}$ is SPD exactly when the columns of $\mathbf{X}$ are linearly independent. State what remains true when they are dependent.

6. **Check.** For the arrow matrix of the example with $n = 5$, draw the elimination graphs for the two orders, and count the nonzero entries of $\mathbf{L}$ in each case.

7. **Derive.** Use Thm 2.22 to prove that, apart from cancellation, $\ell_{ij} \neq 0$ for $i > j$ exactly when G contains a path from i to j whose interior vertices all precede j in the elimination order.

8. **Compute.** Assemble the five-point Poisson matrix on an $N \times N$ grid for $N = 20, 40, 80$. Factor it with a sparse direct solver using the natural order and a minimum degree order, and report the number of nonzero entries in the factors and the factorization times.

9. **Explore.** Show that $\|\boldsymbol{x}\|_{\mathbf{A}} = \|\mathbf{L}^T\boldsymbol{x}\|_2$. Use this identity to interpret the $\mathbf{A}$-unit sphere as the inverse image of the Euclidean unit sphere, without mentioning eigenvalues.

2.5 Reusing Factorizations after Structured Changes

Many scientific calculations solve sequences of related linear systems where the coefficient matrix changes by a low-rank perturbation or exhibits localized block-tridiagonal couplings. Rather than computing fresh factorizations from scratch, we update the existing matrix factors using the Sherman–Morrison and Woodbury identities, and exploit chain connectivity through block sweeps.

2.5.1 Rank-One Changes and the Sherman–Morrison Formula

Let a factorization of a nonsingular matrix $\mathbf{A}$ already be available. Suppose the matrix changes to $\mathbf{A} + \boldsymbol{u}\boldsymbol{v}^T$. A fresh dense factorization costs $O(n^3)$ operations. The rank-one formula replaces it by solves with the old factors and a few inner products.

Thm 2.23 **(Sherman–Morrison Formula)** Let $\mathbf{A} \in \mathbb{R}^{n\times n}$ be nonsingular, and let $\boldsymbol{u}, \boldsymbol{v} \in \mathbb{R}^n$. The updated matrix $\mathbf{A} + \boldsymbol{u}\boldsymbol{v}^T$ is nonsingular if and only if

$$1 + \boldsymbol{v}^T\mathbf{A}^{-1}\boldsymbol{u} \neq 0.$$

When this condition holds,

$$(\mathbf{A} + \boldsymbol{u}\boldsymbol{v}^T)^{-1} = \mathbf{A}^{-1} - \frac{\mathbf{A}^{-1}\boldsymbol{u}\boldsymbol{v}^T\mathbf{A}^{-1}}{1 + \boldsymbol{v}^T\mathbf{A}^{-1}\boldsymbol{u}}.$$

Proof Let $\boldsymbol{z} = \mathbf{A}^{-1}\boldsymbol{u}$ and $\alpha = 1 + \boldsymbol{v}^T\boldsymbol{z}$. If $\alpha = 0$, then

$$(\mathbf{A} + \boldsymbol{u}\boldsymbol{v}^T)\boldsymbol{z} = \boldsymbol{u} + \boldsymbol{u}\boldsymbol{v}^T\boldsymbol{z} = \alpha\boldsymbol{u} = \mathbf{0}.$$

The vector $\boldsymbol{z}$ is nonzero because $\alpha = 0$ implies $\boldsymbol{u} \neq \mathbf{0}$. Thus the updated matrix is singular. Now suppose $\alpha \neq 0$. Direct multiplication gives

$$\begin{aligned}(\mathbf{A} + \boldsymbol{u}\boldsymbol{v}^T)\left(\mathbf{A}^{-1} - \frac{\boldsymbol{z}\boldsymbol{v}^T\mathbf{A}^{-1}}{\alpha}\right) &= \mathbf{I} + \boldsymbol{u}\boldsymbol{v}^T\mathbf{A}^{-1} - \frac{\boldsymbol{u}\boldsymbol{v}^T\mathbf{A}^{-1}}{\alpha} - \frac{\boldsymbol{u}(\boldsymbol{v}^T\boldsymbol{z})\boldsymbol{v}^T\mathbf{A}^{-1}}{\alpha} \\ &= \mathbf{I} + \boldsymbol{u}\boldsymbol{v}^T\mathbf{A}^{-1} - \frac{1 + \boldsymbol{v}^T\boldsymbol{z}}{\alpha}\boldsymbol{u}\boldsymbol{v}^T\mathbf{A}^{-1} = \mathbf{I}.\end{aligned}$$

The displayed matrix is therefore the inverse of $\mathbf{A} + \boldsymbol{u}\boldsymbol{v}^T$. □

Def 2.11 **(Sherman–Morrison Solve)** To solve $(\mathbf{A} + \boldsymbol{u}\boldsymbol{v}^T)\boldsymbol{x} = \boldsymbol{b}$ from stored factors of $\mathbf{A}$:

1. Solve $\mathbf{A}\boldsymbol{z} = \boldsymbol{u}$ once for the update.
2. Compute $\alpha = 1 + \boldsymbol{v}^T\boldsymbol{z}$. Refactor the updated matrix if α is zero or so small that division by it would magnify roundoff.
3. For each right-hand side, solve $\mathbf{A}\boldsymbol{y} = \boldsymbol{b}$.
4. Compute

$$\boldsymbol{x} = \boldsymbol{y} - \boldsymbol{z}\frac{\boldsymbol{v}^T\boldsymbol{y}}{\alpha}.$$

The algorithm never forms $\mathbf{A}^{-1}$.

Example Let

$$\mathbf{A} = \begin{pmatrix} 2 & 0 \\ 0 & 3 \end{pmatrix}, \qquad \boldsymbol{u} = \begin{pmatrix} 1 \\ 1 \end{pmatrix}, \qquad \boldsymbol{v} = \begin{pmatrix} 1 \\ -1 \end{pmatrix}, \qquad \boldsymbol{b} = \begin{pmatrix} 1 \\ 0 \end{pmatrix}.$$

The two solves with $\mathbf{A}$ give $\boldsymbol{z} = (1/2, 1/3)^T$ and $\boldsymbol{y} = (1/2, 0)^T$. Since $\alpha = 1 + 1/2 - 1/3 = 7/6$,

$$\boldsymbol{x} = \boldsymbol{y} - \boldsymbol{z}\frac{1/2}{7/6} = \begin{pmatrix} 2/7 \\ -1/7 \end{pmatrix}.$$

Multiplication by $\mathbf{A} + \boldsymbol{u}\boldsymbol{v}^T = \begin{pmatrix} 3 & -1 \\ 1 & 2 \end{pmatrix}$ verifies the solution.

Remark 2.7 **(Cost of a rank-one change)** With a dense LU factorization, preparing the correction requires one $O(n^2)$ solve. Each new right-hand side requires one further solve and $O(n)$ correction work. A fresh $O(n^3)$ factorization is avoided. For a sparse factorization, the same comparison uses the measured cost of a triangular solve rather than the dense estimate.

2.5.2 Several Changes and the Woodbury Formula

A change described by r pairs of vectors has the form $\mathbf{A} + \mathbf{U}\mathbf{V}^T$, where $\mathbf{U}$ and $\mathbf{V}$ have r columns and $r \ll n$. The correction requires an $r \times r$ system. This small system determines both the cost and the reliability of the update.

Thm 2.24 **(Woodbury Formula)** Let $\mathbf{A} \in \mathbb{R}^{n\times n}$ be nonsingular, and let $\mathbf{U}, \mathbf{V} \in \mathbb{R}^{n\times r}$. Define

$$\mathbf{G} = \mathbf{I}_r + \mathbf{V}^T\mathbf{A}^{-1}\mathbf{U}.$$

The matrix $\mathbf{A} + \mathbf{U}\mathbf{V}^T$ is nonsingular if and only if $\mathbf{G}$ is nonsingular. In that case,

$$(\mathbf{A} + \mathbf{U}\mathbf{V}^T)^{-1} = \mathbf{A}^{-1} - \mathbf{A}^{-1}\mathbf{U}\mathbf{G}^{-1}\mathbf{V}^T\mathbf{A}^{-1}.$$

Proof Suppose $\mathbf{G}\boldsymbol{c} = \mathbf{0}$ for some nonzero $\boldsymbol{c}$. Then $\mathbf{U}\boldsymbol{c} \neq \mathbf{0}$, since otherwise $\mathbf{G}\boldsymbol{c} = \boldsymbol{c}$. Consequently, $\boldsymbol{x} = \mathbf{A}^{-1}\mathbf{U}\boldsymbol{c}$ is nonzero and

$$(\mathbf{A} + \mathbf{U}\mathbf{V}^T)\boldsymbol{x} = \mathbf{U}\boldsymbol{c} + \mathbf{U}\mathbf{V}^T\mathbf{A}^{-1}\mathbf{U}\boldsymbol{c} = \mathbf{U}\mathbf{G}\boldsymbol{c} = \mathbf{0}.$$

Thus a singular $\mathbf{G}$ makes the updated matrix singular.
Conversely, suppose $(\mathbf{A} + \mathbf{U}\mathbf{V}^T)\boldsymbol{x} = \mathbf{0}$ for some nonzero $\boldsymbol{x}$. Let $\boldsymbol{c} = \mathbf{V}^T\boldsymbol{x}$. The vector $\boldsymbol{c}$ cannot be zero, because that would imply $\mathbf{A}\boldsymbol{x} = \mathbf{0}$. From $\boldsymbol{x} = -\mathbf{A}^{-1}\mathbf{U}\boldsymbol{c}$, we obtain

$$\mathbf{G}\boldsymbol{c} = \boldsymbol{c} + \mathbf{V}^T\mathbf{A}^{-1}\mathbf{U}\boldsymbol{c} = \mathbf{V}^T\boldsymbol{x} - \mathbf{V}^T\boldsymbol{x} = \mathbf{0}.$$

Hence the two matrices are nonsingular together.
When $\mathbf{G}$ is nonsingular, direct multiplication verifies the inverse:

$$\begin{aligned}
&(\mathbf{A} + \mathbf{U}\mathbf{V}^T)\left(\mathbf{A}^{-1} - \mathbf{A}^{-1}\mathbf{U}\mathbf{G}^{-1}\mathbf{V}^T\mathbf{A}^{-1}\right) \\
&\qquad = \mathbf{A}\mathbf{A}^{-1} - \mathbf{A}\mathbf{A}^{-1}\mathbf{U}\mathbf{G}^{-1}\mathbf{V}^T\mathbf{A}^{-1} + \mathbf{U}\mathbf{V}^T\mathbf{A}^{-1} - \mathbf{U}\mathbf{V}^T\mathbf{A}^{-1}\mathbf{U}\mathbf{G}^{-1}\mathbf{V}^T\mathbf{A}^{-1} \\
&\qquad = \mathbf{I}_n - \mathbf{U}\mathbf{G}^{-1}\mathbf{V}^T\mathbf{A}^{-1} + \mathbf{U}\mathbf{V}^T\mathbf{A}^{-1} - \mathbf{U}(\mathbf{V}^T\mathbf{A}^{-1}\mathbf{U})\mathbf{G}^{-1}\mathbf{V}^T\mathbf{A}^{-1} \\
&\qquad = \mathbf{I}_n + \mathbf{U}\left(\mathbf{I}_r - (\mathbf{I}_r + \mathbf{V}^T\mathbf{A}^{-1}\mathbf{U})\mathbf{G}^{-1}\right)\mathbf{V}^T\mathbf{A}^{-1} \\
&\qquad = \mathbf{I}_n + \mathbf{U}\left(\mathbf{I}_r - \mathbf{G}\mathbf{G}^{-1}\right)\mathbf{V}^T\mathbf{A}^{-1} \\
&\qquad = \mathbf{I}_n + \mathbf{U}(\mathbf{I}_r - \mathbf{I}_r)\mathbf{V}^T\mathbf{A}^{-1} = \mathbf{I}_n.
\end{aligned}$$

□

Def 2.12 **(Woodbury Solve)** Let stored factors of $\mathbf{A}$ be available.

1. Solve the block system $\mathbf{A}\mathbf{W} = \mathbf{U}$.
2. Form $\mathbf{G} = \mathbf{I}_r + \mathbf{V}^T\mathbf{W}$ and factor the small matrix $\mathbf{G}$.
3. For a right-hand side matrix $\mathbf{B}$, solve $\mathbf{A}\mathbf{Y} = \mathbf{B}$.

4. Solve $\mathbf{GC} = \mathbf{V}^T\mathbf{Y}$ and return

$$\mathbf{X} = \mathbf{Y} - \mathbf{WC}.$$

When several right-hand sides are available, solve them together. This is usually faster than solving one column at a time.

The setup requires r solves with the old factors, the product $\mathbf{V}^T\mathbf{W}$, and one factorization of order r. Each later right-hand side requires one old solve, two products involving an $n \times r$ matrix, and one small solve. For dense factors, the setup costs $O(rn^2 + nr^2 + r^3)$ rather than $O(n^3)$. This comparison favors Woodbury only when r remains small.

Remark 2.8 **(Transpose solves)** The same retained factors also solve the transposed updated system. Compute $\mathbf{Z} = \mathbf{A}^{-T}\mathbf{V}$ and use

$$\mathbf{G}^T = \mathbf{I}_r + \mathbf{U}^T\mathbf{Z}.$$

The transpose solve then has the same four steps with $\mathbf{U}$ and $\mathbf{V}$ interchanged. No factorization of the full transpose is needed when the base factor already supports a transpose solve.

Remark 2.9 **(Updating only the inverse diagonal)** Suppose the diagonal of $\mathbf{A}^{-1}$ is already known. The corrected diagonal follows from

$$\operatorname{diag}\big((\mathbf{A} + \mathbf{UV}^T)^{-1}\big) = \operatorname{diag}(\mathbf{A}^{-1}) - \operatorname{diag}(\mathbf{WG}^{-1}\mathbf{Z}^T),$$

where $\mathbf{W} = \mathbf{A}^{-1}\mathbf{U}$ and $\mathbf{Z} = \mathbf{A}^{-T}\mathbf{V}$. Only the diagonal of the final product is needed. The full inverse should not be formed. When no diagonal is known in advance, Section 6.7 estimates the whole diagonal of $\mathbf{A}^{-1}$ from a few random probes and one factorization.

Prop 2.25 **(Changes in a Few Rows and Columns)** Let $\mathbf{A}$ and $\mathbf{B}$ be square matrices that differ only in rows and columns indexed by a set of size k. Then $\mathbf{B} - \mathbf{A}$ has rank at most $2k$.

Proof Let $\mathbf{E} \in \mathbb{R}^{n\times k}$ contain the columns of the identity matrix that select the changed positions, and let $\mathbf{D} = \mathbf{B} - \mathbf{A}$. Entries of $\mathbf{D}$ outside the selected rows and columns are zero. Therefore

$$\mathbf{D} = \mathbf{E}(\mathbf{E}^T\mathbf{D}) + (\mathbf{I} - \mathbf{EE}^T)\mathbf{DEE}^T.$$

The first term has rank at most k, and the second term has rank at most k. Their sum has rank at most $2k$. □

This rank bound is useful when a model changes by adding, removing, or modifying a few states. The changed rows are stored once, and the changed columns are stored after their overlap with those rows has been removed. Woodbury then corrects the retained factorization with rank at most $2k$.

Remark 2.10 **(When to refactor)** The small matrix $\mathbf{G}$ can be sensitive to roundoff. A reliable implementation estimates that sensitivity, checks the residual of the corrected solution, and

computes a fresh factorization when either check fails. The implementation in `numerics` that motivated this section limits reuse to three changed positions. It also estimates the effect of roundoff in each entry of $\mathbf{G}$ and refactors when the estimated relative error exceeds 10^{-8}. The limit and threshold are choices made in that code, not mathematical rules.

2.5.3 Tridiagonal Systems

Low rank describes how a matrix changes. Banded structure describes which unknowns interact. A tridiagonal matrix couples each unknown only to its two neighbors. Gaussian elimination preserves this pattern when no row interchange is required, so the solve takes linear rather than cubic work.

Def 2.13 **(Thomas Algorithm)** Consider the equations

$$a_i x_{i-1} + d_i x_i + c_i x_{i+1} = r_i,$$

with the missing endpoint terms omitted. Let $\widetilde{d}_1 = d_1$ and $\widetilde{r}_1 = r_1$. For $i = 2, \ldots, n$, compute

$$\begin{aligned} m_i &= \frac{a_i}{\widetilde{d}_{i-1}}, \\ \widetilde{d}_i &= d_i - m_i c_{i-1}, \\ \widetilde{r}_i &= r_i - m_i \widetilde{r}_{i-1}. \end{aligned}$$

Backward substitution gives

$$x_n = \frac{\widetilde{r}_n}{\widetilde{d}_n}, \qquad x_i = \frac{\widetilde{r}_i - c_i x_{i+1}}{\widetilde{d}_i}, \quad i = n-1, \ldots, 1.$$

Prop 2.26 **(Cost of Tridiagonal Elimination)** If every modified pivot $\widetilde{d}_i$ is nonzero, the Thomas algorithm solves an $n \times n$ tridiagonal system in $O(n)$ operations and $O(n)$ storage. The diagonal and right-hand side may be overwritten, so the additional storage can be constant when the input need not be retained.

Proof Each forward step uses a fixed number of arithmetic operations, and each backward step does the same. There are $n - 1$ steps in each direction. The three matrix diagonals and the right-hand side contain $O(n)$ entries. Overwriting d_i and r_i by their modified values removes the need for separate work arrays. □

Remark 2.11 **(Pivoting)** The Thomas algorithm is unpivoted elimination. It is safe for important classes such as symmetric positive definite tridiagonal matrices, but a general tridiagonal matrix can have a zero or dangerously small pivot. In that case, use a tridiagonal solver with pivoting or a general banded factorization, even though pivoting may enlarge the band.

2.5.4 Block-Tridiagonal Systems

Many models place several unknowns at each location or time level. Grouping those unknowns produces a block-tridiagonal matrix. The scalar divisions in the Thomas algorithm become solves with dense diagonal blocks, and the scalar updates become matrix products.

Def 2.14 **(Block-Tridiagonal Matrix)** A block-tridiagonal matrix has the form

$$\mathbf{A} = \begin{pmatrix} \mathbf{D}_1 & \mathbf{U}_1 & & \\ \mathbf{L}_1 & \mathbf{D}_2 & \mathbf{U}_2 & \\ & \ddots & \ddots & \ddots \\ & & \mathbf{L}_{s-1} & \mathbf{D}_s \end{pmatrix}.$$

The diagonal blocks $\mathbf{D}_k$ are square. Neighboring blocks may have different sizes, so $\mathbf{U}_k$ and $\mathbf{L}_k$ may be rectangular.

Thm 2.27 **(Block-Tridiagonal LU Factorization)** Let $\mathbf{S}_1 = \mathbf{D}_1$. For $k = 1, \dots, s-1$, suppose $\mathbf{S}_k$ is nonsingular and compute

$$\mathbf{M}_k \mathbf{S}_k = \mathbf{L}_k, \qquad \mathbf{S}_{k+1} = \mathbf{D}_{k+1} - \mathbf{M}_k \mathbf{U}_k.$$

If the final block $\mathbf{S}_s$ is also nonsingular, then $\mathbf{A} = \mathcal{L}\mathcal{U}$, where $\mathcal{L}$ is block unit lower bidiagonal with blocks $\mathbf{M}_k$, and $\mathcal{U}$ is block upper bidiagonal with diagonal blocks $\mathbf{S}_k$ and upper blocks $\mathbf{U}_k$.

Proof The upper block in position $(k, k+1)$ of $\mathcal{L}\mathcal{U}$ is $\mathbf{U}_k$, and the lower block in position $(k+1, k)$ is $\mathbf{M}_k \mathbf{S}_k = \mathbf{L}_k$. The first diagonal block is $\mathbf{S}_1 = \mathbf{D}_1$. For $k > 1$, the diagonal block of the product is

$$\mathbf{S}_k + \mathbf{M}_{k-1} \mathbf{U}_{k-1} = \mathbf{D}_k$$

by the definition of $\mathbf{S}_k$. All blocks farther from the diagonal vanish because both factors are block bidiagonal. Thus their product is $\mathbf{A}$. $\square$

The equation $\mathbf{M}_k \mathbf{S}_k = \mathbf{L}_k$ should not be evaluated with an inverse. Instead, solve the transposed system

$$\mathbf{S}_k^T \mathbf{M}_k^T = \mathbf{L}_k^T$$

using the stored factorization of $\mathbf{S}_k$. This is how the `numerics` implementation computes $\mathbf{M}_k$.

Def 2.15 **(Block Forward and Backward Sweeps)** Partition the right-hand side into blocks $\boldsymbol{b}_k$. First compute

$$\begin{aligned} \boldsymbol{y}_1 &= \boldsymbol{b}_1, \\ \boldsymbol{y}_k &= \boldsymbol{b}_k - \mathbf{M}_{k-1} \boldsymbol{y}_{k-1}, \qquad k = 2, \dots, s. \end{aligned}$$

Then solve backward:

$$\begin{aligned}\mathbf{S}_s\boldsymbol{x}_s &= \boldsymbol{y}_s,\\ \mathbf{S}_k\boldsymbol{x}_k &= \boldsymbol{y}_k - \mathbf{U}_k\boldsymbol{x}_{k+1}, \qquad k = s-1,\dots,1.\end{aligned}$$

The same recurrences accept several right-hand sides at once.

Remark 2.12 **(Work and storage)** If there are s blocks of comparable size p, the factorization costs $O(sp^3)$ operations and stores $O(sp^2)$ numbers. A solve with q right-hand sides costs $O(sp^2q)$. Since the full order is $n = sp$, the factorization cost is $O(n^3/s^2)$. Unequal neighboring block sizes add rectangular products, so the largest block often controls the cost.

Remark 2.13 **(Transpose and multiple right-hand sides)** The factors also solve $\mathbf{A}^T\boldsymbol{x} = \boldsymbol{b}$. The transpose reverses the order of the two sweeps and interchanges the stored upper and lower blocks. Keep several right-hand sides together throughout the sweeps. This reuses memory and is usually faster than processing the columns separately.

2.5.5 The Symmetric Positive Definite Case

When the block-tridiagonal matrix is symmetric positive definite, block Cholesky removes the need for pivoting and stores only one off-diagonal side. The key fact is that every diagonal Schur complement remains positive definite.

Thm 2.28 **(Positive Definite Schur Complement)** Let

$$\mathbf{A} = \begin{pmatrix}\mathbf{D} & \mathbf{B}^T\\ \mathbf{B} & \mathbf{C}\end{pmatrix}$$

be symmetric positive definite. Then $\mathbf{D}$ is positive definite, and the Schur complement

$$\mathbf{C} - \mathbf{B}\mathbf{D}^{-1}\mathbf{B}^T$$

is positive definite.

Proof For every nonzero $\boldsymbol{x}$, testing with $(\boldsymbol{x}, \mathbf{0})^T$ gives

$$\begin{pmatrix}\boldsymbol{x}\\ \mathbf{0}\end{pmatrix}^T \mathbf{A} \begin{pmatrix}\boldsymbol{x}\\ \mathbf{0}\end{pmatrix} = \boldsymbol{x}^T\mathbf{D}\boldsymbol{x} > 0,$$

so $\mathbf{D}$ is positive definite and $\mathbf{D}^{-1}$ exists. Now let $\boldsymbol{y} \neq \mathbf{0}$, and choose $\boldsymbol{x} = -\mathbf{D}^{-1}\mathbf{B}^T\boldsymbol{y}$. Expanding the block quadratic form:

$$\begin{aligned}\begin{pmatrix}\boldsymbol{x}\\ \boldsymbol{y}\end{pmatrix}^T \begin{pmatrix}\mathbf{D} & \mathbf{B}^T\\ \mathbf{B} & \mathbf{C}\end{pmatrix} \begin{pmatrix}\boldsymbol{x}\\ \boldsymbol{y}\end{pmatrix} &= \boldsymbol{x}^T\mathbf{D}\boldsymbol{x} + \boldsymbol{x}^T\mathbf{B}^T\boldsymbol{y} + \boldsymbol{y}^T\mathbf{B}\boldsymbol{x} + \boldsymbol{y}^T\mathbf{C}\boldsymbol{y}\\ &= (-\mathbf{D}^{-1}\mathbf{B}^T\boldsymbol{y})^T\mathbf{D}(-\mathbf{D}^{-1}\mathbf{B}^T\boldsymbol{y}) + 2(-\mathbf{D}^{-1}\mathbf{B}^T\boldsymbol{y})^T\mathbf{B}^T\boldsymbol{y} + \boldsymbol{y}^T\mathbf{C}\boldsymbol{y}\\ &= \boldsymbol{y}^T\mathbf{B}\mathbf{D}^{-1}\mathbf{B}^T\boldsymbol{y} - 2\boldsymbol{y}^T\mathbf{B}\mathbf{D}^{-1}\mathbf{B}^T\boldsymbol{y} + \boldsymbol{y}^T\mathbf{C}\boldsymbol{y}\\ &= \boldsymbol{y}^T(\mathbf{C} - \mathbf{B}\mathbf{D}^{-1}\mathbf{B}^T)\boldsymbol{y}.\end{aligned}$$

Because $\boldsymbol{y} \neq \mathbf{0}$, the compound vector $(\boldsymbol{x}, \boldsymbol{y})^T$ is nonzero. The positive definiteness of $\mathbf{A}$ implies that this quadratic form is strictly positive:

$$\boldsymbol{y}^T(\mathbf{C} - \mathbf{B}\mathbf{D}^{-1}\mathbf{B}^T)\boldsymbol{y} > 0.$$

The Schur complement $\mathbf{C} - \mathbf{B}\mathbf{D}^{-1}\mathbf{B}^T$ is therefore positive definite. □

Block Cholesky follows the same order as block LU. Factor $\mathbf{S}_k = \mathbf{F}_k\mathbf{F}_k^T$, solve

$$\mathbf{C}_k\mathbf{F}_k^T = \mathbf{L}_k,$$

and update

$$\mathbf{S}_{k+1} = \mathbf{D}_{k+1} - \mathbf{C}_k\mathbf{C}_k^T.$$

The theorem above shows that every $\mathbf{S}_k$ remains positive definite. The stored factors form a block lower-bidiagonal Cholesky factor of $\mathbf{A}$.

2.5.6 Finding the Blocks and Reusing a Prefix

A block order need not be visible in the original row numbering. Make an undirected graph with one vertex for each row and one edge for each nonzero connection. Choose one or more starting vertices and label every vertex by its distance from them. The labels at the ends of an edge differ by at most one. Grouping equal labels therefore produces a block-tridiagonal matrix.

Remark 2.14 **(Block width)** The sizes of the levels matter as much as their number. A poor starting set can place many rows in one level and create an expensive dense block. A useful ordering keeps the largest level small and avoids abrupt changes between neighboring level sizes. Before factorization, check that every nonzero lies in a diagonal block or one of the two neighboring block bands.

Local changes permit a second form of reuse. Suppose block j is the first one that changes. The factors before it remain valid, but the connection between blocks $j-1$ and j touches the changed part. Keep the earlier factors, recompute this connection, and factor blocks j through the end. Keeping the old connection would give the wrong Schur complement.

Remark 2.15 **(Choosing the reuse method)** Use the stored factor directly when only the right-hand side changes. Use Sherman–Morrison or Woodbury when a few rows and columns change throughout the matrix. Recompute from the first changed block when the change begins late in the block order. Compute a fresh factorization when the change can no longer be described by a few vectors, the small Woodbury matrix becomes too sensitive to roundoff, or the block order no longer fits the matrix.

Remark 2.16 **(Implementation details)** A reusable solver should provide ordinary, transpose, and multiple-right-hand-side solves through the same interface. It should retain work arrays

instead of allocating them inside every solve. It should compare the corrected solution with the original equations by a residual. Finally, it should report whether it reused a factorization or computed a new one. These details determine whether the algebraic shortcut reduces actual running time.

2.5.7 Exercises

Ex 2.5

1. **Check. Sherman–Morrison by hand.** Repeat the worked example with $\boldsymbol{b} = (0, 1)^T$. Compute the corrected solution from Def 2.11, and verify it by multiplying with $\mathbf{A} + \boldsymbol{u}\boldsymbol{v}^T$.

2. **Derive. Matrix determinant lemma.** Under the hypotheses of Thm 2.24, prove
$$\det(\mathbf{A} + \mathbf{U}\mathbf{V}^T) = \det(\mathbf{A}) \det(\mathbf{I}_r + \mathbf{V}^T\mathbf{A}^{-1}\mathbf{U}).$$
Begin with $\mathbf{A} + \mathbf{U}\mathbf{V}^T = \mathbf{A}(\mathbf{I} + \mathbf{A}^{-1}\mathbf{U}\mathbf{V}^T)$ and prove the needed identity $\det(\mathbf{I} + \mathbf{X}\mathbf{Y}) = \det(\mathbf{I} + \mathbf{Y}\mathbf{X})$ using a block determinant in two different orders.

3. **Derive. Changed rows and columns.** Work through Prop 2.25 for two changed indices. Construct the two factors with at most four columns, show explicitly how the overlap of the changed rows and columns is counted once, and identify a case in which the actual rank is smaller than four.

4. **Compute. Low-rank solve versus refactorization.** Let $\mathbf{A}$ be a diagonally dominant dense matrix and change k rows and columns. Compare a fresh LU factorization with a Woodbury correction for several values of n and k. Report the initial cost, the time per right-hand side, the relative residual, and the difference from a fresh solve. Find how many right-hand sides are needed before reuse saves time overall.

5. **Explore. A nearly singular correction.** Let $\mathbf{A} = \mathbf{I}$, $\boldsymbol{u} = -(1-\delta)\boldsymbol{e}_1$, and $\boldsymbol{v} = \boldsymbol{e}_1$. Compute the Sherman–Morrison denominator and the updated inverse. Explain why the correction amplifies roundoff as $\delta \to 0$, even though solving with the base matrix is perfectly conditioned.

6. **Derive. Thomas as scalar block elimination.** Take every block in Thm 2.27 to have size one. Derive the three forward recurrences in Def 2.13 and the backward substitution formula. Identify the modified scalar pivot with the scalar Schur complement.

7. **Compute. A two-block Cholesky solve.** Consider
$$\mathbf{D}_1 = \begin{pmatrix} 4 & 1 \\ 1 & 3 \end{pmatrix}, \qquad \mathbf{L}_1 = \begin{pmatrix} -1 & 0 \\ 0 & -1 \end{pmatrix}, \qquad \mathbf{D}_2 = \begin{pmatrix} 3 & 1 \\ 1 & 2 \end{pmatrix},$$
with upper block $\mathbf{L}_1^T$. Compute the first Cholesky block, the scaled lower block, and the second Schur complement. Then solve the system with right-hand side $(1, 0, 0, 1)^T$ and check the residual.

8. **Derive. Transpose block solve.** Starting from $\mathbf{A} = \mathcal{LU}$ in Thm 2.27, derive the two sweeps for $\mathbf{A}^T\boldsymbol{x} = \boldsymbol{b}$. State the direction of each sweep and identify which stored off-diagonal block it uses.

9. **Compute. Graph-distance blocks.** For the five-point grid graph on a 3×3 lattice, label every vertex by its distance from a corner. List the block sizes and verify that every edge joins equal or adjacent levels. Repeat with the center vertex as the seed and compare the largest block.

10. **Explore. Woodbury or partial refactorization.** Construct a sequence of block-tridiagonal matrices in which the final two blocks change at every step. Implement both a low-rank correction and a refactorization that begins at the first changed block. Compare their cost and residual as the block size and update rank vary. Use the operation counts in this chapter to explain when one method becomes faster than the other.

Read **Further reading.** Golub and Van Loan give the standard dense matrix treatment, while Davis develops the sparse direct viewpoint (Golub and Van Loan, 2013; Davis, 2006). Demmel and Higham are useful guides to deciding when an algebraically correct update may still be unsafe in floating-point arithmetic (Demmel, 1997; Higham, 2002).

3 Orthogonality and Least Squares

3.1 Orthogonal Transformations and the QR Factorization

Orthogonal transformations are central to stable numerical linear algebra because they preserve Euclidean lengths and angles. They do not amplify an existing perturbation merely by changing coordinates. Floating-point operations still introduce rounding error, but orthogonal transformations do not add avoidable geometric distortion on top of it.

The QR factorization expresses a matrix as the product of an orthogonal matrix $\mathbf{Q}$ and an upper triangular matrix $\mathbf{R}$. This factorization provides an orthonormal basis for the column space of the matrix, transforming geometric projections and overdetermined least squares problems into triangular solves.

3.1.1 Orthogonality, Isometry, and Invariance

Def 3.1 **(Orthogonal Matrix)** A square matrix $\mathbf{Q} \in \mathbb{R}^{n\times n}$ is termed **orthogonal** if its columns form an orthonormal basis for $\mathbb{R}^n$. Equivalently,

$$\mathbf{Q}^T\mathbf{Q} = \mathbf{Q}\mathbf{Q}^T = \mathbf{I}_n \iff \mathbf{Q}^{-1} = \mathbf{Q}^T.$$

For rectangular matrices $\mathbf{Q} \in \mathbb{R}^{m\times n}$ with $m \geq n$, we say that $\mathbf{Q}$ has orthonormal columns if $\mathbf{Q}^T\mathbf{Q} = \mathbf{I}_n$.

Thm 3.1 **(Geometric Invariance Properties of Orthogonal Transformations)** Let $\mathbf{Q} \in \mathbb{R}^{n\times n}$ be an orthogonal matrix. For all vectors $\boldsymbol{x}, \boldsymbol{y} \in \mathbb{R}^n$, the transformation $\boldsymbol{x} \mapsto \mathbf{Q}\boldsymbol{x}$ satisfies the following properties:

1. **Inner Product Preservation:** $\langle \mathbf{Q}\boldsymbol{x}, \mathbf{Q}\boldsymbol{y}\rangle = (\mathbf{Q}\boldsymbol{x})^T(\mathbf{Q}\boldsymbol{y}) = \boldsymbol{x}^T\boldsymbol{y} = \langle \boldsymbol{x}, \boldsymbol{y}\rangle$.
2. **Isometry (Length Preservation):** $\|\mathbf{Q}\boldsymbol{x}\|_2 = \|\boldsymbol{x}\|_2$.
3. **Matrix 2-Norm and Conditioning:** $\|\mathbf{Q}\|_2 = 1$ and $\kappa_2(\mathbf{Q}) = \|\mathbf{Q}\|_2\|\mathbf{Q}^{-1}\|_2 = 1$.
4. **Determinant:** $\det(\mathbf{Q}) = \pm 1$, where $+1$ corresponds to a pure rotation and -1 corresponds to a reflection.

Proof Using the transpose property of matrix multiplication, $(\mathbf{Q}\boldsymbol{x})^T(\mathbf{Q}\boldsymbol{y}) = \boldsymbol{x}^T(\mathbf{Q}^T\mathbf{Q})\boldsymbol{y} = \boldsymbol{x}^T\mathbf{I}\boldsymbol{y} = \boldsymbol{x}^T\boldsymbol{y}$, which proves the preservation of inner products. Setting $\boldsymbol{x} = \boldsymbol{y}$ gives $\|\mathbf{Q}\boldsymbol{x}\|_2^2 = \boldsymbol{x}^T\boldsymbol{x} = \|\boldsymbol{x}\|_2^2$, proving length preservation. Taking the supremum over unit vectors yields $\|\mathbf{Q}\|_2 = \sup_{\|\boldsymbol{x}\|_2=1} \|\mathbf{Q}\boldsymbol{x}\|_2 = 1$. Because $\mathbf{Q}^{-1} = \mathbf{Q}^T$ is also orthogonal, $\|\mathbf{Q}^{-1}\|_2 = 1$, which establishes $\kappa_2(\mathbf{Q}) = 1 \cdot 1 = 1$. Finally, taking determinants on $\mathbf{Q}^T\mathbf{Q} = \mathbf{I}$ gives $(\det \mathbf{Q})^2 = 1 \implies \det \mathbf{Q} = \pm 1$. □

3.1.2 The QR Factorization

Def 3.2 **(Full and Thin QR Factorizations)** Let $\mathbf{A} \in \mathbb{R}^{m\times n}$ with $m \geq n$.

1. The **Full QR Factorization** is given by

$$\mathbf{A} = \mathbf{Q}\mathbf{R} = \begin{pmatrix}\mathbf{Q}_1 & \mathbf{Q}_2\end{pmatrix}\begin{pmatrix}\mathbf{R}_1 \\ \mathbf{0}\end{pmatrix},$$

   where $\mathbf{Q} \in \mathbb{R}^{m\times m}$ is orthogonal, $\mathbf{Q}_1 \in \mathbb{R}^{m\times n}$, $\mathbf{Q}_2 \in \mathbb{R}^{m\times(m-n)}$, and $\mathbf{R}_1 \in \mathbb{R}^{n\times n}$ is upper triangular.

2. The **Thin (Economy) QR Factorization** retains only the first n columns:

$$\mathbf{A} = \mathbf{Q}_1\mathbf{R}_1,$$

   where $\mathbf{Q}_1 \in \mathbb{R}^{m\times n}$ satisfies $\mathbf{Q}_1^T\mathbf{Q}_1 = \mathbf{I}_n$ and $\mathbf{R}_1 \in \mathbb{R}^{n\times n}$ is upper triangular.

If $\mathbf{A}$ has full column rank, the thin QR factorization is unique when the diagonal entries of $\mathbf{R}_1$ are chosen to be strictly positive.

Remark 3.1 **(Subspace identification via QR)** The columns of $\mathbf{Q}_1$ form an orthonormal basis for the column space (range) $\mathcal{R}(\mathbf{A})$, while the columns of $\mathbf{Q}_2$ form an orthonormal basis for the orthogonal complement $\mathcal{R}(\mathbf{A})^\perp = \mathcal{N}(\mathbf{A}^T)$. For overdetermined least-squares problems where $m \gg n$, the thin factorization requires less storage ($O(mn)$ vs $O(m^2)$) and is used in practical implementations.

3.1.3 Householder Triangularization

The standard method for computing the QR factorization of a dense matrix is the Householder reflection technique. At each step, a Householder reflector is constructed to zero out all subdiagonal entries of a column simultaneously (Householder, 1958).

Def 3.3 **(Householder Reflector)** Let $\boldsymbol{v} \in \mathbb{R}^m$ be a nonzero vector. The Householder reflector associated with $\boldsymbol{v}$ is the matrix

$$\mathbf{H} = \mathbf{I} - 2\frac{\boldsymbol{v}\boldsymbol{v}^T}{\boldsymbol{v}^T\boldsymbol{v}}.$$

Geometrically, $\mathbf{H}$ reflects any vector across the hyperplane orthogonal to $\boldsymbol{v}$. Every Householder reflector is symmetric ($\mathbf{H}^T = \mathbf{H}$) and orthogonal ($\mathbf{H}^T\mathbf{H} = \mathbf{H}^2 = \mathbf{I}$).

Thm 3.2 **(Householder Vector Construction)** Given a vector $\boldsymbol{x} \in \mathbb{R}^m$, we wish to find a reflector $\mathbf{H}$ that aligns $\boldsymbol{x}$ with the first canonical coordinate axis: $\mathbf{H}\boldsymbol{x} = \pm\|\boldsymbol{x}\|_2\boldsymbol{e}_1$. The vector $\boldsymbol{v}$ is chosen as

$$\boldsymbol{v} = \boldsymbol{x} + \operatorname{sign}(x_1)\|\boldsymbol{x}\|_2\boldsymbol{e}_1,$$

where $\operatorname{sign}(x_1) = 1$ if $x_1 \geq 0$ and -1 if $x_1 < 0$. Choosing the sign of the first component ensures that the two added terms have identical signs and avoids cancellation when the first component of $\boldsymbol{v}$ is formed.

Proof Let $\boldsymbol{v} = \boldsymbol{x} - \alpha\boldsymbol{e}_1$. Because $\mathbf{H}$ preserves vector lengths, we must have $\|\mathbf{H}\boldsymbol{x}\|_2 = \|\boldsymbol{x}\|_2$, which requires $\alpha = \pm\|\boldsymbol{x}\|_2$. Computing $\boldsymbol{v}^T\boldsymbol{v}$ gives

$$\boldsymbol{v}^T\boldsymbol{v} = (\boldsymbol{x} - \alpha\boldsymbol{e}_1)^T(\boldsymbol{x} - \alpha\boldsymbol{e}_1) = \|\boldsymbol{x}\|_2^2 - 2\alpha x_1 + \alpha^2 = 2\|\boldsymbol{x}\|_2^2 - 2\alpha x_1 = 2\boldsymbol{v}^T\boldsymbol{x}.$$

Applying $\mathbf{H}$ to $\boldsymbol{x}$ yields

$$\mathbf{H}\boldsymbol{x} = \boldsymbol{x} - 2\frac{\boldsymbol{v}^T\boldsymbol{x}}{\boldsymbol{v}^T\boldsymbol{v}}\boldsymbol{v} = \boldsymbol{x} - \boldsymbol{v} = \alpha\boldsymbol{e}_1 = \pm\|\boldsymbol{x}\|_2\boldsymbol{e}_1.$$

To ensure numerical stability in floating-point arithmetic, we choose $\alpha = -\operatorname{sign}(x_1)\|\boldsymbol{x}\|_2$, so that $v_1 = x_1 + \operatorname{sign}(x_1)\|\boldsymbol{x}\|_2$ avoids subtracting nearly equal quantities. □

Remark 3.2 **(Matrix-free application and complexity)** In numerical algorithms, a Householder matrix $\mathbf{H}$ is usually stored through its defining vector rather than as an explicit $m \times m$ array. To apply $\mathbf{H}$ to a vector $\boldsymbol{y}$, we compute

$$\mathbf{H}\boldsymbol{y} = \boldsymbol{y} - \left(\frac{2\boldsymbol{v}^T\boldsymbol{y}}{\boldsymbol{v}^T\boldsymbol{v}}\right)\boldsymbol{v},$$

which requires only an inner product and a vector update costing $O(m)$ flops. For a matrix $\mathbf{A} \in \mathbb{R}^{m\times n}$, Householder QR requires $2mn^2 - \frac{2}{3}n^3$ flops.

3.1.4 Gram–Schmidt Orthogonalization

The Gram–Schmidt process constructs the thin QR factorization column by column by projecting each column of $\mathbf{A}$ onto the orthogonal complement of the previously computed basis vectors.

Thm 3.3 **(Classical and Modified Gram–Schmidt Algorithms)** Let $\boldsymbol{a}_1, \dots, \boldsymbol{a}_n \in \mathbb{R}^m$ be the columns of $\mathbf{A}$.

1. **Classical Gram–Schmidt (CGS):** For each column $k = 1, \dots, n$, compute all projections against the original vector simultaneously:

$$r_{ik} = \boldsymbol{q}_i^T \boldsymbol{a}_k \quad (i = 1, \dots, k-1), \qquad \boldsymbol{v}_k = \boldsymbol{a}_k - \sum_{i=1}^{k-1} r_{ik} \boldsymbol{q}_i, \qquad r_{kk} = \|\boldsymbol{v}_k\|_2, \qquad \boldsymbol{q}_k = \frac{\boldsymbol{v}_k}{r_{kk}}.$$

2. **Modified Gram–Schmidt (MGS):** For each column k, subtract projections sequentially from the partially updated vector:

$$\boldsymbol{v}_k^{(1)} = \boldsymbol{a}_k, \qquad \boldsymbol{v}_k^{(i+1)} = \boldsymbol{v}_k^{(i)} - (\boldsymbol{q}_i^T \boldsymbol{v}_k^{(i)}) \boldsymbol{q}_i \quad (i = 1, \dots, k-1), \qquad \boldsymbol{q}_k = \frac{\boldsymbol{v}_k^{(k)}}{\|\boldsymbol{v}_k^{(k)}\|_2}.$$

Remark 3.3 **(Stability hierarchy among orthogonalization methods)** In exact arithmetic, CGS and MGS produce the same orthonormal basis, up to signs. In floating-point arithmetic their organization matters:

1. Classical Gram–Schmidt can lose orthogonality when the columns are nearly dependent.
2. Modified Gram–Schmidt retains more orthogonality because each projection is taken from the current residual rather than from the original column.
3. Householder QR is backward stable and normally produces a computed $\mathbf{Q}$ that is orthogonal to working precision.

Precise bounds depend on dimension, scaling, and the condition of the column space. Reorthogonalization can make Gram–Schmidt reliable when an explicit basis is required.

3.1.5 Orthogonal Projectors

Def 3.4 **(Orthogonal Projector)** Let $S \subset \mathbb{R}^m$ be a subspace with orthonormal basis columns stored in $\mathbf{Q}_1 \in \mathbb{R}^{m \times n}$. The orthogonal projector onto S is the matrix

$$\mathbf{P} = \mathbf{Q}_1 \mathbf{Q}_1^T.$$

Every orthogonal projector is symmetric ($\mathbf{P}^T = \mathbf{P}$) and idempotent ($\mathbf{P}^2 = \mathbf{P}$). The complementary projector $\mathbf{P}^\perp = \mathbf{I} - \mathbf{P}$ projects onto the orthogonal complement $S^\perp$.

Thm 3.4 **(Symmetric Idempotents are Orthogonal Projectors)** A square matrix $\mathbf{P}$ is the orthogonal projector onto its range if and only if

$$\mathbf{P}^T = \mathbf{P}, \qquad \mathbf{P}^2 = \mathbf{P}.$$

For such a matrix, every vector has the orthogonal decomposition

$$\boldsymbol{x} = \mathbf{P}\boldsymbol{x} + (\mathbf{I} - \mathbf{P})\boldsymbol{x},$$

with the first term in $\mathcal{R}(\mathbf{P})$ and the second in $\mathcal{N}(\mathbf{P})$.

Proof Suppose first that $\mathbf{P}^T = \mathbf{P}$ and $\mathbf{P}^2 = \mathbf{P}$. The first term lies in the range, and

$$\mathbf{P}(\mathbf{I} - \mathbf{P})\boldsymbol{x} = (\mathbf{P} - \mathbf{P}^2)\boldsymbol{x} = \mathbf{0},$$

so the second lies in the null space. If $\boldsymbol{y} = \mathbf{P}\boldsymbol{u}$ belongs to the range and $\boldsymbol{z}$ belongs to the null space, then

$$\boldsymbol{y}^T\boldsymbol{z} = \boldsymbol{u}^T\mathbf{P}^T\boldsymbol{z} = \boldsymbol{u}^T\mathbf{P}\boldsymbol{z} = 0.$$

Thus the two components are orthogonal. Conversely, an orthogonal projector has the form $\mathbf{Q}_1\mathbf{Q}_1^T$ for an orthonormal basis matrix $\mathbf{Q}_1$, so it is symmetric and idempotent. Its closest-point property was proved in Thm. 1.12. □

Ex 3.1

1. **Compute.** Construct the Householder reflector $\mathbf{H}$ that transforms the vector $\boldsymbol{x} = (3, 4)^T$ into $\alpha\boldsymbol{e}_1 = (-5, 0)^T$. Explicitly write out the 2×2 matrix $\mathbf{H}$ and verify that $\mathbf{H} = \mathbf{H}^T$ and $\mathbf{H}^2 = \mathbf{I}$.

2. **Compute.** Compute the thin QR factorization of $\mathbf{A} = \begin{pmatrix} 1 & 1 \\ 1 & 2 \\ 1 & 3 \end{pmatrix}$ using Modified Gram-Schmidt.

3. **Derive.** Let $\mathbf{Q}$ and $\widehat{\mathbf{Q}}$ have orthonormal columns spanning the same subspace. Prove that $\mathbf{Q}\mathbf{Q}^T = \widehat{\mathbf{Q}}\widehat{\mathbf{Q}}^T$. Thus projection depends on the subspace, not on the chosen orthonormal basis.

4. **Derive.** An idempotent matrix need not be an orthogonal projector. For $\mathbf{P} = \begin{pmatrix} 1 & 1 \\ 0 & 0 \end{pmatrix}$, identify the range and null space, sketch its action, and explain the role of symmetry in Thm. 3.4.

5. **Derive.** Explain why the condition number of any orthogonal matrix is equal to 1. What does this imply about the propagation of errors when applying orthogonal transformations?

3.2 Least Squares as Projection

Suppose the columns of $\mathbf{A}$ describe the responses available to a model. Every prediction has the form $\mathbf{A}\boldsymbol{x}$ and therefore lies in the column space of $\mathbf{A}$. If an observation $\boldsymbol{b}$ lies outside that space, no choice of parameters can fit it exactly. Least squares asks for the point in the column space closest to the observation.

The choice of norm therefore matters. Euclidean least squares regards squared orthogonal distance as the loss. A different norm expresses a different meaning of error and generally leads to a different estimator.

3.2.1 The Geometric Problem

Def 3.5 **(Linear Least-Squares Problem)** For $\mathbf{A} \in \mathbb{R}^{m\times n}$ and $\boldsymbol{b} \in \mathbb{R}^m$, the linear least-squares problem is

$$\min_{\boldsymbol{x}\in\mathbb{R}^n} \|\mathbf{A}\boldsymbol{x} - \boldsymbol{b}\|_2^2 = \min_{\boldsymbol{x}\in\mathbb{R}^n} \sum_{i=1}^{m} \left(\sum_{j=1}^{n} a_{ij}x_j - b_i \right)^2 .$$

The residual associated with $\boldsymbol{x}$ is $\boldsymbol{r}(\boldsymbol{x}) = \boldsymbol{b} - \mathbf{A}\boldsymbol{x}$.

Thm 3.5 **(Orthogonal Characterization and Normal Equations)** A vector $\widehat{\boldsymbol{x}}$ is a least-squares minimizer if and only if its residual is orthogonal to every column of $\mathbf{A}$:

$$\boldsymbol{r} = \boldsymbol{b} - \mathbf{A}\widehat{\boldsymbol{x}} \perp \mathcal{R}(\mathbf{A}).$$

Equivalently,

$$\mathbf{A}^T\boldsymbol{r} = \mathbf{0}, \qquad \mathbf{A}^T\mathbf{A}\widehat{\boldsymbol{x}} = \mathbf{A}^T\boldsymbol{b}.$$

These are the **normal equations**. A minimizer is unique exactly when the columns of $\mathbf{A}$ are linearly independent.

Proof Decompose $\boldsymbol{b} - \mathbf{A}\boldsymbol{x}$ by inserting the candidate fitted vector $\mathbf{A}\widehat{\boldsymbol{x}}$:

$$\boldsymbol{b} - \mathbf{A}\boldsymbol{x} = (\boldsymbol{b} - \mathbf{A}\widehat{\boldsymbol{x}}) + \mathbf{A}(\widehat{\boldsymbol{x}} - \boldsymbol{x}).$$

If $\boldsymbol{b} - \mathbf{A}\widehat{\boldsymbol{x}} \perp \mathcal{R}(\mathbf{A})$, then since $\mathbf{A}(\widehat{\boldsymbol{x}} - \boldsymbol{x}) \in \mathcal{R}(\mathbf{A})$, the two summands are orthogonal:

$$(\boldsymbol{b} - \mathbf{A}\widehat{\boldsymbol{x}})^T\mathbf{A}(\widehat{\boldsymbol{x}} - \boldsymbol{x}) = 0.$$

The Pythagorean identity therefore yields

$$\begin{aligned} \|\boldsymbol{b} - \mathbf{A}\boldsymbol{x}\|_2^2 &= \|(\boldsymbol{b} - \mathbf{A}\widehat{\boldsymbol{x}}) + \mathbf{A}(\widehat{\boldsymbol{x}} - \boldsymbol{x})\|_2^2 \\ &= \|\boldsymbol{b} - \mathbf{A}\widehat{\boldsymbol{x}}\|_2^2 + \|\mathbf{A}(\widehat{\boldsymbol{x}} - \boldsymbol{x})\|_2^2 \geq \|\boldsymbol{b} - \mathbf{A}\widehat{\boldsymbol{x}}\|_2^2. \end{aligned}$$

Equality holds if and only if $\|\mathbf{A}(\widehat{\boldsymbol{x}}-\boldsymbol{x})\|_2^2 = 0$, meaning $\mathbf{A}(\widehat{\boldsymbol{x}}-\boldsymbol{x}) = \mathbf{0}$. Thus $\widehat{\boldsymbol{x}}$ minimizes the residual norm.
Conversely, if $\widehat{\boldsymbol{x}}$ is a minimizer, define for any $\boldsymbol{v} \in \mathbb{R}^n$ and $t \in \mathbb{R}$ the scalar function

$$\begin{aligned}\phi(t) &= \|\boldsymbol{b} - \mathbf{A}(\widehat{\boldsymbol{x}} + t\boldsymbol{v})\|_2^2 \\ &= \|\boldsymbol{r} - t\mathbf{A}\boldsymbol{v}\|_2^2 \\ &= \|\boldsymbol{r}\|_2^2 - 2t\,\boldsymbol{v}^T\mathbf{A}^T\boldsymbol{r} + t^2\|\mathbf{A}\boldsymbol{v}\|_2^2,\end{aligned}$$

where $\boldsymbol{r} = \boldsymbol{b} - \mathbf{A}\widehat{\boldsymbol{x}}$. Because ϕ has a global minimum at $t = 0$,

$$\phi'(0) = -2\boldsymbol{v}^T\mathbf{A}^T\boldsymbol{r} = 0 \quad \text{for all } \boldsymbol{v} \in \mathbb{R}^n.$$

Choosing $\boldsymbol{v} = \boldsymbol{e}_j$ for $j = 1, \ldots, n$ gives $\mathbf{A}^T\boldsymbol{r} = \mathbf{0}$, which means $\boldsymbol{r} \perp \mathcal{R}(\mathbf{A})$. Expanding $\mathbf{A}^T(\boldsymbol{b} - \mathbf{A}\widehat{\boldsymbol{x}}) = \mathbf{0}$ yields the normal equations

$$\mathbf{A}^T\mathbf{A}\widehat{\boldsymbol{x}} = \mathbf{A}^T\boldsymbol{b}.$$

For uniqueness, suppose $\widehat{\boldsymbol{x}}_1$ and $\widehat{\boldsymbol{x}}_2$ are both minimizers. Then $\mathbf{A}(\widehat{\boldsymbol{x}}_1 - \widehat{\boldsymbol{x}}_2) = \mathbf{0}$, so $\widehat{\boldsymbol{x}}_1 - \widehat{\boldsymbol{x}}_2 \in \mathcal{N}(\mathbf{A})$. When $\mathbf{A}$ has full column rank, $\mathcal{N}(\mathbf{A}) = \{\mathbf{0}\}$, forcing $\widehat{\boldsymbol{x}}_1 = \widehat{\boldsymbol{x}}_2$. If $\boldsymbol{z} \in \mathcal{N}(\mathbf{A}) \setminus \{\mathbf{0}\}$, then $\mathbf{A}(\widehat{\boldsymbol{x}} + \boldsymbol{z}) = \mathbf{A}\widehat{\boldsymbol{x}}$, so $\widehat{\boldsymbol{x}} + \boldsymbol{z}$ is also a minimizer. □

Remark 3.4 **(Normal Equations via Differentiation)** For $f(\boldsymbol{x}) = \frac{1}{2}\|\mathbf{A}\boldsymbol{x} - \boldsymbol{b}\|_2^2$, the gradient is $\nabla f(\boldsymbol{x}) = \mathbf{A}^T(\mathbf{A}\boldsymbol{x} - \boldsymbol{b})$. Setting $\nabla f(\boldsymbol{x}) = \mathbf{0}$ recovers $\mathbf{A}^T\mathbf{A}\boldsymbol{x} = \mathbf{A}^T\boldsymbol{b}$. The Pythagorean argument reveals why the condition is sufficient and geometrically necessary.

Remark 3.5 **(Conditioning of Normal Equations)** When $\mathbf{A}$ has full column rank, $\mathbf{A}^T\mathbf{A}$ is SPD because

$$\boldsymbol{x}^T\mathbf{A}^T\mathbf{A}\boldsymbol{x} = \|\mathbf{A}\boldsymbol{x}\|_2^2 > 0$$

for nonzero $\boldsymbol{x}$. This proves that the normal equations have a unique solution.

The normal equations replace the rectangular problem by a square one, but forming $\mathbf{A}^T\mathbf{A}$ hides the geometry and worsens sensitivity. QR instead chooses orthonormal coordinates adapted to the column space.

Thm 3.6 **(Least-Squares Solution by QR)** Let $\mathbf{A} \in \mathbb{R}^{m\times n}$ have full column rank and thin QR factorization $\mathbf{A} = \mathbf{Q}_1\mathbf{R}_1$. Then the unique least-squares solution is obtained from

$$\mathbf{R}_1\widehat{\boldsymbol{x}} = \mathbf{Q}_1^T\boldsymbol{b}.$$

The fitted vector and residual are

$$\widehat{\boldsymbol{b}} = \mathbf{Q}_1\mathbf{Q}_1^T\boldsymbol{b}, \qquad \boldsymbol{r} = (\mathbf{I} - \mathbf{Q}_1\mathbf{Q}_1^T)\boldsymbol{b}.$$

Proof Let $\mathbf{Q} = (\mathbf{Q}_1 \ \mathbf{Q}_2) \in \mathbb{R}^{m\times m}$ be orthogonal, where $\mathbf{Q}_1 \in \mathbb{R}^{m\times n}$ and $\mathbf{Q}_2 \in \mathbb{R}^{m\times(m-n)}$. Since $\mathbf{Q}^T\mathbf{Q} = \mathbf{I}$, the Euclidean norm is orthogonally invariant:

$$\begin{aligned}\|\mathbf{A}\boldsymbol{x} - \boldsymbol{b}\|_2^2 &= \|\mathbf{Q}^T(\mathbf{A}\boldsymbol{x} - \boldsymbol{b})\|_2^2 \\ &= \left\| \begin{pmatrix} \mathbf{Q}_1^T \\ \mathbf{Q}_2^T \end{pmatrix} (\mathbf{Q}_1\mathbf{R}_1\boldsymbol{x} - \boldsymbol{b}) \right\|_2^2 \\ &= \left\| \begin{pmatrix} \mathbf{R}_1\boldsymbol{x} - \mathbf{Q}_1^T\boldsymbol{b} \\ -\mathbf{Q}_2^T\boldsymbol{b} \end{pmatrix} \right\|_2^2 \\ &= \|\mathbf{R}_1\boldsymbol{x} - \mathbf{Q}_1^T\boldsymbol{b}\|_2^2 + \|\mathbf{Q}_2^T\boldsymbol{b}\|_2^2.\end{aligned}$$

The second term $\|\mathbf{Q}_2^T\boldsymbol{b}\|_2^2$ is fixed and independent of $\boldsymbol{x}$. Because $\mathbf{A}$ has full column rank, $\mathbf{R}_1$ is non-singular, and setting $\mathbf{R}_1\widehat{\boldsymbol{x}} = \mathbf{Q}_1^T\boldsymbol{b}$ reduces the first term to zero, attaining the minimum value $\|\mathbf{Q}_2^T\boldsymbol{b}\|_2^2$. The fitted vector and residual follow:

$$\begin{aligned}\widehat{\boldsymbol{b}} &= \mathbf{A}\widehat{\boldsymbol{x}} = \mathbf{Q}_1\mathbf{R}_1(\mathbf{R}_1^{-1}\mathbf{Q}_1^T\boldsymbol{b}) = \mathbf{Q}_1\mathbf{Q}_1^T\boldsymbol{b}, \\ \boldsymbol{r} &= \boldsymbol{b} - \widehat{\boldsymbol{b}} = (\mathbf{I} - \mathbf{Q}_1\mathbf{Q}_1^T)\boldsymbol{b}.\end{aligned}$$

□

Remark 3.6 **(Geometry vs. Conditioning in Householder QR)** For a matrix with full column rank, Thm 4.19 proves $\kappa_2(\mathbf{A}^T\mathbf{A}) = \kappa_2(\mathbf{A})^2$. Householder QR avoids forming $\mathbf{A}^T\mathbf{A}$ and is the safer dense default. The normal equations remain useful when special structure, cost, or repeated updates make the trade worthwhile.

Example Fitting $y = c_0 + c_1 t$ to $(0, 1)$, $(1, 2)$, and $(2, 4)$ gives

$$\mathbf{A} = \begin{pmatrix} 1 & 0 \\ 1 & 1 \\ 1 & 2 \end{pmatrix}, \qquad \boldsymbol{b} = \begin{pmatrix} 1 \\ 2 \\ 4 \end{pmatrix}.$$

The solution is $c_0 = 5/6$ and $c_1 = 3/2$. The residual is orthogonal to both the constant column and the time column. Hence its entries sum to zero and have zero first moment with respect to the sample times.

3.2.2 Rank-Deficient Problems

Thm 3.7 **(All Minimizers and the Minimum-Norm Choice)** Let $\widehat{\boldsymbol{x}}$ be any least-squares minimizer. Then every minimizer has the form

$$\widehat{\boldsymbol{x}} + \boldsymbol{z}, \qquad \boldsymbol{z} \in \mathcal{N}(\mathbf{A}).$$

Among them there is exactly one vector of smallest Euclidean norm. It is characterized by

$$\widehat{\boldsymbol{x}}_{\min} \perp \mathcal{N}(\mathbf{A}), \qquad \text{equivalently} \qquad \widehat{\boldsymbol{x}}_{\min} \in \mathcal{R}(\mathbf{A}^T).$$

Proof Let $\widehat{\boldsymbol{x}}$ be any minimizer. For any $\boldsymbol{z} \in \mathcal{N}(\mathbf{A})$, $\mathbf{A}(\widehat{\boldsymbol{x}}+\boldsymbol{z}) = \mathbf{A}\widehat{\boldsymbol{x}}$, so $\widehat{\boldsymbol{x}}+\boldsymbol{z}$ is also a minimizer. Conversely, if $\boldsymbol{x}'$ is any minimizer, then $\mathbf{A}\boldsymbol{x}' = \mathbf{A}\widehat{\boldsymbol{x}}$ by uniqueness of the orthogonal projection $\widehat{\boldsymbol{b}}$, which gives $\mathbf{A}(\boldsymbol{x}' - \widehat{\boldsymbol{x}}) = \mathbf{0}$, so $\boldsymbol{x}' - \widehat{\boldsymbol{x}} \in \mathcal{N}(\mathbf{A})$.
Every vector decomposes uniquely as $\boldsymbol{x} = \boldsymbol{x}_\perp + \boldsymbol{x}_0$ with $\boldsymbol{x}_0 \in \mathcal{N}(\mathbf{A})$ and $\boldsymbol{x}_\perp \in \mathcal{N}(\mathbf{A})^\perp$. For any minimizer $\widehat{\boldsymbol{x}} + \boldsymbol{z}$, we have $\boldsymbol{z} \in \mathcal{N}(\mathbf{A})$, so

$$\begin{aligned}\|\widehat{\boldsymbol{x}} + \boldsymbol{z}\|_2^2 &= \|(\widehat{\boldsymbol{x}}_\perp + \widehat{\boldsymbol{x}}_0) + \boldsymbol{z}\|_2^2 \\ &= \|\widehat{\boldsymbol{x}}_\perp + (\widehat{\boldsymbol{x}}_0 + \boldsymbol{z})\|_2^2 \\ &= \|\widehat{\boldsymbol{x}}_\perp\|_2^2 + \|\widehat{\boldsymbol{x}}_0 + \boldsymbol{z}\|_2^2 \geq \|\widehat{\boldsymbol{x}}_\perp\|_2^2.\end{aligned}$$

The minimum is attained uniquely when $\boldsymbol{z} = -\widehat{\boldsymbol{x}}_0$, which yields $\widehat{\boldsymbol{x}}_{\min} = \widehat{\boldsymbol{x}}_\perp \in \mathcal{N}(\mathbf{A})^\perp$. By the fundamental theorem of linear algebra, $\mathcal{N}(\mathbf{A})^\perp = \mathcal{R}(\mathbf{A}^T)$. □

Def 3.6 **(Tikhonov Regularization)** For $\lambda > 0$, Tikhonov or ridge regularization solves

$$\min_{\boldsymbol{x}} \left(\|\mathbf{A}\boldsymbol{x} - \boldsymbol{b}\|_2^2 + \lambda \|\boldsymbol{x}\|_2^2 \right).$$

It trades residual fit against parameter size. The problem is equivalently the augmented least-squares problem

$$\min_{\boldsymbol{x}} \left\| \begin{pmatrix} \mathbf{A} \\ \sqrt{\lambda}\mathbf{I} \end{pmatrix} \boldsymbol{x} - \begin{pmatrix} \boldsymbol{b} \\ \mathbf{0} \end{pmatrix} \right\|_2^2,$$

and its normal equations are

$$(\mathbf{A}^T\mathbf{A} + \lambda\mathbf{I})\boldsymbol{x} = \mathbf{A}^T\boldsymbol{b}.$$

Remark 3.7 **(Interpretation of Regularization)** Regularization also adds a modeling assumption. Among models with similar fit, it prefers smaller parameter vectors. Whether that assumption is scientifically appropriate depends on the meaning and scaling of the parameters. Theorem 4.23 identifies the directions that regularization damps and the amount by which each is damped.

3.2.3 Exercises

Ex 3.2

1. **Derive.** Starting only from the orthogonal decomposition theorem, rederive the normal equations and prove both directions of Thm. 3.5. Do not use derivatives.
2. **Compute.** Fit a parabola $y = c_0 + c_1 t + c_2 t^2$ to $(-1, 1)$, $(0, 0)$, $(1, 1)$, and $(2, 5)$ using thin QR. Verify orthogonality of the residual against each column to working precision.

3. **Explore.** Construct two almost parallel columns in $\mathbb{R}^3$. Compare the condition and numerical orthogonality of solutions obtained from normal equations, modified Gram–Schmidt QR, and Householder QR as the angle between the columns decreases.

4. **Derive.** Suppose $\mathbf{A}\boldsymbol{x} = \boldsymbol{b}$ is consistent but underdetermined. Prove that the minimum-norm solution is orthogonal to the null space. Explain why adding any invisible null-space component increases the norm without changing the data fit.

5. **Derive.** Let $\mathbf{W}$ be SPD. Weighted least squares minimizes $\|\mathbf{A}\boldsymbol{x} - \boldsymbol{b}\|_{\mathbf{W}}^2$. Prove that the residual is $\mathbf{W}$-orthogonal to $\mathcal{R}(\mathbf{A})$ and derive $\mathbf{A}^T\mathbf{W}\mathbf{A}\boldsymbol{x} = \mathbf{A}^T\mathbf{W}\boldsymbol{b}$. Explain what statistical or physical information the weight matrix expresses.

6. **Explore.** Rescale one column of a data matrix by 10^6. The set of fitted vectors does not change if its coefficient is rescaled inversely, but the ridge penalty does. Explain why regularization cannot be interpreted independently of units.

Read **Further reading.** Meyer develops the geometry and algebra with unusual clarity (Meyer, 2000). Golub and Van Loan give the algorithmic treatment, while Higham explains what finite precision changes (Golub and Van Loan, 2013; Higham, 2002). Trefethen and Bau provide a concise bridge between the geometry and the computation (Trefethen and Bau, 1997).

4 Eigenvalues and the SVD

4.1 Eigenvalues, Invariant Subspaces, and Their Sensitivity

The eigenvalue problem $\mathbf{A}\boldsymbol{v} = \lambda\boldsymbol{v}$ identifies the invariant one-dimensional subspaces of a linear operator. Along an eigenvector $\boldsymbol{v}$, the action of the matrix $\mathbf{A}$ reduces to scalar scaling by λ. Eigenvalues govern the asymptotic stability of dynamical systems, the resonant frequencies of mechanical and electrical systems, the convergence rates of iterative solvers, and the dominant invariant coordinates in spectral graph theory and data science.

The characteristic equation $\det(\mathbf{A} - \lambda\mathbf{I}) = 0$ is useful for definitions and small examples, but forming its coefficients is not a sound numerical algorithm. Forming the coefficients discards the matrix structure and replaces one problem by a polynomial root problem that may be far more sensitive. Practical methods work with the matrix directly and use orthogonal or unitary transformations whenever possible.

We first define eigenvalues and invariant subspaces and relate them to changes of basis. We then prove the Schur and spectral theorems, characterize the eigenvalues of a symmetric matrix by Rayleigh quotients, and use that characterization to bound how far eigenvalues move when the matrix is perturbed. Algorithms for computing eigenvalues are the subject of Section 4.2.

4.1.1 Invariant Directions and Change of Basis

Def 4.1 **(Eigenvalues and Eigenvectors)** Let $\mathbf{A} \in \mathbb{R}^{n\times n}$. A nonzero vector $\boldsymbol{v} \in \mathbb{C}^n$ is an eigenvector of $\mathbf{A}$ associated with eigenvalue $\lambda \in \mathbb{C}$ if

$$\mathbf{A}\boldsymbol{v} = \lambda\boldsymbol{v}.$$

The set of eigenvalues is the spectrum $\sigma(\mathbf{A})$. When eigenvalues are listed as $\lambda_1, \ldots, \lambda_n$, repetitions are included according to algebraic multiplicity.

Def 4.2 **(Invariant Subspace)** A subspace $S \subseteq \mathbb{C}^n$ is **invariant** under $\mathbf{A}$ if $\mathbf{A}\boldsymbol{x} \in S$ whenever $\boldsymbol{x} \in S$. An eigenvector spans a one-dimensional invariant subspace. Higher-dimensional invariant subspaces remain meaningful even when no basis of eigenvectors exists.

Example Three two-dimensional examples show what the definition can and cannot promise.

1. A diagonal matrix scales the coordinate axes, leaving both axes invariant.
2. The shear $\begin{pmatrix} 1 & 1 \\ 0 & 1 \end{pmatrix}$ has only one independent eigenvector. Repeated application keeps sliding vectors in the same direction, so a repeated eigenvalue need not supply a full eigenbasis.
3. A nontrivial planar rotation has no real invariant line, but it has complex eigenvectors. The complex eigenvalues encode the rotation as multiplication by a complex phase.

Def 4.3 **(Similarity and a Change of Basis)** Matrices $\mathbf{A}$ and $\mathbf{B}$ are **similar** if

$$\mathbf{B} = \mathbf{S}^{-1}\mathbf{A}\mathbf{S}$$

for some nonsingular $\mathbf{S}$. They represent the same linear map in different bases: the columns of $\mathbf{S}$ are the new basis vectors written in the old coordinates.

Thm 4.1 **(Similarity Preserves Eigenvalues)** If $\mathbf{B} = \mathbf{S}^{-1}\mathbf{A}\mathbf{S}$, then $\mathbf{A}$ and $\mathbf{B}$ have the same eigenvalues. More precisely, $\mathbf{A}\boldsymbol{v} = \lambda\boldsymbol{v}$ if and only if

$$\mathbf{B}(\mathbf{S}^{-1}\boldsymbol{v}) = \lambda(\mathbf{S}^{-1}\boldsymbol{v}).$$

Proof Multiplying $\mathbf{A}\boldsymbol{v} = \lambda\boldsymbol{v}$ by $\mathbf{S}^{-1}$ and inserting $\mathbf{S}\mathbf{S}^{-1} = \mathbf{I}$ gives

$$(\mathbf{S}^{-1}\mathbf{A}\mathbf{S})(\mathbf{S}^{-1}\boldsymbol{v}) = \lambda(\mathbf{S}^{-1}\boldsymbol{v}).$$

The reverse implication follows by multiplying by $\mathbf{S}$. □

Thm 4.2 **(Diagonalization Means an Eigenvector Basis)** A matrix $\mathbf{A} \in \mathbb{C}^{n\times n}$ is similar to a diagonal matrix if and only if it has a basis of eigenvectors. If $\mathbf{S} = (\boldsymbol{v}_1 \ \cdots \ \boldsymbol{v}_n)$ contains such a basis, then

$$\mathbf{AS} = \mathbf{S\Lambda}, \qquad \mathbf{A} = \mathbf{S\Lambda S}^{-1}.$$

Proof If the columns of $\mathbf{S}$ are eigenvectors, the jth columns of $\mathbf{AS}$ and $\mathbf{S\Lambda}$ are both $\lambda_j \boldsymbol{v}_j$, which proves the factorization. Conversely, if $\mathbf{A} = \mathbf{S\Lambda S}^{-1}$, then $\mathbf{AS} = \mathbf{S\Lambda}$, so each column of the nonsingular matrix $\mathbf{S}$ is an eigenvector. □

4.1.2 Multiplicity and Schur Form

Def 4.4 **(Algebraic and Geometric Multiplicity)** Let λ be an eigenvalue of $\mathbf{A}$.

1. The algebraic multiplicity $\mu_a(\lambda)$ is the multiplicity of λ as a root of $\det(\mathbf{A} - z\mathbf{I}) = 0$.
2. The geometric multiplicity $\mu_g(\lambda) = \dim \mathcal{N}(\mathbf{A} - \lambda\mathbf{I})$ is the dimension of the corresponding eigenspace.

Every eigenvalue satisfies $1 \le \mu_g(\lambda) \le \mu_a(\lambda)$. When $\mu_g(\lambda) < \mu_a(\lambda)$ for any eigenvalue, $\mathbf{A}$ is defective and does not possess a complete basis of eigenvectors.

Thm 4.3 **(Schur Triangularization Decomposition)** Every square matrix $\mathbf{A} \in \mathbb{C}^{n\times n}$ is unitarily similar to an upper triangular matrix:

$$\mathbf{A} = \mathbf{QTQ}^*,$$

where $\mathbf{Q} \in \mathbb{C}^{n\times n}$ is unitary ($\mathbf{Q}^*\mathbf{Q} = \mathbf{I}$) and $\mathbf{T} \in \mathbb{C}^{n\times n}$ is upper triangular. The diagonal entries of $\mathbf{T}$ are the eigenvalues of $\mathbf{A}$.

Proof We proceed by induction on the matrix dimension n. For $n = 1$, the statement holds trivially with $\mathbf{Q} = [1]$ and $\mathbf{T} = \mathbf{A}$.
Assume the theorem holds for all square matrices of order $n - 1$. By the Fundamental Theorem of Algebra, the characteristic polynomial $\det(\mathbf{A} - z\mathbf{I}) = 0$ possesses at least one complex root λ_1. Let $\boldsymbol{v}_1 \in \mathbb{C}^n$ be an eigenvector associated with λ_1, normalized so that $\|\boldsymbol{v}_1\|_2 = 1$. Using the Gram–Schmidt process or Householder reflectors, construct an orthonormal basis for $\mathbb{C}^n$ whose first column is $\boldsymbol{v}_1$, forming a unitary matrix $\mathbf{U}_1 = [\boldsymbol{v}_1, \mathbf{W}] \in \mathbb{C}^{n\times n}$ with $\mathbf{W} \in \mathbb{C}^{n\times(n-1)}$.
Applying this unitary similarity transformation to $\mathbf{A}$ gives

$$\mathbf{U}_1^*\mathbf{A}\mathbf{U}_1 = \begin{pmatrix} \boldsymbol{v}_1^* \\ \mathbf{W}^* \end{pmatrix} \mathbf{A} \begin{pmatrix} \boldsymbol{v}_1 & \mathbf{W} \end{pmatrix} = \begin{pmatrix} \boldsymbol{v}_1^*\mathbf{A}\boldsymbol{v}_1 & \boldsymbol{v}_1^*\mathbf{A}\mathbf{W} \\ \mathbf{W}^*\mathbf{A}\boldsymbol{v}_1 & \mathbf{W}^*\mathbf{A}\mathbf{W} \end{pmatrix}.$$

Because $\mathbf{A}\boldsymbol{v}_1 = \lambda_1\boldsymbol{v}_1$, the leading entry is $\boldsymbol{v}_1^*(\lambda_1\boldsymbol{v}_1) = \lambda_1\|\boldsymbol{v}_1\|_2^2 = \lambda_1$. The lower-left block is $\mathbf{W}^*(\lambda_1\boldsymbol{v}_1) = \lambda_1(\mathbf{W}^*\boldsymbol{v}_1) = \mathbf{0}$, because the columns of $\mathbf{W}$ are orthogonal to $\boldsymbol{v}_1$. Thus

$$\mathbf{U}_1^*\mathbf{A}\mathbf{U}_1 = \begin{pmatrix} \lambda_1 & \boldsymbol{w}^* \\ \mathbf{0} & \mathbf{A}_1 \end{pmatrix},$$

where $\mathbf{A}_1 = \mathbf{W}^*\mathbf{A}\mathbf{W} \in \mathbb{C}^{(n-1)\times(n-1)}$ and $\boldsymbol{w}^* = \boldsymbol{v}_1^*\mathbf{A}\mathbf{W}$.
By the induction hypothesis, there exists a unitary matrix $\mathbf{Q}_1 \in \mathbb{C}^{(n-1)\times(n-1)}$ such that $\mathbf{Q}_1^*\mathbf{A}_1\mathbf{Q}_1 = \mathbf{T}_1$ is upper triangular. Define the $n \times n$ block-diagonal unitary matrix

$$\mathbf{V} = \begin{pmatrix} 1 & \mathbf{0} \\ \mathbf{0} & \mathbf{Q}_1 \end{pmatrix}.$$

Setting $\mathbf{Q} = \mathbf{U}_1\mathbf{V}$, which is unitary as the product of unitary matrices, we obtain

$$\mathbf{Q}^*\mathbf{A}\mathbf{Q} = \mathbf{V}^*(\mathbf{U}_1^*\mathbf{A}\mathbf{U}_1)\mathbf{V} = \begin{pmatrix} 1 & \mathbf{0} \\ \mathbf{0} & \mathbf{Q}_1^* \end{pmatrix}\begin{pmatrix} \lambda_1 & \boldsymbol{w}^* \\ \mathbf{0} & \mathbf{A}_1 \end{pmatrix}\begin{pmatrix} 1 & \mathbf{0} \\ \mathbf{0} & \mathbf{Q}_1 \end{pmatrix} = \begin{pmatrix} \lambda_1 & \boldsymbol{w}^*\mathbf{Q}_1 \\ \mathbf{0} & \mathbf{Q}_1^*\mathbf{A}_1\mathbf{Q}_1 \end{pmatrix} = \begin{pmatrix} \lambda_1 & \boldsymbol{w}^*\mathbf{Q}_1 \\ \mathbf{0} & \mathbf{T}_1 \end{pmatrix}.$$

Because $\mathbf{T}_1$ is upper triangular, the entire block matrix $\mathbf{T} = \mathbf{Q}^*\mathbf{A}\mathbf{Q}$ is upper triangular. The diagonal entries of $\mathbf{T}$ are λ_1 and the eigenvalues of $\mathbf{A}_1$, which collectively form the eigenvalues of $\mathbf{A}$. □

Thm 4.4 **(Spectral Theorem for Real Symmetric Matrices)** If $\mathbf{A} \in \mathbb{R}^{n\times n}$ is real symmetric ($\mathbf{A} = \mathbf{A}^T$), then all eigenvalues of $\mathbf{A}$ are real, and $\mathbf{A}$ is orthogonally diagonalizable:

$$\mathbf{A} = \mathbf{Q}\mathbf{\Lambda}\mathbf{Q}^T = \sum_{i=1}^{n} \lambda_i \boldsymbol{q}_i \boldsymbol{q}_i^T,$$

where $\mathbf{Q} \in \mathbb{R}^{n\times n}$ is real orthogonal ($\mathbf{Q}^T\mathbf{Q} = \mathbf{I}$) and $\mathbf{\Lambda} = \operatorname{diag}(\lambda_1, \ldots, \lambda_n)$ is real diagonal.

Proof First, let $\lambda \in \mathbb{C}$ and $\boldsymbol{v} \in \mathbb{C}^n \setminus \{\mathbf{0}\}$ satisfy $\mathbf{A}\boldsymbol{v} = \lambda\boldsymbol{v}$. Taking the inner product with $\boldsymbol{v}$,

$$\boldsymbol{v}^*\mathbf{A}\boldsymbol{v} = \boldsymbol{v}^*(\lambda\boldsymbol{v}) = \lambda\|\boldsymbol{v}\|_2^2.$$

Taking the conjugate transpose of this scalar and using $\mathbf{A} = \mathbf{A}^T = \mathbf{A}^*$,

$$\boldsymbol{v}^*\mathbf{A}\boldsymbol{v} = (\boldsymbol{v}^*\mathbf{A}\boldsymbol{v})^* = (\lambda\|\boldsymbol{v}\|_2^2)^* = \bar{\lambda}\|\boldsymbol{v}\|_2^2.$$

Subtracting these two scalar expressions gives

$$(\lambda - \bar{\lambda})\|\boldsymbol{v}\|_2^2 = 0.$$

Because $\boldsymbol{v} \neq \mathbf{0}$, $\|\boldsymbol{v}\|_2^2 > 0$, forcing $\lambda = \bar{\lambda} \in \mathbb{R}$. Thus all eigenvalues are real.
Because $\lambda_1 \in \mathbb{R}$ and $\mathbf{A}$ is real, writing $\boldsymbol{v} = \boldsymbol{x} + i\boldsymbol{y}$ gives $\mathbf{A}\boldsymbol{x} = \lambda_1\boldsymbol{x}$ and $\mathbf{A}\boldsymbol{y} = \lambda_1\boldsymbol{y}$. At least one of $\boldsymbol{x}, \boldsymbol{y}$ is nonzero, yielding a real unit eigenvector $\boldsymbol{q}_1$. Extend $\boldsymbol{q}_1$ to an orthogonal matrix $\mathbf{Q}_1 = [\boldsymbol{q}_1, \mathbf{W}] \in \mathbb{R}^{n\times n}$ with $\mathbf{W} \in \mathbb{R}^{n\times(n-1)}$. Because $\mathbf{A}\boldsymbol{q}_1 = \lambda_1\boldsymbol{q}_1$ and $\mathbf{W}^T\boldsymbol{q}_1 = \mathbf{0}$:

$$\mathbf{Q}_1^T\mathbf{A}\mathbf{Q}_1 = \begin{pmatrix} \boldsymbol{q}_1^T \\ \mathbf{W}^T \end{pmatrix}\mathbf{A}\begin{pmatrix} \boldsymbol{q}_1 & \mathbf{W} \end{pmatrix} = \begin{pmatrix} \boldsymbol{q}_1^T\mathbf{A}\boldsymbol{q}_1 & \boldsymbol{q}_1^T\mathbf{A}\mathbf{W} \\ \mathbf{W}^T\mathbf{A}\boldsymbol{q}_1 & \mathbf{W}^T\mathbf{A}\mathbf{W} \end{pmatrix} = \begin{pmatrix} \lambda_1 & (\mathbf{W}^T\mathbf{A}\boldsymbol{q}_1)^T \\ \mathbf{0} & \mathbf{A}_1 \end{pmatrix} = \begin{pmatrix} \lambda_1 & \mathbf{0}^T \\ \mathbf{0} & \mathbf{A}_1 \end{pmatrix},$$

where $\mathbf{A}_1 = \mathbf{W}^T\mathbf{A}\mathbf{W} \in \mathbb{R}^{(n-1)\times(n-1)}$ is symmetric. By induction on dimension, there exists an orthogonal $\mathbf{Q}_2 \in \mathbb{R}^{(n-1)\times(n-1)}$ such that $\mathbf{Q}_2^T\mathbf{A}_1\mathbf{Q}_2 = \operatorname{diag}(\lambda_2, \ldots, \lambda_n)$. The matrix

$$\mathbf{Q} = \mathbf{Q}_1\begin{pmatrix} 1 & \mathbf{0} \\ \mathbf{0} & \mathbf{Q}_2 \end{pmatrix}$$

is orthogonal and satisfies $\mathbf{Q}^T\mathbf{A}\mathbf{Q} = \mathbf{\Lambda}$, so $\mathbf{A} = \mathbf{Q}\mathbf{\Lambda}\mathbf{Q}^T$. □

Thm 4.5 **(Courant–Fischer Minimax Theorem)** Let $\mathbf{A} \in \mathbb{R}^{n\times n}$ be real symmetric with eigenvalues ordered as $\lambda_1 \le \lambda_2 \le \cdots \le \lambda_n$. Then for each $k \in \{1, \dots, n\}$,

$$\lambda_k = \min_{\dim(S)=k} \max_{\boldsymbol{x}\in S,\, \boldsymbol{x}\neq \mathbf{0}} \frac{\boldsymbol{x}^T\mathbf{A}\boldsymbol{x}}{\boldsymbol{x}^T\boldsymbol{x}} = \max_{\dim(S)=n-k+1} \min_{\boldsymbol{x}\in S,\, \boldsymbol{x}\neq \mathbf{0}} \frac{\boldsymbol{x}^T\mathbf{A}\boldsymbol{x}}{\boldsymbol{x}^T\boldsymbol{x}}.$$

Proof Let $\boldsymbol{q}_1, \dots, \boldsymbol{q}_n$ be an orthonormal basis of eigenvectors of $\mathbf{A}$ with $\mathbf{A}\boldsymbol{q}_i = \lambda_i \boldsymbol{q}_i$. For any nonzero $\boldsymbol{x} = \sum_{i=1}^n c_i \boldsymbol{q}_i$, the Rayleigh quotient satisfies

$$R(\boldsymbol{x}) = \frac{\boldsymbol{x}^T\mathbf{A}\boldsymbol{x}}{\boldsymbol{x}^T\boldsymbol{x}} = \frac{\sum_{i=1}^n \lambda_i c_i^2}{\sum_{i=1}^n c_i^2}.$$

Define the subspace $V_k = \operatorname{span}\{\boldsymbol{q}_1, \dots, \boldsymbol{q}_k\}$, which has dimension k. For any nonzero $\boldsymbol{x} \in V_k$, $c_i = 0$ for all $i > k$, so

$$R(\boldsymbol{x}) = \frac{\sum_{i=1}^k \lambda_i c_i^2}{\sum_{i=1}^k c_i^2} \le \lambda_k \frac{\sum_{i=1}^k c_i^2}{\sum_{i=1}^k c_i^2} = \lambda_k.$$

Because $\boldsymbol{q}_k \in V_k$ achieves $R(\boldsymbol{q}_k) = \lambda_k$, we have $\max_{\boldsymbol{x}\in V_k,\, \boldsymbol{x}\neq\mathbf{0}} R(\boldsymbol{x}) = \lambda_k$.
Now let S be any arbitrary subspace of $\mathbb{R}^n$ with $\dim(S) = k$. Define $W_{k-1} = \operatorname{span}\{\boldsymbol{q}_k, \dots, \boldsymbol{q}_n\}$, which has dimension $n - k + 1$. By the dimension formula,

$$\dim(S \cap W_{k-1}) = \dim(S) + \dim(W_{k-1}) - \dim(S + W_{k-1}) \ge k + (n-k+1) - n = 1.$$

Therefore, there exists a nonzero vector $\boldsymbol{y} \in S \cap W_{k-1}$. Because $\boldsymbol{y} \in W_{k-1}$, its expansion $\boldsymbol{y} = \sum_{i=k}^n d_i \boldsymbol{q}_i$ contains only modes $i \ge k$, so

$$R(\boldsymbol{y}) = \frac{\sum_{i=k}^n \lambda_i d_i^2}{\sum_{i=k}^n d_i^2} \ge \lambda_k \frac{\sum_{i=k}^n d_i^2}{\sum_{i=k}^n d_i^2} = \lambda_k.$$

Because $\boldsymbol{y} \in S$, the maximum over S satisfies $\max_{\boldsymbol{x}\in S,\, \boldsymbol{x}\neq\mathbf{0}} R(\boldsymbol{x}) \ge R(\boldsymbol{y}) \ge \lambda_k$. Taking the minimum over all k-dimensional subspaces S proves the first equality. The dual maximum-minimum characterization follows similarly by replacing $\mathbf{A}$ with $-\mathbf{A}$. □

Thm 4.6 **(Cauchy Interlacing Theorem)** Let $\mathbf{A} \in \mathbb{R}^{n\times n}$ be symmetric, and let $\mathbf{B} \in \mathbb{R}^{(n-1)\times(n-1)}$ be a principal submatrix. If

$$\lambda_1 \le \cdots \le \lambda_n, \qquad \mu_1 \le \cdots \le \mu_{n-1}$$

are the eigenvalues of $\mathbf{A}$ and $\mathbf{B}$, respectively, then

$$\lambda_k \le \mu_k \le \lambda_{k+1}, \qquad k = 1, \dots, n-1.$$

Proof After a simultaneous permutation of rows and columns, we may assume that $\mathbf{B}$ is obtained by deleting the last row and column. Identify $\mathbb{R}^{n-1}$ with the coordinate hyperplane

$$H = \{\boldsymbol{x} \in \mathbb{R}^n : x_n = 0\}.$$

For every nonzero $\boldsymbol{x} \in H$, the Rayleigh quotients of $\mathbf{A}$ and $\mathbf{B}$ agree.
Apply the first Courant–Fischer characterization from Thm 4.5. The minimization defining μ_k ranges only over k-dimensional subspaces of H, whereas the minimization defining λ_k ranges over all k-dimensional subspaces of $\mathbb{R}^n$. Restricting the admissible set cannot decrease the minimum, so $\lambda_k \le \mu_k$.
For the other inequality, use the dual characterization. Since $\dim(H) = n - 1$,

$$\mu_k = \max_{\substack{S \subset H \\ \dim(S) = n-k}} \min_{\boldsymbol{x} \in S,\, \boldsymbol{x} \neq \mathbf{0}} R_{\mathbf{A}}(\boldsymbol{x}) \le \max_{\substack{S \subset \mathbb{R}^n \\ \dim(S) = n-k}} \min_{\boldsymbol{x} \in S,\, \boldsymbol{x} \neq \mathbf{0}} R_{\mathbf{A}}(\boldsymbol{x}) = \lambda_{k+1}.$$

Together the two bounds give the interlacing inequalities. □

4.1.3 Sensitivity of Eigenvalues

Every eigenvalue algorithm computes the eigenvalues of a matrix that differs from $\mathbf{A}$ by rounding errors, and every iterative eigensolver stops with an approximate eigenpair whose residual is small but nonzero. We therefore need to know how far eigenvalues move when the matrix is perturbed. For symmetric matrices the answer follows from Thm 4.5, and eigenvalues move no farther than the norm of the perturbation. For a general diagonalizable matrix the answer also depends on the condition number of the eigenvector basis.

Thm 4.7 **(Weyl's Perturbation Theorem)** Let $\mathbf{A}$ and $\mathbf{E}$ be real symmetric $n \times n$ matrices. Let $\lambda_1 \le \cdots \le \lambda_n$ be the eigenvalues of $\mathbf{A}$ and $\mu_1 \le \cdots \le \mu_n$ the eigenvalues of $\mathbf{A} + \mathbf{E}$. Then

$$|\mu_k - \lambda_k| \le \|\mathbf{E}\|_2, \qquad k = 1, \ldots, n.$$

Proof For every nonzero $\boldsymbol{x}$, the Rayleigh quotients of the two matrices satisfy

$$\frac{\boldsymbol{x}^T(\mathbf{A} + \mathbf{E})\boldsymbol{x}}{\boldsymbol{x}^T\boldsymbol{x}} = \frac{\boldsymbol{x}^T\mathbf{A}\boldsymbol{x}}{\boldsymbol{x}^T\boldsymbol{x}} + \frac{\boldsymbol{x}^T\mathbf{E}\boldsymbol{x}}{\boldsymbol{x}^T\boldsymbol{x}}, \qquad \frac{|\boldsymbol{x}^T\mathbf{E}\boldsymbol{x}|}{\boldsymbol{x}^T\boldsymbol{x}} \le \frac{\|\boldsymbol{x}\|_2 \|\mathbf{E}\boldsymbol{x}\|_2}{\|\boldsymbol{x}\|_2^2} \le \|\mathbf{E}\|_2,$$

where the second relation uses Cauchy–Schwarz and the definition of the induced norm. The Rayleigh quotient of $\mathbf{A} + \mathbf{E}$ therefore lies within $\|\mathbf{E}\|_2$ of that of $\mathbf{A}$ for every $\boldsymbol{x}$. Taking the maximum over a k-dimensional subspace and then the minimum over all such subspaces preserves this relation, so the first characterization in Thm 4.5 gives $\lambda_k - \|\mathbf{E}\|_2 \le \mu_k \le \lambda_k + \|\mathbf{E}\|_2$. □

Cor 4.8 **(Residual Bounds for Symmetric Matrices)** Let $\mathbf{A}$ be real symmetric, let $\boldsymbol{x}$ be a unit vector, let θ be real, and let $\boldsymbol{r} = \mathbf{A}\boldsymbol{x} - \theta\boldsymbol{x}$.

1. Some eigenvalue λ of $\mathbf{A}$ satisfies $|\lambda - \theta| \le \|\boldsymbol{r}\|_2$.

2. Suppose $\theta = \boldsymbol{x}^T \mathbf{A} \boldsymbol{x}$, let λ be an eigenvalue of $\mathbf{A}$ nearest to θ, and let $\delta > 0$ be the distance from θ to the remaining eigenvalues. Then

$$|\lambda - \theta| \le \frac{\|\boldsymbol{r}\|_2^2}{\delta}.$$

Proof Expand $\boldsymbol{x} = \sum_i c_i \boldsymbol{q}_i$ in an orthonormal eigenvector basis, so that $\sum_i c_i^2 = 1$ and $\boldsymbol{r} = \sum_i (\lambda_i - \theta) c_i \boldsymbol{q}_i$. Then

$$\|\boldsymbol{r}\|_2^2 = \sum_{i=1}^{n} (\lambda_i - \theta)^2 c_i^2 \ge \min_i (\lambda_i - \theta)^2,$$

which proves the first claim.
For the second claim, the Rayleigh quotient condition is $\sum_i (\lambda_i - \theta) c_i^2 = 0$. Suppose $\lambda \le \theta$, since the case $\lambda \ge \theta$ follows by applying the argument to $-\mathbf{A}$. Let $\beta = \theta + \delta$. An eigenvalue $\lambda_i \ne \lambda$ cannot lie in $(\lambda, \theta]$, because it would then be closer to θ than λ. Such an eigenvalue also cannot lie in (θ, β), by the definition of δ. Hence every such eigenvalue satisfies $\lambda_i < \lambda$ or $\lambda_i \ge \beta$, and in both cases $(\lambda_i - \lambda)(\lambda_i - \beta) \ge 0$. The product vanishes when $\lambda_i = \lambda$. Writing $\lambda_i - \lambda = (\lambda_i - \theta) + (\theta - \lambda)$ and $\lambda_i - \beta = (\lambda_i - \theta) - \delta$ and expanding gives

$$0 \le \sum_{i=1}^{n} (\lambda_i - \lambda)(\lambda_i - \beta) c_i^2 = \|\boldsymbol{r}\|_2^2 + (\theta - \lambda - \delta) \sum_{i=1}^{n} (\lambda_i - \theta) c_i^2 - (\theta - \lambda)\delta = \|\boldsymbol{r}\|_2^2 - (\theta - \lambda)\delta.$$

Therefore $(\theta - \lambda)\delta \le \|\boldsymbol{r}\|_2^2$. □

The first bound states that a small residual locates an eigenvalue of a symmetric matrix to within the residual norm. The second bound shows that the Rayleigh quotient is much more accurate than its residual when the eigenvalue is well separated from the rest of the spectrum. An eigenvector residual of 10^{-4} and a gap of order one, for example, give an eigenvalue error of order 10^{-8}.

Thm 4.9 **(Bauer–Fike Theorem)** Let $\mathbf{A} = \mathbf{X} \mathbf{\Lambda} \mathbf{X}^{-1}$ be diagonalizable with eigenvalues $\lambda_1, \dots, \lambda_n$, and let $\mathbf{E}$ be any $n \times n$ matrix. Every eigenvalue μ of $\mathbf{A} + \mathbf{E}$ satisfies

$$\min_{1 \le i \le n} |\lambda_i - \mu| \le \kappa_2(\mathbf{X}) \|\mathbf{E}\|_2,$$

where $\kappa_2(\mathbf{X}) = \|\mathbf{X}\|_2 \|\mathbf{X}^{-1}\|_2$ is computed in the Euclidean norm on $\mathbb{C}^n$.

Proof If μ is an eigenvalue of $\mathbf{A}$, there is nothing to prove. Otherwise $\mathbf{\Lambda} - \mu \mathbf{I}$ is nonsingular. Because $\mathbf{A} + \mathbf{E} - \mu \mathbf{I}$ is singular, so is

$$\mathbf{X}^{-1} (\mathbf{A} + \mathbf{E} - \mu \mathbf{I}) \mathbf{X} = (\mathbf{\Lambda} - \mu \mathbf{I}) \big(\mathbf{I} + (\mathbf{\Lambda} - \mu \mathbf{I})^{-1} \mathbf{X}^{-1} \mathbf{E} \mathbf{X} \big).$$

The second factor is singular, so Lem 2.6, whose proof applies without change on $\mathbb{C}^n$, shows that $\|(\mathbf{\Lambda} - \mu\mathbf{I})^{-1}\mathbf{X}^{-1}\mathbf{E}\mathbf{X}\|_2 \geq 1$. The norm of the diagonal matrix $(\mathbf{\Lambda} - \mu\mathbf{I})^{-1}$ is $1/\min_i |\lambda_i - \mu|$. Submultiplicativity therefore gives

$$1 \leq \frac{\|\mathbf{X}^{-1}\|_2 \|\mathbf{E}\|_2 \|\mathbf{X}\|_2}{\min_i |\lambda_i - \mu|},$$

which is the stated bound. □

Cor 4.10 **(Residual Bound for Diagonalizable Matrices)** Let $\mathbf{A} = \mathbf{X}\mathbf{\Lambda}\mathbf{X}^{-1}$, let $\boldsymbol{x}$ be a unit vector, let $\theta \in \mathbb{C}$, and let $\boldsymbol{r} = \mathbf{A}\boldsymbol{x} - \theta\boldsymbol{x}$. Then some eigenvalue λ of $\mathbf{A}$ satisfies $|\lambda - \theta| \leq \kappa_2(\mathbf{X})\|\boldsymbol{r}\|_2$.

Proof The matrix $\mathbf{E} = -\boldsymbol{r}\boldsymbol{x}^*$ satisfies $(\mathbf{A} + \mathbf{E})\boldsymbol{x} = \mathbf{A}\boldsymbol{x} - \boldsymbol{r} = \theta\boldsymbol{x}$, so θ is an eigenvalue of $\mathbf{A} + \mathbf{E}$. Since $\|\mathbf{E}\|_2 = \|\boldsymbol{r}\|_2\|\boldsymbol{x}\|_2 = \|\boldsymbol{r}\|_2$, Thm 4.9 gives the bound. □

For a symmetric matrix, Thm 4.4 provides an orthogonal $\mathbf{X}$ with $\kappa_2(\mathbf{X}) = 1$, and Cor 4.10 reduces to the first part of Cor 4.8. For a nonnormal matrix, $\kappa_2(\mathbf{X})$ can be large, and a small residual then locates an eigenvalue only loosely. Near a defective matrix the eigenvalues can move much farther than the perturbation, as the following example shows.

Example **(Eigenvalues of a nearly defective matrix)** The matrix

$$\mathbf{A}_\varepsilon = \begin{pmatrix} 0 & 1 \\ \varepsilon & 0 \end{pmatrix}$$

differs from the defective matrix $\mathbf{A}_0$ by a perturbation of norm ε, but its eigenvalues are $\pm\sqrt{\varepsilon}$. For $\varepsilon = 10^{-10}$, a perturbation at the level of 10^{-10} moves the double eigenvalue zero by 10^{-5}. The eigenvectors $(1, \pm\sqrt{\varepsilon})^T$ of $\mathbf{A}_\varepsilon$ become parallel as $\varepsilon \to 0$, so $\kappa_2(\mathbf{X})$ grows without bound, in agreement with Thm 4.9.

4.1.4 Exercises

Ex 4.1

1. **Derive. (Rayleigh quotient stationary points)** For a real symmetric matrix $\mathbf{A}$, define the Rayleigh quotient $R(\boldsymbol{x}) = \frac{\boldsymbol{x}^T\mathbf{A}\boldsymbol{x}}{\boldsymbol{x}^T\boldsymbol{x}}$ on $\mathbb{R}^n \setminus \{\mathbf{0}\}$.
   (a) Compute the gradient $\nabla R(\boldsymbol{x})$ directly.
   (b) Prove that $\nabla R(\boldsymbol{x}) = \mathbf{0}$ if and only if $\boldsymbol{x}$ is an eigenvector of $\mathbf{A}$.
   (c) Restrict the Hessian $\nabla^2 R(\boldsymbol{q}_i)$ to directions orthogonal to $\boldsymbol{q}_i$. Show that its eigenvalues on this tangent space are $\frac{2}{\|\boldsymbol{q}_i\|_2^2}(\lambda_j - \lambda_i)$ for $j \neq i$. Explain why the radial direction has zero curvature, and conclude that the smallest and largest eigenvalues are the global extrema of R.

2. **Derive. (Similarity is a coordinate change)** Choose a nonorthogonal basis of $\mathbb{R}^2$, form its basis matrix $\mathbf{S}$, and compute $\mathbf{S}^{-1}\mathbf{A}\mathbf{S}$ for a simple diagonal matrix $\mathbf{A}$. Verify directly that eigenvalues are preserved although lengths and angles generally are not.

3. **Explore. (A defective shear)** For $\mathbf{A} = \begin{pmatrix} 1 & 1 \\ 0 & 1 \end{pmatrix}$, find the algebraic and geometric multiplicities of the eigenvalue. Compute $\mathbf{A}^k$ and explain how its growth is invisible if one records only the eigenvalue 1.

4. **Compute. (Cauchy interlacing on a concrete matrix)** For

$$\mathbf{A} = \begin{pmatrix} 2 & 1 & 0 \\ 1 & 2 & 1 \\ 0 & 1 & 2 \end{pmatrix},$$

compute the eigenvalues of $\mathbf{A}$ and of the leading 2×2 principal submatrix. Verify each inequality in Thm 4.6. Repeat after deleting the middle row and column.

4.2 Vector Iterations and the QR Algorithm

Section 4.1 characterized eigenvalues but did not compute them. Every polynomial is the characteristic polynomial of its companion matrix, so the Abel–Ruffini theorem implies that no formula built from finitely many arithmetic operations and root extractions gives the eigenvalues of a general matrix with $n \geq 5$. Every eigenvalue algorithm is therefore iterative. We first study iterations that apply $\mathbf{A}$ or $(\mathbf{A} - \sigma\mathbf{I})^{-1}$ to a single vector. These iterations compute one eigenpair at a time and form the basis of the sparse methods in later sections. We then describe the QR algorithm, which computes all eigenvalues of a dense matrix by a sequence of orthogonal similarity transformations.

4.2.1 Power Iteration and Shifted Inverse Iteration

Def 4.5 **(Power Iteration)** Let $\mathbf{A}$ be real symmetric. Given a unit vector $\boldsymbol{v}^{(0)}$, power iteration constructs

$$\boldsymbol{w}^{(k)} = \mathbf{A}\boldsymbol{v}^{(k-1)}, \qquad \boldsymbol{v}^{(k)} = \frac{\boldsymbol{w}^{(k)}}{\|\boldsymbol{w}^{(k)}\|_2}, \qquad \lambda^{(k)} = (\boldsymbol{v}^{(k)})^T \mathbf{A}\boldsymbol{v}^{(k)}.$$

Order the eigenvalues by magnitude. If $|\lambda_1| > |\lambda_2| \geq \cdots \geq |\lambda_n|$ and $\boldsymbol{q}_1^T \boldsymbol{v}^{(0)} \neq 0$, the direction of the iterate converges linearly:

$$\|\boldsymbol{v}^{(k)} - (\pm \boldsymbol{q}_1)\|_2 = O\left(\left|\frac{\lambda_2}{\lambda_1}\right|^k\right).$$

For a general nonnormal matrix, an eigenvector basis may be poorly conditioned or may not exist, and this simple orthogonal expansion no longer justifies the claim.

Def 4.6 **(Shifted Inverse Iteration)** For a real symmetric matrix, an eigenvalue near a target σ can be isolated by applying power iteration implicitly to $(\mathbf{A}-\sigma\mathbf{I})^{-1}$:

$$(\mathbf{A}-\sigma\mathbf{I})\boldsymbol{w}^{(k)}=\boldsymbol{v}^{(k-1)},\qquad \boldsymbol{v}^{(k)}=\frac{\boldsymbol{w}^{(k)}}{\|\boldsymbol{w}^{(k)}\|_2}.$$

The eigenvalues of $(\mathbf{A}-\sigma\mathbf{I})^{-1}$ are $(\lambda_i-\sigma)^{-1}$. Provided the shift is not itself an eigenvalue and the starting vector contains the desired component, the mode closest to σ dominates. The matrix inverse is never formed. Each step solves a shifted linear system instead.

Def 4.7 **(Rayleigh Quotient Iteration (RQI))** For symmetric matrices, updating the shift dynamically to the current Rayleigh estimate $\sigma_k=(\boldsymbol{v}^{(k)})^T\mathbf{A}\boldsymbol{v}^{(k)}$ defines Rayleigh Quotient Iteration:

$$(\mathbf{A}-\sigma_k\mathbf{I})\boldsymbol{w}^{(k+1)}=\boldsymbol{v}^{(k)},\qquad \boldsymbol{v}^{(k+1)}=\frac{\boldsymbol{w}^{(k+1)}}{\|\boldsymbol{w}^{(k+1)}\|_2},\qquad \sigma_{k+1}=(\boldsymbol{v}^{(k+1)})^T\mathbf{A}\boldsymbol{v}^{(k+1)}.$$

Thm 4.11 **(Local Cubic Convergence of Rayleigh Quotient Iteration)** Let $\mathbf{A}$ be real symmetric, and let λ_* be a simple eigenvalue with unit eigenvector $\boldsymbol{q}_*$. If a unit RQI iterate $\boldsymbol{v}^{(k)}$ is sufficiently close to $\boldsymbol{q}_*$, then

$$\sin\angle(\boldsymbol{v}^{(k+1)},\boldsymbol{q}_*)\le C\sin^3\angle(\boldsymbol{v}^{(k)},\boldsymbol{q}_*).$$

If λ_* is an extremal eigenvalue, the Rayleigh shifts also satisfy

$$|\sigma_{k+1}-\lambda_*|\le C'|\sigma_k-\lambda_*|^3$$

for iterates sufficiently close to $\boldsymbol{q}_*$.

Proof Write the current unit vector as

$$\boldsymbol{v}=\cos\theta\,\boldsymbol{q}_*+\sin\theta\,\boldsymbol{z},\qquad \boldsymbol{z}\perp\boldsymbol{q}_*,\qquad \|\boldsymbol{z}\|_2=1.$$

Its Rayleigh quotient $\sigma=\boldsymbol{v}^T\mathbf{A}\boldsymbol{v}$ obeys

$$\begin{aligned}\sigma&=(\cos\theta\,\boldsymbol{q}_*+\sin\theta\,\boldsymbol{z})^T\mathbf{A}(\cos\theta\,\boldsymbol{q}_*+\sin\theta\,\boldsymbol{z})\\&=\cos^2\theta\,\lambda_*+2\cos\theta\sin\theta\,\boldsymbol{q}_*^T\mathbf{A}\boldsymbol{z}+\sin^2\theta\,\boldsymbol{z}^T\mathbf{A}\boldsymbol{z}.\end{aligned}$$

Because $\mathbf{A}\boldsymbol{q}_*=\lambda_*\boldsymbol{q}_*$ and $\boldsymbol{q}_*^T\boldsymbol{z}=0$, the cross-term vanishes: $\boldsymbol{q}_*^T\mathbf{A}\boldsymbol{z}=\lambda_*\boldsymbol{q}_*^T\boldsymbol{z}=0$. Substituting $\cos^2\theta=1-\sin^2\theta$,

$$\sigma-\lambda_*=(1-\sin^2\theta)\lambda_*+\sin^2\theta\,\boldsymbol{z}^T\mathbf{A}\boldsymbol{z}-\lambda_*=\sin^2\theta\,(\boldsymbol{z}^T\mathbf{A}\boldsymbol{z}-\lambda_*),$$

so $|\sigma-\lambda_*|\le C_0\sin^2\theta$.
Let $\gamma=\min_{\lambda_j\neq\lambda_*}|\lambda_j-\lambda_*|>0$. Once θ is small enough, $|\lambda_j-\sigma|\ge\gamma/2$ for every other eigenvalue. Expanding the inverse-iteration vector $\boldsymbol{w}=(\mathbf{A}-\sigma\mathbf{I})^{-1}\boldsymbol{v}$ in an orthonormal eigenbasis, its component along $\boldsymbol{q}_*$ has magnitude

$$\frac{|\cos\theta|}{|\lambda_*-\sigma|},$$

whereas the norm of its orthogonal component is at most $2|\sin\theta|/\gamma$. Normalization does not change their ratio. Therefore,

$$\tan\angle(\boldsymbol{v}^+, \boldsymbol{q}_*) \leq \frac{2|\lambda_* - \sigma|}{\gamma|\cos\theta|}|\sin\theta| \leq C_1|\sin\theta|^3.$$

For small angles, $\sin\theta$ and $\tan\theta$ are comparable, which proves cubic convergence of the eigenvector direction.
If λ_* is extremal, the spectral gap also gives two-sided local bounds $c\sin^2\theta \leq |\sigma - \lambda_*| \leq C\sin^2\theta$. Applying the Rayleigh-quotient identity to the next iterate and using the cubic angle bound yields

$$|\sigma^+ - \lambda_*| \leq C_2\sin^2\angle(\boldsymbol{v}^+, \boldsymbol{q}_*) \leq C_3\sin^6\theta \leq C_4|\sigma - \lambda_*|^3.$$

□

4.2.2 The Practical QR Algorithm

Def 4.8 **(Basic QR Iteration)** Given $\mathbf{A}_0 = \mathbf{A}$, the unshifted QR iteration generates orthogonally similar matrices via

$$\mathbf{A}_k = \mathbf{Q}_k\mathbf{R}_k, \qquad \mathbf{A}_{k+1} = \mathbf{R}_k\mathbf{Q}_k = \mathbf{Q}_k^T\mathbf{A}_k\mathbf{Q}_k.$$

All iterates share the same eigenvalues. Under suitable separation conditions the subdiagonal entries tend to zero and the iterates reveal a Schur form. Shifts and deflation are essential for a reliable practical method.

Lem 4.12 **(Invariance of Hessenberg Structure under QR Iteration)** Let $\mathbf{H} \in \mathbb{R}^{n\times n}$ be an upper Hessenberg matrix ($H_{ij} = 0$ for all $i > j + 1$). If $\mathbf{H} = \mathbf{QR}$ is the QR factorization of $\mathbf{H}$, then the reversed product $\mathbf{H}_+ = \mathbf{RQ}$ is also upper Hessenberg.

Proof Because $\mathbf{H} = \mathbf{QR}$ and $\mathbf{R}$ is upper triangular and invertible (assuming $\mathbf{H}$ is nonsingular, or extending by continuity), $\mathbf{Q} = \mathbf{HR}^{-1}$. Because the inverse of an upper triangular matrix is upper triangular, $\mathbf{R}^{-1}$ is upper triangular. The product of an upper Hessenberg matrix $\mathbf{H}$ and an upper triangular matrix $\mathbf{R}^{-1}$ is upper Hessenberg. Therefore, $\mathbf{Q}$ is upper Hessenberg.
Now consider $\mathbf{H}_+ = \mathbf{RQ}$. The matrix $\mathbf{R}$ is upper triangular and $\mathbf{Q}$ is upper Hessenberg. For entry (i, j) with $i > j + 1$:

$$(\mathbf{H}_+)_{ij} = \sum_{k=1}^{n} R_{ik}Q_{kj}.$$

Because $\mathbf{R}$ is upper triangular, $R_{ik} = 0$ for all $k < i$. For the remaining terms $k \geq i$, we have $k \geq i > j + 1$, which implies $k > j + 1$. Because $\mathbf{Q}$ is upper Hessenberg, $Q_{kj} = 0$ for all $k > j + 1$. Thus every term in the sum vanishes, proving $(\mathbf{H}_+)_{ij} = 0$ for $i > j + 1$. □

Thm 4.13 **(Practical Shifted QR Framework)** A standard dense nonsymmetric eigenvalue computation has two phases:

1. **Hessenberg reduction:** Compute $\mathbf{H}_0 = \mathbf{U}_0^T \mathbf{A} \mathbf{U}_0$ with Householder reflectors. This one-time reduction costs $O(n^3)$ operations and preserves the eigenvalues.

2. **Implicit shifted QR steps:** Apply shifts while preserving Hessenberg form. A schematic single-shift step is

$$\mathbf{H}_k - \mu_k \mathbf{I} = \mathbf{Q}_k \mathbf{R}_k, \qquad \mathbf{H}_{k+1} = \mathbf{R}_k \mathbf{Q}_k + \mu_k \mathbf{I}.$$

   Lem. 4.12 explains why each step costs $O(n^2)$ rather than $O(n^3)$. Real nonsymmetric codes use implicit single or double shifts so that complex conjugate pairs can remain in real arithmetic. A small subdiagonal entry permits deflation and splits the problem into smaller blocks.

For a real symmetric matrix, Hessenberg form reduces to tridiagonal form. Wilkinson shifts from the trailing two by two block and implicit bulge chasing then give a particularly efficient symmetric algorithm.

4.2.3 The Jacobi Eigenvalue Algorithm

For a symmetric matrix, an alternative to the QR algorithm drives the off-diagonal entries to zero directly. Each step applies an orthogonal similarity by a plane rotation chosen to annihilate one off-diagonal entry. The method, due to Jacobi in 1846, is slower than shifted QR for a general dense matrix. It remains useful because it can compute the small eigenvalues of graded positive definite matrices to high relative accuracy (Demmel, 1997), and because rotations acting on disjoint pairs of indices can be applied in parallel.

Def 4.9 **(Givens Plane Rotation Matrix)** The matrix $\mathbf{G}(p, q, \theta) \in \mathbb{R}^{n \times n}$ is an identity matrix except for the four entries at the intersections of rows and columns p and q:

$$g_{pp} = c, \qquad g_{pq} = s, \qquad g_{qp} = -s, \qquad g_{qq} = c,$$

where $c = \cos\theta$ and $s = \sin\theta$. The transformation $\mathbf{A}' = \mathbf{G}^T \mathbf{A} \mathbf{G}$ modifies only rows and columns p and q of $\mathbf{A}$.

Thm 4.14 **(Derivation of the Optimal Rotation Angle)** To zero the transformed off-diagonal entry $a'_{pq} = 0$, we consider the 2×2 orthogonal similarity transformation on the submatrix at indices $\{p, q\}$:

$$\begin{aligned}\begin{pmatrix} a'_{pp} & a'_{pq} \\ a'_{pq} & a'_{qq} \end{pmatrix} &= \begin{pmatrix} c & -s \\ s & c \end{pmatrix} \begin{pmatrix} a_{pp} & a_{pq} \\ a_{pq} & a_{qq} \end{pmatrix} \begin{pmatrix} c & s \\ -s & c \end{pmatrix} \\ &= \begin{pmatrix} c^2 a_{pp} - 2cs a_{pq} + s^2 a_{qq} & (c^2 - s^2) a_{pq} + cs(a_{pp} - a_{qq}) \\ (c^2 - s^2) a_{pq} + cs(a_{pp} - a_{qq}) & s^2 a_{pp} + 2cs a_{pq} + c^2 a_{qq} \end{pmatrix}.\end{aligned}$$

Setting $a'_{pq} = 0$ yields the trigonometric condition

$$(c^2 - s^2) a_{pq} + cs(a_{pp} - a_{qq}) = 0 \implies \frac{\cos(2\theta)}{\sin(2\theta)} = \frac{a_{qq} - a_{pp}}{2a_{pq}}.$$

Defining $\tau = \cot(2\theta) = \frac{a_{qq} - a_{pp}}{2a_{pq}}$ and using $\cot(2\theta) = \frac{1 - \tan^2\theta}{2\tan\theta}$, setting $t = \tan\theta$ produces

$$t^2 + 2\tau t - 1 = 0.$$

To minimize the perturbation to $\mathbf{A}$ and avoid cancellation, we select the root of smaller absolute value:

$$t = \frac{\operatorname{sign}(\tau)}{|\tau| + \sqrt{1 + \tau^2}}, \qquad c = \frac{1}{\sqrt{1 + t^2}}, \qquad s = ct.$$

Proof The quadratic formula yields roots $t = -\tau \pm \sqrt{1 + \tau^2}$. Multiplying the root of smaller magnitude by its algebraic conjugate yields

$$t = \frac{(-\tau \pm \sqrt{1 + \tau^2})(\tau \pm \sqrt{1 + \tau^2})}{\tau \pm \sqrt{1 + \tau^2}} = \frac{1}{\tau + \operatorname{sign}(\tau)\sqrt{1 + \tau^2}} = \frac{\operatorname{sign}(\tau)}{|\tau| + \sqrt{1 + \tau^2}}.$$

This formula satisfies $|t| \leq 1$, corresponding to a rotation angle $|\theta| \leq \pi/4$, and avoids subtracting nearly equal floating-point numbers. □

A rotation that annihilates a_{pq} may make a previously annihilated entry nonzero again, so the method does not terminate after $n(n-1)/2$ steps. Convergence follows instead from a quantity that decreases at every step.

Thm 4.15 **(Monotone Decrease of the Off-Diagonal Frobenius Norm)** Let the off-diagonal squared norm be defined by

$$\operatorname{off}(\mathbf{A})^2 = \sum_{i \neq j} a_{ij}^2 = \|\mathbf{A}\|_F^2 - \sum_{i=1}^{n} a_{ii}^2.$$

Applying the Jacobi rotation $\mathbf{A}' = \mathbf{G}(p, q, \theta)^T \mathbf{A} \mathbf{G}(p, q, \theta)$ with the optimal angle strictly reduces the off-diagonal norm:

$$\operatorname{off}(\mathbf{A}')^2 = \operatorname{off}(\mathbf{A})^2 - 2a_{pq}^2.$$

Proof Because $\mathbf{G}$ is orthogonal, the total Frobenius norm is preserved: $\|\mathbf{A}'\|_F^2 = \|\mathbf{A}\|_F^2$. The rotation affects only entries in rows and columns p and q. For the 2×2 submatrix, the sum of squares is invariant under orthogonal similarity:

$$(a'_{pp})^2 + (a'_{qq})^2 + 2(a'_{pq})^2 = a_{pp}^2 + a_{qq}^2 + 2a_{pq}^2.$$

Because the rotation enforces $a'_{pq} = 0$, the sum of squared diagonal entries increases:

$$(a'_{pp})^2 + (a'_{qq})^2 = a_{pp}^2 + a_{qq}^2 + 2a_{pq}^2.$$

All other diagonal entries a_{ii} $(i \neq p, q)$ are untouched. Substituting this into $\operatorname{off}(\mathbf{A}')^2 = \|\mathbf{A}'\|_F^2 - \sum (a'_{ii})^2$ proves that $\operatorname{off}(\mathbf{A}')^2 = \operatorname{off}(\mathbf{A})^2 - 2a_{pq}^2$. $\square$

Def 4.10 **(Cyclic Jacobi Algorithm)** Let $\mathbf{A}^{(0)} = \mathbf{A}$ and $\mathbf{V} = \mathbf{I}_n$. While $\operatorname{off}(\mathbf{A}) > \texttt{tol}\, \|\mathbf{A}^{(0)}\|_F$, sweep through the pairs $1 \le p < q \le n$ in a fixed order. For each pair, compute c and s from Thm 4.14, replace $\mathbf{A}$ by $\mathbf{G}(p, q, \theta)^T \mathbf{A} \mathbf{G}(p, q, \theta)$, and replace $\mathbf{V}$ by $\mathbf{V}\mathbf{G}(p, q, \theta)$. On termination, the diagonal of $\mathbf{A}$ approximates the eigenvalues and the columns of $\mathbf{V}$ the eigenvectors.

Theorem 4.15 shows that $\operatorname{off}(\mathbf{A})^2$ decreases by $2a_{pq}^2$ at every rotation. Choosing the largest off-diagonal entry at each step gives linear convergence, as the exercises show. The cyclic order avoids that search, and once the off-diagonal entries are small compared with the gaps between the eigenvalues, cyclic Jacobi converges quadratically, measured sweep by sweep (Golub and Van Loan, 2013). Each sweep costs $O(n^3)$ flops.

4.2.4 Shifted Systems from One Hessenberg Reduction

Several calculations require the solution of $(s\mathbf{I} - \mathbf{A})\boldsymbol{x} = \boldsymbol{b}$ for many complex shifts s with the same $\mathbf{A}$ and $\boldsymbol{b}$. The vector $(s\mathbf{I} - \mathbf{A})^{-1}\boldsymbol{b}$ is the resolvent of $\mathbf{A}$ applied to $\boldsymbol{b}$. Resolvents appear in Laplace transforms of solutions of linear differential equations, in frequency responses, and in contour-integral formulas for matrix functions. Factoring $s\mathbf{I} - \mathbf{A}$ separately for each of N shifts costs about $\frac{2}{3}n^3$ flops per shift. The Hessenberg reduction of Thm 4.13 removes this repetition, because adding a multiple of the identity does not change Hessenberg structure.

Thm 4.16 **(Shifted Solves with a Hessenberg Matrix)** Let $\mathbf{A} = \mathbf{Q}\mathbf{H}\mathbf{Q}^T$ with $\mathbf{Q}$ orthogonal and $\mathbf{H}$ upper Hessenberg, and let s not be an eigenvalue of $\mathbf{A}$. Then $\boldsymbol{x} = (s\mathbf{I} - \mathbf{A})^{-1}\boldsymbol{b}$ is given by

$$(s\mathbf{I} - \mathbf{H})\boldsymbol{y} = \mathbf{Q}^T\boldsymbol{b}, \qquad \boldsymbol{x} = \mathbf{Q}\boldsymbol{y}.$$

Gaussian elimination with partial pivoting solves the Hessenberg system in $O(n^2)$ operations. At each step, the only possible row interchange is between the pivot row and the row directly below it.

Proof Since $\mathbf{Q}\mathbf{Q}^T = \mathbf{I}$, we have $s\mathbf{I} - \mathbf{A} = \mathbf{Q}(s\mathbf{I} - \mathbf{H})\mathbf{Q}^T$, so $\boldsymbol{x} = \mathbf{Q}(s\mathbf{I} - \mathbf{H})^{-1}\mathbf{Q}^T\boldsymbol{b}$. The matrix $s\mathbf{I} - \mathbf{H}$ differs from $-\mathbf{H}$ only on the diagonal, so it is upper Hessenberg. Suppose that after $k-1$ elimination steps the active submatrix is upper Hessenberg. Its first column then has a single nonzero entry below the diagonal, in row $k+1$. Partial pivoting compares only rows k and $k+1$, computes one multiplier, and updates one row of length at most $n - k + 1$. Rows $k+2, \ldots, n$ are untouched, so the next active submatrix is again upper Hessenberg. Summing the work over $k = 1, \ldots, n-1$ gives $O(n^2)$ operations, and the triangular solve that follows costs another $O(n^2)$. $\square$

Reducing $\mathbf{A}$ to Hessenberg form with Householder reflectors costs about $\frac{10}{3}n^3$ flops when $\mathbf{Q}$ is kept as a product of reflectors, and this reduction is done once (Golub and Van Loan,

2013). Applying $\mathbf{Q}^T$ and $\mathbf{Q}$ to a vector costs $O(n^2)$ flops. For N shifts, the total cost is therefore about $\frac{10}{3}n^3 + O(Nn^2)$ flops, compared with about $\frac{2}{3}Nn^3$ for separate factorizations. The reduction pays for itself after five shifts, and with one hundred shifts it reduces the cost by a factor of about twenty.

Remark 4.1 **(Contour integrals for the matrix exponential)** The Cauchy integral formula for matrix functions gives

$$e^{t\mathbf{A}}\boldsymbol{b} = \frac{1}{2\pi \mathrm{i}} \int_{\Gamma} e^{ts}(s\mathbf{I} - \mathbf{A})^{-1}\boldsymbol{b}\, ds,$$

where Γ is a contour that encloses the spectrum of $\mathbf{A}$ (Higham, 2008). When the spectrum lies in the left half-plane, as for Markov generators and discretized diffusion operators, Γ can be chosen to wrap around the negative real axis so that e^{ts} decays along it. After a parametrization of Γ, the trapezoid rule with N nodes approximates the integral by a sum of N resolvent applications, each computed by Thm 4.16. For suitable Talbot-type contours the error decreases geometrically in N, and a few tens of nodes commonly reach double-precision accuracy (Weideman and Trefethen, 2007). The geometric convergence is the same mechanism as the spectral accuracy of the trapezoid rule for periodic analytic integrands in Section 5.3. For large sparse matrices, the Krylov approximations of Section 4.4 are usually preferable, because they avoid the dense Hessenberg reduction.

4.2.5 Exercises

Ex 4.2

1. **Compute. (Observed cubic convergence of RQI)** Apply RQI to a symmetric matrix with distinct eigenvalues, starting from several unit vectors close to the same eigenvector.
   (a) Record the angle errors $\theta_k = \angle(\boldsymbol{v}^{(k)}, \boldsymbol{q}_*)$ and the ratios $\sin\theta_{k+1}/\sin^3\theta_k$.
   (b) Record the Rayleigh errors and verify the cubic relation in Thm 4.11 before rounding error dominates.
   (c) Repeat with two nearly repeated eigenvalues and explain how the spectral gap enters the constants in the proof.

2. **Explore. (Failure of unshifted QR on an exchange matrix)** Let $\mathbf{P} = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}$.
   (a) Compute the QR factorization $\mathbf{P} = \mathbf{Q}_0\mathbf{R}_0$.
   (b) Compute $\mathbf{P}_1 = \mathbf{R}_0\mathbf{Q}_0$ and show that $\mathbf{P}_1 = \mathbf{P}$.
   (c) Repeat the step with a nonzero shift. Determine which shifts break the cycle, and explain how an eigenvalue shift from the trailing two by two block permits immediate deflation in this example.

3. **Compute. (Givens rotation Hessenberg QR)** Write a Python function that executes one step of shifted QR on an upper Hessenberg matrix $\mathbf{H} \in \mathbb{R}^{n\times n}$ using $n-1$ Givens rotations. Verify that your implementation requires $O(n^2)$ flops rather than $O(n^3)$.

4. **Compute.** Perform one Jacobi rotation by hand to zero out the off-diagonal entry of

$$\mathbf{A} = \begin{pmatrix} 3 & 2 \\ 2 & 6 \end{pmatrix}.$$

Explicitly compute τ, t, c, s, and the resulting diagonal matrix $\mathbf{A}'$.

5. **Compute.** Verify that the eigenvalues computed on the diagonal of $\mathbf{A}'$ match the exact roots of $\det(\mathbf{A} - \lambda\mathbf{I}) = 0$.

6. **Derive.** Prove that for an $n \times n$ matrix, if the largest off-diagonal entry is selected at each step (Classical Jacobi), then $\operatorname{off}(\mathbf{A}^{(k+1)})^2 \le \left(1 - \frac{2}{n(n-1)}\right) \operatorname{off}(\mathbf{A}^{(k)})^2$.

7. **Explore. (Many shifted solves)** For a random 200×200 matrix, compare the time to solve $(s_k\mathbf{I} - \mathbf{A})\boldsymbol{x}_k = \boldsymbol{b}$ for $k = 1, \dots, N$ by separate LU factorizations and by one Hessenberg reduction followed by Thm 4.16. Find the value of N at which the two approaches cost the same, and compare it with the flop estimate in the text.

4.3 The Singular Value Decomposition

Several questions from earlier chapters remain open. The Euclidean norm and the condition number of a matrix were defined by optimization problems that we have not yet evaluated, and the least-squares theory of Section 3.2 gave a computable solution only for matrices with full column rank. The singular value decomposition (SVD) answers these questions. It separates a linear map into orthogonal input directions, nonnegative scale factors, and orthogonal output directions, and unlike an eigendecomposition it exists for every rectangular matrix $\mathbf{A} \in \mathbb{R}^{m\times n}$.

Geometrically, the SVD shows that every linear map sends the unit sphere in $\mathbb{R}^n$ to a hyperellipsoid in $\mathbb{R}^m$. The right singular vectors $\boldsymbol{v}_i$ are the orthogonal directions in the domain that are mapped to the semi-axes, the singular values σ_i are the semi-axis lengths, and the left singular vectors $\boldsymbol{u}_i$ are the directions of the semi-axes in the codomain.

We first prove that the SVD exists and use it to identify the four fundamental subspaces. We then compute norms and condition numbers, construct the best low-rank approximation, and solve rank-deficient and regularized least-squares problems. The section ends with the way the SVD is computed in practice.

(Running example: data matrices) In a fitting problem with noisy observations, the singular values rank the directions in parameter space by how strongly the data determine

them. Truncation and regularization treat the weakly determined directions differently, and principal component analysis interprets the same decomposition as variance.

4.3.1 The SVD Theorem and Subspace Characterization

Thm 4.17 **(The Singular Value Decomposition Theorem)** Let $\mathbf{A} \in \mathbb{R}^{m\times n}$ with $m \geq n$. There exist orthogonal matrices $\mathbf{U} \in \mathbb{R}^{m\times m}$ and $\mathbf{V} \in \mathbb{R}^{n\times n}$, and a rectangular diagonal matrix $\mathbf{\Sigma} \in \mathbb{R}^{m\times n}$, such that

$$\mathbf{A} = \mathbf{U}\mathbf{\Sigma}\mathbf{V}^T,$$

where the diagonal entries of $\mathbf{\Sigma} = \operatorname{diag}(\sigma_1, \sigma_2, \ldots, \sigma_n)$ are real, non-negative, and sorted in non-increasing order:

$$\sigma_1 \geq \sigma_2 \geq \cdots \geq \sigma_r > \sigma_{r+1} = \cdots = \sigma_n = 0.$$

The number of strictly positive singular values $r = \operatorname{rank}(\mathbf{A})$ equals the rank of $\mathbf{A}$. In dyadic outer-product form,

$$\mathbf{A} = \sum_{i=1}^{r} \sigma_i \boldsymbol{u}_i \boldsymbol{v}_i^T.$$

Proof Consider the Gram matrix $\mathbf{A}^T\mathbf{A} \in \mathbb{R}^{n\times n}$. Because $(\mathbf{A}^T\mathbf{A})^T = \mathbf{A}^T\mathbf{A}$, it is symmetric, and for every $\boldsymbol{x} \in \mathbb{R}^n$, $\boldsymbol{x}^T(\mathbf{A}^T\mathbf{A})\boldsymbol{x} = \|\mathbf{A}\boldsymbol{x}\|_2^2 \geq 0$, so $\mathbf{A}^T\mathbf{A}$ is positive semidefinite.
By the Spectral Theorem (Thm 4.4), $\mathbf{A}^T\mathbf{A}$ has an orthonormal basis of eigenvectors $\boldsymbol{v}_1, \ldots, \boldsymbol{v}_n \in \mathbb{R}^n$ with corresponding real eigenvalues $\lambda_1 \geq \lambda_2 \geq \cdots \geq \lambda_n \geq 0$. Define the singular values $\sigma_i = \sqrt{\lambda_i} \geq 0$ for $i = 1, \ldots, n$, and let r denote the number of strictly positive singular values, so that $\sigma_r > 0$ and $\sigma_{r+1} = \cdots = \sigma_n = 0$.
For each index $i \in \{1, \ldots, r\}$, define the normalized vectors in $\mathbb{R}^m$:

$$\boldsymbol{u}_i = \frac{1}{\sigma_i}\mathbf{A}\boldsymbol{v}_i.$$

We verify that $\{\boldsymbol{u}_1, \ldots, \boldsymbol{u}_r\}$ form an orthonormal set in $\mathbb{R}^m$:

$$\boldsymbol{u}_i^T\boldsymbol{u}_j = \frac{1}{\sigma_i\sigma_j}(\mathbf{A}\boldsymbol{v}_i)^T(\mathbf{A}\boldsymbol{v}_j) = \frac{1}{\sigma_i\sigma_j}\boldsymbol{v}_i^T(\mathbf{A}^T\mathbf{A}\boldsymbol{v}_j) = \frac{\lambda_j}{\sigma_i\sigma_j}\boldsymbol{v}_i^T\boldsymbol{v}_j = \frac{\sigma_j^2}{\sigma_i\sigma_j}\delta_{ij} = \delta_{ij}.$$

Using Gram–Schmidt orthonormalization, extend $\{\boldsymbol{u}_1, \ldots, \boldsymbol{u}_r\}$ to a complete orthonormal basis $\{\boldsymbol{u}_1, \ldots, \boldsymbol{u}_m\}$ for $\mathbb{R}^m$. Assemble the orthogonal matrices $\mathbf{U} = [\boldsymbol{u}_1, \ldots, \boldsymbol{u}_m] \in \mathbb{R}^{m\times m}$ and $\mathbf{V} = [\boldsymbol{v}_1, \ldots, \boldsymbol{v}_n] \in \mathbb{R}^{n\times n}$.
Evaluating the (i, j) entry of $\mathbf{U}^T\mathbf{A}\mathbf{V}$:

$$(\mathbf{U}^T\mathbf{A}\mathbf{V})_{ij} = \boldsymbol{u}_i^T\mathbf{A}\boldsymbol{v}_j.$$

For $j \le r$, $\mathbf{A}\boldsymbol{v}_j = \sigma_j \boldsymbol{u}_j$, so $\boldsymbol{u}_i^T \mathbf{A}\boldsymbol{v}_j = \sigma_j \boldsymbol{u}_i^T \boldsymbol{u}_j = \sigma_j \delta_{ij}$. For $j > r$, $\lambda_j = 0$, so $\|\mathbf{A}\boldsymbol{v}_j\|_2^2 = \boldsymbol{v}_j^T \mathbf{A}^T \mathbf{A}\boldsymbol{v}_j = 0$, which implies $\mathbf{A}\boldsymbol{v}_j = \mathbf{0}$ and $\boldsymbol{u}_i^T \mathbf{A}\boldsymbol{v}_j = 0$. Thus $\mathbf{U}^T\mathbf{A}\mathbf{V} = \mathbf{\Sigma}$, which gives $\mathbf{A} = \mathbf{U}\mathbf{\Sigma}\mathbf{V}^T$. □

Thm 4.18 **(Thin SVD and the Four Fundamental Subspaces)** Let $\mathbf{A} \in \mathbb{R}^{m \times n}$ have rank $r \le n \le m$. The thin SVD is

$$\mathbf{A} = \mathbf{U}_1 \mathbf{\Sigma}_1 \mathbf{V}_1^T,$$

where $\mathbf{U}_1 \in \mathbb{R}^{m \times r}$ and $\mathbf{V}_1 \in \mathbb{R}^{n \times r}$ have orthonormal columns, and $\mathbf{\Sigma}_1 = \operatorname{diag}(\sigma_1, \ldots, \sigma_r) \in \mathbb{R}^{r \times r}$ has strictly positive entries. The complete singular vectors partition the domain and codomain into the four fundamental subspaces:

1. **Column space (range):** $\mathcal{R}(\mathbf{A}) = \operatorname{span}\{\boldsymbol{u}_1, \ldots, \boldsymbol{u}_r\} \subset \mathbb{R}^m$.
2. **Left null space:** $\mathcal{N}(\mathbf{A}^T) = \operatorname{span}\{\boldsymbol{u}_{r+1}, \ldots, \boldsymbol{u}_m\} \subset \mathbb{R}^m$.
3. **Row space:** $\mathcal{R}(\mathbf{A}^T) = \operatorname{span}\{\boldsymbol{v}_1, \ldots, \boldsymbol{v}_r\} \subset \mathbb{R}^n$.
4. **Null space (kernel):** $\mathcal{N}(\mathbf{A}) = \operatorname{span}\{\boldsymbol{v}_{r+1}, \ldots, \boldsymbol{v}_n\} \subset \mathbb{R}^n$.

Proof From $\mathbf{A} = \mathbf{U}\mathbf{\Sigma}\mathbf{V}^T$, we have $\mathbf{A}\boldsymbol{v}_i = \sigma_i \boldsymbol{u}_i$ for $i \le r$ and $\mathbf{A}\boldsymbol{v}_i = \mathbf{0}$ for $i > r$. Any $\boldsymbol{x} \in \mathbb{R}^n$ expands uniquely as $\boldsymbol{x} = \sum_{i=1}^n (\boldsymbol{v}_i^T \boldsymbol{x}) \boldsymbol{v}_i$. Applying $\mathbf{A}$ gives

$$\mathbf{A}\boldsymbol{x} = \sum_{i=1}^{r} \sigma_i (\boldsymbol{v}_i^T \boldsymbol{x}) \boldsymbol{u}_i.$$

This expresses $\mathbf{A}\boldsymbol{x}$ as a linear combination of $\{\boldsymbol{u}_1, \ldots, \boldsymbol{u}_r\}$, proving $\mathcal{R}(\mathbf{A}) = \operatorname{span}\{\boldsymbol{u}_1, \ldots, \boldsymbol{u}_r\}$. Furthermore, $\mathbf{A}\boldsymbol{x} = \mathbf{0}$ if and only if $\boldsymbol{v}_i^T \boldsymbol{x} = 0$ for all $i \le r$, which proves that $\mathcal{N}(\mathbf{A}) = \operatorname{span}\{\boldsymbol{v}_{r+1}, \ldots, \boldsymbol{v}_n\}$. Repeating the argument for $\mathbf{A}^T = \mathbf{V}\mathbf{\Sigma}^T\mathbf{U}^T$ completes the characterization for $\mathcal{R}(\mathbf{A}^T)$ and $\mathcal{N}(\mathbf{A}^T)$. □

4.3.2 Norms and Condition Numbers

The induced Euclidean norm of Def 1.21 is the largest amplification of a unit vector, and the condition number of Def 1.24 compares the largest and the smallest amplification. Both were defined as optimization problems over the unit sphere. Because orthogonal matrices preserve Euclidean length, the SVD reduces these problems to a diagonal matrix, where they can be solved by inspection.

Thm 4.19 **(Norms and Condition Numbers from the SVD)** Let $\mathbf{A} \in \mathbb{R}^{m \times n}$ with $m \ge n$ have singular values $\sigma_1 \ge \cdots \ge \sigma_n \ge 0$. Then

$$\|\mathbf{A}\|_2 = \sigma_1, \qquad \min_{\|\boldsymbol{x}\|_2 = 1} \|\mathbf{A}\boldsymbol{x}\|_2 = \sigma_n, \qquad \|\mathbf{A}\|_F = \Big(\sum_{i=1}^{n} \sigma_i^2 \Big)^{1/2}.$$

If $\mathbf{A}$ has full column rank, we define $\kappa_2(\mathbf{A}) = \sigma_1/\sigma_n$. The condition number satisfies

$$\kappa_2(\mathbf{A}^T\mathbf{A}) = \kappa_2(\mathbf{A})^2.$$

If $\mathbf{A}$ is square and nonsingular, then $\|\mathbf{A}^{-1}\|_2 = 1/\sigma_n$, so $\kappa_2(\mathbf{A}) = \|\mathbf{A}\|_2\|\mathbf{A}^{-1}\|_2$ agrees with Def 1.24.

Proof Let $\boldsymbol{x}$ be a unit vector and let $\boldsymbol{y} = \mathbf{V}^T\boldsymbol{x}$, which is also a unit vector. Since $\mathbf{U}$ preserves Euclidean length,

$$\|\mathbf{A}\boldsymbol{x}\|_2^2 = \|\mathbf{U}\mathbf{\Sigma}\mathbf{V}^T\boldsymbol{x}\|_2^2 = \|\mathbf{\Sigma}\boldsymbol{y}\|_2^2 = \sum_{i=1}^{n} \sigma_i^2 y_i^2.$$

The sum is a weighted average of the σ_i^2, so it lies between σ_n^2 and σ_1^2. The upper value is attained at $\boldsymbol{y} = \boldsymbol{e}_1$, that is, at $\boldsymbol{x} = \boldsymbol{v}_1$, and the lower value is attained at $\boldsymbol{x} = \boldsymbol{v}_n$. These two cases prove the first two identities.
For the Frobenius norm, $\|\mathbf{A}\|_F^2 = \operatorname{trace}(\mathbf{A}^T\mathbf{A})$ and $\mathbf{A}^T\mathbf{A} = \mathbf{V}\mathbf{\Sigma}^T\mathbf{\Sigma}\mathbf{V}^T$. The trace is invariant under the similarity by $\mathbf{V}$, so $\|\mathbf{A}\|_F^2 = \operatorname{trace}(\mathbf{\Sigma}^T\mathbf{\Sigma}) = \sum_i \sigma_i^2$.
The factorization $\mathbf{A}^T\mathbf{A} = \mathbf{V}(\mathbf{\Sigma}^T\mathbf{\Sigma})\mathbf{V}^T$ is itself an SVD of $\mathbf{A}^T\mathbf{A}$ with singular values $\sigma_1^2 \geq \cdots \geq \sigma_n^2$. When $\mathbf{A}$ has full column rank these are positive, and $\kappa_2(\mathbf{A}^T\mathbf{A}) = \sigma_1^2/\sigma_n^2 = \kappa_2(\mathbf{A})^2$. Finally, if $\mathbf{A}$ is square and nonsingular, then $\mathbf{A}^{-1} = \mathbf{V}\mathbf{\Sigma}^{-1}\mathbf{U}^T$ is an SVD whose largest singular value is $1/\sigma_n$. □

Cor 4.20 **(Symmetric Positive Definite Matrices)** If $\mathbf{A}$ is symmetric positive definite, its singular values are its eigenvalues, and $\kappa_2(\mathbf{A}) = \lambda_{\max}(\mathbf{A})/\lambda_{\min}(\mathbf{A})$. If $\mathbf{A} = \mathbf{L}\mathbf{L}^T$ is the Cholesky factorization of Thm 2.18, then $\kappa_2(\mathbf{L}) = \sqrt{\kappa_2(\mathbf{A})}$.

Proof By Thm 4.4, $\mathbf{A} = \mathbf{Q}\mathbf{\Lambda}\mathbf{Q}^T$ with $\mathbf{Q}$ orthogonal and positive diagonal $\mathbf{\Lambda}$. After the eigenvalues are sorted in decreasing order, this factorization is an SVD, which proves the first claim. For the second, $\mathbf{A} = (\mathbf{L}^T)^T\mathbf{L}^T$, so the argument in the proof of Thm 4.19 shows that the singular values of $\mathbf{L}^T$ are the square roots of the eigenvalues of $\mathbf{A}$. Transposition does not change singular values, so $\kappa_2(\mathbf{L}) = \kappa_2(\mathbf{L}^T) = \sqrt{\kappa_2(\mathbf{A})}$. □

The identity $\kappa_2(\mathbf{A}^T\mathbf{A}) = \kappa_2(\mathbf{A})^2$ explains the preference for QR in Section 3.2. When the normal equations are formed and solved by a backward stable method, the forward error can be as large as a modest multiple of $\kappa_2(\mathbf{A})^2 u$, where u is the unit roundoff. Householder QR works with $\mathbf{A}$ directly, and when the least-squares residual is small its forward error is a modest multiple of $\kappa_2(\mathbf{A})u$ (Higham, 2002). For $\kappa_2(\mathbf{A}) = 10^6$ in double precision, these two estimates are of order 10^{-4} and 10^{-10}.

4.3.3 Low-Rank Matrix Approximation

Thm 4.21 **(Eckart–Young–Mirsky Low-Rank Approximation Theorem)** Let $\mathbf{A} \in \mathbb{R}^{m\times n}$ have singular value decomposition $\mathbf{A} = \sum_{i=1}^{r} \sigma_i \boldsymbol{u}_i \boldsymbol{v}_i^T$. For any $k < r$, let the rank-k truncated

SVD be

$$\mathbf{A}_k = \sum_{i=1}^{k} \sigma_i \boldsymbol{u}_i \boldsymbol{v}_i^T.$$

Then $\mathbf{A}_k$ is the best rank-k approximation to $\mathbf{A}$ in the spectral norm:

$$\min_{\operatorname{rank}(\mathbf{B})\le k} \|\mathbf{A}-\mathbf{B}\|_2 = \|\mathbf{A}-\mathbf{A}_k\|_2 = \sigma_{k+1},$$

and in the Frobenius norm:

$$\min_{\operatorname{rank}(\mathbf{B})\le k} \|\mathbf{A}-\mathbf{B}\|_F = \|\mathbf{A}-\mathbf{A}_k\|_F = \sqrt{\sum_{j=k+1}^{r} \sigma_j^2}.$$

Proof First, evaluating the spectral norm of $\mathbf{A}-\mathbf{A}_k$:

$$\mathbf{A}-\mathbf{A}_k = \sum_{i=k+1}^{r} \sigma_i \boldsymbol{u}_i \boldsymbol{v}_i^T = \mathbf{U}\,\operatorname{diag}(0,\dots,0,\sigma_{k+1},\dots,\sigma_r,0,\dots,0)\mathbf{V}^T.$$

Because the 2-norm is unitarily invariant, $\|\mathbf{A}-\mathbf{A}_k\|_2 = \sigma_{k+1}$.
Now let $\mathbf{B} \in \mathbb{R}^{m\times n}$ be any arbitrary matrix with $\operatorname{rank}(\mathbf{B}) \le k$. By the Rank-Nullity Theorem, $\dim \mathcal{N}(\mathbf{B}) = n - \operatorname{rank}(\mathbf{B}) \ge n-k$. Define the $(k+1)$-dimensional subspace $W_{k+1} = \operatorname{span}\{\boldsymbol{v}_1,\dots,\boldsymbol{v}_{k+1}\} \subset \mathbb{R}^n$. By the dimension formula,

$$\dim(\mathcal{N}(\mathbf{B}) \cap W_{k+1}) = \dim \mathcal{N}(\mathbf{B}) + \dim W_{k+1} - \dim(\mathcal{N}(\mathbf{B}) + W_{k+1}) \ge (n-k) + (k+1) - n = 1.$$

Therefore, there exists a nonzero vector $\boldsymbol{w} \in \mathcal{N}(\mathbf{B}) \cap W_{k+1}$, normalized so that $\|\boldsymbol{w}\|_2 = 1$.
Because $\boldsymbol{w} \in \mathcal{N}(\mathbf{B})$, $\mathbf{B}\boldsymbol{w} = \mathbf{0}$. Expanding $\boldsymbol{w} = \sum_{j=1}^{k+1} c_j \boldsymbol{v}_j$ with $\sum_{j=1}^{k+1} c_j^2 = 1$,

$$\begin{aligned}
\|(\mathbf{A}-\mathbf{B})\boldsymbol{w}\|_2^2 &= \|\mathbf{A}\boldsymbol{w} - \mathbf{B}\boldsymbol{w}\|_2^2 = \|\mathbf{A}\boldsymbol{w}\|_2^2 \\
&= \left\| \mathbf{A} \sum_{j=1}^{k+1} c_j \boldsymbol{v}_j \right\|_2^2 = \left\| \sum_{j=1}^{k+1} c_j \sigma_j \boldsymbol{u}_j \right\|_2^2 \\
&= \sum_{j=1}^{k+1} c_j^2 \sigma_j^2 \ge \sigma_{k+1}^2 \sum_{j=1}^{k+1} c_j^2 = \sigma_{k+1}^2.
\end{aligned}$$

Consequently, the operator norm satisfies

$$\|\mathbf{A}-\mathbf{B}\|_2 = \max_{\|\boldsymbol{x}\|_2=1} \|(\mathbf{A}-\mathbf{B})\boldsymbol{x}\|_2 \ge \|(\mathbf{A}-\mathbf{B})\boldsymbol{w}\|_2 \ge \sigma_{k+1}.$$

This proves that $\|\mathbf{A}-\mathbf{B}\|_2 \ge \sigma_{k+1}$ for every matrix $\mathbf{B}$ of rank at most k.
For the Frobenius norm, let $\mathbf{P}$ be the orthogonal projector onto $\mathcal{N}(\mathbf{B})$. Since $\operatorname{rank}(\mathbf{B}) \le k$, this null space has dimension at least $n-k$. Hence

$$\|\mathbf{A}-\mathbf{B}\|_F^2 \ge \|(\mathbf{A}-\mathbf{B})\mathbf{P}\|_F^2 = \|\mathbf{A}\mathbf{P}\|_F^2.$$

Choose an $(n-k)$-dimensional subspace of $\mathcal{N}(\mathbf{B})$ if the null space is larger, and let $\mathbf{P}$ project onto that subspace. Expanding in the right singular-vector basis gives

$$\|\mathbf{AP}\|_F^2 = \operatorname{tr}(\mathbf{P}^T\mathbf{A}^T\mathbf{AP}) = \operatorname{tr}\left(\mathbf{P}\left(\sum_{i=1}^{n}\sigma_i^2\boldsymbol{v}_i\boldsymbol{v}_i^T\right)\mathbf{P}\right)$$
$$= \sum_{i=1}^{n}\sigma_i^2\operatorname{tr}(\mathbf{P}\boldsymbol{v}_i\boldsymbol{v}_i^T\mathbf{P}) = \sum_{i=1}^{n}\sigma_i^2\|\mathbf{P}\boldsymbol{v}_i\|_2^2.$$

The weights $d_i = \|\mathbf{P}\boldsymbol{v}_i\|_2^2$ satisfy $0 \le d_i \le 1$ and $\sum_i d_i = \operatorname{tr}(\mathbf{P}) = n-k$. The weighted sum is minimized by placing unit weight on the $n-k$ smallest singular values. Therefore

$$\|\mathbf{A}-\mathbf{B}\|_F^2 \ge \sum_{i=k+1}^{r}\sigma_i^2.$$

The truncated SVD attains this bound, completing the proof. □

Remark 4.2 **(Numerical rank and noise filtering)** In measured or discretized data, singular values rarely vanish exactly. Given a tolerance $\tau > 0$, define the numerical rank by

$$r_\tau = \#\{i : \sigma_i > \tau\}.$$

A common floating-point default is $\tau = \varepsilon_{\text{mach}}\max(m,n)\sigma_1$, but a scientifically meaningful threshold should reflect the noise and discretization scales. Truncation removes components below the chosen threshold. Calling those components noise requires a model of the data.

4.3.4 Least Squares and Regularization

The least-squares theory of Section 3.2 characterized every minimizer and the minimum-norm minimizer, but it computed a solution only when $\mathbf{A}$ has full column rank. The SVD computes all minimizers in every case. It also shows which components of the solution are sensitive to errors in $\boldsymbol{b}$, and how truncation and Tikhonov regularization modify those components.

Def 4.11 **(Moore–Penrose Pseudoinverse)** Let $\mathbf{A} \in \mathbb{R}^{m\times n}$ have rank r and thin SVD $\mathbf{A} = \mathbf{U}_1\mathbf{\Sigma}_1\mathbf{V}_1^T$. The Moore–Penrose pseudoinverse of $\mathbf{A}$ is

$$\mathbf{A}^\dagger = \mathbf{V}_1\mathbf{\Sigma}_1^{-1}\mathbf{U}_1^T = \sum_{i=1}^{r}\frac{1}{\sigma_i}\boldsymbol{v}_i\boldsymbol{u}_i^T \in \mathbb{R}^{n\times m}.$$

Thm 4.22 **(Least Squares by the SVD)** Let $\mathbf{A} = \mathbf{U}\mathbf{\Sigma}\mathbf{V}^T$ have rank r. A vector $\boldsymbol{x}$ minimizes $\|\mathbf{A}\boldsymbol{x}-\boldsymbol{b}\|_2$ if and only if

$$\boldsymbol{v}_i^T\boldsymbol{x} = \frac{\boldsymbol{u}_i^T\boldsymbol{b}}{\sigma_i}, \qquad i = 1,\dots,r.$$

The minimum residual norm is $\left(\sum_{i=r+1}^{m}(\boldsymbol{u}_i^T\boldsymbol{b})^2\right)^{1/2}$, and the unique minimizer of smallest Euclidean norm is

$$\mathbf{A}^\dagger\boldsymbol{b} = \sum_{i=1}^{r}\frac{\boldsymbol{u}_i^T\boldsymbol{b}}{\sigma_i}\,\boldsymbol{v}_i.$$

Proof Let $\boldsymbol{y} = \mathbf{V}^T\boldsymbol{x}$ and $\boldsymbol{c} = \mathbf{U}^T\boldsymbol{b}$. Since $\mathbf{U}^T$ preserves Euclidean length,

$$\|\mathbf{A}\boldsymbol{x}-\boldsymbol{b}\|_2^2 = \|\mathbf{U}^T(\mathbf{U}\boldsymbol{\Sigma}\mathbf{V}^T\boldsymbol{x}-\boldsymbol{b})\|_2^2 = \|\boldsymbol{\Sigma}\boldsymbol{y}-\boldsymbol{c}\|_2^2 = \sum_{i=1}^{r}(\sigma_i y_i - c_i)^2 + \sum_{i=r+1}^{m} c_i^2.$$

The second sum does not depend on $\boldsymbol{x}$. The first sum vanishes exactly when $y_i = c_i/\sigma_i$ for $i \le r$, which proves the characterization and the value of the minimum. The components $y_{r+1},\dots,y_n$ are unrestricted. Because $\|\boldsymbol{x}\|_2 = \|\boldsymbol{y}\|_2$, the norm is smallest when these components are zero, and the resulting vector $\boldsymbol{x} = \mathbf{V}\boldsymbol{y}$ is $\mathbf{A}^\dagger\boldsymbol{b}$. □

The coefficient $\boldsymbol{u}_i^T\boldsymbol{b}/\sigma_i$ shows how errors in the data enter the solution. If $\boldsymbol{b}$ is replaced by $\boldsymbol{b}+\boldsymbol{e}$, the component of the minimum-norm solution along $\boldsymbol{v}_i$ changes by $\boldsymbol{u}_i^T\boldsymbol{e}/\sigma_i$. Directions with small singular values amplify the noise by $1/\sigma_i$, which is the mechanism behind the ill-conditioning measured by $\kappa_2(\mathbf{A})$. Regularization reduces this amplification by modifying the coefficients for small σ_i.

Thm 4.23 **(Filter Factors for Truncation and Tikhonov Regularization)** Let $\mathbf{A}$ have rank r and SVD $\mathbf{A} = \sum_{i=1}^{r}\sigma_i\boldsymbol{u}_i\boldsymbol{v}_i^T$. For $k \le r$, the minimum-norm least-squares solution for the truncated matrix $\mathbf{A}_k$ of Thm 4.21 is

$$\boldsymbol{x}_k = \mathbf{A}_k^\dagger\boldsymbol{b} = \sum_{i=1}^{k}\frac{\boldsymbol{u}_i^T\boldsymbol{b}}{\sigma_i}\,\boldsymbol{v}_i.$$

For $\lambda > 0$, the Tikhonov solution of Def 3.6 is

$$\boldsymbol{x}_\lambda = \sum_{i=1}^{r}\frac{\sigma_i^2}{\sigma_i^2+\lambda}\,\frac{\boldsymbol{u}_i^T\boldsymbol{b}}{\sigma_i}\,\boldsymbol{v}_i.$$

Proof The truncated matrix $\mathbf{A}_k$ has rank k and SVD $\sum_{i=1}^{k}\sigma_i\boldsymbol{u}_i\boldsymbol{v}_i^T$, so Thm 4.22 gives $\boldsymbol{x}_k$. For the Tikhonov solution, substitute $\mathbf{A} = \mathbf{U}\boldsymbol{\Sigma}\mathbf{V}^T$ into the normal equations $(\mathbf{A}^T\mathbf{A}+\lambda\mathbf{I})\boldsymbol{x} = \mathbf{A}^T\boldsymbol{b}$. This gives

$$\mathbf{V}(\boldsymbol{\Sigma}^T\boldsymbol{\Sigma}+\lambda\mathbf{I})\mathbf{V}^T\boldsymbol{x} = \mathbf{V}\boldsymbol{\Sigma}^T\mathbf{U}^T\boldsymbol{b}.$$

Multiplying by $\mathbf{V}^T$ and solving the diagonal system gives $\boldsymbol{v}_i^T\boldsymbol{x}_\lambda = \sigma_i\,\boldsymbol{u}_i^T\boldsymbol{b}/(\sigma_i^2+\lambda)$, which is zero for $i > r$. Writing $\sigma_i/(\sigma_i^2+\lambda)$ as $\sigma_i^2/(\sigma_i^2+\lambda)$ times $1/\sigma_i$ gives the stated form. □

Both regularized solutions multiply the unregularized coefficient $\boldsymbol{u}_i^T\boldsymbol{b}/\sigma_i$ by a filter factor. Truncation uses the factor one for $i \le k$ and zero for $i > k$, so it removes the weak

directions entirely. Tikhonov regularization uses $\sigma_i^2/(\sigma_i^2+\lambda)$, which is close to one when $\sigma_i^2 \gg \lambda$ and close to σ_i^2/λ when $\sigma_i^2 \ll \lambda$. The amplification $1/\sigma_i$ is therefore replaced by at most $1/(2\sqrt{\lambda})$, and the transition between kept and damped directions occurs near $\sigma_i = \sqrt{\lambda}$. The choice between the two methods, and the choice of k or λ, depends on the noise level in $\boldsymbol{b}$ and on the assumption about the solution discussed after Def 3.6.

Remark 4.3 **(Computing the SVD)** The proof of Thm 4.17 constructs the SVD from the eigendecomposition of $\mathbf{A}^T\mathbf{A}$, but practical codes do not compute it that way. A backward stable eigensolver applied to $\mathbf{A}^T\mathbf{A}$ computes its eigenvalues with absolute errors of order $u\sigma_1^2$, so singular values below about $\sqrt{u}\,\sigma_1$ lose all relative accuracy. Golub and Kahan instead reduce $\mathbf{A}$ to an upper bidiagonal matrix $\mathbf{B} = \mathbf{U}_0^T\mathbf{A}\mathbf{V}_0$ by applying Householder reflections alternately from the left and from the right (Golub and Kahan, 1965). A shifted QR iteration applied implicitly to $\mathbf{B}$ then computes the singular values, and standard dense libraries follow this two-phase approach. For a large sparse matrix, the bidiagonal matrix can instead be built one column at a time by a recurrence that uses only products with $\mathbf{A}$ and $\mathbf{A}^T$. The recurrence is closely related to the Lanczos iteration of Section 4.5 applied to $\mathbf{A}^T\mathbf{A}$.

4.3.5 Exercises

Ex 4.3

1. **Compute. (Hand computation of full and thin SVD)** For $\mathbf{A} = \begin{pmatrix} 3 & 3 \\ 4 & 4 \end{pmatrix}$:
   (a) Compute $\mathbf{A}^T\mathbf{A}$ and find its eigenvalues λ_1, λ_2 and orthonormal eigenvectors $\boldsymbol{v}_1, \boldsymbol{v}_2$.
   (b) Compute σ_1, σ_2 and the left singular vector $\boldsymbol{u}_1 = \frac{1}{\sigma_1}\mathbf{A}\boldsymbol{v}_1$.
   (c) Complete $\{\boldsymbol{u}_1\}$ to an orthonormal basis of $\mathbb{R}^2$ to find $\boldsymbol{u}_2$.
   (d) Write down the full SVD $\mathbf{A} = \mathbf{U}\mathbf{\Sigma}\mathbf{V}^T$ and verify that $\mathbf{U}\mathbf{\Sigma}\mathbf{V}^T = \mathbf{A}$.

2. **Compute. (Truncation versus Tikhonov regularization)** Let $\mathbf{A} = \mathrm{diag}(1, 10^{-2}, 10^{-4})$, let $\boldsymbol{x}_* = (1,1,1)^T$, and let $\boldsymbol{b} = \mathbf{A}\boldsymbol{x}_* + 10^{-3}(1,-1,1)^T$.
   (a) Compute the least-squares solution $\mathbf{A}^{-1}\boldsymbol{b}$ and its error $\|\mathbf{A}^{-1}\boldsymbol{b} - \boldsymbol{x}_*\|_2$. Identify the component responsible for the error using the coefficients $\boldsymbol{u}_i^T\boldsymbol{b}/\sigma_i$.
   (b) Compute the truncated solutions $\boldsymbol{x}_1, \boldsymbol{x}_2, \boldsymbol{x}_3$ of Thm 4.23 and their errors.
   (c) Compute the Tikhonov solutions for $\lambda = 10^{-2}, 10^{-4}, 10^{-6}$. For each λ, list the three filter factors and explain which components are kept and which are damped.

3. **Compute. (Low-rank error accounting)** For a diagonal matrix $\mathbf{A} = \mathrm{diag}(9, 4, 1)$, compute the best rank-one and rank-two approximations. Verify both the spectral- and Frobenius-norm errors predicted by Thm 4.21.

4. **Derive. (Verification of the four Moore–Penrose conditions)** Using the thin SVD $\mathbf{A} = \mathbf{U}_1\mathbf{\Sigma}_1\mathbf{V}_1^T$ and $\mathbf{A}^\dagger = \mathbf{V}_1\mathbf{\Sigma}_1^{-1}\mathbf{U}_1^T$, prove algebraically:

   (a) $\mathbf{A}\mathbf{A}^\dagger\mathbf{A} = \mathbf{A}$.

   (b) $\mathbf{A}^\dagger\mathbf{A}\mathbf{A}^\dagger = \mathbf{A}^\dagger$.

   (c) $(\mathbf{A}\mathbf{A}^\dagger)^T = \mathbf{A}\mathbf{A}^\dagger$ (orthogonal projector onto $\mathcal{R}(\mathbf{A})$).

   (d) $(\mathbf{A}^\dagger\mathbf{A})^T = \mathbf{A}^\dagger\mathbf{A}$ (orthogonal projector onto $\mathcal{R}(\mathbf{A}^T)$).

5. **Compute. (SVD image compression and energy decay)** In Python, load a grayscale image as a matrix $\mathbf{A} \in \mathbb{R}^{m\times n}$.

   (a) Compute the full SVD using `scipy.linalg.svd`.

   (b) Plot the singular value spectrum σ_i on a semilog scale and the cumulative energy ratio $E(k) = \frac{\sum_{i=1}^{k}\sigma_i^2}{\sum_{i=1}^{r}\sigma_i^2}$ as a function of k.

   (c) Reconstruct the image at ranks $k \in \{5, 20, 50\}$. Compare memory storage against the original image.

4.4 Krylov Subspaces and the Arnoldi Process

The QR algorithm of Thm 4.13 computes every eigenvalue of a dense matrix in $O(n^3)$ operations. Many applications instead require a few eigenvalues of a large sparse matrix, and the matrix is often available only through the product $\boldsymbol{v} \mapsto \mathbf{A}\boldsymbol{v}$. Power iteration (Def 4.5) uses only such products, but at step k it retains only the last vector $\mathbf{A}^{k-1}\boldsymbol{v}$ and discards the earlier ones.

The Arnoldi process keeps the entire sequence $\boldsymbol{v}, \mathbf{A}\boldsymbol{v}, \ldots, \mathbf{A}^{k-1}\boldsymbol{v}$. We first define the subspace spanned by this sequence and explain why its natural basis is unusable in floating-point arithmetic. We then construct an orthonormal basis one vector at a time and derive the Arnoldi relation. The same relation underlies the Lanczos algorithm in the next section, the GMRES method for linear systems, and the Krylov approximation of the matrix exponential.

4.4.1 From Power Iteration to Krylov Subspaces

Def 4.12 **(Krylov Subspace)** Let $\mathbf{A} \in \mathbb{R}^{n\times n}$ and let $\boldsymbol{v} \in \mathbb{R}^n$ be nonzero. The kth Krylov subspace generated by $\mathbf{A}$ and $\boldsymbol{v}$ is

$$\mathcal{K}_k(\mathbf{A}, \boldsymbol{v}) = \operatorname{span}\{\boldsymbol{v}, \mathbf{A}\boldsymbol{v}, \ldots, \mathbf{A}^{k-1}\boldsymbol{v}\}.$$

Equivalently, $\mathcal{K}_k(\mathbf{A}, \boldsymbol{v}) = \{p(\mathbf{A})\boldsymbol{v} : p \in \mathcal{P}_{k-1}\}$, where $\mathcal{P}_{k-1}$ denotes the polynomials of degree at most $k-1$.

The kth power iterate is a multiple of $\mathbf{A}^{k-1}\boldsymbol{v}$, so it lies in $\mathcal{K}_k(\mathbf{A},\boldsymbol{v})$. Power iteration therefore selects one particular vector from the Krylov subspace, namely the one produced by the polynomial $p(t)=t^{k-1}$. Any method that searches the whole subspace can do at least as well with the same number of matrix–vector products. Section 4.5 makes this comparison precise for symmetric matrices.

Lem 4.24 **(Krylov Subspaces Stop Growing at an Invariant Subspace)** If $\mathcal{K}_{k+1}(\mathbf{A},\boldsymbol{v}) = \mathcal{K}_k(\mathbf{A},\boldsymbol{v})$, then $\mathcal{K}_k(\mathbf{A},\boldsymbol{v})$ is invariant under $\mathbf{A}$ in the sense of Def 4.2, and $\mathcal{K}_j(\mathbf{A},\boldsymbol{v}) = \mathcal{K}_k(\mathbf{A},\boldsymbol{v})$ for every $j \geq k$.

Proof Every vector in $\mathcal{K}_k$ has the form $p(\mathbf{A})\boldsymbol{v}$ with $\deg p \leq k-1$. Multiplying by $\mathbf{A}$ gives $\mathbf{A}p(\mathbf{A})\boldsymbol{v}$, which is a polynomial of degree at most k in $\mathbf{A}$ applied to $\boldsymbol{v}$, and therefore lies in $\mathcal{K}_{k+1}$. Hence $\mathbf{A}\mathcal{K}_k \subseteq \mathcal{K}_{k+1} = \mathcal{K}_k$, which is the invariance claim. Since $\mathcal{K}_{k+2} = \mathcal{K}_{k+1} + \mathbf{A}\mathcal{K}_{k+1} = \mathcal{K}_k + \mathbf{A}\mathcal{K}_k = \mathcal{K}_k$, induction gives the second claim. □

Remark 4.4 **(Ill-conditioned monomial basis)** The vectors $\boldsymbol{v}, \mathbf{A}\boldsymbol{v}, \ldots, \mathbf{A}^{k-1}\boldsymbol{v}$ form a basis of $\mathcal{K}_k$ whenever the subspace has dimension k, but the basis is useless in practice. By the argument behind power iteration, the normalized vectors $\mathbf{A}^j\boldsymbol{v}/\|\mathbf{A}^j\boldsymbol{v}\|_2$ converge to the dominant eigenvector, so the later columns of the basis matrix become nearly parallel. Its condition number grows geometrically with k, and rounding errors destroy the information carried by the smaller components.

4.4.2 The Arnoldi Process

The remedy is to orthonormalize the Krylov sequence as it is generated. If $\boldsymbol{v}_1, \ldots, \boldsymbol{v}_j$ is an orthonormal basis of $\mathcal{K}_j$, we obtain a new direction by multiplying the latest basis vector by $\mathbf{A}$ instead of forming $\mathbf{A}^j\boldsymbol{v}$. We then remove its components along the existing basis by Gram–Schmidt orthogonalization (Thm 3.3).

Def 4.13 **(The Arnoldi Iteration)** Let $\mathbf{A} \in \mathbb{R}^{n\times n}$ and let $\boldsymbol{v} \in \mathbb{R}^n$ be nonzero. Set $\boldsymbol{v}_1 = \boldsymbol{v}/\|\boldsymbol{v}\|_2$. For $j = 1, 2, \ldots, k$:

1. Compute $\boldsymbol{w} = \mathbf{A}\boldsymbol{v}_j$.
2. For $i = 1, \ldots, j$, compute $h_{ij} = \boldsymbol{v}_i^T\boldsymbol{w}$ and update $\boldsymbol{w} \leftarrow \boldsymbol{w} - h_{ij}\boldsymbol{v}_i$.
3. Compute $h_{j+1,j} = \|\boldsymbol{w}\|_2$.
4. If $h_{j+1,j} = 0$, stop. Otherwise, set $\boldsymbol{v}_{j+1} = \boldsymbol{w}/h_{j+1,j}$.

Step 2 is modified Gram–Schmidt. The coefficients h_{ij} are collected in the $k \times k$ upper Hessenberg matrix $\mathbf{H}_k$ and in the $(k+1)\times k$ matrix $\bar{\mathbf{H}}_k$, which is $\mathbf{H}_k$ with the additional row $h_{k+1,k}\boldsymbol{e}_k^T$.

Thm 4.25 **(The Arnoldi Relation)** Suppose the Arnoldi iteration completes k steps with $h_{j+1,j} \neq 0$ for $j = 1, \ldots, k$, and let $\mathbf{V}_k = (\boldsymbol{v}_1 \ \cdots \ \boldsymbol{v}_k)$. Then:

1. The columns of $\mathbf{V}_k$ are orthonormal and span $\mathcal{K}_k(\mathbf{A}, \boldsymbol{v})$.

2. The matrices satisfy

$$\mathbf{A}\mathbf{V}_k = \mathbf{V}_k\mathbf{H}_k + h_{k+1,k}\boldsymbol{v}_{k+1}\boldsymbol{e}_k^T = \mathbf{V}_{k+1}\bar{\mathbf{H}}_k. \tag{1}$$

3. The Hessenberg matrix is the projection of $\mathbf{A}$ onto the Krylov subspace, $\mathbf{H}_k = \mathbf{V}_k^T\mathbf{A}\mathbf{V}_k$.

Proof Step 2 of the iteration subtracts from $\boldsymbol{w}$ its components along $\boldsymbol{v}_1, \ldots, \boldsymbol{v}_j$ one at a time. After the subtraction, $\boldsymbol{w}$ is orthogonal to each of these vectors, and normalizing it gives a unit vector $\boldsymbol{v}_{j+1}$ orthogonal to $\boldsymbol{v}_1, \ldots, \boldsymbol{v}_j$. Induction on j proves orthonormality.
Rearranging the definition of $\boldsymbol{v}_{j+1}$ gives the column relation

$$\mathbf{A}\boldsymbol{v}_j = \sum_{i=1}^{j+1} h_{ij}\boldsymbol{v}_i, \qquad j = 1, \ldots, k.$$

Placing these k columns side by side gives $\mathbf{A}\mathbf{V}_k = \mathbf{V}_{k+1}\bar{\mathbf{H}}_k$. Separating the last row of $\bar{\mathbf{H}}_k$ gives the first form of (1).
For the span, assume $\operatorname{span}\{\boldsymbol{v}_1, \ldots, \boldsymbol{v}_j\} = \mathcal{K}_j$. The column relation expresses $\boldsymbol{v}_{j+1}$ as a combination of $\mathbf{A}\boldsymbol{v}_j$ and $\boldsymbol{v}_1, \ldots, \boldsymbol{v}_j$, and $\mathbf{A}\boldsymbol{v}_j \in \mathbf{A}\mathcal{K}_j \subseteq \mathcal{K}_{j+1}$. Hence $\boldsymbol{v}_{j+1} \in \mathcal{K}_{j+1}$. The $j+1$ orthonormal vectors $\boldsymbol{v}_1, \ldots, \boldsymbol{v}_{j+1}$ are linearly independent and lie in $\mathcal{K}_{j+1}$, whose dimension is at most $j+1$, so they span it.
Finally, multiplying (1) on the left by $\mathbf{V}_k^T$ and using $\mathbf{V}_k^T\mathbf{V}_k = \mathbf{I}_k$ and $\mathbf{V}_k^T\boldsymbol{v}_{k+1} = \mathbf{0}$ gives $\mathbf{V}_k^T\mathbf{A}\mathbf{V}_k = \mathbf{H}_k$. □

Remark 4.5 **(Arnoldi cost)** Step j requires one product with $\mathbf{A}$ and j inner products and vector updates of length n. After k steps the iteration has used k matrix–vector products and $O(k^2 n)$ additional operations, and it stores $k+1$ vectors of length n. The orthogonalization cost and storage both grow with k. For a symmetric matrix, Section 4.5 shows that most of the coefficients h_{ij} vanish and that the growth disappears.

4.4.3 Ritz Pairs and Breakdown

The matrix $\mathbf{H}_k$ is small, so its eigenvalues can be computed by the dense QR algorithm. Its eigenpairs, lifted back to $\mathbb{R}^n$ through $\mathbf{V}_k$, serve as approximate eigenpairs of $\mathbf{A}$. The Arnoldi relation measures their quality without any further products with $\mathbf{A}$.

Def 4.14 **(Ritz Pairs)** Let $\mathbf{H}_k\boldsymbol{s} = \theta\boldsymbol{s}$ with $\|\boldsymbol{s}\|_2 = 1$. The scalar θ is a **Ritz value** and the unit vector $\tilde{\boldsymbol{u}} = \mathbf{V}_k\boldsymbol{s} \in \mathcal{K}_k(\mathbf{A}, \boldsymbol{v})$ is the corresponding **Ritz vector** of $\mathbf{A}$ with respect to $\mathcal{K}_k(\mathbf{A}, \boldsymbol{v})$.

Thm 4.26 **(Residual of a Ritz Pair)** Let $(\theta, \tilde{\boldsymbol{u}})$ be a Ritz pair with $\tilde{\boldsymbol{u}} = \mathbf{V}_k\boldsymbol{s}$, and let s_k denote the last entry of $\boldsymbol{s}$. Then

$$\mathbf{A}\tilde{\boldsymbol{u}} - \theta\tilde{\boldsymbol{u}} = h_{k+1,k}s_k\,\boldsymbol{v}_{k+1}, \qquad \|\mathbf{A}\tilde{\boldsymbol{u}} - \theta\tilde{\boldsymbol{u}}\|_2 = h_{k+1,k}|s_k|.$$

Proof Multiplying (1) on the right by $\boldsymbol{s}$ gives

$$\mathbf{A}\tilde{\boldsymbol{u}} = \mathbf{V}_k\mathbf{H}_k\boldsymbol{s} + h_{k+1,k}\boldsymbol{v}_{k+1}(\boldsymbol{e}_k^T\boldsymbol{s}) = \theta\tilde{\boldsymbol{u}} + h_{k+1,k}s_k\boldsymbol{v}_{k+1}.$$

Subtracting $\theta\tilde{\boldsymbol{u}}$ and using $\|\boldsymbol{v}_{k+1}\|_2 = 1$ gives both claims. □

The residual norm depends only on $h_{k+1,k}$ and the last entry of a k-dimensional eigenvector. We can therefore monitor convergence of every Ritz pair at each step and form $\tilde{\boldsymbol{u}} = \mathbf{V}_k\boldsymbol{s}$ only for the pairs that have converged. A small residual shows that $(\theta, \tilde{\boldsymbol{u}})$ is an exact eigenpair of a nearby matrix. For a symmetric matrix, Cor 4.8 then places an eigenvalue within the residual norm of θ. For a nonnormal diagonalizable matrix, Cor 4.10 multiplies this distance by the condition number of the eigenvector basis, which can be large.

Cor 4.27 **(Breakdown Gives Exact Eigenvalues)** If the Arnoldi iteration stops at step k because $h_{k+1,k} = 0$, then $\mathcal{K}_k(\mathbf{A}, \boldsymbol{v})$ is invariant under $\mathbf{A}$, and every Ritz pair is an exact eigenpair of $\mathbf{A}$.

Proof When $h_{k+1,k} = 0$, Thm 4.26 gives $\mathbf{A}\tilde{\boldsymbol{u}} = \theta\tilde{\boldsymbol{u}}$ for every Ritz pair. The relation $\mathbf{A}\mathbf{V}_k = \mathbf{V}_k\mathbf{H}_k$ also shows that $\mathbf{A}$ maps the column space of $\mathbf{V}_k$, which is $\mathcal{K}_k$, into itself. □

4.4.4 Exercises

Ex 4.4

1. **Check. (Shift invariance)** Show that $\mathcal{K}_k(\mathbf{A} - \sigma\mathbf{I}, \boldsymbol{v}) = \mathcal{K}_k(\mathbf{A}, \boldsymbol{v})$ for every scalar σ. Conclude that the Arnoldi iteration applied to $\mathbf{A} - \sigma\mathbf{I}$ produces the same basis $\mathbf{V}_k$ and the Hessenberg matrix $\mathbf{H}_k - \sigma\mathbf{I}$.

2. **Derive. (Exact termination)** Suppose $\boldsymbol{v}$ is a combination of exactly m eigenvectors of a diagonalizable matrix $\mathbf{A}$, with nonzero coefficients and distinct eigenvalues. Prove that $\dim \mathcal{K}_k(\mathbf{A}, \boldsymbol{v}) = \min(k, m)$, and use Lem 4.24 and Cor 4.27 to conclude that the Arnoldi iteration stops at step m with those m eigenvalues as its Ritz values.

3. **Compute. (Monomial versus Arnoldi basis)** Let $\mathbf{A} = \operatorname{diag}(1, 2, \ldots, 100)$ and $\boldsymbol{v} = (1, \ldots, 1)^T$. For $k = 5, 10, 15, 20$, compute the condition number of the matrix with columns $\boldsymbol{v}, \mathbf{A}\boldsymbol{v}, \ldots, \mathbf{A}^{k-1}\boldsymbol{v}$, and the quantity $\|\mathbf{V}_k^T\mathbf{V}_k - \mathbf{I}\|_2$ for the Arnoldi basis. Explain the growth of the first quantity using power iteration.

4. **Compute. (Implementing Arnoldi)** Implement Def 4.13 for a function that applies $\mathbf{A}$ to a vector. Test it on a random 200×200 nonsymmetric matrix by checking $\|\mathbf{A}\mathbf{V}_k - \mathbf{V}_{k+1}\bar{\mathbf{H}}_k\|_2$ and by comparing the residuals of Thm 4.26 with directly computed residuals $\|\mathbf{A}\tilde{\boldsymbol{u}} - \theta\tilde{\boldsymbol{u}}\|_2$.

4.5 The Lanczos Algorithm

When $\mathbf{A}$ is real symmetric, the Arnoldi process of Section 4.4 simplifies in two ways. First, the projected matrix $\mathbf{H}_k = \mathbf{V}_k^T \mathbf{A} \mathbf{V}_k$ is symmetric as well as Hessenberg, so it is tridiagonal. Each new basis vector then needs to be orthogonalized only against the two preceding ones, and the cost per step no longer grows with k. Second, the Rayleigh quotient characterization of Thm 4.5 applies to the projected matrix. The characterization shows that the extreme Ritz values are the best estimates of the extreme eigenvalues available from the Krylov subspace.

We first derive the three-term recurrence that defines the Lanczos algorithm (Lanczos, 1950). We then show that the extreme Ritz values move monotonically toward the extreme eigenvalues and that they are never worse than the power-iteration estimate obtained with the same number of products. A numerical example compares the two methods. The section ends with the loss of orthogonality that occurs in floating-point arithmetic.

4.5.1 Symmetry Gives a Three-Term Recurrence

Thm 4.28 **(Tridiagonal Projection)** Let $\mathbf{A} \in \mathbb{R}^{n \times n}$ be symmetric, and suppose the Arnoldi iteration (Def 4.13) completes k steps. Then $\mathbf{H}_k$ is symmetric tridiagonal. Writing $\alpha_j = h_{jj}$ and $\beta_j = h_{j+1,j}$, we have

$$\mathbf{H}_k = \mathbf{T}_k = \begin{pmatrix} \alpha_1 & \beta_1 & & \\ \beta_1 & \alpha_2 & \ddots & \\ & \ddots & \ddots & \beta_{k-1} \\ & & \beta_{k-1} & \alpha_k \end{pmatrix},$$

and the Arnoldi coefficients satisfy $h_{ij} = 0$ for $i < j - 1$ and $h_{j-1,j} = \beta_{j-1}$.

Proof By Thm 4.25, $\mathbf{H}_k = \mathbf{V}_k^T \mathbf{A} \mathbf{V}_k$. Since $\mathbf{A}^T = \mathbf{A}$,

$$\mathbf{H}_k^T = \mathbf{V}_k^T \mathbf{A}^T \mathbf{V}_k = \mathbf{V}_k^T \mathbf{A} \mathbf{V}_k = \mathbf{H}_k.$$

The matrix $\mathbf{H}_k$ is upper Hessenberg, so $h_{ij} = 0$ for $i > j+1$. Symmetry gives $h_{ij} = h_{ji} = 0$ for $i < j - 1$ as well, which leaves only the diagonal and the two adjacent diagonals. Symmetry also gives $h_{j-1,j} = h_{j,j-1} = \beta_{j-1}$. □

In the Arnoldi iteration, the coefficient h_{ij} is the component of $\mathbf{A}\boldsymbol{v}_j$ along $\boldsymbol{v}_i$. Theorem 4.28 shows that these components vanish for $i < j - 1$ in exact arithmetic, so the inner loop of Step 2 reduces to two subtractions. The coefficient for $i = j - 1$ is already known from the previous step. Substituting these facts into Def 4.13 gives the Lanczos iteration.

Def 4.15 **(The Lanczos Iteration)** Let $\mathbf{A} \in \mathbb{R}^{n \times n}$ be symmetric and let $\boldsymbol{v}_1$ be a unit vector. Set $\boldsymbol{v}_0 = \mathbf{0}$ and $\beta_0 = 0$. For $j = 1, 2, \ldots, k$:

1. Compute $\boldsymbol{w} = \mathbf{A}\boldsymbol{v}_j - \beta_{j-1}\boldsymbol{v}_{j-1}$.
2. Compute $\alpha_j = \boldsymbol{v}_j^T \boldsymbol{w}$ and update $\boldsymbol{w} \leftarrow \boldsymbol{w} - \alpha_j \boldsymbol{v}_j$.
3. Compute $\beta_j = \|\boldsymbol{w}\|_2$.
4. If $\beta_j = 0$, stop. Otherwise, set $\boldsymbol{v}_{j+1} = \boldsymbol{w}/\beta_j$.

Cor 4.29 **(The Lanczos Relation)** After k steps of the Lanczos iteration without breakdown, the matrix $\mathbf{V}_k = (\boldsymbol{v}_1 \ \cdots \ \boldsymbol{v}_k)$ has orthonormal columns spanning $\mathcal{K}_k(\mathbf{A}, \boldsymbol{v}_1)$, and

$$\mathbf{A}\mathbf{V}_k = \mathbf{V}_k \mathbf{T}_k + \beta_k \boldsymbol{v}_{k+1} \boldsymbol{e}_k^T, \qquad \mathbf{T}_k = \mathbf{V}_k^T \mathbf{A} \mathbf{V}_k.$$

Proof In exact arithmetic the Lanczos iteration computes the same vectors and coefficients as the Arnoldi iteration, because it omits only the coefficients that Thm 4.28 shows to be zero. The claims are therefore Thm 4.25 with $\mathbf{H}_k = \mathbf{T}_k$ and $h_{k+1,k} = \beta_k$. □

Remark 4.6 **(Lanczos cost)** Each step requires one product with $\mathbf{A}$ and $O(n)$ further operations, independent of k. If only eigenvalues are wanted, the iteration needs to keep just $\boldsymbol{v}_{j-1}$, $\boldsymbol{v}_j$, and the scalars α_j, β_j. If Ritz vectors are also wanted, we either store $\mathbf{V}_k$ or run the recurrence a second time to form $\mathbf{V}_k \boldsymbol{s}$.

4.5.2 Ritz Values as Optimal Estimates

The Ritz pairs of Def 4.14 are now the eigenpairs of the symmetric tridiagonal matrix $\mathbf{T}_k$, so the Ritz values are real. Theorem 4.26 gives their residuals as $\beta_k |s_k|$. Symmetry also lets us say how good the Ritz values are. Throughout, let $\lambda_1 \le \cdots \le \lambda_n$ be the eigenvalues of $\mathbf{A}$ and let $\theta_1^{(k)} \le \cdots \le \theta_k^{(k)}$ be the eigenvalues of $\mathbf{T}_k$.

Thm 4.30 **(Rayleigh–Ritz Optimality)** Let $R(\boldsymbol{x}) = \boldsymbol{x}^T \mathbf{A} \boldsymbol{x} / \boldsymbol{x}^T \boldsymbol{x}$. The extreme Ritz values are the extreme values of the Rayleigh quotient over the Krylov subspace,

$$\theta_k^{(k)} = \max_{\boldsymbol{x} \in \mathcal{K}_k,\, \boldsymbol{x} \neq \mathbf{0}} R(\boldsymbol{x}), \qquad \theta_1^{(k)} = \min_{\boldsymbol{x} \in \mathcal{K}_k,\, \boldsymbol{x} \neq \mathbf{0}} R(\boldsymbol{x}).$$

Consequently, $\lambda_1 \le \theta_1^{(k)}$ and $\theta_k^{(k)} \le \lambda_n$.

Proof Every nonzero $\boldsymbol{x} \in \mathcal{K}_k$ has the form $\boldsymbol{x} = \mathbf{V}_k \boldsymbol{y}$ with $\boldsymbol{y} \in \mathbb{R}^k$ nonzero. Orthonormality of the columns of $\mathbf{V}_k$ gives $\boldsymbol{x}^T \boldsymbol{x} = \boldsymbol{y}^T \boldsymbol{y}$, and Cor 4.29 gives $\boldsymbol{x}^T \mathbf{A} \boldsymbol{x} = \boldsymbol{y}^T \mathbf{T}_k \boldsymbol{y}$. Hence

$$R(\boldsymbol{x}) = \frac{\boldsymbol{y}^T \mathbf{T}_k \boldsymbol{y}}{\boldsymbol{y}^T \boldsymbol{y}}.$$

Applying Thm 4.5 to the symmetric matrix $\mathbf{T}_k$ with $k = 1$ and with the largest index shows that the minimum and maximum of this quotient over $\boldsymbol{y} \neq \mathbf{0}$ are $\theta_1^{(k)}$ and $\theta_k^{(k)}$.

Applied to $\mathbf{A}$, the same characterization shows that $R(\boldsymbol{x})$ lies in $[\lambda_1, \lambda_n]$ for every nonzero $\boldsymbol{x} \in \mathbb{R}^n$, which gives the final inequalities. $\square$

Cor 4.31 **(Monotone Convergence of the Extreme Ritz Values)** The Ritz values at consecutive steps interlace,

$$\theta_i^{(k+1)} \le \theta_i^{(k)} \le \theta_{i+1}^{(k+1)}, \qquad i = 1, \dots, k.$$

In particular, $\theta_k^{(k)}$ increases toward λ_n and $\theta_1^{(k)}$ decreases toward λ_1 as k grows.

Proof The matrix $\mathbf{T}_k$ is obtained from $\mathbf{T}_{k+1}$ by deleting its last row and column. Applying Thm 4.6 with $\mathbf{T}_{k+1}$ in place of $\mathbf{A}$ and $\mathbf{T}_k$ in place of $\mathbf{B}$ gives the interlacing inequalities. Taking $i = k$ in the right inequality gives $\theta_k^{(k)} \le \theta_{k+1}^{(k+1)}$, and taking $i = 1$ in the left inequality gives $\theta_1^{(k+1)} \le \theta_1^{(k)}$. Theorem 4.30 bounds the two sequences by λ_n and λ_1. $\square$

Cor 4.32 **(Lanczos Improves on Power Iteration)** Let $\mathbf{A}$ be symmetric positive definite, and let $\lambda^{(k-1)}$ be the Rayleigh quotient estimate of Def 4.5 after $k-1$ power steps from $\boldsymbol{v}^{(0)} = \boldsymbol{v}_1$. Both $\lambda^{(k-1)}$ and $\theta_k^{(k)}$ require k products with $\mathbf{A}$, and

$$\lambda^{(k-1)} \le \theta_k^{(k)} \le \lambda_n.$$

Proof The power iterate $\boldsymbol{v}^{(k-1)}$ is a multiple of $\mathbf{A}^{k-1}\boldsymbol{v}_1$, so it lies in $\mathcal{K}_k(\mathbf{A}, \boldsymbol{v}_1)$. Its Rayleigh quotient is $\lambda^{(k-1)} = R(\boldsymbol{v}^{(k-1)})$, which is at most the maximum in Thm 4.30. Power iteration uses $k-1$ products to form $\boldsymbol{v}^{(k-1)}$ and one more to evaluate its Rayleigh quotient. Lanczos uses one product per step. Positive definiteness makes λ_n the eigenvalue of largest magnitude, so the two methods approximate the same eigenvalue. $\square$

The polynomial form of Def 4.12 explains the size of the improvement. Every vector in $\mathcal{K}_k$ is $p(\mathbf{A})\boldsymbol{v}_1$ for some $p \in \mathcal{P}_{k-1}$. Expanding $\boldsymbol{v}_1 = \sum_i c_i \boldsymbol{q}_i$ in the eigenvectors of $\mathbf{A}$ gives

$$\theta_k^{(k)} = \max_{p \in \mathcal{P}_{k-1}} R(p(\mathbf{A})\boldsymbol{v}_1) = \max_{p \in \mathcal{P}_{k-1}} \frac{\sum_{i=1}^n \lambda_i \, p(\lambda_i)^2 c_i^2}{\sum_{i=1}^n p(\lambda_i)^2 c_i^2}.$$

The quotient is close to λ_n when p is large at λ_n and small on the rest of the spectrum. Power iteration is restricted to $p(t) = t^{k-1}$, which separates λ_n from λ_{n-1} only by the factor $(\lambda_{n-1}/\lambda_n)^{k-1}$. Lanczos implicitly selects the best polynomial. A shifted Chebyshev polynomial, which is bounded by one on $[\lambda_1, \lambda_{n-1}]$ and grows rapidly outside it, gives the following bound.

Remark 4.7 **(Kaniel–Paige bound)** Let ϕ be the angle between $\boldsymbol{v}_1$ and the eigenvector $\boldsymbol{q}_n$, let $\gamma = (\lambda_n - \lambda_{n-1})/(\lambda_{n-1} - \lambda_1)$, and let $T_m(x) = \cosh(m \operatorname{arccosh} x)$ for $x \ge 1$. Then

$$0 \le \lambda_n - \theta_k^{(k)} \le (\lambda_n - \lambda_1) \left(\frac{\tan \phi}{T_{k-1}(1 + 2\gamma)} \right)^2.$$

A proof is given by Golub and Van Loan (Golub and Van Loan, 2013). Since $T_m(1+2\gamma)$ grows roughly like $(1+2\sqrt{\gamma})^m$ when γ is small, the error decreases geometrically at a rate set by the square root of the relative gap. A square root of the same kind appears in the conjugate gradient bound of Thm 6.13. The analogous bound for $\theta_1^{(k)}$ follows by applying the result to $-\mathbf{A}$. Interior eigenvalues admit weaker bounds and converge more slowly, because no low-degree polynomial can be large at an interior point and small on both sides of it.

Example **(Lanczos and power iteration on a model spectrum)** Let $\mathbf{A} = \operatorname{diag}(1, 2, \dots, 100)$ and $\boldsymbol{v}_1 = (1, \dots, 1)^T/10$. We run Lanczos with full reorthogonalization, so that the computed Ritz values agree with exact arithmetic to rounding level, and compare the error in the largest Ritz value with the error in the power-iteration estimate after the same number of products.

k	$\lambda_n - \theta_k^{(k)}$	$\lambda_n - \lambda^{(k-1)}$	Kaniel–Paige bound
10	8.6×10^{-1}	4.5	9.9×10^{2}
20	5.0×10^{-2}	2.0	1.8×10^{1}
30	6.4×10^{-4}	1.2	3.3×10^{-1}
40	7.2×10^{-7}	8.2×10^{-1}	5.8×10^{-3}

The relative gap is $\gamma = 1/98$, and the power-iteration ratio is $\lambda_{99}/\lambda_{100} = 0.99$. After 40 products the power-iteration estimate has not yet reached one correct digit, whereas the largest Ritz value is correct to about eight significant digits. The bound is valid but conservative at every step. By the symmetry of this spectrum about 50.5 and of the starting vector, the error in $\theta_1^{(k)}$ is the same as the error in $\theta_k^{(k)}$.
At $k = 40$ the residual of Thm 4.26 for the largest Ritz pair is 3.7×10^{-3}, about four orders of magnitude larger than the eigenvalue error. Corollary 4.8 explains the difference. The corollary bounds the distance from the Ritz value to the nearest eigenvalue by the squared residual divided by the distance δ from the Ritz value to the other eigenvalues. Here δ is about one, and the bound 1.4×10^{-5} is consistent with the observed error.

Remark 4.8 **(Lanczos and conjugate gradients)** The conjugate gradient method of Section 6.4 builds the same Krylov subspace $\mathcal{K}_k(\mathbf{A}, \boldsymbol{r}^{(0)})$ for a symmetric positive definite system. Its residuals are multiples of the Lanczos vectors started from $\boldsymbol{v}_1 = \boldsymbol{r}^{(0)}/\|\boldsymbol{r}^{(0)}\|_2$, and its coefficients determine the entries of $\mathbf{T}_k$. Section 6.4 derives this correspondence and uses it to estimate the extreme eigenvalues during a CG solve.

4.5.3 Loss of Orthogonality and Ghost Eigenvalues

Theorem 4.28 relies on the exact orthogonality of $\boldsymbol{v}_{j+1}$ to $\boldsymbol{v}_1, \dots, \boldsymbol{v}_{j-2}$. The Lanczos iteration never enforces this orthogonality explicitly, so rounding errors are free to destroy it.

Remark 4.9 **(Paige's analysis)** In floating-point arithmetic, the computed Lanczos vectors gradually lose mutual orthogonality. Paige showed that the loss is not random (Paige, 1976). Let

$(\theta_i, \tilde{\boldsymbol{u}}_i)$ be a Ritz pair computed from $\mathbf{T}_k$, and let u denote the unit roundoff. Paige's analysis gives

$$|\tilde{\boldsymbol{u}}_i^T \boldsymbol{v}_{k+1}| \approx \frac{u\,\|\mathbf{A}\|_2}{\beta_k |s_{k,i}|}.$$

The denominator is the residual of Thm 4.26. The new Lanczos vector therefore acquires a component along $\tilde{\boldsymbol{u}}_i$ precisely as that Ritz pair converges, and orthogonality is lost in the directions of converged Ritz vectors. Once orthogonality to a converged Ritz vector is lost, the iteration begins to approximate the same eigenvalue again and produces repeated copies of it, known as **ghost eigenvalues**. In the example above, running the iteration without reorthogonalization for $k = 150$ steps gives three Ritz values within 10^{-6} of $\lambda_{100} = 100$ and three within 10^{-6} of $\lambda_1 = 1$. At $k = 200$ there are four copies of each.

Remark 4.10 **(Remedies for ghost eigenvalues)** Practical implementations use one of three strategies.

1. **Full reorthogonalization.** We orthogonalize each new vector against all previous Lanczos vectors. This restores the behavior of exact arithmetic at the Arnoldi cost of $O(k^2 n)$ operations and $O(kn)$ storage.

2. **Selective reorthogonalization.** We orthogonalize only against converged Ritz vectors, and only when Paige's analysis indicates that orthogonality to them has been lost. The cost is proportional to the number of converged Ritz vectors.

3. **Cullum–Willoughby identification.** We run the iteration without reorthogonalization and discard spurious Ritz values afterward by comparing the eigenvalues of $\mathbf{T}_k$ with those of the matrix obtained by deleting its first row and column.

4.5.4 Exercises

Ex 4.5

1. **Check.** Carry out two steps of the Lanczos iteration by hand for $\mathbf{A} = \operatorname{diag}(4, 2, 1)$ and $\boldsymbol{v}_1 = (1, 1, 1)^T/\sqrt{3}$. Write down α_1, β_1, α_2, and $\mathbf{T}_2$, and compute β_2.

2. **Check.** For $\mathbf{T}_2$ from the previous exercise, compute both Ritz values and their residuals using Thm 4.26. Verify the interlacing inequalities of Cor 4.31 between the Ritz value $\theta_1^{(1)} = \alpha_1$, the two eigenvalues of $\mathbf{T}_2$, and the eigenvalues of $\mathbf{A}$.

3. **Derive.** Using Thm 4.30, show that $\theta_{k-1}^{(k)} = \max_S \min_{\boldsymbol{x} \in S,\, \boldsymbol{x} \neq \mathbf{0}} R(\boldsymbol{x})$, where S ranges over two-dimensional subspaces of $\mathcal{K}_k$. Conclude that $\theta_{k-1}^{(k)} \le \lambda_{n-1}$.

4. **Derive.** Let $\mathbf{A}$ be symmetric positive definite. Show that the power-iteration estimate satisfies $\lambda_n - \lambda^{(k-1)} \le (\lambda_n - \lambda_1)\tan^2\phi\,(\lambda_{n-1}/\lambda_n)^{2(k-1)}$, where ϕ is the angle between $\boldsymbol{v}_1$ and $\boldsymbol{q}_n$. Compare the rate with the Kaniel–Paige bound when γ is small.

5. **Compute.** Implement Def 4.15 with an option for full reorthogonalization. Reproduce the table in the example above, and plot all Ritz values against k for $k \le 200$ with and without reorthogonalization. Identify the steps at which ghost copies of λ_{100} appear, and relate them to the step at which the largest Ritz value converged.

6. **Explore.** Repeat the previous computation with $\mathbf{A} = \mathrm{diag}(1, 2, \dots, 99, 110)$. Explain, using the polynomial characterization, why the largest Ritz value now converges much faster while the convergence of $\theta_1^{(k)}$ is essentially unchanged.

Read **Further reading.** Horn and Johnson are a reference for the matrix structure behind the spectral results (Horn and Johnson, 2012). The computational treatment in Golub and Van Loan includes symmetric eigensolvers and the SVD (Golub and Van Loan, 2013). The original Lanczos and Golub–Kahan papers show how two algorithms in these chapters first appeared (Lanczos, 1950; Golub and Kahan, 1965).

5 Approximation and Integration

5.1 Polynomial and Spline Interpolation

Interpolation is the process of constructing a continuous function that passes exactly through a discrete collection of given data points. Given $n+1$ distinct abscissas (or nodes) $x_0, x_1, \dots, x_n \in \mathbb{R}$ and corresponding values $y_0, y_1, \dots, y_n \in \mathbb{R}$, polynomial interpolation seeks a polynomial P_n of degree at most n such that

$$P_n(x_i) = y_i, \qquad i = 0, 1, \dots, n.$$

Polynomials are useful local and global approximants because finite polynomial representations can be evaluated, differentiated, and integrated by finite algorithms.

5.1.1 Existence, Uniqueness, and the Vandermonde Matrix

Thm 5.1 **(Existence and Uniqueness of the Interpolating Polynomial)** Let $x_0, x_1, \dots, x_n$ be $n+1$ pairwise distinct real numbers, and let $y_0, y_1, \dots, y_n$ be arbitrary real numbers. There exists a unique polynomial $P_n \in \mathcal{P}_n$ of degree at most n such that $P_n(x_i) = y_i$ for all $i = 0, 1, \dots, n$.

Proof We establish existence and uniqueness using the fundamental properties of polynomials.

1. **Uniqueness:** Suppose there exist two polynomials $P_n, Q_n \in \mathcal{P}_n$ that satisfy the interpolation conditions, so $P_n(x_i) = Q_n(x_i) = y_i$ for $i = 0, \dots, n$. Define the difference polynomial $R(x) = P_n(x) - Q_n(x)$. Because $\mathcal{P}_n$ is a vector space, R has degree at most n. Furthermore,

$$R(x_i) = P_n(x_i) - Q_n(x_i) = y_i - y_i = 0, \qquad i = 0, \dots, n.$$

Thus $R(x)$ has at least $n+1$ distinct real roots. By the Fundamental Theorem of Algebra, a non-zero polynomial of degree at most n can have at most n roots. Therefore, $R(x)$ must be the zero polynomial, which implies $P_n(x) \equiv Q_n(x)$.

2. **Existence via Linear Algebra:** Express $P_n(x)$ in the standard monomial basis:

$$P_n(x) = c_0 + c_1 x + c_2 x^2 + \cdots + c_n x^n = \sum_{j=0}^{n} c_j x^j.$$

The interpolation conditions $P_n(x_i) = y_i$ form a square $(n+1)\times(n+1)$ linear system $\mathbf{V}\boldsymbol{c} = \boldsymbol{y}$:

$$\begin{pmatrix} 1 & x_0 & x_0^2 & \cdots & x_0^n \\ 1 & x_1 & x_1^2 & \cdots & x_1^n \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & x_n & x_n^2 & \cdots & x_n^n \end{pmatrix} \begin{pmatrix} c_0 \\ c_1 \\ \vdots \\ c_n \end{pmatrix} = \begin{pmatrix} y_0 \\ y_1 \\ \vdots \\ y_n \end{pmatrix}.$$

The matrix $\mathbf{V}$ is the Vandermonde matrix. Its determinant evaluates to

$$\det(\mathbf{V}) = \prod_{0 \le i < j \le n} (x_j - x_i).$$

Because the nodes x_i are pairwise distinct, $x_j - x_i \neq 0$ for all $j > i$, so $\det(\mathbf{V}) \neq 0$. Consequently, $\mathbf{V}$ is invertible, and a unique coefficient vector $\boldsymbol{c} = \mathbf{V}^{-1}\boldsymbol{y}$ exists for any data vector $\boldsymbol{y}$.

□

Remark 5.1 Although the Vandermonde matrix provides a constructive proof of existence, solving $\mathbf{V}\boldsymbol{c} = \boldsymbol{y}$ in the monomial basis is often a poor numerical representation. Its condition number depends on the nodes and their scaling and can grow rapidly with degree. A large coefficient error does not imply that the interpolation problem itself is ill-posed. It may instead reflect an ill-conditioned choice of basis. The Lagrange and barycentric forms avoid solving for monomial coefficients.

5.1.2 The Lagrange Form

The Lagrange formulation circumvents the ill-conditioned Vandermonde linear system by constructing a basis of polynomials tailored to the specific nodes x_i.

Def 5.1 **(Lagrange Basis Polynomials)** Given $n+1$ distinct nodes $x_0, x_1, \ldots, x_n$, the i-th Lagrange basis polynomial $L_i \in \mathcal{P}_n$ is defined by

$$L_i(x) = \prod_{\substack{j=0 \\ j \neq i}}^{n} \frac{x - x_j}{x_i - x_j} = \frac{(x-x_0)\cdots(x-x_{i-1})(x-x_{i+1})\cdots(x-x_n)}{(x_i-x_0)\cdots(x_i-x_{i-1})(x_i-x_{i+1})\cdots(x_i-x_n)}.$$

The Lagrange basis polynomials satisfy the cardinal Kronecker delta property:

$$L_i(x_j) = \delta_{ij} = \begin{cases} 1, & \text{if } i = j, \\ 0, & \text{if } i \neq j. \end{cases}$$

The interpolating polynomial in Lagrange form is

$$P_n(x) = \sum_{i=0}^{n} y_i L_i(x).$$

Example To interpolate the three data points $(-1, 1)$, $(0, 0)$, and $(1, 1)$, we construct the degree-2 Lagrange basis polynomials on $x_0 = -1, x_1 = 0, x_2 = 1$:

$$\begin{aligned} L_0(x) &= \frac{(x-0)(x-1)}{(-1-0)(-1-1)} = \frac{x(x-1)}{2} = \frac{x^2 - x}{2}, \\ L_1(x) &= \frac{(x-(-1))(x-1)}{(0-(-1))(0-1)} = \frac{(x+1)(x-1)}{-1} = 1 - x^2, \\ L_2(x) &= \frac{(x-(-1))(x-0)}{(1-(-1))(1-0)} = \frac{x(x+1)}{2} = \frac{x^2 + x}{2}. \end{aligned}$$

Multiplying by the data values $y_0 = 1, y_1 = 0, y_2 = 1$ gives

$$P_2(x) = 1 \cdot \left(\frac{x^2 - x}{2}\right) + 0 \cdot (1 - x^2) + 1 \cdot \left(\frac{x^2 + x}{2}\right) = \frac{x^2 - x + x^2 + x}{2} = x^2.$$

Remark 5.2 The standard Lagrange form has two practical shortcomings. First, evaluating $P_n(x)$ at a single point x requires $O(n^2)$ floating-point operations. Second, adding a new node (x_{n+1}, y_{n+1}) alters every single basis polynomial $L_i(x)$, requiring a complete recomputation from scratch.

5.1.3 The Barycentric Interpolation Formula

Barycentric interpolation reorganizes the Lagrange formula into a stable, computationally optimal form requiring only $O(n)$ operations per evaluation.

Thm 5.2 **(Barycentric Interpolation Formulas)** Let $\ell(x) = \prod_{j=0}^{n}(x - x_j)$ denote the nodal polynomial. For each node x_i, define the barycentric weights

$$w_i = \frac{1}{\prod_{j \neq i}(x_i - x_j)} = \frac{1}{\ell'(x_i)}, \qquad i = 0, 1, \ldots, n.$$

Then the interpolating polynomial satisfies:

1. **First Barycentric Form:**

$$P_n(x) = \ell(x) \sum_{i=0}^{n} \frac{w_i}{x - x_i} y_i.$$

2. **Second (True) Barycentric Form:**

$$P_n(x) = \frac{\sum_{i=0}^{n} \frac{w_i}{x-x_i} y_i}{\sum_{i=0}^{n} \frac{w_i}{x-x_i}}.$$

If $x = x_j$ for some node index j, the formula evaluates to $P_n(x_j) = y_j$.

Proof We express the Lagrange basis polynomials in terms of $\ell(x)$ and w_i. By definition,

$$L_i(x) = \prod_{j \neq i} \frac{x - x_j}{x_i - x_j} = \left(\prod_{j \neq i} (x - x_j) \right) \left(\frac{1}{\prod_{j \neq i} (x_i - x_j)} \right) = \frac{\ell(x)}{x - x_i} w_i.$$

Substituting this relation into $P_n(x) = \sum_{i=0}^{n} y_i L_i(x)$ yields the First Barycentric Formula:

$$P_n(x) = \sum_{i=0}^{n} y_i \left(\frac{\ell(x)}{x - x_i} w_i \right) = \ell(x) \sum_{i=0}^{n} \frac{w_i}{x - x_i} y_i.$$

To derive the Second Barycentric Formula, consider interpolating the constant function $f(x) \equiv 1$, where $y_i = 1$ for all i. Because the unique degree-n interpolant of a constant is the constant itself, the First Barycentric Formula gives

$$1 = \ell(x) \sum_{i=0}^{n} \frac{w_i}{x - x_i}.$$

Dividing the expression for $P_n(x)$ by this identity cancels the nodal polynomial $\ell(x)$, yielding

$$P_n(x) = \frac{P_n(x)}{1} = \frac{\ell(x) \sum_{i=0}^{n} \frac{w_i}{x-x_i} y_i}{\ell(x) \sum_{i=0}^{n} \frac{w_i}{x-x_i}} = \frac{\sum_{i=0}^{n} \frac{w_i}{x-x_i} y_i}{\sum_{i=0}^{n} \frac{w_i}{x-x_i}}.$$

□

Remark 5.3 The second barycentric formula is numerically stable. Higham (2004) proved that barycentric interpolation is backward stable, meaning the computed value $\hat{P}_n(x)$ is the exact value of a polynomial interpolating slightly perturbed data $y_i(1+\varepsilon_i)$. Furthermore, common multiplicative factors in the weights w_i cancel out in the numerator and denominator, allowing simple analytical expressions for standard node families (such as Chebyshev nodes).

5.1.4 The Interpolation Error Formula

Thm 5.3 **(Cauchy Remainder Error Theorem)** Let $f \in C^{n+1}[a, b]$, and let $P_n \in \mathcal{P}_n$ interpolate f at $n + 1$ distinct nodes $x_0, x_1, \ldots, x_n \in [a, b]$. For each point $x \in [a, b]$, there exists a point $\xi_x \in (a, b)$ such that

$$f(x) - P_n(x) = \frac{f^{(n+1)}(\xi_x)}{(n+1)!} \prod_{j=0}^{n} (x - x_j) = \frac{f^{(n+1)}(\xi_x)}{(n+1)!} \omega_{n+1}(x),$$

where $\omega_{n+1}(x) = \prod_{j=0}^{n}(x - x_j)$ is the nodal error polynomial.

Proof If $x = x_i$ for some node x_i, both sides vanish and the formula holds trivially for any ξ_x. Now fix $x \in [a, b]$ with $x \neq x_i$ for all i. Define the constant

$$\lambda = \frac{f(x) - P_n(x)}{\omega_{n+1}(x)},$$

and consider the auxiliary function $g : [a, b] \to \mathbb{R}$ defined by

$$g(t) = f(t) - P_n(t) - \lambda\omega_{n+1}(t).$$

Because $f \in C^{n+1}[a, b]$ and polynomials are infinitely differentiable, $g \in C^{n+1}[a, b]$. We evaluate g at the $n + 2$ distinct points $x_0, x_1, \ldots, x_n$ and x:

$$g(x_i) = f(x_i) - P_n(x_i) - \lambda\omega_{n+1}(x_i) = y_i - y_i - 0 = 0, \qquad i = 0, \ldots, n,$$
$$g(x) = f(x) - P_n(x) - \left(\frac{f(x) - P_n(x)}{\omega_{n+1}(x)}\right)\omega_{n+1}(x) = 0.$$

Thus $g(t)$ vanishes at $n + 2$ distinct points in $[a, b]$.
By Rolle's Theorem, between any two adjacent zeros of $g(t)$, its derivative $g'(t)$ has at least one zero. Therefore, $g'(t)$ has at least $n + 1$ distinct zeros in (a, b). Applying Rolle's Theorem successively:

- $g''(t)$ has at least n distinct zeros in (a, b),
- $g'''(t)$ has at least $n - 1$ distinct zeros in (a, b),
- Continuing inductively, $g^{(n+1)}(t)$ has at least one zero in (a, b), which we denote by ξ_x.

We now differentiate $g(t)$ exactly $n + 1$ times with respect to t:

$$g^{(n+1)}(t) = f^{(n+1)}(t) - P_n^{(n+1)}(t) - \lambda\frac{d^{n+1}}{dt^{n+1}}[\omega_{n+1}(t)].$$

Since P_n has degree at most n, its $(n + 1)$-th derivative is identically zero: $P_n^{(n+1)}(t) \equiv 0$. The nodal polynomial $\omega_{n+1}(t) = t^{n+1} + \ldots$ is a monic polynomial of degree $n + 1$, so its $(n + 1)$-th derivative is $(n + 1)!$. Evaluating at $t = \xi_x$ gives

$$0 = g^{(n+1)}(\xi_x) = f^{(n+1)}(\xi_x) - 0 - \lambda(n + 1)!.$$

Solving for λ produces

$$\lambda = \frac{f^{(n+1)}(\xi_x)}{(n + 1)!}.$$

Equating this to our definition of $\lambda = (f(x) - P_n(x))/\omega_{n+1}(x)$ completes the proof. □

5.1.5 Runge's Phenomenon and Chebyshev Nodes

The error bound contains two decoupled components:

$$\|f - P_n\|_\infty \leq \frac{\|f^{(n+1)}\|_\infty}{(n+1)!}\|\omega_{n+1}\|_\infty.$$

The derivative term $\|f^{(n+1)}\|_\infty$ depends entirely on the analytical smoothness of f, while $\|\omega_{n+1}\|_\infty = \max_{x\in[a,b]} |\prod_{j=0}^{n}(x - x_j)|$ depends exclusively on the geometric placement of the interpolation nodes.

Remark 5.4 **(Runge phenomenon)** Analyticity alone does not make equally spaced interpolation converge. For Runge's function $f(x) = 1/(1 + 25x^2)$, the interpolants on equally spaced nodes develop increasing endpoint oscillations and satisfy

$$\lim_{n\to\infty} \|f - P_n\|_\infty = \infty,$$

even though f is analytic on the real interval. The growing Lebesgue constant amplifies the approximation error, while the nearby complex poles at $\pm i/5$ limit the rate of polynomial approximation. The derivative remainder bound above warns that convergence is not automatic, but a divergent upper bound by itself would not prove divergence.

The nodal factor can be made small by placing the nodes well. On $[-1, 1]$, the best choice is the set of Chebyshev points $x_k = \cos\big((2k+1)\pi/(2n+2)\big)$, $k = 0, \ldots, n$, the roots of the Chebyshev polynomial T_{n+1}. For these nodes $\|\omega_{n+1}\|_\infty = 2^{-n}$, the smallest value attained by any monic polynomial of degree $n + 1$. Section 5.2 proves this minimax property and shows that interpolation at Chebyshev points converges geometrically for functions that are analytic in a neighborhood of $[-1, 1]$.

Remark 5.5 The Lebesgue constant $\Lambda_n = \max_{x\in[a,b]} \sum_{i=0}^{n} |L_i(x)|$ bounds the stability of polynomial interpolation: $\|f - P_n\|_\infty \leq (1 + \Lambda_n) \min_{Q\in\mathcal{P}_n} \|f - Q\|_\infty$.

- For equally spaced nodes, $\Lambda_n \sim O\left(\frac{2^n}{n \log n}\right)$ grows exponentially.
- For Chebyshev nodes, $\Lambda_n \sim \frac{2}{\pi}\log(n+1)+O(1)$ grows only logarithmically, remaining near-optimal.

5.1.6 Piecewise Polynomials: Cubic Splines

When interpolating large sets of experimental data without prior knowledge of analytical smoothness, high-degree global polynomials are abandoned in favor of piecewise low-degree polynomials. A spline is a piecewise polynomial that maintains global derivative continuity across all internal nodes.

Def 5.2 **(Cubic Spline Interpolant)** Given $n + 1$ distinct points $a = x_0 < x_1 < \cdots < x_n = b$ and values $y_0, y_1, \ldots, y_n$, a cubic spline $S(x)$ is a function such that:

1. On each subinterval $[x_i, x_{i+1}]$, $S(x) = S_i(x)$ is a cubic polynomial:
$$S_i(x) = a_i + b_i(x - x_i) + c_i(x - x_i)^2 + d_i(x - x_i)^3, \qquad i = 0, \ldots, n-1.$$
2. Interpolation conditions: $S(x_i) = y_i$ for all $i = 0, \ldots, n$.
3. C^2 continuity: $S(x)$, $S'(x)$, and $S''(x)$ are continuous on the entire interval $[a, b]$.

Thm 5.4 **(The Tridiagonal Spline System)** Let $h_i = x_{i+1} - x_i$ and let $M_i = S''(x_i)$ denote the second derivative of the spline at node x_i. The condition that $S'(x)$ is continuous at each interior node x_i $(i = 1, \ldots, n-1)$ leads to the linear system
$$h_{i-1}M_{i-1} + 2(h_{i-1} + h_i)M_i + h_i M_{i+1} = 6\left(\frac{y_{i+1} - y_i}{h_i} - \frac{y_i - y_{i-1}}{h_{i-1}}\right).$$
To close the system of $n-1$ equations in $n+1$ unknowns, two boundary conditions are imposed:

1. **Natural (Free) Spline:** $M_0 = M_n = 0$.
2. **Clamped (Complete) Spline:** $S'(x_0) = f'(a)$ and $S'(x_n) = f'(b)$.
3. **Periodic Spline:** $S'(x_0) = S'(x_n)$ and $M_0 = M_n$.

For natural and clamped boundary conditions, the resulting linear system is strictly diagonally dominant and symmetric positive definite. It can be solved in $O(n)$ time using the Thomas algorithm.

Proof Since $S_i(x)$ is a cubic polynomial on $[x_i, x_{i+1}]$, its second derivative $S_i''(x)$ is a linear function interpolating $M_i = S''(x_i)$ and $M_{i+1} = S''(x_{i+1})$:
$$S_i''(x) = M_i\frac{x_{i+1} - x}{h_i} + M_{i+1}\frac{x - x_i}{h_i}.$$
Integrating twice with respect to x gives
$$S_i(x) = \frac{M_i(x_{i+1} - x)^3}{6h_i} + \frac{M_{i+1}(x - x_i)^3}{6h_i} + C_i(x_{i+1} - x) + D_i(x - x_i).$$
Applying the interpolation conditions $S_i(x_i) = y_i$ and $S_i(x_{i+1}) = y_{i+1}$ determines the constants C_i and D_i:
$$C_i = \frac{y_i}{h_i} - \frac{M_i h_i}{6}, \qquad D_i = \frac{y_{i+1}}{h_i} - \frac{M_{i+1}h_i}{6}.$$
Differentiating $S_i(x)$ to obtain $S_i'(x)$ and evaluating at the endpoints of $[x_i, x_{i+1}]$ yields
$$S_i'(x_i) = -\frac{M_i h_i}{3} - \frac{M_{i+1}h_i}{6} + \frac{y_{i+1} - y_i}{h_i},$$
$$S_{i-1}'(x_i) = \frac{M_{i-1}h_{i-1}}{6} + \frac{M_i h_{i-1}}{3} + \frac{y_i - y_{i-1}}{h_{i-1}}.$$

Equating $S'_{i-1}(x_i) = S'_i(x_i)$ for continuity of the first derivative and gathering terms produces the stated tridiagonal equation. Strict diagonal dominance follows from $2(h_{i-1} + h_i) > h_{i-1} + h_i$, which guarantees non-singularity and unconditional stability of Gaussian elimination without pivoting. $\square$

Thm 5.5 **(Minimum Curvature (Energy Minimization) Property)** Let $S(x)$ be the natural cubic spline interpolating $f(x)$ at $x_0, x_1, \ldots, x_n$. Let $g \in C^2[a, b]$ be any other function satisfying $g(x_i) = y_i$ for all $i = 0, \ldots, n$. Then

$$\int_a^b [S''(x)]^2 \, dx \leq \int_a^b [g''(x)]^2 \, dx.$$

Natural cubic splines are the smoothest possible interpolants in the sense of minimizing total bending energy.

Proof Let $h(x) = g(x) - S(x)$. Since both g and S interpolate f at the knots $x_0, x_1, \ldots, x_n$, we have

$$h(x_i) = g(x_i) - S(x_i) = y_i - y_i = 0, \qquad i = 0, 1, \ldots, n.$$

Expanding the squared second derivative of $g = S + h$:

$$\begin{aligned} \int_a^b [g''(x)]^2 \, dx &= \int_a^b [S''(x) + h''(x)]^2 \, dx \\ &= \int_a^b [S''(x)]^2 \, dx + 2 \int_a^b S''(x) h''(x) \, dx + \int_a^b [h''(x)]^2 \, dx. \end{aligned}$$

We evaluate the cross term by integrating by parts on each subinterval $[x_i, x_{i+1}]$:

$$\int_{x_i}^{x_{i+1}} S''(x) h''(x) \, dx = [S''(x) h'(x)]_{x_i}^{x_{i+1}} - \int_{x_i}^{x_{i+1}} S'''(x) h'(x) \, dx.$$

Because $S(x)$ is a cubic polynomial on $[x_i, x_{i+1}]$, its third derivative $S'''_i(x)$ is constant on (x_i, x_{i+1}). Let $S'''_i(x) = c_i$. Then

$$\int_{x_i}^{x_{i+1}} S'''(x) h'(x) \, dx = c_i \int_{x_i}^{x_{i+1}} h'(x) \, dx = c_i [h(x_{i+1}) - h(x_i)] = 0,$$

since $h(x_{i+1}) = h(x_i) = 0$. Summing the boundary terms over all subintervals:

$$\begin{aligned} \int_a^b S''(x) h''(x) \, dx &= \sum_{i=0}^{n-1} \left(S''(x_{i+1}^-) h'(x_{i+1}) - S''(x_i^+) h'(x_i) \right) \\ &= S''(b) h'(b) - S''(a) h'(a) + \sum_{i=1}^{n-1} \left(S''(x_i^-) - S''(x_i^+) \right) h'(x_i). \end{aligned}$$

Because $S \in C^2[a,b]$, the second derivative is continuous across interior knots: $S''(x_i^-) - S''(x_i^+) = 0$. For a natural cubic spline, $S''(a) = S''(b) = 0$, so the boundary terms vanish as well. Therefore

$$\int_a^b S''(x)h''(x)\,dx = 0.$$

Substituting this back yields

$$\int_a^b [g''(x)]^2\,dx = \int_a^b [S''(x)]^2\,dx + \int_a^b [h''(x)]^2\,dx \geq \int_a^b [S''(x)]^2\,dx,$$

with equality if and only if $h''(x) \equiv 0$, meaning h is linear and hence identically zero because $h(x_0) = h(x_n) = 0$. □

5.1.7 Exercises

Ex 5.1

1. **Compute.** Let $x_0 = 0, x_1 = 1, x_2 = 3$ and $y_0 = 1, y_1 = 3, y_2 = 2$. Construct the interpolating polynomial $P_2(x)$ using:

   (a) The Vandermonde matrix system.

   (b) The Lagrange basis form.

   (c) The Barycentric formula.

   Verify that all three approaches yield identical polynomials.

2. **Derive.** Compute the barycentric weights $w_0, \ldots, w_n$ explicitly for $n+1$ Chebyshev nodes on $[-1,1]$. Show that $w_k = (-1)^k \sin\left(\frac{2k+1}{2n+2}\pi\right)$ (or $(-1)^k\delta_k$ with $\delta_k = 1/2$ at boundary nodes for Chebyshev-Lobatto points).

3. **Check.** Use Thm 5.3 to bound the maximum error when $f(x) = \sin(x)$ is interpolated by a polynomial of degree 4 on $[0, \pi/2]$ using equally spaced nodes.

4. **Compute.** Set up the 2×2 tridiagonal linear system for the natural cubic spline interpolating $(0,0), (1,1), (2,0), (3,1)$. Solve for M_1 and M_2, and write down the explicit cubic polynomial for each subinterval.

5. **Explore.** For the spline in the previous exercise, compute its bending energy $\int_0^3 (S'')^2\,dx$ and compare it with the energy of one other twice-differentiable interpolant.

5.2 Chebyshev Polynomials and Minimax Approximation

Chebyshev polynomials provide an orthogonal polynomial basis on $[-1,1]$ and a principled choice of nodes for high-degree interpolation. Under the coordinate mapping $x = \cos\theta$,

algebraic polynomials become cosine series. The scaled polynomial $2^{1-n}T_n(x)$ uniquely minimizes the uniform norm among monic polynomials of degree n. Chebyshev nodes replace the exponential growth of the uniform-grid Lebesgue constant by logarithmic growth, and analytic functions then admit geometrically convergent approximations.

5.2.1 Two Failures of Polynomial Interpolation

Approximating a continuous function $f \in C([-1,1])$ by polynomial interpolation encounters two distinct failure modes. The first is algebraic. Vandermonde systems in the monomial basis are often ill-conditioned, so a coefficient calculation may lose accuracy even when the interpolating polynomial is well defined. The second is analytical. For functions such as Runge's example, interpolation at equally spaced nodes diverges near the endpoints even in exact arithmetic. These are different failures and require different remedies.

For the analytical failure, Thm 5.3 gives the remedy. With n nodes $x_1, \dots, x_n$ and the monic nodal polynomial $\omega_n(x) = \prod_{j=1}^{n}(x - x_j)$, the theorem bounds the interpolation error by

$$\|f - p_{n-1}\|_\infty \le \frac{\|f^{(n)}\|_\infty}{n!} \, \|\omega_n\|_\infty.$$

The first factor depends only on f, and the second only on the nodes. Choosing the nodes to minimize $\|\omega_n\|_\infty$ is therefore a minimax problem over monic polynomials of degree n, which this section solves.

5.2.2 Chebyshev Polynomials and Fundamental Identities

Def 5.3 **(Chebyshev Polynomials of the First Kind)** For $x \in [-1,1]$, the Chebyshev polynomial of degree k is defined by

$$T_k(x) = \cos(k \arccos x).$$

Under the substitution $x = \cos\theta$ with $\theta \in [0, \pi]$,

$$T_k(\cos\theta) = \cos(k\theta).$$

Lem 5.6 **(Chebyshev Three-Term Recurrence)** The functions $T_k(x)$ are algebraic polynomials satisfying the three-term recurrence relation

$$T_0(x) = 1, \qquad T_1(x) = x, \qquad T_{k+1}(x) = 2xT_k(x) - T_{k-1}(x) \quad \text{for } k \ge 1.$$

Furthermore, for every $k \ge 1$, the leading coefficient of $T_k(x)$ is 2^{k-1}.

Proof Using the trigonometric angle sum and difference identities,

$$\cos((k+1)\theta) + \cos((k-1)\theta) = 2\cos\theta\cos(k\theta).$$

Substituting $\theta = \arccos x$, so that $\cos\theta = x$, yields

$$T_{k+1}(x) + T_{k-1}(x) = 2xT_k(x),$$

which establishes the recurrence.
For the initial conditions, $T_0(x) = \cos(0) = 1$ and $T_1(x) = \cos(\arccos x) = x$. Proceeding by induction, if $T_k(x)$ has degree k with leading coefficient 2^{k-1}, then $2xT_k(x)$ has degree $k+1$ with leading coefficient $2 \cdot 2^{k-1} = 2^k$. Because T_{k-1} has degree $k-1 < k+1$, it does not affect the leading coefficient, confirming that $T_{k+1}(x)$ has leading coefficient 2^k. □

Thm 5.7 **(Orthogonality of Chebyshev Polynomials)** Chebyshev polynomials are orthogonal on $[-1, 1]$ with respect to the singular weight function $w(x) = (1-x^2)^{-1/2}$:

$$\int_{-1}^{1} T_j(x)T_k(x)\frac{dx}{\sqrt{1-x^2}} = \begin{cases} 0, & j \neq k, \\ \pi, & j = k = 0, \\ \frac{\pi}{2}, & j = k \geq 1. \end{cases}$$

Proof Substitute $x = \cos\theta$. As x ranges from -1 to 1, θ ranges from π to 0. The differential transforms as $dx = -\sin\theta\, d\theta$, and the denominator becomes $\sqrt{1-x^2} = \sin\theta$. The integral transforms to

$$\int_{-1}^{1} T_j(x)T_k(x)\frac{dx}{\sqrt{1-x^2}} = \int_{\pi}^{0} \cos(j\theta)\cos(k\theta)\frac{-\sin\theta\, d\theta}{\sin\theta} = \int_{0}^{\pi} \cos(j\theta)\cos(k\theta)\, d\theta.$$

By the orthogonality of the trigonometric cosine basis on $[0, \pi]$, this integral evaluates to 0 when $j \neq k$, to π when $j = k = 0$, and to $\pi/2$ when $j = k \geq 1$. □

5.2.3 Roots, Extrema, and the Nodal Error Polynomial

Thm 5.8 **(Zeros and Extreme Points of T_n)** For each $n \geq 1$, the polynomial $T_n(x)$ has:

1. Exactly n distinct real zeros in $(-1, 1)$, given by

$$x_j = \cos\left(\frac{2j-1}{2n}\pi\right), \qquad j = 1, \ldots, n.$$

2. Exactly $n+1$ alternating extreme points on $[-1, 1]$, given by

$$y_j = \cos\left(\frac{j\pi}{n}\right), \qquad j = 0, \ldots, n,$$

at which $T_n(y_j) = (-1)^j$.

Proof The condition $T_n(x) = 0$ corresponds to $\cos(n\theta) = 0$ for $\theta \in [0, \pi]$. The cosine function vanishes when $n\theta = (j - 1/2)\pi = \frac{2j-1}{2}\pi$ for integers j. Restricting $\theta \in [0, \pi]$ restricts $j \in \{1, \dots, n\}$. Setting $x_j = \cos\theta_j$ gives the n distinct roots.
Similarly, $T_n(x)$ attains its extreme values ± 1 when $|\cos(n\theta)| = 1$, which requires $n\theta = j\pi$ for $j \in \{0, \dots, n\}$. At these points, $T_n(y_j) = \cos(j\pi) = (-1)^j$. □

Def 5.4 **(Chebyshev Nodes)** The Chebyshev interpolation nodes of degree n are the n roots of $T_n(x)$:

$$x_j = \cos\left(\frac{2j-1}{2n}\pi\right), \qquad j = 1, \dots, n.$$

These nodes cluster quadratically near the interval endpoints ± 1, with local separation $x_{j+1} - x_j \sim O(1/n^2)$ near the boundaries and $O(1/n)$ in the interior.

Thm 5.9 **(Representation of the Chebyshev Nodal Polynomial)** Let $\{x_j\}_{j=1}^n$ be the Chebyshev nodes from Def 5.4. The monic nodal polynomial satisfies

$$\omega_n(x) = \prod_{j=1}^{n}(x - x_j) = 2^{1-n}T_n(x).$$

Consequently, the uniform norm of the nodal polynomial is

$$\|\omega_n\|_\infty = 2^{1-n}\|T_n\|_\infty = 2^{1-n}.$$

Proof Both $\omega_n(x)$ and $T_n(x)$ are polynomials of degree n sharing the same n distinct roots $x_1, \dots, x_n$. Therefore, they are proportional: $T_n(x) = C\omega_n(x)$ for some nonzero constant C. By Lem 5.6, the leading coefficient of $T_n(x)$ is 2^{n-1}, whereas $\omega_n(x)$ is monic with leading coefficient 1. Hence $C = 2^{n-1}$, which gives $\omega_n(x) = 2^{1-n}T_n(x)$.
Because $T_n(\cos\theta) = \cos(n\theta)$, $|T_n(x)| \le 1$ for all $x \in [-1, 1]$, and this maximum is attained at $x = 1$. Thus $\|T_n\|_\infty = 1$, which confirms that $\|\omega_n\|_\infty = 2^{1-n}$. □

5.2.4 The Minimax Property and Equioscillation

Thm 5.10 **(Chebyshev Minimax Theorem)** Let $\widetilde{\mathcal{P}}_n$ denote the set of all monic polynomials of degree n with real coefficients. Then $2^{1-n}T_n$ is the unique solution to the minimax problem on $[-1, 1]$:

$$\min_{q \in \widetilde{\mathcal{P}}_n} \|q\|_\infty = \|2^{1-n}T_n\|_\infty = 2^{1-n}.$$

Proof Let $m(x) = 2^{1-n}T_n(x) \in \widetilde{\mathcal{P}}_n$. By Thm 5.9, $\|m\|_\infty = 2^{1-n}$. At the $n + 1$ extreme points $y_j = \cos(j\pi/n)$ for $j = 0, \dots, n$,

$$m(y_j) = 2^{1-n}(-1)^j.$$

Suppose there exists a monic polynomial $q \in \widetilde{\mathcal{P}}_n$ such that $\|q\|_\infty < 2^{1-n}$. Consider the difference polynomial

$$r(x) = m(x) - q(x).$$

Because both m and q are monic polynomials of degree n, the leading terms x^n cancel. Thus $r \in \mathcal{P}_{n-1}$ has degree at most $n-1$.
Evaluating $r(x)$ at each extreme point y_j:

$$r(y_j) = m(y_j) - q(y_j) = 2^{1-n}(-1)^j - q(y_j).$$

Because $|q(y_j)| \le \|q\|_\infty < 2^{1-n}$, the sign of $r(y_j)$ is dictated entirely by $m(y_j)$:

- When j is even, $m(y_j) = 2^{1-n}$, so $r(y_j) = 2^{1-n} - q(y_j) > 0$.
- When j is odd, $m(y_j) = -2^{1-n}$, so $r(y_j) = -2^{1-n} - q(y_j) < 0$.

Thus $r(x)$ alternates sign across the $n+1$ ordered points $1 = y_0 > y_1 > \cdots > y_n = -1$. By the Intermediate Value Theorem, $r(x)$ must vanish at least once in each of the n disjoint open intervals (y_{j+1}, y_j) for $j = 0, \ldots, n-1$.
Consequently, $r(x)$ has at least n distinct real zeros in $(-1, 1)$. However, a nonzero polynomial of degree at most $n-1$ cannot have n roots. Therefore, $r(x)$ must be identically zero, which contradicts the strict inequality $\|q\|_\infty < 2^{1-n}$. This proves that $\|q\|_\infty \ge 2^{1-n}$ for all monic $q \in \widetilde{\mathcal{P}}_n$.
To establish uniqueness, suppose $\|q\|_\infty = 2^{1-n}$ and retain $r = m - q \in \mathcal{P}_{n-1}$. The weak inequalities

$$(-1)^j r(y_j) \ge 0, \qquad j = 0, \ldots, n,$$

follow from $m(y_j) = 2^{1-n}(-1)^j$ and $|q(y_j)| \le 2^{1-n}$. Because r has degree at most $n-1$, its divided difference of order n at the distinct points $y_0, \ldots, y_n$ is zero:

$$0 = r[y_0, \ldots, y_n] = \sum_{j=0}^{n} \frac{r(y_j)}{\prod_{k \ne j}(y_j - y_k)}.$$

Because the nodes are ordered $1 = y_0 > y_1 > \cdots > y_n = -1$, the product in the denominator has exactly j negative factors:

$$\operatorname{sign}\left(\prod_{k \ne j}(y_j - y_k)\right) = \operatorname{sign}\left(\prod_{k<j}(y_j - y_k)\prod_{k>j}(y_j - y_k)\right) = (-1)^j.$$

Therefore

$$\frac{r(y_j)}{\prod_{k \ne j}(y_j - y_k)} = \frac{(-1)^j r(y_j)}{\left|\prod_{k \ne j}(y_j - y_k)\right|} \ge 0.$$

A sum of non-negative terms can vanish only if each numerator $(-1)^j r(y_j)$ is zero. Thus $r(y_j) = 0$ for all $j = 0, \ldots, n$. Since $r \in \mathcal{P}_{n-1}$ has $n+1$ distinct roots, $r \equiv 0$, proving $q \equiv m$. □

Remark 5.6 **(Lebesgue constant growth)** For any grid of n interpolation nodes X_n, the interpolation projection $I_n : C([-1,1]) \to \mathcal{P}_{n-1}$ satisfies the Lebesgue inequality $\|f - I_n f\|_\infty \leq (1+\Lambda_n) \min_{p\in\mathcal{P}_{n-1}} \|f-p\|_\infty$, where Λ_n is the Lebesgue constant. For equally spaced nodes, $\Lambda_n \sim \frac{2^n}{en\log n}$, which diverges exponentially and induces Runge's phenomenon. For Chebyshev nodes, $\Lambda_n \leq \frac{2}{\pi}\log n + 1$, guaranteeing that Chebyshev interpolation converges at a rate within a logarithmic factor of the optimal polynomial approximation.

5.2.5 Chebyshev Series and Spectral Accuracy

Def 5.5 **(Chebyshev Series Expansion)** A function with a convergent Chebyshev expansion is written using the primed-sum convention

$$f(x) = \sum_{k=0}^{\infty}{}' c_k T_k(x),$$

where the prime on the summation indicates that the initial term $k = 0$ carries a factor of $1/2$:

$$c_k = \frac{2}{\pi}\int_{-1}^{1} f(x)T_k(x)\frac{dx}{\sqrt{1-x^2}} = \frac{2}{\pi}\int_0^\pi f(\cos\theta)\cos(k\theta)\,d\theta.$$

Thm 5.11 **(Geometric Convergence for Analytic Functions)** Let f be analytic on and inside a Bernstein ellipse whose boundary is

$$\mathcal{E}_\rho = \left\{ z \in \mathbb{C} : z = \frac{w + w^{-1}}{2},\ |w| = \rho \right\}$$

with $\rho > 1$, semi-axes $a = \frac{\rho+\rho^{-1}}{2}$ and $b = \frac{\rho-\rho^{-1}}{2}$. Suppose $|f(z)| \leq M$ on this boundary. Then the Chebyshev coefficients satisfy

$$|c_k| \leq \frac{2M}{\rho^k} \qquad \text{for all } k \geq 0.$$

Consequently, the truncated polynomial approximation $p_n(x) = \sum_{k=0}^{n}{}' c_k T_k(x)$ converges geometrically:

$$\|f - p_n\|_\infty \leq \frac{2M}{\rho - 1}\rho^{-n}.$$

Proof Under the conformal map $z = \frac{w+w^{-1}}{2}$, the circle $|w| = r$ for $1 < r < \rho$ maps to the ellipse $\mathcal{E}_r$ in the complex z-plane. The transformed function $F(w) = f\left(\frac{w+w^{-1}}{2}\right)$ is analytic in the annulus $r^{-1} < |w| < r$ and satisfies $F(w) = F(w^{-1})$.

Expanding $F(w)$ in its Laurent series $F(w) = \sum_{k=-\infty}^{\infty} a_k w^k$, the symmetry $F(w) = F(w^{-1})$ implies $a_k = a_{-k}$. On the unit circle $w = e^{i\theta}$, $F(e^{i\theta}) = f(\cos\theta) = a_0 + 2\sum_{k=1}^{\infty} a_k \cos(k\theta)$, so the Chebyshev coefficients are related to the Laurent coefficients by $c_k = 2a_k$ for $k \geq 1$. By Cauchy's Integral Formula, for any $1 < r < \rho$,

$$a_k = \frac{1}{2\pi i} \oint_{|w|=r} \frac{F(w)}{w^{k+1}}\, dw.$$

Taking absolute values and using $|F(w)| \leq M$ gives $|a_k| \leq Mr^{-k}$. Taking the infimum over $r \to \rho^-$ yields $|c_k| \leq 2M\rho^{-k}$. Summing the tail of the series gives the uniform error bound:

$$\|f - p_n\|_\infty \leq \sum_{k=n+1}^{\infty} |c_k| \leq 2M \sum_{k=n+1}^{\infty} \rho^{-k} = \frac{2M\rho^{-n}}{\rho - 1}.$$

□

5.2.6 Exercises

Ex 5.2

1. **Derive. (Clenshaw's evaluation algorithm)** Evaluating a Chebyshev sum $p(x) = \sum_{k=0}^{n} c_k T_k(x)$ by explicitly forming $T_k(x)$ is inefficient and prone to floating-point error. Clenshaw's recurrence evaluates the sum in $O(n)$ flops without computing trigonometric functions.

   (a) Define the backward sequence $b_{n+2} = b_{n+1} = 0$, and for $k = n, n-1, \ldots, 1$,

   $$b_k = 2xb_{k+1} - b_{k+2} + c_k.$$

   Show that $p(x) = c_0 + xb_1 - b_2$.

   (b) Prove that this backward recurrence is algebraically equivalent to evaluating $p(x)$ by substituting the three-term relation $T_{k+1} = 2xT_k - T_{k-1}$.

   (c) Implement Clenshaw's algorithm in Python and compare its runtime against explicit monomial expansion for $n = 50$.

2. **Check. (Equioscillation on a concrete polynomial)** Let $m(x) = 2^{1-4}T_4(x)$.

   (a) Expand m in the monomial basis and verify that it is monic.

   (b) Evaluate m at $y_j = \cos(j\pi/4)$ for $j = 0, \ldots, 4$ and verify the alternating values $\pm 1/8$.

   (c) Choose any monic quartic $q \neq m$, evaluate it on a fine grid, and confirm numerically that $\|q\|_\infty > 1/8$, as predicted by Thm 5.10.

3. **Compute. (Lebesgue constant computation and Runge comparison)** The Lebesgue function is $\lambda_n(x) = \sum_{j=1}^{n} |\ell_j(x)|$, where $\ell_j(x)$ are the Lagrange cardinal polynomials.

(a) Write a script to evaluate $\lambda_n(x)$ on a fine grid of 10^4 points in $[-1, 1]$ for uniform nodes and Chebyshev nodes for $n \in \{5, 10, 15, 20\}$.

(b) Plot the maximum $\Lambda_n = \max_{x\in[-1,1]} \lambda_n(x)$ versus n on a semilog scale. Verify that Λ_n grows exponentially for uniform nodes and logarithmically for Chebyshev nodes.

(c) Interpolate Runge's function $f(x) = 1/(1 + 25x^2)$ with $n = 20$ using both node sets. Explain the boundary behavior in terms of the Bernstein ellipse parameter ρ.

4. **Derive. (Chebyshev differentiation matrix)** Let $\boldsymbol{x} = (x_0, x_1, \ldots, x_n)^T$ be the Chebyshev extreme nodes $x_j = \cos(j\pi/n)$.

(a) Derive the entries of the spectral differentiation matrix $\mathbf{D}_n \in \mathbb{R}^{(n+1)\times(n+1)}$ that maps the nodal values $\mathbf{f} = (f(x_0), \ldots, f(x_n))^T$ to the exact derivative values of the polynomial interpolant $\mathbf{f}' = \mathbf{D}_n\mathbf{f}$.

(b) Show that the diagonal entries satisfy $D_{00} = \frac{2n^2+1}{6}$ and $D_{nn} = -\frac{2n^2+1}{6}$.

(c) Verify that the row sums of $\mathbf{D}_n$ vanish identically: $\mathbf{D}_n\mathbf{1} = \mathbf{0}$.

5. **Explore. (Bernstein ellipse parameter for Runge's function)** Consider $f(x) = \frac{1}{1+25x^2}$ on $[-1, 1]$.

(a) Find the complex poles of $f(z)$ in $\mathbb{C}$.

(b) Determine the largest value of ρ such that the Bernstein ellipse $\mathcal{E}_\rho$ does not contain either pole.

(c) Using Thm 5.11, compute the asymptotic error decay factor ρ^{-1} and predict how many Chebyshev modes are needed to achieve machine precision $\|f - p_n\|_\infty \leq 10^{-16}$.

5.3 Numerical Quadrature

Numerical quadrature approximates the definite integral of a continuous function $f : [a, b] \to \mathbb{R}$ by a finite linear combination of sampled function values:

$$I(f) = \int_a^b f(x)\,dx \approx Q_n(f) = \sum_{i=0}^{n} w_i f(x_i).$$

The points $x_i \in [a, b]$ are the quadrature nodes, and the scalar coefficients $w_i \in \mathbb{R}$ are the quadrature weights. The design of quadrature rules balances algebraic precision, computational cost, and numerical stability.

5.3.1 Degree of Exactness and Interpolatory Quadrature

The standard criterion for evaluating a quadrature rule is its algebraic precision, defined by the highest degree of polynomials it integrates without error.

Def 5.6 **(Degree of Exactness (Algebraic Precision))** A quadrature rule Q_n has degree of exactness m if

$$Q_n(p) = \int_a^b p(x)\,dx \qquad \text{for all } p \in \mathcal{P}_m,$$

and there exists at least one polynomial $q \in \mathcal{P}_{m+1}$ for which $Q_n(q) \neq \int_a^b q(x)\,dx$.

Thm 5.12 **(Interpolatory Quadrature Rules)** Let $x_0, x_1, \ldots, x_n$ be $n+1$ distinct nodes in $[a,b]$. The interpolatory quadrature rule approximates $I(f)$ by integrating the polynomial interpolant $P_n(x) = \sum_{i=0}^n f(x_i)L_i(x)$ in Lagrange form:

$$Q_n(f) = \int_a^b P_n(x)\,dx = \sum_{i=0}^n w_i f(x_i), \qquad w_i = \int_a^b L_i(x)\,dx.$$

Every interpolatory quadrature rule with $n+1$ distinct nodes has degree of exactness at least n.

Proof Let $p \in \mathcal{P}_n$. By the uniqueness of polynomial interpolation (Thm 5.1), the interpolant of p is identical to p itself: $P_n(x) \equiv p(x)$. Integrating this identity yields

$$\int_a^b p(x)\,dx = \int_a^b P_n(x)\,dx = \int_a^b \left(\sum_{i=0}^n p(x_i)L_i(x)\right) dx = \sum_{i=0}^n p(x_i)\int_a^b L_i(x)\,dx = \sum_{i=0}^n w_i p(x_i) = Q_n(p).$$

Therefore, Q_n integrates all polynomials of degree at most n exactly. □

5.3.2 Newton–Cotes Formulas and Composite Rules

Newton–Cotes rules are interpolatory quadrature formulas built on equally spaced nodes. Because high-order single-panel Newton–Cotes rules suffer from negative weights and Runge's phenomenon, practical computations use composite rules on small subintervals of uniform width $h = (b-a)/N$.

Def 5.7 **(Composite Trapezoidal and Simpson's Rules)** Dividing $[a,b]$ into N subintervals with uniform width $h = (b-a)/N$ and grid points $x_j = a + jh$:

1. The **Composite Trapezoidal Rule** is

$$T_N(f) = h\left[\frac{f(a)+f(b)}{2} + \sum_{j=1}^{N-1} f(x_j)\right].$$

2. For an even number of panels N, the **Composite Simpson's Rule** is

$$S_N(f) = \frac{h}{3}\left[f(x_0) + 4\sum_{\substack{j=1\\ j \text{ odd}}}^{N-1} f(x_j) + 2\sum_{\substack{j=2\\ j \text{ even}}}^{N-2} f(x_j) + f(x_N)\right].$$

Thm 5.13 **(Composite Error Formulas)** Let $f \in C^2[a,b]$ for the trapezoidal rule, and let $f \in C^4[a,b]$ for Simpson's rule.

1. The composite trapezoidal error satisfies

$$I(f) - T_N(f) = -\frac{b-a}{12}h^2 f''(\xi_T), \qquad \xi_T \in (a,b).$$

2. The composite Simpson error satisfies

$$I(f) - S_N(f) = -\frac{b-a}{180}h^4 f^{(4)}(\xi_S), \qquad \xi_S \in (a,b).$$

Proof For the trapezoidal rule on panel $[x_j, x_{j+1}]$, the Cauchy interpolation error formula (Thm 5.3) gives

$$f(x) - P_1(x) = \frac{f''(\xi_j(x))}{2}(x - x_j)(x - x_{j+1}).$$

Because $(x - x_j)(x - x_{j+1}) \le 0$ on this panel, the Mean Value Theorem for Integrals implies there exists $\mu_j \in (x_j, x_{j+1})$ such that

$$\int_{x_j}^{x_{j+1}} [f(x) - P_1(x)]\,dx = \frac{f''(\mu_j)}{2}\int_{x_j}^{x_{j+1}} (x - x_j)(x - x_{j+1})\,dx = -\frac{h^3}{12}f''(\mu_j).$$

Summing across all N panels and applying the Intermediate Value Theorem to the average $\frac{1}{N}\sum_{j=0}^{N-1} f''(\mu_j) = f''(\xi_T)$ yields the trapezoidal error formula:

$$I(f) - T_N(f) = -\frac{h^3}{12}\sum_{j=0}^{N-1} f''(\mu_j) = -\frac{Nh^3}{12}f''(\xi_T) = -\frac{b-a}{12}h^2 f''(\xi_T).$$

For Simpson's rule, expand $f(x_{2j+1} + t)$ about the midpoint $c_j = x_{2j+1}$ through fourth order for $t \in [-h, h]$:

$$f(c_j + t) = f(c_j) + tf'(c_j) + \frac{t^2}{2}f''(c_j) + \frac{t^3}{6}f'''(c_j) + \frac{t^4}{24}f^{(4)}(c_j) + O(t^5).$$

Integrating over $[-h, h]$, odd powers vanish by symmetry:

$$\int_{c_j-h}^{c_j+h} f(x)\,dx = \int_{-h}^{h} f(c_j + t)\,dt = 2hf(c_j) + \frac{h^3}{3}f''(c_j) + \frac{h^5}{60}f^{(4)}(c_j) + O(h^7).$$

The Simpson rule on this double-panel evaluates to

$$\begin{aligned} S_{\text{panel}} &= \frac{h}{3}\left[f(c_j-h)+4f(c_j)+f(c_j+h)\right] \\ &= \frac{h}{3}\left[6f(c_j)+h^2 f''(c_j)+\frac{h^4}{12}f^{(4)}(c_j)+O(h^6)\right] \\ &= 2hf(c_j)+\frac{h^3}{3}f''(c_j)+\frac{h^5}{36}f^{(4)}(c_j)+O(h^7). \end{aligned}$$

Subtracting gives the single-panel error:

$$\int_{c_j-h}^{c_j+h} f(x)\,dx - S_{\text{panel}} = \left(\frac{1}{60}-\frac{1}{36}\right)h^5 f^{(4)}(\mu_j) = -\frac{h^5}{90}f^{(4)}(\mu_j).$$

Summing over all $N/2$ double panels and applying the Intermediate Value Theorem gives

$$I(f)-S_N(f) = -\frac{h^5}{90}\sum_{j=0}^{N/2-1} f^{(4)}(\mu_j) = -\frac{Nh^5}{180}f^{(4)}(\xi_S) = -\frac{b-a}{180}h^4 f^{(4)}(\xi_S).$$

□

5.3.3 Peano Kernel Error Representation

The Peano kernel theorem provides a unified framework for representing the error of any quadrature rule with algebraic degree of exactness m.

Thm 5.14 **(Peano Kernel Theorem)** Let Q be a quadrature rule on $[a,b]$ with error functional $E(f)=\int_a^b f(x)\,dx - Q(f)$. If $E(p)=0$ for all $p\in\mathcal{P}_m$, then for every $f\in C^{m+1}[a,b]$,

$$E(f)=\int_a^b f^{(m+1)}(t)K(t)\,dt,$$

where the Peano kernel $K(t)$ is defined by applying the error functional to the truncated power function:

$$K(t)=\frac{1}{m!}E_x\left((x-t)_+^m\right), \qquad (x-t)_+=\max(0,x-t).$$

If $K(t)$ does not change sign on $[a,b]$, then there exists $\xi\in(a,b)$ such that

$$E(f)=\frac{f^{(m+1)}(\xi)}{(m+1)!}E(x^{m+1}).$$

Proof By Taylor's theorem with integral remainder expanded about a:

$$f(x)=\sum_{k=0}^{m}\frac{f^{(k)}(a)}{k!}(x-a)^k+\frac{1}{m!}\int_a^x f^{(m+1)}(t)(x-t)^m\,dt.$$

Extending the upper integration limit using the truncated power notation $(x-t)_+^m$ allows writing the remainder as $\frac{1}{m!}\int_a^b f^{(m+1)}(t)(x-t)_+^m\,dt$. Applying the linear functional E to both sides, the polynomial terms vanish because $E(p)=0$ for all $p\in\mathcal{P}_m$. Exchanging the order of integration and the functional gives

$$E(f)=E_x\left(\frac{1}{m!}\int_a^b f^{(m+1)}(t)(x-t)_+^m\,dt\right)=\int_a^b f^{(m+1)}(t)\left[\frac{1}{m!}E_x\left((x-t)_+^m\right)\right]dt=\int_a^b f^{(m+1)}(t)K(t)\,d$$

If $K(t)$ is of constant sign, the Mean Value Theorem for Integrals implies $E(f)=f^{(m+1)}(\xi)\int_a^b K(t)\,dt$. Testing the identity with $f(x)=x^{m+1}$, we have $f^{(m+1)}(x)=(m+1)!$, which yields $E(x^{m+1})=(m+1)!\int_a^b K(t)\,dt$. Substituting this integral proves the claim. □

5.3.4 Gauss–Legendre Quadrature

Newton–Cotes rules fix equispaced nodes and optimize only the weights. Gaussian quadrature optimizes both the nodes $x_1,\dots,x_p$ and the weights $w_1,\dots,w_p$, providing $2p$ degrees of freedom to maximize the degree of exactness.

Lem 5.15 **(Roots of Orthogonal Polynomials)** Let $P_p(x)$ be the degree-p Legendre polynomial orthogonal on $[-1,1]$ with respect to weight $w(x)\equiv 1$:

$$\int_{-1}^{1} P_p(x)q(x)\,dx=0 \qquad \text{for all } q\in\mathcal{P}_{p-1}.$$

Then $P_p(x)$ has exactly p distinct, real roots, and all roots lie in the open interval $(-1,1)$.

Proof Let $x_1,\dots,x_k$ denote the points in $(-1,1)$ where $P_p(x)$ changes sign. Because P_p is continuous and orthogonal to the constant polynomial $q(x)=1$, we have $\int_{-1}^1 P_p(x)\,dx=0$, so P_p must change sign at least once in $(-1,1)$, meaning $k\geq 1$. Suppose for contradiction that $k<p$. Construct the polynomial $q(x)=\prod_{j=1}^k(x-x_j)\in\mathcal{P}_k$. By construction, $P_p(x)$ and $q(x)$ change sign at identical points in $(-1,1)$, so their product $P_p(x)q(x)$ has constant sign on $[-1,1]$ and is not identically zero. Consequently,

$$\int_{-1}^{1} P_p(x)q(x)\,dx\neq 0.$$

However, because $\deg(q)=k<p$, orthogonality requires $\int_{-1}^1 P_p(x)q(x)\,dx=0$, producing a contradiction. Therefore, $k=p$, proving that P_p has p distinct real roots in $(-1,1)$. □

Thm 5.16 **(Fundamental Theorem of Gaussian Quadrature)** Let $x_1,\dots,x_p$ be the roots of the degree-p Legendre polynomial $P_p(x)$ on $[-1,1]$, and let $w_i=\int_{-1}^1 L_i(x)\,dx$. The p-point Gauss–Legendre quadrature rule

$$\int_{-1}^{1} f(x)\,dx\approx\sum_{i=1}^{p} w_i f(x_i)$$

has degree of exactness $2p-1$. No p-point quadrature rule can achieve a degree of exactness higher than $2p-1$. Furthermore, all weights are strictly positive: $w_i > 0$ for all $i = 1, \ldots, p$.

Proof Let $f \in \mathcal{P}_{2p-1}$. Polynomial long division of $f(x)$ by $P_p(x)$ yields

$$f(x) = q(x)P_p(x) + r(x), \qquad \deg(q) \leq p-1, \quad \deg(r) \leq p-1.$$

Integrating both sides over $[-1, 1]$:

$$\int_{-1}^{1} f(x)\,dx = \int_{-1}^{1} q(x)P_p(x)\,dx + \int_{-1}^{1} r(x)\,dx.$$

Because $\deg(q) \leq p-1$, orthogonality of P_p against $\mathcal{P}_{p-1}$ implies $\int_{-1}^{1} q(x)P_p(x)\,dx = 0$, giving

$$\int_{-1}^{1} f(x)\,dx = \int_{-1}^{1} r(x)\,dx.$$

Applying the p-point quadrature sum at the Legendre roots $\{x_i\}_{i=1}^{p}$ where $P_p(x_i) = 0$:

$$\begin{aligned}\sum_{i=1}^{p} w_i f(x_i) &= \sum_{i=1}^{p} w_i \left[q(x_i)P_p(x_i) + r(x_i)\right] \\ &= \sum_{i=1}^{p} w_i \left[0 + r(x_i)\right] = \sum_{i=1}^{p} w_i r(x_i).\end{aligned}$$

Because $\deg(r) \leq p-1$, interpolatory quadrature on p distinct nodes integrates $r(x)$ exactly by Thm 5.12:

$$\sum_{i=1}^{p} w_i r(x_i) = \int_{-1}^{1} r(x)\,dx = \int_{-1}^{1} f(x)\,dx.$$

Thus $\sum_{i=1}^{p} w_i f(x_i) = \int_{-1}^{1} f(x)\,dx$ for all $f \in \mathcal{P}_{2p-1}$. To prove that $2p-1$ is the maximal degree, define the nodal polynomial

$$g(x) = \prod_{i=1}^{p} (x - x_i)^2 \in \mathcal{P}_{2p}.$$

Because $g(x) \geq 0$ and vanishes only at isolated points, its integral is strictly positive:

$$\int_{-1}^{1} g(x)\,dx > 0.$$

However, evaluating the quadrature sum yields

$$\sum_{i=1}^{p} w_i g(x_i) = \sum_{i=1}^{p} w_i \cdot 0 = 0 \neq \int_{-1}^{1} g(x)\,dx.$$

Therefore no p-point quadrature rule can integrate all degree-$2p$ polynomials. For weight positivity, let $L_i \in \mathcal{P}_{p-1}$ be the i-th Lagrange basis polynomial. The square satisfies $\deg(L_i^2) = 2p-2 \le 2p-1$. The rule integrates L_i^2 exactly:

$$0 < \int_{-1}^{1} L_i^2(x)\,dx = \sum_{j=1}^{p} w_j L_i^2(x_j) = \sum_{j=1}^{p} w_j \delta_{ij} = w_i.$$

Every weight w_i is strictly positive. □

5.3.5 Richardson Extrapolation and Romberg Integration

When the error of a numerical method admits an asymptotic expansion in powers of h, Richardson extrapolation algebraically cancels the leading error terms.

Thm 5.17 **(Euler–Maclaurin Summation and Romberg Integration)** Let $f \in C^{2m+2}[a,b]$. The composite trapezoidal rule satisfies the asymptotic expansion

$$T(h) = I(f) + c_2 h^2 + c_4 h^4 + c_6 h^6 + \cdots + c_{2m} h^{2m} + O(h^{2m+2}),$$

where $c_{2k} = \frac{B_{2k}}{(2k)!}[f^{(2k-1)}(b) - f^{(2k-1)}(a)]$ depend on the Bernoulli numbers B_{2k}. Romberg integration systematically eliminates powers of h^2 via the tableau recurrence

$$R(k,0) = T(h_k), \qquad R(k,j) = \frac{4^j R(k,j-1) - R(k-1,j-1)}{4^j - 1} \quad (1 \le j \le k),$$

where $h_k = (b-a)/2^k$. Column j achieves order $O(h^{2j+2})$.

Proof At column 1, $R(k,0) = I(f) + c_2 h_k^2 + O(h_k^4)$ and $R(k-1,0) = I(f) + c_2(2h_k)^2 + O(h_k^4) = I(f) + 4c_2 h_k^2 + O(h_k^4)$. Forming the linear combination

$$\frac{4R(k,0) - R(k-1,0)}{3} = \frac{4(I(f) + c_2 h_k^2) - (I(f) + 4c_2 h_k^2)}{3} + O(h_k^4) = I(f) + O(h_k^4)$$

cancels the h^2 error term. Induction on j shows that combining step sizes differing by a factor of 2 cancels the leading h^{2j} error term with weights $\frac{4^j}{4^j-1}$ and $-\frac{1}{4^j-1}$. □

5.3.6 Periodic Functions and Spectral Convergence

Thm 5.18 **(Spectral Accuracy of the Trapezoidal Rule on Periodic Domains)** Let f be a smooth, 2π-periodic function on $\mathbb{R}$. All endpoint derivatives in the Euler–Maclaurin expansion cancel identically:

$$f^{(2k-1)}(2\pi) - f^{(2k-1)}(0) = 0 \qquad \text{for all } k \ge 1.$$

Consequently, all algebraic error coefficients $c_2, c_4, c_6, \ldots$ vanish. If f is analytic in a strip $\{z \in \mathbb{C} : |\operatorname{Im}(z)| < d\}$, the composite trapezoidal rule converges exponentially fast:

$$|I(f) - T_N(f)| \le Ce^{-dN}.$$

On smooth periodic domains, the composite trapezoidal rule achieves spectral accuracy.

Proof Because f is periodic and smooth, $f^{(m)}(2\pi) = f^{(m)}(0)$ for all $m \geq 0$. Every term c_{2k} in the Euler–Maclaurin formula is proportional to $f^{(2k-1)}(2\pi) - f^{(2k-1)}(0) = 0$, so the error decays faster than any power of $h = 2\pi/N$. To quantify the exponential rate, expand $f(x)$ in its Fourier series:

$$f(x) = \sum_{k=-\infty}^{\infty} \hat{f}_k e^{ikx}, \qquad \hat{f}_k = \frac{1}{2\pi} \int_0^{2\pi} f(x) e^{-ikx}\, dx.$$

The exact integral is $I(f) = \int_0^{2\pi} f(x)\, dx = 2\pi \hat{f}_0$. The composite trapezoidal rule with $h = 2\pi/N$ and nodes $x_j = 2\pi j/N$ for $j = 0, \ldots, N-1$ gives

$$\begin{aligned} T_N(f) = \frac{2\pi}{N} \sum_{j=0}^{N-1} f(x_j) &= \frac{2\pi}{N} \sum_{j=0}^{N-1} \sum_{k=-\infty}^{\infty} \hat{f}_k e^{2\pi ijk/N} \\ &= 2\pi \sum_{k=-\infty}^{\infty} \hat{f}_k \left(\frac{1}{N} \sum_{j=0}^{N-1} e^{2\pi ijk/N} \right). \end{aligned}$$

The inner sum satisfies discrete orthogonality:

$$\frac{1}{N} \sum_{j=0}^{N-1} e^{2\pi ijk/N} = \begin{cases} 1, & k = mN \text{ for some } m \in \mathbb{Z}, \\ 0, & k \not\equiv 0 \pmod{N}. \end{cases}$$

Substituting this indicator isolates the aliased modes:

$$T_N(f) = 2\pi \sum_{m=-\infty}^{\infty} \hat{f}_{mN} = 2\pi \hat{f}_0 + 2\pi \sum_{m \neq 0} \hat{f}_{mN}.$$

The quadrature error is therefore the tail sum:

$$I(f) - T_N(f) = -2\pi \sum_{m \neq 0} \hat{f}_{mN}.$$

Because f is analytic and bounded by M_0 in the strip $|\operatorname{Im}(z)| < d$, shifting the integration path in the definition of $\hat{f}_k$ gives the geometric decay $|\hat{f}_k| \leq M e^{-d|k|}$. Applying this bound:

$$|I(f) - T_N(f)| \leq 2\pi M \sum_{m \neq 0} e^{-d|m|N} = 4\pi M \sum_{m=1}^{\infty} (e^{-dN})^m = 4\pi M \frac{e^{-dN}}{1 - e^{-dN}} = O(e^{-dN}).$$

□

5.3.7 Exercises

Ex 5.3

1. **Derive. Method of undetermined coefficients.** Find the nodes and weights for a symmetric 3-point rule on $[-1,1]$ of the form $\int_{-1}^{1} f(x)\,dx \approx w_0 f(-\alpha) + w_1 f(0) + w_2 f(\alpha)$.

   (a) For fixed $\alpha = 1$, enforce exactness for $f(x) = 1, x, x^2$ to derive the weights of Simpson's rule: $w_0 = 1/3, w_1 = 4/3, w_2 = 1/3$.

   (b) Let α be a free parameter. Enforce exactness for $f(x) = 1, x^2, x^4$ to derive the Gauss–Legendre parameter $\alpha = 1/\sqrt{3}$ and weights $w_0 = w_2 = 1, w_1 = 0$.

   (c) Compare the degree of exactness of both rules.

2. **Compute. Romberg tableau construction.** Approximate the integral $I = \int_0^1 e^{-x^2}\,dx$.

   (a) Compute $T(h)$ for $h = 1, 1/2, 1/4$ using the composite trapezoidal rule.

   (b) Construct the first three rows of the Romberg tableau to evaluate $R(1,1), R(2,1)$, and $R(2,2)$.

   (c) Confirm that $R(1,1)$ matches composite Simpson's rule with $N = 2$.

3. **Compute. Change of interval and Gaussian quadrature.** Let $\int_a^b f(t)\,dt$ be an integral on an arbitrary interval $[a,b]$.

   (a) Derive the affine substitution $t(x) = \frac{b-a}{2}x + \frac{a+b}{2}$ mapping $x \in [-1,1]$ to $t \in [a,b]$.

   (b) Transform the 2-point Gauss–Legendre rule to $[0,3]$ and approximate $\int_0^3 \sqrt{1+t^3}\,dt$.

   (c) Compare the result with Simpson's rule using the same number of function evaluations.

4. **Derive. Peano kernel of the trapezoidal rule.** Consider the single-panel trapezoidal rule on $[0,h]$ with error functional $E(f) = \int_0^h f(x)\,dx - \frac{h}{2}[f(0) + f(h)]$.

   (a) Show that $E(1) = 0$ and $E(x) = 0$, so $m = 1$.

   (b) Evaluate the Peano kernel $K(t) = E_x((x-t)_+) = \int_0^h (x-t)_+\,dx - \frac{h}{2}[(0-t)_+ + (h-t)_+]$.

   (c) Show that $K(t) = \frac{1}{2}t(t-h) \le 0$ for all $t \in [0,h]$.

   (d) Apply Thm 5.14 to compute $\int_0^h K(t)\,dt = -\frac{h^3}{12}$ and recover the exact error term $-\frac{h^3}{12}f''(\xi)$.

5. **Compute. Spectral accuracy on periodic integrals.** Consider the integral $I = \int_0^{2\pi} \frac{dx}{2+\cos x} = \frac{2\pi}{\sqrt{3}}$.

   (a) Identify the complex poles of $f(z) = \frac{1}{2+\cos z}$ by solving $\cos z = -2$.

   (b) Determine the half-width d of the strip of analyticity around the real axis.

   (c) Evaluate $T_N(f)$ for $N = 4, 8, 16$. Confirm that $|I - T_N(f)|$ decays as e^{-dN}.

Read **Further reading.** Stoer and Bulirsch give a classical numerical-analysis treatment of interpolation and quadrature (Stoer and Bulirsch, 2002). Trefethen is especially useful for polynomial approximation, and de Boor for splines (Trefethen, 2013; de Boor, 2001). The short paper by Clenshaw and Curtis is a good example of an algorithmic quadrature paper that remains approachable (Clenshaw and Curtis, 1960).

5.4 Fourier Analysis as Change of Basis

Fourier analysis provides the mathematical framework for decomposing functions, discrete signals, and linear operators into frequency components. In scientific computing, the Fourier transform diagonalizes shift-invariant operators, accelerates convolution from $O(n^2)$ to $O(n \log n)$ operations, and yields spectral differentiation methods with exponential convergence for smooth periodic solutions.

(Running example: Poisson matrix) On a periodic grid, Fourier modes diagonalize the discrete Laplacian. With Dirichlet boundary conditions, sine modes play the same role for the Poisson matrix of Section 6.1. Their eigenvalues determine the conditioning of that matrix and the smoothing behavior used by multigrid in Section 6.6.

5.4.1 The Discrete Fourier Transform

We consider discrete signals defined on a uniform periodic grid of n points. The Discrete Fourier Transform (DFT) expands a vector in the orthonormal basis of discrete complex exponentials.

Def 5.8 **(Periodic Grid Signal)** A discrete signal $\boldsymbol{x} \in \mathbb{C}^n$ represents samples of a periodic function on the uniform grid $x_j = 2\pi j/n$ for $j = 0, 1, \ldots, n-1$, with the periodic index convention

$$x_{j+n} = x_j \qquad \text{for all } j \in \mathbb{Z}.$$

Def 5.9 **(DFT Matrix)** Let $\omega_n = e^{-2\pi i/n}$ denote the primitive n-th root of unity. The Discrete Fourier Transform matrix $\mathbf{F} \in \mathbb{C}^{n \times n}$ has entries

$$F_{jk} = \frac{1}{\sqrt{n}} \omega_n^{jk} = \frac{1}{\sqrt{n}} e^{-2\pi i jk/n}, \qquad 0 \le j, k \le n-1.$$

The forward transform maps a spatial signal $\boldsymbol{x} \in \mathbb{C}^n$ to its frequency representation $\hat{\boldsymbol{x}} = \mathbf{F}\boldsymbol{x}$.

Lem 5.19 **(Unitarity of the DFT Matrix)** The normalized DFT matrix $\mathbf{F}$ is unitary:

$$\mathbf{F}^*\mathbf{F} = \mathbf{F}\mathbf{F}^* = \mathbf{I}_n.$$

Consequently, the inverse transform is given by the conjugate transpose: $\mathbf{F}^{-1} = \mathbf{F}^*$.

Proof We evaluate the (j, ℓ) entry of the Gram matrix product $\mathbf{F}^*\mathbf{F}$. Because $(\mathbf{F}^*)_{jk} = \overline{F_{kj}} = \frac{1}{\sqrt{n}}\omega_n^{-jk}$, we have

$$(\mathbf{F}^*\mathbf{F})_{j\ell} = \sum_{k=0}^{n-1} (\mathbf{F}^*)_{jk} F_{k\ell} = \frac{1}{n} \sum_{k=0}^{n-1} \omega_n^{-jk} \omega_n^{k\ell} = \frac{1}{n} \sum_{k=0}^{n-1} \left(\omega_n^{\ell-j}\right)^k.$$

If $j = \ell$, each term in the summation equals $\omega_n^0 = 1$, yielding $\frac{1}{n}\sum_{k=0}^{n-1} 1 = 1$. If $j \neq \ell$, let $r = \omega_n^{\ell-j}$. Because $-(n-1) \leq \ell - j \leq n-1$ and $j \neq \ell$, the base $r \neq 1$, while $r^n = (\omega_n^n)^{\ell-j} = 1^{\ell-j} = 1$. The finite geometric series sums to

$$\sum_{k=0}^{n-1} r^k = \frac{1 - r^n}{1 - r} = \frac{1 - 1}{1 - r} = 0.$$

Therefore, $(\mathbf{F}^*\mathbf{F})_{j\ell} = \delta_{j\ell}$, which establishes $\mathbf{F}^*\mathbf{F} = \mathbf{I}_n$. □

Thm 5.20 **(Parseval's Identity and Isometry)** The Discrete Fourier Transform is an isometry on $\mathbb{C}^n$ with respect to the Euclidean norm:

$$\|\mathbf{F}\boldsymbol{x}\|_2 = \|\boldsymbol{x}\|_2 \qquad \text{for all } \boldsymbol{x} \in \mathbb{C}^n.$$

The inner product is likewise preserved: $\langle \mathbf{F}\boldsymbol{x}, \mathbf{F}\boldsymbol{y} \rangle = \langle \boldsymbol{x}, \boldsymbol{y} \rangle$.

Proof Using Lem 5.19, the Euclidean norm of the transformed vector satisfies

$$\|\mathbf{F}\boldsymbol{x}\|_2^2 = (\mathbf{F}\boldsymbol{x})^*(\mathbf{F}\boldsymbol{x}) = \boldsymbol{x}^*\mathbf{F}^*\mathbf{F}\boldsymbol{x} = \boldsymbol{x}^*\mathbf{I}\boldsymbol{x} = \boldsymbol{x}^*\boldsymbol{x} = \|\boldsymbol{x}\|_2^2.$$

Taking positive square roots proves the claim. For general vectors, $\langle \mathbf{F}\boldsymbol{x}, \mathbf{F}\boldsymbol{y} \rangle = (\mathbf{F}\boldsymbol{x})^*(\mathbf{F}\boldsymbol{y}) = \boldsymbol{x}^*\mathbf{F}^*\mathbf{F}\boldsymbol{y} = \boldsymbol{x}^*\boldsymbol{y} = \langle \boldsymbol{x}, \boldsymbol{y} \rangle$. □

Def 5.10 **(Frequency Bins and Nyquist Limit)** For a signal sampled on n points with uniform spacing $h = 2\pi/n$, integer frequency index k corresponds to the continuous Fourier mode e^{ikx}. The distinct resolvable frequencies are ordered as

$$0, 1, 2, \ldots, \lfloor n/2 \rfloor, -\lfloor (n-1)/2 \rfloor, \ldots, -1.$$

The maximum resolvable frequency is the Nyquist frequency $k_{\text{Nyq}} = \lfloor n/2 \rfloor$. Any continuous mode with frequency $|k| > k_{\text{Nyq}}$ aliases to a lower grid frequency satisfying $k' \equiv k \pmod{n}$.

Remark 5.7 **(Normalization convention)** Numerical libraries such as NumPy's `np.fft.fft` define the forward transform without the factor $1/\sqrt{n}$, so that $\hat{\boldsymbol{x}}_{\text{numpy}} = \sum_{j=0}^{n-1} x_j \omega_n^{jk}$, and place the full factor $1/n$ in the inverse transform `np.fft.ifft`. This choice scales the entries by $\sqrt{n}$ relative to the unitary operator $\mathbf{F}$ defined here.

5.4.2 Circulant Matrices and the Convolution Theorem

A linear system or filtering operation that commutes with spatial translation is represented by a circulant matrix. The Discrete Fourier Transform diagonalizes every circulant matrix.

Def 5.11 **(Circulant Matrix)** Let $\boldsymbol{c} = (c_0, c_1, \ldots, c_{n-1})^T \in \mathbb{C}^n$. The circulant matrix $\mathbf{C} = \operatorname{circ}(\boldsymbol{c}) \in \mathbb{C}^{n\times n}$ has entries $C_{jk} = c_{(j-k) \pmod n}$:

$$\mathbf{C} = \begin{pmatrix} c_0 & c_{n-1} & c_{n-2} & \cdots & c_1 \\ c_1 & c_0 & c_{n-1} & \cdots & c_2 \\ c_2 & c_1 & c_0 & \cdots & c_3 \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ c_{n-1} & c_{n-2} & c_{n-3} & \cdots & c_0 \end{pmatrix}.$$

Thm 5.21 **(Spectral Diagonalization of Circulant Matrices)** Every circulant matrix $\mathbf{C} = \operatorname{circ}(\boldsymbol{c})$ is diagonalized by the unitary DFT matrix:

$$\mathbf{C} = \mathbf{F}^* \mathbf{\Lambda} \mathbf{F},$$

where the diagonal eigenvalue matrix $\mathbf{\Lambda} = \operatorname{diag}(\lambda_0, \ldots, \lambda_{n-1})$ has entries equal to the unnormalized DFT coefficients of the first column:

$$\lambda_k = \sqrt{n}(\mathbf{F}\boldsymbol{c})_k = \sum_{j=0}^{n-1} c_j \omega_n^{jk}.$$

Proof Let $\mathbf{S} = \operatorname{circ}(\boldsymbol{e}_2) \in \mathbb{R}^{n\times n}$ be the cyclic permutation matrix that shifts coordinate indices forward by one: $\mathbf{S}\boldsymbol{x} = (x_{n-1}, x_0, x_1, \ldots, x_{n-2})^T$. Powers of $\mathbf{S}$ shift indices by j positions, satisfying $\mathbf{S}^n = \mathbf{I}$. Every circulant matrix is a polynomial in $\mathbf{S}$:

$$\mathbf{C} = \sum_{j=0}^{n-1} c_j \mathbf{S}^j.$$

Let $\boldsymbol{v}_k$ denote the k-th column of $\mathbf{F}^*$, with entries $(\boldsymbol{v}_k)_j = \frac{1}{\sqrt{n}}\omega_n^{-jk}$. Multiplying by the shift matrix gives

$$(\mathbf{S}\boldsymbol{v}_k)_j = (\boldsymbol{v}_k)_{(j-1) \pmod n} = \frac{1}{\sqrt{n}}\omega_n^{-(j-1)k} = \omega_n^k \left(\frac{1}{\sqrt{n}}\omega_n^{-jk}\right) = \omega_n^k (\boldsymbol{v}_k)_j.$$

Thus $\boldsymbol{v}_k$ is an eigenvector of $\mathbf{S}$ with eigenvalue ω_n^k. Applying the polynomial representation of $\mathbf{C}$ to $\boldsymbol{v}_k$ yields

$$\mathbf{C}\boldsymbol{v}_k = \sum_{j=0}^{n-1} c_j \mathbf{S}^j \boldsymbol{v}_k = \sum_{j=0}^{n-1} c_j (\omega_n^k)^j \boldsymbol{v}_k = \left(\sum_{j=0}^{n-1} c_j \omega_n^{jk}\right) \boldsymbol{v}_k = \lambda_k \boldsymbol{v}_k.$$

Collecting all n eigenvector equations in matrix form gives $\mathbf{C}\mathbf{F}^* = \mathbf{F}^*\mathbf{\Lambda}$. Multiplying on the right by $\mathbf{F}$ yields $\mathbf{C} = \mathbf{F}^*\mathbf{\Lambda}\mathbf{F}$. □

Thm 5.22 **(Discrete Convolution Theorem)** Let $\boldsymbol{c}, \boldsymbol{x} \in \mathbb{C}^n$. The circular convolution $\boldsymbol{z} = \boldsymbol{c} * \boldsymbol{x}$ has components

$$z_j = \sum_{k=0}^{n-1} c_k x_{(j-k) \pmod n} = (\mathbf{C}\boldsymbol{x})_j, \qquad \mathbf{C} = \operatorname{circ}(\boldsymbol{c}).$$

Applying the Discrete Fourier Transform converts convolution to entrywise multiplication:

$$\mathbf{F}(\boldsymbol{c} * \boldsymbol{x}) = \sqrt{n}(\mathbf{F}\boldsymbol{c}) \odot (\mathbf{F}\boldsymbol{x}),$$

where $\odot$ denotes the Hadamard (entrywise) vector product.

Proof Because $\boldsymbol{z} = \mathbf{C}\boldsymbol{x}$ and $\mathbf{C} = \mathbf{F}^*\mathbf{\Lambda}\mathbf{F}$ by Thm 5.21, we have

$$\mathbf{F}\boldsymbol{z} = \mathbf{F}(\mathbf{C}\boldsymbol{x}) = \mathbf{F}(\mathbf{F}^*\mathbf{\Lambda}\mathbf{F}\boldsymbol{x}) = \mathbf{\Lambda}(\mathbf{F}\boldsymbol{x}).$$

Because $\mathbf{\Lambda}$ is diagonal with entries $\lambda_k = \sqrt{n}(\mathbf{F}\boldsymbol{c})_k$, matrix-vector multiplication by $\mathbf{\Lambda}$ scales each component of $\mathbf{F}\boldsymbol{x}$ by $\sqrt{n}(\mathbf{F}\boldsymbol{c})_k$:

$$(\mathbf{F}\boldsymbol{z})_k = \lambda_k(\mathbf{F}\boldsymbol{x})_k = \sqrt{n}(\mathbf{F}\boldsymbol{c})_k(\mathbf{F}\boldsymbol{x})_k.$$

Writing this component identity in vector form yields $\mathbf{F}(\boldsymbol{c} * \boldsymbol{x}) = \sqrt{n}(\mathbf{F}\boldsymbol{c}) \odot (\mathbf{F}\boldsymbol{x})$. □

5.4.3 The Fast Fourier Transform

Computing a matrix-vector product with a general dense $n \times n$ matrix requires $2n^2$ flops. The Cooley–Tukey Fast Fourier Transform (FFT) factors the DFT matrix into sparse structured factors, reducing the complexity to $O(n \log n)$ operations.

Thm 5.23 **(Radix-2 Cooley–Tukey Decomposition)** Let n be an even integer, and let $\boldsymbol{x} \in \mathbb{C}^n$. Partition $\boldsymbol{x}$ into even-indexed and odd-indexed subvectors $\boldsymbol{x}_E, \boldsymbol{x}_O \in \mathbb{C}^{n/2}$ defined by $(\boldsymbol{x}_E)_j = x_{2j}$ and $(\boldsymbol{x}_O)_j = x_{2j+1}$ for $j = 0, \ldots, n/2 - 1$. The n-point unnormalized DFT $\hat{x}_k = \sum_{j=0}^{n-1} x_j \omega_n^{jk}$ satisfies the butterfly identity

$$\begin{aligned} \hat{x}_k &= (\hat{\boldsymbol{x}}_E)_k + \omega_n^k (\hat{\boldsymbol{x}}_O)_k, \\ \hat{x}_{k+n/2} &= (\hat{\boldsymbol{x}}_E)_k - \omega_n^k (\hat{\boldsymbol{x}}_O)_k, \end{aligned}$$

for each $k = 0, 1, \ldots, n/2-1$, where $\hat{\boldsymbol{x}}_E, \hat{\boldsymbol{x}}_O$ are the $(n/2)$-point DFTs of $\boldsymbol{x}_E$ and $\boldsymbol{x}_O$. When $n = 2^p$, applying this reduction recursively requires $O(n \log n)$ floating-point operations.

Proof Splitting the DFT sum into even and odd indices gives

$$\hat{x}_k = \sum_{m=0}^{n/2-1} x_{2m} \omega_n^{2mk} + \sum_{m=0}^{n/2-1} x_{2m+1} \omega_n^{(2m+1)k}.$$

Because $\omega_n^2 = e^{-4\pi i/n} = e^{-2\pi i/(n/2)} = \omega_{n/2}$, the first sum is the $(n/2)$-point transform of $\boldsymbol{x}_E$:

$$\sum_{m=0}^{n/2-1} x_{2m}\omega_{n/2}^{mk} = (\hat{\boldsymbol{x}}_E)_k.$$

Factoring ω_n^k out of the second sum gives $\omega_n^k \sum_{m=0}^{n/2-1} x_{2m+1}\omega_{n/2}^{mk} = \omega_n^k(\hat{\boldsymbol{x}}_O)_k$. This establishes the first identity:

$$\hat{x}_k = (\hat{\boldsymbol{x}}_E)_k + \omega_n^k(\hat{\boldsymbol{x}}_O)_k.$$

For the second half of the spectrum $k+n/2$, the periodicity $\omega_{n/2}^{m(k+n/2)} = \omega_{n/2}^{mk}(\omega_{n/2}^{n/2})^m = \omega_{n/2}^{mk}$ and the half-period phase shift

$$\omega_n^{k+n/2} = \omega_n^k\omega_n^{n/2} = \omega_n^k e^{-\pi i} = -\omega_n^k$$

give the complementary butterfly equation:

$$\hat{x}_{k+n/2} = (\hat{\boldsymbol{x}}_E)_k - \omega_n^k(\hat{\boldsymbol{x}}_O)_k.$$

In matrix form, introducing the diagonal twiddle matrix $\boldsymbol{\Omega}_{n/2} = \operatorname{diag}(1, \omega_n, \ldots, \omega_n^{n/2-1})$ and the even–odd permutation matrix $\mathbf{P}_n$:

$$\hat{\boldsymbol{x}} = \begin{pmatrix} \mathbf{I}_{n/2} & \boldsymbol{\Omega}_{n/2} \\ \mathbf{I}_{n/2} & -\boldsymbol{\Omega}_{n/2} \end{pmatrix} \begin{pmatrix} \hat{\boldsymbol{x}}_E \\ \hat{\boldsymbol{x}}_O \end{pmatrix} = \begin{pmatrix} \mathbf{I}_{n/2} & \boldsymbol{\Omega}_{n/2} \\ \mathbf{I}_{n/2} & -\boldsymbol{\Omega}_{n/2} \end{pmatrix} \begin{pmatrix} \mathbf{W}_{n/2} & \mathbf{0} \\ \mathbf{0} & \mathbf{W}_{n/2} \end{pmatrix} \mathbf{P}_n \boldsymbol{x}.$$

At each recursion stage, the butterfly recombination performs $n/2$ complex multiplications and n additions. The arithmetic complexity satisfies the recurrence

$$T(n) = 2T(n/2) + cn.$$

For $n = 2^p$, the depth of the recursion tree is $\log_2 n$, yielding total cost $T(n) = cn\log_2 n = O(n\log n)$. □

5.4.4 Spectral Differentiation

For smooth periodic functions, differentiation in the Fourier domain replaces finite-difference stencils with exact multiplication in frequency.

Thm 5.24 **(Spectral Differentiation of Periodic Functions)** Let $u : \mathbb{R} \to \mathbb{C}$ be a 2π-periodic function with continuous derivatives through order m. Representing u by its Fourier series $u(x) = \sum_{k=-\infty}^{\infty} \hat{u}_k e^{ikx}$, the derivative has Fourier coefficients

$$\widehat{(u')}_k = ik\hat{u}_k.$$

If u is analytic in a strip $\{z \in \mathbb{C} : |\operatorname{Im}(z)| < d\}$, the truncation error of the discrete spectral derivative converges geometrically to zero:

$$\max_{x_j} |u'(x_j) - (D_n \boldsymbol{u})_j| = O(e^{-dn/2}).$$

Proof Differentiating the series term by term gives $\frac{d}{dx}\sum_k \hat{u}_k e^{ikx} = \sum_k (ik\hat{u}_k)e^{ikx}$, which proves the coefficient formula. If u is analytic in the strip of half-width d, shifting the contour of integration in $\hat{u}_k = \frac{1}{2\pi}\int_0^{2\pi} u(x)e^{-ikx}dx$ to $x - i\,\mathrm{sign}(k)d$ yields the bound $|\hat{u}_k| \le Me^{-d|k|}$. The aliased grid modes satisfy $|k| \ge n/2$. Summing the tail of the neglected high-frequency modes gives $\sum_{|k|\ge n/2} |k||\hat{u}_k| \le 2M\sum_{k=n/2}^{\infty} ke^{-dk} = O(e^{-dn/2})$. □

5.4.5 Exercises

Ex 5.4

1. **Check. Unitary coordinates and geometric sums.** Let $\mathbf{F} \in \mathbb{C}^{4\times 4}$ be the 4-point DFT matrix. Write out its sixteen numerical entries explicitly using powers of $\omega_4 = -i$. Verify directly that columns 0 and 2 are orthogonal, and confirm that $\mathbf{F}^*\mathbf{F} = \mathbf{I}_4$.

2. **Derive. Circulant spectral inversion for the periodic Poisson problem.** Consider the discrete periodic Poisson equation $-\frac{u_{j-1}-2u_j+u_{j+1}}{h^2} = f_j$ on n points with periodic boundary conditions.

   (a) Express the coefficient matrix as a circulant matrix $\mathbf{C} = \operatorname{circ}(\boldsymbol{c})$ and identify the entries of $\boldsymbol{c}$.

   (b) Compute the eigenvalues λ_k of $\mathbf{C}$ using Thm 5.21. Show that $\lambda_0 = 0$ and $\lambda_k > 0$ for $k = 1, \dots, n-1$.

   (c) Explain why the system is solvable if and only if $\sum_{j=0}^{n-1} f_j = 0$, and formulate an $O(n \log n)$ solver using the FFT.

3. **Derive. Fast linear convolution via zero padding.** Let $\boldsymbol{x} \in \mathbb{R}^p$ and $\boldsymbol{y} \in \mathbb{R}^q$ be two discrete sequences. Their linear convolution has length $m = p + q - 1$.

   (a) Explain how zero-padding $\boldsymbol{x}$ and $\boldsymbol{y}$ to length $N \ge p + q - 1$ prevents periodic wraparound in circular convolution.

   (b) Compare the arithmetic operation count of direct linear convolution ($2pq$ flops) against FFT-based circular convolution ($O(N \log N)$ flops).

   (c) For $p = q = 10^5$, estimate the speedup obtained by the FFT algorithm.

4. **Derive. Cooley–Tukey butterfly and bit-reversal permutation.** Consider the radix-2 FFT algorithm with $n = 8$.

   (a) Write the indices $0, 1, \dots, 7$ in 3-bit binary notation $(b_2 b_1 b_0)$.

(b) Reverse the bits $(b_0 b_1 b_2)$ to construct the bit-reversal permutation vector $\mathbf{p}$.

(c) Show that sorting the input by $\mathbf{p}$ allows the recursive Cooley–Tukey butterflies to be evaluated in place without allocating auxiliary arrays.

5. **Compute. Spectral differentiation and the Runge phenomenon.** Implement the Fourier spectral differentiation matrix for $n = 16, 32, 64$ points on $[0, 2\pi)$.

(a) Apply the spectral derivative to $u(x) = \sin(x)$ and measure the maximum pointwise error against $\cos(x)$. Confirm that error reaches machine precision for small n.

(b) Apply the spectral derivative to $v(x) = \frac{1}{2+\cos(x)}$. Estimate the decay constant d from Thm 5.24 and confirm geometric convergence.

(c) Explain why applying Fourier differentiation to a non-periodic function such as $w(x) = x$ causes Gibbs oscillations and algebraic error decay $O(1/n)$.

Read **Further reading.** The Cooley–Tukey paper is short and still worth reading as an example of how reorganizing a calculation changes its cost (Cooley and Tukey, 1965). Trefethen's approximation text gives a gentle route from polynomial approximation to spectral accuracy (Trefethen, 2013).

6 Sparse Systems and Iterative Methods

6.1 Linear Operators and the Poisson Equation

(Running example: Poisson matrix) The matrix constructed here is the model sparse system for the rest of this part. The following sections solve it by stationary iteration, conjugate gradients, and multigrid, and each method uses a different property of the same matrix. Its eigenvectors are discrete sine modes (Thm 6.1), and its eigenvalues explain the convergence rate of each method.

Differential equations become matrix problems after discretization. The finite matrix should preserve the main structure of the operator: sparsity, symmetry, definiteness, spectrum, and conditioning.

6.1.1 Differential Operators and Discretization

Def 6.1 **(Linear Operator)** Map $\mathcal{A} : X \to Y$ between function spaces satisfying:

$$\mathcal{A}(\alpha u + v) = \alpha \mathcal{A} u + \mathcal{A} v.$$

Matrices are finite-dimensional operators, whereas $\frac{d}{dx}$ and ∇^2 are infinite-dimensional.

Def 6.2 **(Dirichlet Poisson Problem)** The one-dimensional Poisson problem with homogeneous Dirichlet boundary data is

$$-u''(x) = f(x), \qquad 0 < x < 1, \qquad u(0) = u(1) = 0.$$

The boundary conditions are part of the problem data.

Def 6.3 **(Grid Function)** For n interior grid points, let $h = 1/(n+1)$ and $x_i = ih$, $i = 1, \ldots, n$. A grid function is the vector

$$\boldsymbol{u} = (u_1, \ldots, u_n)^T, \qquad u_i \approx u(x_i).$$

The boundary values are fixed separately by the boundary conditions.

Def 6.4 **(Finite Difference Stencil)** Approximating $u''(x)$ on a grid with spacing h:

$$u''(x_k) \approx \frac{u(x_{k-1}) - 2u(x_k) + u(x_{k+1})}{h^2}.$$

Accuracy: Second-order $O(h^2)$ via Taylor expansion.

Ex 6.1 **Derive.**

1. Taylor expand $u(x+h)$ and $u(x-h)$ about x.
2. Add the two expansions.
3. Solve for $u''(x)$.
4. Identify the leading error term and prove the $O(h^2)$ claim in Def 6.4.

Def 6.5 **(1-D Discrete Laplacian)** Discretizing $-d^2/dx^2$ on $[0,1]$ with $u(0) = u(1) = 0$ yields the tridiagonal system $\mathbf{Tu} = \mathbf{f}$:

$$\mathbf{T} = \frac{1}{h^2}\text{tridiag}(-1, 2, -1) \in \mathbb{R}^{n \times n}.$$

Remark 6.1 **(Sparsity)** Each row of $\mathbf{T}$ uses only the grid point and its two neighbors. This locality is why finite difference matrices are sparse.

Thm 6.1 **(Spectrum Convergence)** Eigenvalues of $\mathbf{T}$ are $\lambda_k = \frac{4}{h^2}\sin^2(\frac{k\pi}{2(n+1)})$. As $n \to \infty$, $\lambda_k \to k^2\pi^2$ (eigenvalues of continuous $-d^2/dx^2$).

Ex 6.2 **Derive.**

1. Let $(\boldsymbol{v}_k)_j = \sin(jk\pi h)$ with $h = 1/(n+1)$.
2. Compute the j-th entry of $\mathbf{T}\boldsymbol{v}_k$.

3. Use $\sin(A-B)+\sin(A+B)=2\sin A\cos B$.
4. Use $1-\cos\theta=2\sin^2(\theta/2)$.
5. Derive the eigenvalue formula in Thm 6.1.
6. Let $h\to 0$ with fixed k and recover $k^2\pi^2$.

Thm 6.2 **(Discrete Poisson Structure)** The matrix $\mathbf{T}$ in Def 6.5 is sparse, symmetric positive definite, and has condition number

$$\kappa_2(\mathbf{T})=O(h^{-2}).$$

Proof Sparsity and symmetry follow directly from the tridiagonal stencil $\mathbf{T}=\frac{1}{h^2}\,\mathrm{tridiag}(-1,2,-1)$. For any grid vector $\boldsymbol{u}\in\mathbb{R}^n$ with boundary conditions $u_0=u_{n+1}=0$, the quadratic form expands as a sum of squared forward differences:

$$\begin{aligned}\boldsymbol{u}^T\mathbf{T}\boldsymbol{u}&=\frac{1}{h^2}\sum_{i=1}^{n}u_i(2u_i-u_{i-1}-u_{i+1})\\&=\frac{1}{h^2}\sum_{i=1}^{n}(u_i^2-u_iu_{i-1}+u_i^2-u_iu_{i+1})\\&=\frac{1}{h^2}\sum_{i=0}^{n}(u_{i+1}-u_i)^2.\end{aligned}$$

Because $u_0=0$, if $\boldsymbol{u}^T\mathbf{T}\boldsymbol{u}=0$, then $u_{i+1}-u_i=0$ for all i, forcing $u_1=\cdots=u_n=0$. Hence $\boldsymbol{u}^T\mathbf{T}\boldsymbol{u}>0$ for all $\boldsymbol{u}\neq\mathbf{0}$, proving that $\mathbf{T}$ is SPD.
By Thm 6.1, the extremal eigenvalues are

$$\begin{aligned}\lambda_{\min}=\lambda_1&=\frac{4}{h^2}\sin^2\left(\frac{\pi h}{2}\right)=\frac{4}{h^2}\left(\frac{\pi h}{2}+O(h^3)\right)^2=\pi^2+O(h^2),\\\lambda_{\max}=\lambda_n&=\frac{4}{h^2}\sin^2\left(\frac{n\pi h}{2}\right)=\frac{4}{h^2}\cos^2\left(\frac{\pi h}{2}\right)=\frac{4}{h^2}(1-O(h^2)).\end{aligned}$$

The 2-norm condition number is therefore

$$\kappa_2(\mathbf{T})=\frac{\lambda_{\max}}{\lambda_{\min}}=\frac{4/h^2+O(1)}{\pi^2+O(h^2)}=\frac{4}{\pi^2h^2}+O(1)=O(h^{-2}).$$

□

Ex 6.3 **Derive.**

1. Use the stencil to count the number of nonzeros in $\mathbf{T}$.

2. Use the quadratic form $\boldsymbol{u}^T\mathbf{T}\boldsymbol{u}$ to show positive definiteness for nonzero interior grid functions.

3. Use Thm 6.1 to estimate $\lambda_{\min}$ and $\lambda_{\max}$.

4. Derive the condition number estimate in Thm 6.2.

6.1.2 Operator Norms and Conditioning

Def 6.6 **(Boundedness)** An operator is **bounded** if $\|\mathcal{A}u\|_Y \leq C\|u\|_X$ for some $C < \infty$.

- Matrices are always bounded.
- Differential operators are **unbounded**.

Remark 6.2 **(Ill-Conditioning from Unboundedness)** Because the continuous Laplacian is unbounded, its discretization $\mathbf{T}$ becomes increasingly ill-conditioned as $h \to 0$:

$$\kappa(\mathbf{T}) = \frac{\lambda_{\max}}{\lambda_{\min}} \approx \frac{4/h^2}{\pi^2} = O(h^{-2}).$$

Refining the grid $2\times$ quadruples the condition number.

Remark 6.3 **(Solver consequence)** Dense Gaussian elimination ignores sparsity. For Poisson matrices, sparse Cholesky, CG with preconditioning, multigrid, or FFT-based solvers are the natural algorithms, depending on geometry and boundary conditions.

6.1.3 Exercises

Ex 6.4

1. **Compute.** Build $\mathbf{T}$ for $n = 50$ and plot its eigenvalues vs. $k^2\pi^2$ using Thm 6.1.

2. **Derive.** Show that $\kappa(\mathbf{T}) \to \infty$ as $n \to \infty$ using Thm 6.2.

3. **Compute.** Implement a 2-D Poisson solver on a unit square. Verify $O(N^{3/2})$ cost for direct sparse solve.

4. **Derive.** Prove the differentiation operator is unbounded on $L^2[0,1]$ using $u_k(x) = \sin(k\pi x)$.

Read **Further reading.** LeVeque develops finite differences with close attention to consistency, stability, and boundary conditions (LeVeque, 2007). Strang connects the same model problems to broader ideas in applied mathematics and computation (Strang, 2007).

6.2 Iterative Methods for Linear Systems

Direct methods such as Gaussian elimination and Cholesky factorization compute exact solutions up to machine roundoff in a finite number of arithmetic operations. However, for the large-scale linear systems arising from the discretization of three-dimensional partial differential equations or network models, direct factorizations become computationally prohibitive. Section 2.4 showed that the Cholesky factor of a three-dimensional Poisson matrix has $O(n^{4/3})$ nonzero entries and costs $O(n^2)$ flops even with a nested dissection ordering, so the factor can exceed the available memory long before the matrix does.

Iterative methods avoid this storage cost. Instead of transforming the matrix through elimination, an iterative solver generates a sequence of approximations $\boldsymbol{x}^{(0)}, \boldsymbol{x}^{(1)}, \boldsymbol{x}^{(2)}, \ldots$ to the solution $\boldsymbol{x}^* = \mathbf{A}^{-1}\boldsymbol{b}$. The central computational primitive is the matrix–vector product $\boldsymbol{v} \mapsto \mathbf{A}\boldsymbol{v}$. For a stored sparse matrix this product requires $O(\mathrm{nnz}(\mathbf{A}))$ operations and storage, whereas for a matrix-free operator it may require only a small number of work vectors. Whether the complete method scales well also depends on convergence and preconditioning.

(Running example: Poisson matrix) Return to the tridiagonal Poisson matrix of Section 6.1. The matrix is sparse enough for cheap matrix–vector products, but its condition number grows as the mesh is refined. One example therefore separates methods that use only sparsity from methods that also use symmetry, definiteness, or multiple grid scales.

6.2.1 Matrix Splittings and Stationary Iterations

The classical approach to designing an iterative solver is matrix splitting. We decompose the coefficient matrix $\mathbf{A}$ into the difference of two matrices,

$$\mathbf{A} = \mathbf{M} - \mathbf{N},$$

where the splitting matrix $\mathbf{M}$ is nonsingular and easily invertible. The linear system $\mathbf{A}\boldsymbol{x} = \boldsymbol{b}$ is then algebraically equivalent to $\mathbf{M}\boldsymbol{x} = \mathbf{N}\boldsymbol{x} + \boldsymbol{b}$.

Def 6.7 **(Stationary Iteration)** Given a splitting $\mathbf{A} = \mathbf{M} - \mathbf{N}$, the corresponding stationary iteration computes successive approximations via the fixed-point recurrence

$$\mathbf{M}\boldsymbol{x}^{(k+1)} = \mathbf{N}\boldsymbol{x}^{(k)} + \boldsymbol{b} \iff \boldsymbol{x}^{(k+1)} = \mathbf{T}\boldsymbol{x}^{(k)} + \boldsymbol{c},$$

where $\mathbf{T} = \mathbf{M}^{-1}\mathbf{N} = \mathbf{I} - \mathbf{M}^{-1}\mathbf{A}$ is the **iteration matrix** and $\boldsymbol{c} = \mathbf{M}^{-1}\boldsymbol{b}$ is the modified right-hand side vector.

Remark 6.4 **(Fixed-point interpretation)** A stationary iteration is a linear fixed-point mapping. The exact solution $\boldsymbol{x}^*$ satisfies $\boldsymbol{x}^* = \mathbf{T}\boldsymbol{x}^* + \boldsymbol{c}$. The effectiveness of the method depends entirely on the choice of $\mathbf{M}$: the matrix $\mathbf{M}$ should be as close to $\mathbf{A}$ as possible to minimize the magnitude of $\mathbf{T}$, while remaining computationally cheap to invert at each step.

6.2.2 Convergence Theory and the Spectral Radius

The convergence of a linear stationary iteration is determined entirely by the spectral properties of the iteration matrix $\mathbf{T}$.

Def 6.8 **(Spectral Radius)** Let $\mathbf{T} \in \mathbb{R}^{n\times n}$ with spectrum $\sigma(\mathbf{T}) \subset \mathbb{C}$. The spectral radius of $\mathbf{T}$ is defined as the maximum modulus among all its eigenvalues:

$$\rho(\mathbf{T}) = \max\{|\lambda| : \lambda \in \sigma(\mathbf{T})\}.$$

Thm 6.3 **(Gelfand's Spectral Radius Theorem)** For any matrix norm $\|\cdot\|$ induced by a vector norm on $\mathbb{C}^n$ and any matrix $\mathbf{T} \in \mathbb{R}^{n\times n}$,

$$\rho(\mathbf{T}) = \lim_{k\to\infty} \|\mathbf{T}^k\|^{1/k}.$$

In particular, for any $\varepsilon > 0$, there exists an induced matrix norm $\|\cdot\|_*$ such that $\|\mathbf{T}\|_* \leq \rho(\mathbf{T}) + \varepsilon$.

Thm 6.4 **(Fundamental Convergence Criterion for Stationary Iterations)** The stationary iteration $\boldsymbol{x}^{(k+1)} = \mathbf{T}\boldsymbol{x}^{(k)} + \boldsymbol{c}$ converges to the unique solution $\boldsymbol{x}^* = \mathbf{A}^{-1}\boldsymbol{b}$ for every initial guess $\boldsymbol{x}^{(0)} \in \mathbb{R}^n$ if and only if

$$\rho(\mathbf{T}) < 1.$$

Proof Let $\boldsymbol{e}^{(k)} = \boldsymbol{x}^{(k)} - \boldsymbol{x}^*$ denote the error vector at step k. Subtracting the fixed-point identity $\boldsymbol{x}^* = \mathbf{T}\boldsymbol{x}^* + \boldsymbol{c}$ from the iteration formula $\boldsymbol{x}^{(k+1)} = \mathbf{T}\boldsymbol{x}^{(k)} + \boldsymbol{c}$:

$$\begin{aligned}\boldsymbol{e}^{(k+1)} &= \boldsymbol{x}^{(k+1)} - \boldsymbol{x}^* \\ &= (\mathbf{T}\boldsymbol{x}^{(k)} + \boldsymbol{c}) - (\mathbf{T}\boldsymbol{x}^* + \boldsymbol{c}) \\ &= \mathbf{T}(\boldsymbol{x}^{(k)} - \boldsymbol{x}^*) = \mathbf{T}\boldsymbol{e}^{(k)}.\end{aligned}$$

Applying this recurrence recursively yields

$$\boldsymbol{e}^{(k)} = \mathbf{T}^k \boldsymbol{e}^{(0)}.$$

The error $\boldsymbol{e}^{(k)} \to \mathbf{0}$ for all initial errors $\boldsymbol{e}^{(0)} \in \mathbb{R}^n$ if and only if $\lim_{k\to\infty} \mathbf{T}^k = \mathbf{0}$.
If $\rho(\mathbf{T}) < 1$, choose $\varepsilon = \frac{1-\rho(\mathbf{T})}{2} > 0$. By Gelfand's theorem (Thm 6.3), there exists an induced matrix norm $\|\cdot\|_*$ such that

$$\|\mathbf{T}\|_* \leq \rho(\mathbf{T}) + \varepsilon = \frac{1 + \rho(\mathbf{T})}{2} < 1.$$

Then $\|\mathbf{T}^k\|_* \leq \|\mathbf{T}\|_*^k \to 0$ as $k \to \infty$, so $\mathbf{T}^k \to \mathbf{0}$ and $\|\boldsymbol{e}^{(k)}\|_* \leq \|\mathbf{T}\|_*^k \|\boldsymbol{e}^{(0)}\|_* \to 0$.

Conversely, suppose $\mathbf{T}^k \to \mathbf{0}$. If $\lambda \in \sigma(\mathbf{T})$ with eigenvector $\boldsymbol{v} \neq \mathbf{0}$, then $\mathbf{T}^k\boldsymbol{v} = \lambda^k\boldsymbol{v}$. For this sequence to converge to $\mathbf{0}$, we must have $|\lambda|^k \to 0$, forcing $|\lambda| < 1$. Because this holds for all eigenvalues, $\rho(\mathbf{T}) < 1$. □

Remark 6.5 **(Asymptotic convergence rate and iteration count)** The quantity $-\log_{10}\rho(\mathbf{T})$ represents the asymptotic rate of convergence, measuring the number of decimal digits of accuracy gained per iteration. To reduce the initial error by a relative tolerance factor ε, the required number of iterations satisfies

$$k \approx \frac{\ln(1/\varepsilon)}{-\ln\rho(\mathbf{T})}.$$

If $\rho(\mathbf{T}) = 0.99$, we have $-\ln(0.99) \approx 0.01005$, which requires approximately 230 iterations to reduce the error by a factor of ten. As $\rho(\mathbf{T}) \to 1$, convergence slows drastically.

6.2.3 Krylov Subspaces and Matrix-Free Solvers

A stationary method applies the same splitting at every step. A Krylov method instead chooses the iterate $\boldsymbol{x}^{(k)}$ from the affine space $\boldsymbol{x}^{(0)} + \mathcal{K}_k(\mathbf{A}, \boldsymbol{r}^{(0)})$, where $\mathcal{K}_k$ is the Krylov subspace of Def 4.12 generated by the initial residual. The basis for this space is built by the Arnoldi process of Section 4.4, or by the Lanczos recurrence when $\mathbf{A}$ is symmetric. The conjugate gradient and GMRES methods differ in the optimality criterion used to select $\boldsymbol{x}^{(k)}$.

Thm 6.5 **(Krylov Polynomial Representation)** Let $\boldsymbol{x}^{(0)}$ be an initial guess with residual $\boldsymbol{r}^{(0)} = \boldsymbol{b} - \mathbf{A}\boldsymbol{x}^{(0)}$. Any iterate $\boldsymbol{x}^{(k)}$ belonging to the affine Krylov space $\boldsymbol{x}^{(0)} + \mathcal{K}_k(\mathbf{A}, \boldsymbol{r}^{(0)})$ can be represented as

$$\boldsymbol{x}^{(k)} = \boldsymbol{x}^{(0)} + p_{k-1}(\mathbf{A})\boldsymbol{r}^{(0)},$$

where p_{k-1} is a polynomial of degree at most $k-1$. The corresponding residual vector satisfies

$$\boldsymbol{r}^{(k)} = \boldsymbol{b} - \mathbf{A}\boldsymbol{x}^{(k)} = q_k(\mathbf{A})\boldsymbol{r}^{(0)},$$

where $q_k(t) = 1 - tp_{k-1}(t)$ is a polynomial of degree at most k satisfying the normalization constraint $q_k(0) = 1$.

Remark 6.6 **(Matrix-free computation)** Krylov subspace methods never require explicit access to the individual entries of $\mathbf{A}$. They interact with the linear operator exclusively by computing matrix–vector products $\boldsymbol{v} \mapsto \mathbf{A}\boldsymbol{v}$. Consequently, linear systems can be solved efficiently even when the matrix is defined implicitly as a differential operator or a numerical simulation routine.

Def 6.9 **(Stopping Criteria for Iterative Solvers)** Because the true error $\boldsymbol{e}^{(k)} = \boldsymbol{x}^{(k)} - \boldsymbol{x}^*$ is unobservable, termination is determined using the computable residual vector $\boldsymbol{r}^{(k)} = \boldsymbol{b} - \mathbf{A}\boldsymbol{x}^{(k)}$. Common stopping criteria include:

1. **Relative residual test:** $\frac{\|\boldsymbol{r}^{(k)}\|_2}{\|\boldsymbol{b}\|_2} \leq \texttt{tol}$.

2. **Preconditioned relative residual test:** $\frac{\|\mathbf{M}^{-1}\boldsymbol{r}^{(k)}\|_2}{\|\mathbf{M}^{-1}\boldsymbol{b}\|_2} \leq \texttt{tol}$.

3. **Step difference test:** $\frac{\|\boldsymbol{x}^{(k+1)}-\boldsymbol{x}^{(k)}\|_2}{\|\boldsymbol{x}^{(k+1)}\|_2} \leq \texttt{tol}$.

Ex 6.5

1. **Compute.** Consider the matrix $\mathbf{A} = \begin{pmatrix} 4 & 1 \\ 2 & 3 \end{pmatrix}$ and the splitting $\mathbf{M} = 4\mathbf{I}$. Compute the iteration matrix $\mathbf{T} = \mathbf{I} - \mathbf{M}^{-1}\mathbf{A}$, find its eigenvalues, and evaluate its spectral radius $\rho(\mathbf{T})$.

2. **Check.** For the splitting above, determine the number of iterations required to reduce the initial error by a factor of 10^{-6}.

3. **Derive.** Prove that if $\|\mathbf{T}\| < 1$ for some induced matrix norm, then the error satisfies $\|\boldsymbol{e}^{(k)}\| \leq \frac{\|\mathbf{T}\|}{1-\|\mathbf{T}\|}\|\boldsymbol{x}^{(k)} - \boldsymbol{x}^{(k-1)}\|$.

4. **Derive.** Let $\mathbf{A} \in \mathbb{R}^{n\times n}$ and let $\boldsymbol{r}^{(0)} \in \mathbb{R}^n$. Lem 4.24 shows that $\dim \mathcal{K}_k(\mathbf{A}, \boldsymbol{r}^{(0)})$ stops growing once the Krylov subspace is invariant. Prove the complementary statement: before that step, the dimension increases by exactly one at each step.

6.3 Jacobi, Gauss-Seidel, and SOR Methods

The classical stationary iterations represent the earliest systematic approaches to solving linear systems iteratively. They partition the matrix $\mathbf{A}$ into its diagonal, strictly lower triangular, and strictly upper triangular components:

$$\mathbf{A} = \mathbf{D} - \mathbf{L}_A - \mathbf{U}_A,$$

where $\mathbf{D} = \operatorname{diag}(a_{11}, \ldots, a_{nn})$, $-\mathbf{L}_A$ is the strictly lower triangular part of $\mathbf{A}$, and $-\mathbf{U}_A$ is the strictly upper triangular part of $\mathbf{A}$. By choosing different combinations of these components for the splitting matrix $\mathbf{M}$, we obtain the Jacobi, Gauss–Seidel, and Successive Over-Relaxation (SOR) algorithms.

6.3.1 The Jacobi Method

The Jacobi method assumes that the diagonal elements dominate the matrix. It computes the $(k+1)$-th estimate of each unknown x_i exclusively from the k-th estimates of all other variables.

Def 6.10 **(Jacobi Iteration)** Let $\mathbf{A} \in \mathbb{R}^{n\times n}$ with nonzero diagonal entries $a_{ii} \neq 0$. The Jacobi method corresponds to the splitting $\mathbf{M} = \mathbf{D}$ and $\mathbf{N} = \mathbf{L}_A + \mathbf{U}_A$. In component form, the update is

$$x_i^{(k+1)} = \frac{1}{a_{ii}} \left(b_i - \sum_{j\neq i} a_{ij} x_j^{(k)} \right), \qquad i = 1, 2, \ldots, n.$$

In matrix notation, the iteration is given by

$$\boldsymbol{x}^{(k+1)} = \mathbf{T}_J \boldsymbol{x}^{(k)} + \boldsymbol{c}_J,$$

where $\mathbf{T}_J = \mathbf{D}^{-1}(\mathbf{L}_A + \mathbf{U}_A) = \mathbf{I} - \mathbf{D}^{-1}\mathbf{A}$ is the Jacobi iteration matrix and $\boldsymbol{c}_J = \mathbf{D}^{-1}\boldsymbol{b}$.

Remark 6.7 **(Parallel execution and memory)** Because each component $x_i^{(k+1)}$ depends only on values from the previous iteration vector $\boldsymbol{x}^{(k)}$, all n components can be computed concurrently. This fine-grained parallelism makes Jacobi attractive on vector and GPU hardware. However, it requires double buffering in memory to store $\boldsymbol{x}^{(k)}$ and $\boldsymbol{x}^{(k+1)}$ simultaneously.

6.3.2 The Gauss–Seidel Method

The Gauss–Seidel method improves upon Jacobi by using updated variable values as soon as they become available within the current sweep.

Def 6.11 **(Gauss–Seidel Iteration)** Let $\mathbf{A} \in \mathbb{R}^{n\times n}$ with nonzero diagonal entries. The Gauss–Seidel method corresponds to the triangular splitting $\mathbf{M} = \mathbf{D} - \mathbf{L}_A$ and $\mathbf{N} = \mathbf{U}_A$. In component form, the update is

$$x_i^{(k+1)} = \frac{1}{a_{ii}} \left(b_i - \sum_{j<i} a_{ij} x_j^{(k+1)} - \sum_{j>i} a_{ij} x_j^{(k)} \right), \qquad i = 1, 2, \ldots, n.$$

In matrix notation, the iteration is given by

$$(\mathbf{D} - \mathbf{L}_A)\boldsymbol{x}^{(k+1)} = \mathbf{U}_A \boldsymbol{x}^{(k)} + \boldsymbol{b} \iff \boldsymbol{x}^{(k+1)} = \mathbf{T}_{GS}\boldsymbol{x}^{(k)} + \boldsymbol{c}_{GS},$$

where $\mathbf{T}_{GS} = (\mathbf{D}-\mathbf{L}_A)^{-1}\mathbf{U}_A$ is the Gauss–Seidel iteration matrix and $\boldsymbol{c}_{GS} = (\mathbf{D}-\mathbf{L}_A)^{-1}\boldsymbol{b}$.

Remark 6.8 **(Sequential dependencies and in-place updates)** Because the calculation of $x_i^{(k+1)}$ requires the newly computed values $x_1^{(k+1)}, \ldots, x_{i-1}^{(k+1)}$, the standard Gauss–Seidel sweep is inherently sequential. However, it permits in-place updates, in which new values immediately overwrite old values in a single vector. For many classes of matrices, Gauss–Seidel converges approximately twice as fast as Jacobi.

Example **(Comparison of one Jacobi and Gauss–Seidel step)** Consider the linear system

$$\begin{pmatrix} 4 & 1 & 0 \\ 1 & 4 & 1 \\ 0 & 1 & 4 \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \\ x_3 \end{pmatrix} = \begin{pmatrix} 5 \\ 6 \\ 5 \end{pmatrix},$$

whose exact solution is $\boldsymbol{x}^* = (1, 1, 1)^T$. Let the initial guess be $\boldsymbol{x}^{(0)} = (0, 0, 0)^T$.

- **Jacobi Step:**

$$
\begin{aligned}
x_1^{(1)} &= \frac{1}{4}(5 - 0 - 0) = 1.25, \\
x_2^{(1)} &= \frac{1}{4}(6 - 0 - 0) = 1.50, \\
x_3^{(1)} &= \frac{1}{4}(5 - 0 - 0) = 1.25, \qquad \Longrightarrow \ \boldsymbol{x}_J^{(1)} = \begin{pmatrix} 1.25 \\ 1.50 \\ 1.25 \end{pmatrix}.
\end{aligned}
$$

- **Gauss–Seidel Step:**

$$
\begin{aligned}
x_1^{(1)} &= \frac{1}{4}(5 - 0 - 0) = 1.25, \\
x_2^{(1)} &= \frac{1}{4}(6 - (1.25) - 0) = \frac{4.75}{4} = 1.1875, \\
x_3^{(1)} &= \frac{1}{4}(5 - (1.1875) - 0) = \frac{3.8125}{4} \approx 0.9531, \qquad \Longrightarrow \ \boldsymbol{x}_{GS}^{(1)} \approx \begin{pmatrix} 1.2500 \\ 1.1875 \\ 0.9531 \end{pmatrix}.
\end{aligned}
$$

The Gauss–Seidel iterate is closer to the true solution $(1, 1, 1)^T$ because the update for x_2 immediately used the new value of x_1, and the update for x_3 used the new value of x_2.

6.3.3 Convergence Theorems

Thm 6.6 **(Convergence Under Strict Diagonal Dominance)** A matrix $\mathbf{A} \in \mathbb{R}^{n \times n}$ is strictly diagonally dominant if

$$
|a_{ii}| > \sum_{j \neq i} |a_{ij}| \qquad \text{for all } i = 1, 2, \ldots, n.
$$

If $\mathbf{A}$ is strictly diagonally dominant, then both the Jacobi iteration and the Gauss–Seidel iteration converge for any initial guess $\boldsymbol{x}^{(0)} \in \mathbb{R}^n$.

Proof We show that the matrix ∞-norm of the iteration matrix $\mathbf{T}_J = \mathbf{D}^{-1}(\mathbf{L}_A + \mathbf{U}_A)$ is strictly bounded by one. The entries of $\mathbf{T}_J$ are

$$
(\mathbf{T}_J)_{ij} = \begin{cases} -\frac{a_{ij}}{a_{ii}}, & i \neq j, \\ 0, & i = j. \end{cases}
$$

The induced ∞-norm is the maximum absolute row sum:

$$
\|\mathbf{T}_J\|_\infty = \max_{1 \leq i \leq n} \sum_{j=1}^{n} |(\mathbf{T}_J)_{ij}| = \max_{1 \leq i \leq n} \sum_{j \neq i} \frac{|a_{ij}|}{|a_{ii}|}.
$$

Because $\mathbf{A}$ is strictly diagonally dominant, $|a_{ii}| > \sum_{j\neq i} |a_{ij}|$ for each $i \in \{1, \dots, n\}$, so

$$\sum_{j\neq i} \frac{|a_{ij}|}{|a_{ii}|} < 1 \implies \|\mathbf{T}_J\|_\infty < 1.$$

By Gelfand's spectral radius theorem, $\rho(\mathbf{T}_J) \leq \|\mathbf{T}_J\|_\infty < 1$, which establishes convergence of the Jacobi iteration.
For Gauss–Seidel, let λ be an eigenvalue of $\mathbf{T}_{GS} = (\mathbf{D} - \mathbf{L}_A)^{-1}\mathbf{U}_A$ with eigenvector $\boldsymbol{v} \neq \mathbf{0}$, normalized so that $\|\boldsymbol{v}\|_\infty = |v_p| = 1$. From $\mathbf{U}_A\boldsymbol{v} = \lambda(\mathbf{D} - \mathbf{L}_A)\boldsymbol{v}$, row p reads

$$-\sum_{j>p} a_{pj}v_j = \lambda a_{pp}v_p + \lambda \sum_{j<p} a_{pj}v_j.$$

Rearranging and applying the triangle inequality with $|v_j| \leq 1$ yields

$$|\lambda||a_{pp}| \leq \sum_{j>p} |a_{pj}| + |\lambda| \sum_{j<p} |a_{pj}|.$$

If $|\lambda| \geq 1$, then

$$|a_{pp}| \leq \frac{1}{|\lambda|} \sum_{j>p} |a_{pj}| + \sum_{j<p} |a_{pj}| \leq \sum_{j\neq p} |a_{pj}|,$$

which contradicts strict diagonal dominance. Hence $|\lambda| < 1$, proving $\rho(\mathbf{T}_{GS}) < 1$. □

Thm 6.7 **(Convergence of Gauss–Seidel for Symmetric Positive Definite Systems)** If $\mathbf{A} \in \mathbb{R}^{n\times n}$ is symmetric positive definite, then the Gauss–Seidel method converges for every initial vector $\boldsymbol{x}^{(0)} \in \mathbb{R}^n$. The Jacobi method, however, is not guaranteed to converge for arbitrary SPD matrices.

6.3.4 Successive Over-Relaxation (SOR)

The Successive Over-Relaxation method accelerates the convergence of Gauss–Seidel by taking a weighted linear combination of the previous iterate and the new Gauss–Seidel update.

Def 6.12 **(Successive Over-Relaxation Scheme)** Let $\omega > 0$ be a relaxation parameter. The SOR method computes the updated components via

$$x_i^{(k+1)} = (1-\omega)x_i^{(k)} + \frac{\omega}{a_{ii}} \left(b_i - \sum_{j<i} a_{ij}x_j^{(k+1)} - \sum_{j>i} a_{ij}x_j^{(k)} \right).$$

In matrix notation, this corresponds to the splitting $\mathbf{M}_\omega = \frac{1}{\omega}\mathbf{D} - \mathbf{L}_A$ and $\mathbf{N}_\omega = \left(\frac{1}{\omega} - 1\right)\mathbf{D} + \mathbf{U}_A$. The iteration matrix is

$$\mathbf{T}_\omega = (\mathbf{D} - \omega\mathbf{L}_A)^{-1} \left((1-\omega)\mathbf{D} + \omega\mathbf{U}_A\right).$$

Thm 6.8 **(Ostrowski–Reich Theorem and the Optimal Parameter)** If $\mathbf{A}$ is symmetric positive definite, then the SOR iteration converges if and only if the relaxation parameter satisfies

$$0 < \omega < 2.$$

Furthermore, if $\mathbf{A}$ is consistently ordered (such as the standard tridiagonal or block-tridiagonal Poisson discretization matrices) and the Jacobi spectral radius is $\rho(\mathbf{T}_J) < 1$, then the optimal relaxation parameter that minimizes $\rho(\mathbf{T}_\omega)$ is

$$\omega^* = \frac{2}{1 + \sqrt{1 - \rho(\mathbf{T}_J)^2}}.$$

With this optimal choice, the spectral radius satisfies $\rho(\mathbf{T}_{\omega^*}) = \omega^* - 1$.

Remark 6.9 **(Acceleration for discrete Poisson equations)** For the 1D discrete Laplacian on a grid with spacing h, the Jacobi spectral radius is $\rho(\mathbf{T}_J) = \cos(\pi h) \approx 1 - \frac{1}{2}\pi^2 h^2$. Consequently, Jacobi and Gauss–Seidel require $O(h^{-2}) = O(n^2)$ iterations to converge. With optimal SOR, $\omega^* \approx 2 - 2\pi h$, yielding $\rho(\mathbf{T}_{\omega^*}) \approx 1 - 2\pi h$. This reduces the required iteration count to $O(h^{-1}) = O(n)$, providing an order-of-magnitude computational speedup.

Ex 6.6

1. **Compute.** Compute the Jacobi and Gauss-Seidel iteration matrices for $\mathbf{A} = \begin{pmatrix} 2 & 1 \\ 1 & 2 \end{pmatrix}$. Find their spectral radii and verify that $\rho(\mathbf{T}_{GS}) = \rho(\mathbf{T}_J)^2$.

2. **Explore.** Let $\mathbf{A} = \begin{pmatrix} 1 & 2 \\ 2 & 1 \end{pmatrix}$. Show that $\mathbf{A}$ is symmetric but indefinite, and prove that both Jacobi and Gauss-Seidel diverge.

3. **Derive.** Prove the Kahan theorem: for any matrix $\mathbf{A}$ with nonzero diagonal entries, $\rho(\mathbf{T}_\omega) \geq |\omega - 1|$, which implies that SOR can only converge if $0 < \omega < 2$.

4. **Compute.** Write a Python script to solve the 1D discrete Poisson equation with $n = 50$ grid points using Jacobi, Gauss-Seidel, and optimal SOR. Plot the residual norm versus iteration number on a semilogarithmic scale.

6.4 The Conjugate Gradient Method

The Conjugate Gradient (CG) method of Hestenes and Stiefel is a standard iterative solver for large, sparse, symmetric positive definite systems. Unlike a stationary iteration determined by one fixed splitting, CG selects the approximation $\boldsymbol{x}^{(k)}$ that minimizes the error in the $\mathbf{A}$-norm over the affine Krylov subspace $\boldsymbol{x}^{(0)} + \mathcal{K}_k(\mathbf{A}, \boldsymbol{r}^{(0)})$.

Symmetry and conjugacy reduce this expanding-subspace minimization to a short recurrence. The resulting algorithm stores only a few vectors of length n and uses one matrix–vector product per iteration.

(Running example: Poisson matrix) The discrete Poisson matrix is SPD, so CG applies without changing the model. Its slowly varying eigenmodes explain why the unpreconditioned iteration deteriorates under refinement and why the coarse-grid ideas of Section 6.6 are relevant.

6.4.1 Quadratic Optimization and Steepest Descent

The mathematical foundation of the Conjugate Gradient method lies in the equivalence between solving an SPD linear system and minimizing a convex quadratic energy functional.

Thm 6.9 **(Variational Equivalence for SPD Systems)** Let $\mathbf{A} \in \mathbb{R}^{n\times n}$ be a symmetric positive definite matrix and let $\boldsymbol{b} \in \mathbb{R}^n$. Define the quadratic functional $\phi : \mathbb{R}^n \to \mathbb{R}$ by

$$\phi(\boldsymbol{x}) = \frac{1}{2}\boldsymbol{x}^T\mathbf{A}\boldsymbol{x} - \boldsymbol{b}^T\boldsymbol{x}.$$

The gradient of ϕ is $\nabla\phi(\boldsymbol{x}) = \mathbf{A}\boldsymbol{x} - \boldsymbol{b} = -\boldsymbol{r}(\boldsymbol{x})$, where $\boldsymbol{r}(\boldsymbol{x}) = \boldsymbol{b} - \mathbf{A}\boldsymbol{x}$ is the residual. Because $\mathbf{A}$ is positive definite, the Hessian matrix $\nabla^2\phi(\boldsymbol{x}) = \mathbf{A}$ is strictly positive definite everywhere, and ϕ has a unique global minimizer $\boldsymbol{x}^*$ satisfying

$$\nabla\phi(\boldsymbol{x}^*) = \mathbf{0} \iff \mathbf{A}\boldsymbol{x}^* = \boldsymbol{b}.$$

Furthermore, the functional error is directly proportional to the squared $\mathbf{A}$-norm of the solution error:

$$\phi(\boldsymbol{x}) - \phi(\boldsymbol{x}^*) = \frac{1}{2}\|\boldsymbol{x} - \boldsymbol{x}^*\|_{\mathbf{A}}^2.$$

Proof Taylor expanding $\phi(\boldsymbol{x})$ about the exact solution $\boldsymbol{x}^* = \mathbf{A}^{-1}\boldsymbol{b}$ gives

$$\begin{aligned}\phi(\boldsymbol{x}) &= \phi(\boldsymbol{x}^*) + \nabla\phi(\boldsymbol{x}^*)^T(\boldsymbol{x} - \boldsymbol{x}^*) + \frac{1}{2}(\boldsymbol{x} - \boldsymbol{x}^*)^T\mathbf{A}(\boldsymbol{x} - \boldsymbol{x}^*) \\ &= \phi(\boldsymbol{x}^*) + \mathbf{0} + \frac{1}{2}\|\boldsymbol{x} - \boldsymbol{x}^*\|_{\mathbf{A}}^2.\end{aligned}$$

Since $\mathbf{A}$ is positive definite, $\|\boldsymbol{x} - \boldsymbol{x}^*\|_{\mathbf{A}}^2 > 0$ for all $\boldsymbol{x} \neq \boldsymbol{x}^*$. Thus, minimizing $\phi(\boldsymbol{x})$ over any subspace is strictly equivalent to minimizing the energy norm of the error $\|\boldsymbol{x} - \boldsymbol{x}^*\|_{\mathbf{A}}$. □

Remark 6.10 **(The limitation of steepest descent)** The method of steepest descent chooses the negative gradient $-\nabla\phi(\boldsymbol{x}^{(k)}) = \boldsymbol{r}^{(k)}$ as its search direction at each step, computing $\boldsymbol{x}^{(k+1)} =$

$\boldsymbol{x}^{(k)} + \alpha_k \boldsymbol{r}^{(k)}$ where α_k is the exact line minimizer. Because successive gradients are mutually orthogonal in the Euclidean metric ($\boldsymbol{r}^{(k+1)T}\boldsymbol{r}^{(k)} = 0$), the iterates oscillate back and forth along narrow, elongated energy valleys. The convergence rate of steepest descent depends on $(\kappa - 1)/(\kappa + 1)$, leading to severe slowdown when $\kappa_2(\mathbf{A}) \gg 1$.

6.4.2 Conjugate Directions and Subspace Decoupling

To eliminate the inefficient zig-zagging of steepest descent, we replace orthogonal search directions with directions that are orthogonal with respect to the $\mathbf{A}$-inner product.

Def 6.13 **(A-Conjugate Vectors)** Let $\mathbf{A} \in \mathbb{R}^{n \times n}$ be symmetric positive definite. A set of nonzero vectors $\{\boldsymbol{p}_0, \boldsymbol{p}_1, \ldots, \boldsymbol{p}_{k-1}\} \subset \mathbb{R}^n$ is termed **A-conjugate** (or $\mathbf{A}$-orthogonal) if

$$\boldsymbol{p}_i^T \mathbf{A} \boldsymbol{p}_j = 0 \qquad \text{for all } i \neq j.$$

Because $\mathbf{A}$ is positive definite, any set of mutually conjugate nonzero vectors is linearly independent.

Thm 6.10 **(Subspace Decoupling and Exact Minimization)** Let $\{\boldsymbol{p}_0, \ldots, \boldsymbol{p}_{n-1}\}$ be an $\mathbf{A}$-conjugate basis for $\mathbb{R}^n$. The exact solution $\boldsymbol{x}^* = \boldsymbol{x}^{(0)} + \sum_{i=0}^{n-1} \alpha_i \boldsymbol{p}_i$ can be computed by performing n independent one-dimensional line minimizations. The optimal step lengths are given by

$$\alpha_k = \frac{\boldsymbol{p}_k^T \boldsymbol{r}^{(0)}}{\boldsymbol{p}_k^T \mathbf{A} \boldsymbol{p}_k} = \frac{\boldsymbol{p}_k^T \boldsymbol{r}^{(k)}}{\boldsymbol{p}_k^T \mathbf{A} \boldsymbol{p}_k}.$$

Furthermore, for any $k \leq n$, the partial sum $\boldsymbol{x}^{(k)} = \boldsymbol{x}^{(0)} + \sum_{i=0}^{k-1} \alpha_i \boldsymbol{p}_i$ is the unique vector that minimizes $\|\boldsymbol{x} - \boldsymbol{x}^*\|_{\mathbf{A}}$ over the affine subspace $\boldsymbol{x}^{(0)} + \operatorname{span}\{\boldsymbol{p}_0, \ldots, \boldsymbol{p}_{k-1}\}$.

Proof Express the error in the conjugate basis:

$$\boldsymbol{x} - \boldsymbol{x}^* = \sum_{i=0}^{n-1} c_i \boldsymbol{p}_i.$$

Because the directions are $\mathbf{A}$-conjugate, $\boldsymbol{p}_i^T \mathbf{A} \boldsymbol{p}_j = 0$ for $i \neq j$. The squared $\mathbf{A}$-norm decouples into a sum of scalar terms:

$$\begin{aligned} \|\boldsymbol{x} - \boldsymbol{x}^*\|_{\mathbf{A}}^2 &= \left(\sum_{i=0}^{n-1} c_i \boldsymbol{p}_i \right)^T \mathbf{A} \left(\sum_{j=0}^{n-1} c_j \boldsymbol{p}_j \right) \\ &= \sum_{i=0}^{n-1} \sum_{j=0}^{n-1} c_i c_j \boldsymbol{p}_i^T \mathbf{A} \boldsymbol{p}_j = \sum_{i=0}^{n-1} c_i^2 \boldsymbol{p}_i^T \mathbf{A} \boldsymbol{p}_i. \end{aligned}$$

Minimizing this sum over the first k coordinates is equivalent to minimizing each term $c_i^2 \boldsymbol{p}_i^T \mathbf{A} \boldsymbol{p}_i$ independently.

For the initial guess $\boldsymbol{x}^{(0)}$, write $\boldsymbol{x}^{(0)} - \boldsymbol{x}^* = -\sum_{i=0}^{n-1} \alpha_i \boldsymbol{p}_i$. Taking the $\mathbf{A}$-inner product with $\boldsymbol{p}_k$:

$$\boldsymbol{p}_k^T \mathbf{A}(\boldsymbol{x}^{(0)} - \boldsymbol{x}^*) = -\sum_{i=0}^{n-1} \alpha_i \boldsymbol{p}_k^T \mathbf{A} \boldsymbol{p}_i = -\alpha_k \boldsymbol{p}_k^T \mathbf{A} \boldsymbol{p}_k.$$

Since $\mathbf{A}\boldsymbol{x}^* = \boldsymbol{b}$, the left-hand side is $\boldsymbol{p}_k^T(\mathbf{A}\boldsymbol{x}^{(0)} - \boldsymbol{b}) = -\boldsymbol{p}_k^T \boldsymbol{r}^{(0)}$. Solving for α_k:

$$\alpha_k = \frac{\boldsymbol{p}_k^T \boldsymbol{r}^{(0)}}{\boldsymbol{p}_k^T \mathbf{A} \boldsymbol{p}_k}.$$

Because $\boldsymbol{r}^{(k)} = \boldsymbol{r}^{(0)} - \sum_{i=0}^{k-1} \alpha_i \mathbf{A}\boldsymbol{p}_i$ and $\boldsymbol{p}_k^T \mathbf{A} \boldsymbol{p}_i = 0$ for $i < k$, we have $\boldsymbol{p}_k^T \boldsymbol{r}^{(k)} = \boldsymbol{p}_k^T \boldsymbol{r}^{(0)}$, giving $\alpha_k = \frac{\boldsymbol{p}_k^T \boldsymbol{r}^{(k)}}{\boldsymbol{p}_k^T \mathbf{A} \boldsymbol{p}_k}$. □

6.4.3 The Conjugate Gradient Algorithm

The Conjugate Gradient algorithm generates the conjugate directions $\{\boldsymbol{p}_k\}$ dynamically from the sequence of residuals $\{\boldsymbol{r}^{(k)}\}$ via a Gram–Schmidt process. Because $\mathbf{A}$ is symmetric, all Gram–Schmidt orthogonalization coefficients against previous directions vanish except for the immediately preceding one, yielding a simple two-term recurrence.

Thm 6.11 **(The Conjugate Gradient Algorithm)** Let $\mathbf{A} \in \mathbb{R}^{n \times n}$ be symmetric positive definite and let $\boldsymbol{b} \in \mathbb{R}^n$. Given an initial estimate $\boldsymbol{x}^{(0)}$, compute $\boldsymbol{r}^{(0)} = \boldsymbol{b} - \mathbf{A}\boldsymbol{x}^{(0)}$ and set $\boldsymbol{p}^{(0)} = \boldsymbol{r}^{(0)}$. For $k = 0, 1, 2, \ldots$:

1. Compute the matrix–vector product $\boldsymbol{w}^{(k)} = \mathbf{A}\boldsymbol{p}^{(k)}$.
2. Compute the step length:
$$\alpha_k = \frac{\boldsymbol{r}^{(k)T} \boldsymbol{r}^{(k)}}{\boldsymbol{p}^{(k)T} \boldsymbol{w}^{(k)}}.$$
3. Update the solution iterate:
$$\boldsymbol{x}^{(k+1)} = \boldsymbol{x}^{(k)} + \alpha_k \boldsymbol{p}^{(k)}.$$
4. Update the residual vector:
$$\boldsymbol{r}^{(k+1)} = \boldsymbol{r}^{(k)} - \alpha_k \boldsymbol{w}^{(k)}.$$
5. Check termination: if $\|\boldsymbol{r}^{(k+1)}\|_2 / \|\boldsymbol{b}\|_2 \le \texttt{tol}$, terminate.
6. Compute the orthogonalization coefficient:
$$\beta_k = \frac{\boldsymbol{r}^{(k+1)T} \boldsymbol{r}^{(k+1)}}{\boldsymbol{r}^{(k)T} \boldsymbol{r}^{(k)}}.$$

7. Compute the next search direction:

$$\boldsymbol{p}^{(k+1)} = \boldsymbol{r}^{(k+1)} + \beta_k \boldsymbol{p}^{(k)}.$$

Thm 6.12 **(Orthogonality Properties of CG)** In exact arithmetic, the sequences generated by the Conjugate Gradient algorithm satisfy the following orthogonality relations for all $i \neq j$:

$$\boldsymbol{r}^{(i)T}\boldsymbol{r}^{(j)} = 0 \qquad \text{and} \qquad \boldsymbol{p}^{(i)T}\mathbf{A}\boldsymbol{p}^{(j)} = 0.$$

Moreover, the residual vectors and search directions span the Krylov subspace:

$$\operatorname{span}\{\boldsymbol{r}^{(0)}, \boldsymbol{r}^{(1)}, \ldots, \boldsymbol{r}^{(k)}\} = \operatorname{span}\{\boldsymbol{p}^{(0)}, \boldsymbol{p}^{(1)}, \ldots, \boldsymbol{p}^{(k)}\} = \mathcal{K}_{k+1}(\mathbf{A}, \boldsymbol{r}^{(0)}).$$

Consequently, the method is guaranteed to reach the exact solution in at most n iterations in exact arithmetic.

Remark 6.11 **(Connection with Lanczos)** By Thm 6.12, the residuals $\boldsymbol{r}^{(0)}, \ldots, \boldsymbol{r}^{(k-1)}$ are mutually orthogonal and the first j of them span $\mathcal{K}_j(\mathbf{A}, \boldsymbol{r}^{(0)})$. The Lanczos iteration of Def 4.15, started from $\boldsymbol{v}_1 = \boldsymbol{r}^{(0)}/\|\boldsymbol{r}^{(0)}\|_2$, produces an orthonormal basis with the same nested spans. Such a basis is unique up to the signs of its vectors, and the signs work out to

$$\boldsymbol{v}_{j+1} = (-1)^j \frac{\boldsymbol{r}^{(j)}}{\|\boldsymbol{r}^{(j)}\|_2}, \qquad j = 0, \ldots, k-1.$$

The CG coefficients also determine the Lanczos matrix $\mathbf{T}_k$ of Cor 4.29. Its entries are

$$(\mathbf{T}_k)_{11} = \frac{1}{\alpha_0}, \qquad (\mathbf{T}_k)_{j+1,j+1} = \frac{1}{\alpha_j} + \frac{\beta_{j-1}}{\alpha_{j-1}}, \qquad (\mathbf{T}_k)_{j+1,j} = (\mathbf{T}_k)_{j,j+1} = \frac{\sqrt{\beta_{j-1}}}{\alpha_{j-1}},$$

where α_j and β_j are the CG step length and direction coefficient. Conjugate gradients may therefore be read as the Lanczos algorithm applied to a linear system.
The correspondence also describes the CG iterate directly. Write $\boldsymbol{x}^{(k)} = \boldsymbol{x}^{(0)} + \mathbf{V}_k \boldsymbol{y}_k$. Since $\boldsymbol{r}^{(k)}$ is orthogonal to $\mathcal{K}_k$, and since $\mathbf{V}_k^T \boldsymbol{r}^{(0)} = \|\boldsymbol{r}^{(0)}\|_2 \boldsymbol{e}_1$ and $\mathbf{V}_k^T \mathbf{A} \mathbf{V}_k = \mathbf{T}_k$, we obtain

$$\mathbf{0} = \mathbf{V}_k^T \boldsymbol{r}^{(k)} = \mathbf{V}_k^T(\boldsymbol{r}^{(0)} - \mathbf{A}\mathbf{V}_k \boldsymbol{y}_k) = \|\boldsymbol{r}^{(0)}\|_2 \boldsymbol{e}_1 - \mathbf{T}_k \boldsymbol{y}_k.$$

The CG iterate is thus obtained by solving the projected system $\mathbf{T}_k \boldsymbol{y}_k = \|\boldsymbol{r}^{(0)}\|_2 \boldsymbol{e}_1$. In addition, the extreme eigenvalues of $\mathbf{T}_k$ approach $\lambda_{\min}$ and $\lambda_{\max}$ by Thm 4.30, so the CG coefficients give a running estimate of $\kappa_2(\mathbf{A})$ at no extra cost in products with $\mathbf{A}$.

Example **(Complete hand trace of a 2×2 CG solve)** Consider solving $\mathbf{A}\boldsymbol{x} = \boldsymbol{b}$ with

$$\mathbf{A} = \begin{pmatrix} 2 & 1 \\ 1 & 2 \end{pmatrix}, \qquad \boldsymbol{b} = \begin{pmatrix} 4 \\ 5 \end{pmatrix}, \qquad \boldsymbol{x}^{(0)} = \begin{pmatrix} 0 \\ 0 \end{pmatrix}.$$

1. **Initialization ($k = 0$):**

$$\boldsymbol{r}^{(0)} = \boldsymbol{b} - \mathbf{A}\boldsymbol{x}^{(0)} = \begin{pmatrix} 4 \\ 5 \end{pmatrix}, \qquad \boldsymbol{p}^{(0)} = \boldsymbol{r}^{(0)} = \begin{pmatrix} 4 \\ 5 \end{pmatrix}.$$

2. **First Iteration ($k = 0$):**

$$\begin{aligned}
\mathbf{A}\boldsymbol{p}^{(0)} &= \begin{pmatrix} 2 & 1 \\ 1 & 2 \end{pmatrix} \begin{pmatrix} 4 \\ 5 \end{pmatrix} = \begin{pmatrix} 13 \\ 14 \end{pmatrix}, \\
\boldsymbol{r}^{(0)T}\boldsymbol{r}^{(0)} &= 4^2 + 5^2 = 41, \\
\boldsymbol{p}^{(0)T}\mathbf{A}\boldsymbol{p}^{(0)} &= 4(13) + 5(14) = 52 + 70 = 122, \\
\alpha_0 &= \frac{41}{122} \approx 0.3361, \\
\boldsymbol{x}^{(1)} &= \boldsymbol{x}^{(0)} + \alpha_0 \boldsymbol{p}^{(0)} = \frac{41}{122} \begin{pmatrix} 4 \\ 5 \end{pmatrix} = \begin{pmatrix} 164/122 \\ 205/122 \end{pmatrix} \approx \begin{pmatrix} 1.3443 \\ 1.6803 \end{pmatrix}, \\
\boldsymbol{r}^{(1)} &= \boldsymbol{r}^{(0)} - \alpha_0 \mathbf{A}\boldsymbol{p}^{(0)} = \begin{pmatrix} 4 \\ 5 \end{pmatrix} - \frac{41}{122} \begin{pmatrix} 13 \\ 14 \end{pmatrix} = \begin{pmatrix} -45/122 \\ 36/122 \end{pmatrix} \approx \begin{pmatrix} -0.3689 \\ 0.2951 \end{pmatrix}.
\end{aligned}$$

The inner product $\boldsymbol{r}^{(1)T}\boldsymbol{r}^{(0)} = (-45)(4)+(36)(5) = -180+180 = 0$ confirms residual orthogonality.

3. **Direction Update ($k = 0 \to 1$):**

$$\begin{aligned}
\boldsymbol{r}^{(1)T}\boldsymbol{r}^{(1)} &= \frac{(-45)^2 + 36^2}{122^2} = \frac{2025 + 1296}{14884} = \frac{3321}{14884}, \\
\beta_0 &= \frac{\boldsymbol{r}^{(1)T}\boldsymbol{r}^{(1)}}{\boldsymbol{r}^{(0)T}\boldsymbol{r}^{(0)}} = \frac{3321/14884}{41} = \frac{81}{14884}, \\
\boldsymbol{p}^{(1)} &= \boldsymbol{r}^{(1)} + \beta_0 \boldsymbol{p}^{(0)} = \frac{1}{14884} \begin{pmatrix} -5166 \\ 4797 \end{pmatrix}.
\end{aligned}$$

Checking conjugacy with $\mathbf{A}\boldsymbol{p}^{(0)} = (13, 14)^T$ gives

$$\boldsymbol{p}^{(1)T}\mathbf{A}\boldsymbol{p}^{(0)} = \frac{-5166(13) + 4797(14)}{14884} = 0.$$

4. **Second Iteration ($k = 1$):** Computing step α_1 and updating $\boldsymbol{x}^{(2)}$ arrives at the exact solution $\boldsymbol{x}^{(2)} = (1, 2)^T$ with residual $\boldsymbol{r}^{(2)} = \mathbf{0}$.

6.4.4 Convergence Rate and Preconditioning

Although CG terminates in n steps in exact arithmetic, its primary utility in scientific computing is as a rapid iterative solver for which $k \ll n$.

Thm 6.13 **(Chebyshev Error Bound for Conjugate Gradient)** Let $\mathbf{A}$ be symmetric positive definite with 2-norm condition number $\kappa = \lambda_{\max}/\lambda_{\min}$. The error at step k satisfies

$$\frac{\|\boldsymbol{x}^{(k)} - \boldsymbol{x}^*\|_{\mathbf{A}}}{\|\boldsymbol{x}^{(0)} - \boldsymbol{x}^*\|_{\mathbf{A}}} \leq 2 \left(\frac{\sqrt{\kappa} - 1}{\sqrt{\kappa} + 1} \right)^k.$$

Cor 6.14 **(Termination with Distinct Eigenvalues)** If $\mathbf{A}$ has only m distinct eigenvalues, CG reaches the exact solution in at most m iterations in exact arithmetic.

Proof Let the distinct eigenvalues be $\lambda_1, \ldots, \lambda_m$. The polynomial

$$p_m(t) = \prod_{j=1}^{m} \left(1 - \frac{t}{\lambda_j}\right)$$

has degree m, satisfies $p_m(0) = 1$, and vanishes on the spectrum of $\mathbf{A}$. The spectral theorem therefore gives $p_m(\mathbf{A}) = \mathbf{0}$. CG minimizes the energy-norm error over all vectors whose error has the form $p(\mathbf{A})\boldsymbol{e}^{(0)}$ with $p(0) = 1$. Since $p_m(\mathbf{A})\boldsymbol{e}^{(0)} = \mathbf{0}$ is available after m steps, the minimum error is zero. □

Remark 6.12 **(The square root condition number advantage)** The convergence bound for the Conjugate Gradient method depends on $\sqrt{\kappa}$, whereas stationary iterations depend directly on κ. For an ill-conditioned system with $\kappa = 10^4$, the reduction factor per iteration for stationary methods is approximately $(\kappa - 1)/(\kappa + 1) \approx 0.9998$, whereas for CG it is $(\sqrt{\kappa} - 1)/(\sqrt{\kappa} + 1) = 99/101 \approx 0.9802$, converging fifty times faster per step.

Remark 6.13 **(Superlinear convergence)** The Chebyshev bound is a worst-case guarantee for an arbitrary spectrum. When the eigenvalues of $\mathbf{A}$ are grouped into tight clusters with only a few isolated outliers, CG eliminates the outlying eigenmodes in the first few iterations and then converges at a rate governed by the clustered eigenvalues. This behavior is termed superlinear convergence.

Def 6.14 **(Preconditioned Conjugate Gradient (PCG))** Let $\mathbf{M}$ be a symmetric positive definite preconditioner. The Preconditioned Conjugate Gradient algorithm solves $\mathbf{A}\boldsymbol{x} = \boldsymbol{b}$ by transforming the geometry using the inner product induced by $\mathbf{M}^{-1}$. At each step, a linear system $\mathbf{M}\boldsymbol{z}^{(k)} = \boldsymbol{r}^{(k)}$ is solved, and the step lengths and direction updates become

$$\alpha_k = \frac{\boldsymbol{r}^{(k)T}\boldsymbol{z}^{(k)}}{\boldsymbol{p}^{(k)T}\mathbf{A}\boldsymbol{p}^{(k)}}, \qquad \beta_k = \frac{\boldsymbol{r}^{(k+1)T}\boldsymbol{z}^{(k+1)}}{\boldsymbol{r}^{(k)T}\boldsymbol{z}^{(k)}}.$$

A well-chosen preconditioner clusters the eigenvalues of $\mathbf{M}^{-1}\mathbf{A}$ near 1, ensuring rapid convergence in few iterations.

Ex 6.7

1. **Check.** For a diagonal SPD matrix with three distinct eigenvalues, construct the polynomial in Cor 6.14 and verify that it annihilates the matrix.
2. **Derive.** Derive the formula for α_k by explicitly minimizing the single-variable quadratic function $g(\alpha) = \phi(\boldsymbol{x}^{(k)} + \alpha\boldsymbol{p}^{(k)})$ with respect to α.
3. **Compute.** Implement the Preconditioned Conjugate Gradient algorithm in Python using an incomplete Cholesky preconditioner, and test it on a 2D discrete Laplacian of size 100×100.

6.5 The Generalized Minimal Residual (GMRES) Method

When a linear system $\mathbf{A}\boldsymbol{x} = \boldsymbol{b}$ is nonsymmetric or indefinite, the quadratic energy minimization framework of the Conjugate Gradient method ceases to apply. The Faber–Manteuffel theorem characterizes the exceptional matrix classes that admit an optimal Krylov method with a fixed short recurrence. No such recurrence gives optimal residuals for arbitrary nonsymmetric matrices, so GMRES retains the full Arnoldi basis.

The Generalized Minimal Residual (GMRES) method of Saad and Schultz chooses, at every step k, a vector $\boldsymbol{x}^{(k)}$ in the affine Krylov subspace $\boldsymbol{x}^{(0)} + \mathcal{K}_k(\mathbf{A}, \boldsymbol{r}^{(0)})$ that minimizes the Euclidean residual norm $\|\boldsymbol{b} - \mathbf{A}\boldsymbol{x}\|_2$ (Saad and Schultz, 1986). GMRES constructs an orthonormal basis for the growing Krylov subspace by the Arnoldi process of Section 4.4 (Arnoldi, 1951) and reduces the original problem to a small upper-Hessenberg least-squares problem. The minimizing residual is unique, and the iterate is unique when the reduced least-squares problem has full column rank.

6.5.1 Reduction to Hessenberg Least Squares

Let $\boldsymbol{r}^{(0)} = \boldsymbol{b} - \mathbf{A}\boldsymbol{x}^{(0)}$ be nonzero and let $\beta = \|\boldsymbol{r}^{(0)}\|_2$. We run the Arnoldi iteration of Def 4.13 from $\boldsymbol{v}_1 = \boldsymbol{r}^{(0)}/\beta$. By Thm 4.25, after k steps the columns of $\mathbf{V}_k$ form an orthonormal basis of $\mathcal{K}_k(\mathbf{A}, \boldsymbol{r}^{(0)})$ and $\mathbf{A}\mathbf{V}_k = \mathbf{V}_{k+1}\bar{\mathbf{H}}_k$. These two facts reduce the residual minimization in $\mathbb{R}^n$ to a small least-squares problem.

Thm 6.15 **(The GMRES Projected Least-Squares Problem)** Let any vector in the affine space $\boldsymbol{x}^{(0)} + \mathcal{K}_k(\mathbf{A}, \boldsymbol{r}^{(0)})$ be parameterized as $\boldsymbol{x} = \boldsymbol{x}^{(0)} + \mathbf{V}_k\boldsymbol{y}$ for some vector $\boldsymbol{y} \in \mathbb{R}^k$. The residual vector satisfies

$$\boldsymbol{r}(\boldsymbol{x}) = \boldsymbol{b} - \mathbf{A}(\boldsymbol{x}^{(0)} + \mathbf{V}_k\boldsymbol{y}) = \boldsymbol{r}^{(0)} - \mathbf{A}\mathbf{V}_k\boldsymbol{y} = \beta\boldsymbol{v}_1 - \mathbf{V}_{k+1}\bar{\mathbf{H}}_k\boldsymbol{y} = \mathbf{V}_{k+1}(\beta\boldsymbol{e}_1 - \bar{\mathbf{H}}_k\boldsymbol{y}).$$

Because the columns of $\mathbf{V}_{k+1}$ are orthonormal, $\mathbf{V}_{k+1}$ is an isometry in the Euclidean norm:

$$\|\boldsymbol{b} - \mathbf{A}\boldsymbol{x}\|_2 = \|\beta\boldsymbol{e}_1 - \bar{\mathbf{H}}_k\boldsymbol{y}\|_2,$$

where $\boldsymbol{e}_1 = (1, 0, \ldots, 0)^T \in \mathbb{R}^{k+1}$. The GMRES iterate is $\boldsymbol{x}^{(k)} = \boldsymbol{x}^{(0)} + \mathbf{V}_k\boldsymbol{y}_k$, where $\boldsymbol{y}_k$ uniquely solves the $(k+1) \times k$ upper Hessenberg least-squares problem

$$\boldsymbol{y}_k = \arg\min_{\boldsymbol{y} \in \mathbb{R}^k} \|\beta\boldsymbol{e}_1 - \bar{\mathbf{H}}_k\boldsymbol{y}\|_2.$$

Proof Express any $\boldsymbol{x} \in \boldsymbol{x}^{(0)} + \mathcal{K}_k(\mathbf{A}, \boldsymbol{r}^{(0)})$ as $\boldsymbol{x} = \boldsymbol{x}^{(0)} + \mathbf{V}_k\boldsymbol{y}$ for $\boldsymbol{y} \in \mathbb{R}^k$. The residual vector is

$$\begin{aligned} \boldsymbol{r}(\boldsymbol{x}) &= \boldsymbol{b} - \mathbf{A}(\boldsymbol{x}^{(0)} + \mathbf{V}_k\boldsymbol{y}) \\ &= (\boldsymbol{b} - \mathbf{A}\boldsymbol{x}^{(0)}) - \mathbf{A}\mathbf{V}_k\boldsymbol{y} \\ &= \boldsymbol{r}^{(0)} - \mathbf{V}_{k+1}\bar{\mathbf{H}}_k\boldsymbol{y}. \end{aligned}$$

Since $\boldsymbol{v}_1 = \boldsymbol{r}^{(0)}/\beta$, we have $\boldsymbol{r}^{(0)} = \beta\boldsymbol{v}_1 = \mathbf{V}_{k+1}(\beta\boldsymbol{e}_1)$, where $\boldsymbol{e}_1 = (1, 0, \ldots, 0)^T \in \mathbb{R}^{k+1}$. Factoring $\mathbf{V}_{k+1}$ gives

$$\boldsymbol{r}(\boldsymbol{x}) = \mathbf{V}_{k+1}(\beta\boldsymbol{e}_1 - \bar{\mathbf{H}}_k\boldsymbol{y}).$$

Because the columns of $\mathbf{V}_{k+1}$ are orthonormal, $\mathbf{V}_{k+1}^T\mathbf{V}_{k+1} = \mathbf{I}_{k+1}$, and the Euclidean norm satisfies

$$\begin{aligned}\|\boldsymbol{r}(\boldsymbol{x})\|_2^2 &= (\beta\boldsymbol{e}_1 - \bar{\mathbf{H}}_k\boldsymbol{y})^T\mathbf{V}_{k+1}^T\mathbf{V}_{k+1}(\beta\boldsymbol{e}_1 - \bar{\mathbf{H}}_k\boldsymbol{y}) \\ &= \|\beta\boldsymbol{e}_1 - \bar{\mathbf{H}}_k\boldsymbol{y}\|_2^2.\end{aligned}$$

Thus $\|\boldsymbol{r}(\boldsymbol{x})\|_2 = \|\beta\boldsymbol{e}_1 - \bar{\mathbf{H}}_k\boldsymbol{y}\|_2$, reducing the n-dimensional optimization to the stated $(k+1) \times k$ Hessenberg least-squares problem. □

Thm 6.16 **(Polynomial Residual Characterization and Spectral Bound)** The k-th GMRES residual satisfies the polynomial minimization property

$$\|\boldsymbol{r}^{(k)}\|_2 = \min_{p \in \mathcal{P}_k,\, p(0)=1} \|p(\mathbf{A})\boldsymbol{r}^{(0)}\|_2.$$

If $\mathbf{A}$ is diagonalizable with $\mathbf{A} = \mathbf{X}\mathbf{\Lambda}\mathbf{X}^{-1}$, where $\mathbf{\Lambda} = \operatorname{diag}(\lambda_1, \ldots, \lambda_n)$, then the relative residual is bounded by

$$\frac{\|\boldsymbol{r}^{(k)}\|_2}{\|\boldsymbol{r}^{(0)}\|_2} \leq \kappa_2(\mathbf{X}) \min_{p \in \mathcal{P}_k,\, p(0)=1} \max_{\lambda \in \sigma(\mathbf{A})} |p(\lambda)|.$$

Proof Any vector $\boldsymbol{x} \in \boldsymbol{x}^{(0)} + \mathcal{K}_k(\mathbf{A}, \boldsymbol{r}^{(0)})$ takes the form $\boldsymbol{x} = \boldsymbol{x}^{(0)} + q(\mathbf{A})\boldsymbol{r}^{(0)}$ for some polynomial $q \in \mathcal{P}_{k-1}$. The resulting residual is

$$\boldsymbol{r}(\boldsymbol{x}) = \boldsymbol{b} - \mathbf{A}\boldsymbol{x} = \boldsymbol{r}^{(0)} - \mathbf{A}q(\mathbf{A})\boldsymbol{r}^{(0)} = (\mathbf{I} - \mathbf{A}q(\mathbf{A}))\boldsymbol{r}^{(0)}.$$

Define $p(z) = 1 - zq(z)$. Then $p \in \mathcal{P}_k$ and $p(0) = 1$. Conversely, any polynomial $p \in \mathcal{P}_k$ satisfying $p(0) = 1$ can be written as $p(z) = 1 - zq(z)$ for some $q \in \mathcal{P}_{k-1}$. Because GMRES minimizes $\|\boldsymbol{r}(\boldsymbol{x})\|_2$ over all such vectors, the first equality holds.
If $\mathbf{A} = \mathbf{X}\mathbf{\Lambda}\mathbf{X}^{-1}$, then $p(\mathbf{A}) = \mathbf{X}p(\mathbf{\Lambda})\mathbf{X}^{-1}$. Taking norms gives

$$\|p(\mathbf{A})\boldsymbol{r}^{(0)}\|_2 \leq \|\mathbf{X}\|_2\|p(\mathbf{\Lambda})\|_2\|\mathbf{X}^{-1}\|_2\|\boldsymbol{r}^{(0)}\|_2 = \kappa_2(\mathbf{X}) \left(\max_{\lambda \in \sigma(\mathbf{A})} |p(\lambda)| \right) \|\boldsymbol{r}^{(0)}\|_2,$$

which yields the spectral bound upon taking the minimum over all normalized polynomials $p \in \mathcal{P}_k$. □

6.5.2 Solution via Givens Rotations

Thm 6.17 **(Incremental QR Factorization of the Hessenberg Matrix)** At step k, applying the previously accumulated Givens rotations $\mathbf{G}_1, \ldots, \mathbf{G}_{k-1}$ transforms the first $k-1$ entries

of the new column of $\bar{\mathbf{H}}_k$. The subdiagonal entry at position $(k+1,k)$ is then eliminated using a new Givens rotation $\mathbf{G}_k$ defined by

$$\mathbf{G}_k = \begin{pmatrix} c_k & s_k \\ -s_k & c_k \end{pmatrix}, \qquad c_k = \frac{h_{kk}^{(k-1)}}{\sqrt{(h_{kk}^{(k-1)})^2 + h_{k+1,k}^2}}, \quad s_k = \frac{h_{k+1,k}}{\sqrt{(h_{kk}^{(k-1)})^2 + h_{k+1,k}^2}}.$$

Applying $\mathbf{G}_k \dots \mathbf{G}_1$ transforms $\bar{\mathbf{H}}_k$ into an upper triangular matrix $\mathbf{R}_k$ with a zero final row, and transforms the right-hand side vector $\beta \boldsymbol{e}_1$ into $\boldsymbol{g}_{k+1} = (\gamma_1, \dots, \gamma_k, \gamma_{k+1})^T$. The residual norm at iteration k is obtained without forming $\boldsymbol{x}^{(k)}$:

$$\|\boldsymbol{r}^{(k)}\|_2 = |\gamma_{k+1}|.$$

Proof Let $\mathbf{Q}_k = \mathbf{G}_k \dots \mathbf{G}_1 \in \mathbb{R}^{(k+1)\times(k+1)}$ denote the product of Givens rotations. Because each Givens rotation is an orthogonal transformation on $\mathbb{R}^{k+1}$, $\mathbf{Q}_k$ is orthogonal. The least-squares problem satisfies

$$\min_{\boldsymbol{y}\in\mathbb{R}^k} \|\beta \boldsymbol{e}_1 - \bar{\mathbf{H}}_k \boldsymbol{y}\|_2^2 = \min_{\boldsymbol{y}\in\mathbb{R}^k} \|\mathbf{Q}_k(\beta \boldsymbol{e}_1 - \bar{\mathbf{H}}_k \boldsymbol{y})\|_2^2 = \min_{\boldsymbol{y}\in\mathbb{R}^k} \left\| \begin{pmatrix} \gamma_{1:k} \\ \gamma_{k+1} \end{pmatrix} - \begin{pmatrix} \mathbf{R}_k \\ \mathbf{0}^T \end{pmatrix} \boldsymbol{y} \right\|_2^2.$$

Expanding the squared Euclidean norm:

$$\min_{\boldsymbol{y}\in\mathbb{R}^k} \left(\|\gamma_{1:k} - \mathbf{R}_k \boldsymbol{y}\|_2^2 + |\gamma_{k+1}|^2 \right).$$

Because $\mathbf{R}_k \in \mathbb{R}^{k\times k}$ is upper triangular and nonsingular (assuming no earlier breakdown), we can choose $\boldsymbol{y}_k = \mathbf{R}_k^{-1}\gamma_{1:k}$ to make the first term identically zero. The remaining squared residual is exactly $|\gamma_{k+1}|^2$, proving that $\|\boldsymbol{r}^{(k)}\|_2 = |\gamma_{k+1}|$. □

Remark 6.14 **(Happy breakdown vs. stagnation)** If the subdiagonal entry $h_{k+1,k}$ vanishes, Cor 4.27 shows that the Krylov subspace is invariant under $\mathbf{A}$. When $\mathbf{A}$ is nonsingular, its restriction to this invariant subspace is nonsingular, so the GMRES least-squares problem has residual zero and $\boldsymbol{x}^{(k)}$ is the exact solution. Termination for this reason is called happy breakdown. Stagnation is a different phenomenon. The Krylov space may continue to grow while its image under $\mathbf{A}$ contributes no useful correction to the residual. Non-normality, an unfavorable polynomial approximation problem, or a poor preconditioner can all cause this behavior, and the residual history alone does not identify which one is responsible.

6.5.3 Restarted GMRES and Practical Considerations

Def 6.15 **(Restarted GMRES(m))** Full GMRES requires storing k basis vectors of length n, incurring $O(kn)$ memory storage and $O(k^2 n)$ orthogonalization flops. To avoid memory exhaustion on large systems, the algorithm is restarted every m steps:

1. Run m steps of GMRES starting from $\boldsymbol{x}^{(0)}$ to produce $\boldsymbol{x}^{(m)}$.

2. Compute the residual $\boldsymbol{r}^{(m)} = \boldsymbol{b} - \mathbf{A}\boldsymbol{x}^{(m)}$.

3. Clear the Krylov basis vectors, set $\boldsymbol{x}^{(0)} \leftarrow \boldsymbol{x}^{(m)}$, and repeat.

Remark 6.15 **(The trade-off of restarting)** Restarted GMRES(m) bounds memory usage to $O(mn)$ and bounds orthogonalization cost per cycle. However, restarting discards the accumulated Krylov subspace and destroys the global optimality property. If m is chosen too small, the algorithm may stagnate completely.

6.5.4 Exercises

Ex 6.8

1. **Compute. (Arnoldi factorization by hand)** For the matrix and initial residual

$$\mathbf{A} = \begin{pmatrix} 1 & 1 & 0 \\ 0 & 2 & 1 \\ 0 & 0 & 3 \end{pmatrix}, \qquad \boldsymbol{r}^{(0)} = \begin{pmatrix} 1 \\ 0 \\ 1 \end{pmatrix},$$

perform two steps of the Arnoldi iteration by hand.

   (a) Compute $\boldsymbol{v}_1$ and $\boldsymbol{w}_1 = \mathbf{A}\boldsymbol{v}_1$. Find h_{11}, h_{21} and $\boldsymbol{v}_2$.

   (b) Compute $\boldsymbol{w}_2 = \mathbf{A}\boldsymbol{v}_2$. Find h_{12}, h_{22}, h_{32} and $\boldsymbol{v}_3$.

   (c) Explicitly assemble the matrices $\mathbf{V}_2$ and the 3×2 upper Hessenberg matrix $\bar{\mathbf{H}}_2$.

2. **Compute. (Hessenberg QR and residual norm via Givens)** Using the 3×2 matrix $\bar{\mathbf{H}}_2$ from Exercise 1:

   (a) Form the Givens rotation $\mathbf{G}_1$ that eliminates h_{21}, and the rotation $\mathbf{G}_2$ that eliminates h_{32}.

   (b) Apply these rotations to the right-hand side vector $\beta\boldsymbol{e}_1 = (\sqrt{2}, 0, 0)^T$.

   (c) Read off the residual norm $\|\boldsymbol{r}^{(2)}\|_2$ from the transformed vector without computing $\boldsymbol{x}^{(2)}$.

3. **Derive. (GMRES polynomial minimax on an ellipse)** Suppose the eigenvalues of $\mathbf{A}$ are enclosed within an ellipse in the complex plane centered at $c > 0$ with focal distance $d < c$ that does not contain the origin.

   (a) Using the scaled Chebyshev polynomial $p(z) = T_k((c - z)/d)/T_k(c/d)$, show that $p(0) = 1$.

   (b) Apply Thm 6.16 to derive an explicit upper bound for $\|\boldsymbol{r}^{(k)}\|_2/\|\boldsymbol{r}^{(0)}\|_2$ in terms of the semi-axes of the ellipse.

4. **Explore. (Exact stagnation for a cyclic permutation)** Let $\mathbf{A}$ be the cyclic permutation matrix defined by $\mathbf{A}\boldsymbol{e}_j = \boldsymbol{e}_{j+1}$ for $j < n$ and $\mathbf{A}\boldsymbol{e}_n = \boldsymbol{e}_1$. Take $\boldsymbol{x}^{(0)} = \mathbf{0}$ and $\boldsymbol{b} = \boldsymbol{e}_1$.

   (a) Prove that $\mathcal{K}_k(\mathbf{A}, \boldsymbol{b}) = \operatorname{span}\{\boldsymbol{e}_1, \ldots, \boldsymbol{e}_k\}$ for $k < n$.

   (b) Show that $\mathbf{A}\mathcal{K}_k = \operatorname{span}\{\boldsymbol{e}_2, \ldots, \boldsymbol{e}_{k+1}\}$ is orthogonal to $\boldsymbol{b}$. Conclude that $\|\boldsymbol{r}^{(k)}\|_2 = 1$ for every $k < n$.

   (c) Show that $\boldsymbol{x} = \boldsymbol{e}_n$ becomes available at step n and gives zero residual. Construct the degree-n residual polynomial that vanishes on the spectrum of $\mathbf{A}$, and explain why no polynomial of smaller degree can do so.

5. **Explore. (Left versus right preconditioning)** Compare left preconditioning ($\mathbf{M}^{-1}\mathbf{A}\boldsymbol{x} = \mathbf{M}^{-1}\boldsymbol{b}$) and right preconditioning ($\mathbf{A}\mathbf{M}^{-1}\boldsymbol{y} = \boldsymbol{b}$, $\boldsymbol{x} = \mathbf{M}^{-1}\boldsymbol{y}$). Show that right-preconditioned GMRES minimizes the true residual $\|\boldsymbol{b} - \mathbf{A}\boldsymbol{x}\|_2$, whereas left-preconditioned GMRES minimizes $\|\mathbf{M}^{-1}(\boldsymbol{b} - \mathbf{A}\boldsymbol{x})\|_2$. Explain why a code that uses left preconditioning may need to monitor both quantities.

6.6 Multigrid Methods

(Running example: Poisson matrix) We again use the Poisson matrix, now together with the hierarchy of grids from which it arose. Stationary iteration exposed the slow modes, whereas multigrid succeeds by changing the scale on which those modes are represented.

6.6.1 Oscillating Error and the Smoothing Property

Classical stationary iterations such as Jacobi and Gauss–Seidel converge slowly because the *smooth* (low-frequency) error components decay slowly. High-frequency, oscillatory components decay within a few sweeps. A Fourier analysis of the error makes this asymmetry precise, and multigrid is built on it. The smoother removes the oscillatory error components, and the remaining smooth error is passed to a coarser grid, where it becomes oscillatory and cheap to resolve.

Frequency-Dependent Error Decay in Jacobi Iteration For the 1D model problem, the Jacobi update reads

$$x_i^{(k+1)} = \tfrac{1}{2}\big(x_{i-1}^{(k)} + x_{i+1}^{(k)}\big) + \tfrac{1}{2}b_i.$$

Each new value is the *average of its two neighbors* plus a source term. Consider two extreme error profiles at some iteration:

- *Oscillatory error*: neighboring values often have opposite signs, so averaging damps many high-frequency modes rapidly. The exact alternating mode is an exception for unweighted Jacobi: its sign flips without shrinking. Weighted Jacobi or Gauss–Seidel supplies the smoothing used in multigrid.

- *Smooth error* ($e_i \propto \sin(j\pi h)$ for small j, slowly varying): neighbors of node i carry nearly the same value as e_i itself, so their average barely moves things. The error barely changes per sweep.

Jacobi acts as a local *averaging filter* that pulls each value toward the mean of its neighbors. Oscillations cancel in that average, whereas smooth trends do not. Starting from a random initial guess, the error spans all frequencies. After a handful of sweeps the oscillatory components are gone, but the smooth residual remains, requiring a coarser grid for resolution.

Remark 6.16 **(Choosing a solver)** The structure of the coefficient matrix determines the method. The following summary collects the recommendations of the preceding sections.

1. **Dense symmetric positive definite.** Cholesky factorization (Thm 2.18), with $n^3/3$ flops and no pivoting.
2. **Large sparse symmetric positive definite.** Preconditioned conjugate gradients (Section 6.4) or multigrid, with $O(\text{nnz}(\mathbf{A}))$ work per iteration.
3. **Dense general square.** Gaussian elimination with partial pivoting (Def 2.6), with $2n^3/3$ flops and the backward error of Thm 2.15.
4. **Large sparse general square.** Preconditioned GMRES (Section 6.5).
5. **Overdetermined with full column rank.** Householder QR (Thm 3.6), which avoids the squared condition number of the normal equations.
6. **Rank-deficient or nearly rank-deficient.** The SVD (Thm 4.22), which determines the numerical rank and the minimum-norm solution.

Ex 6.9 **Explore. (Observing oscillating error)** Set $n = 63$, $\boldsymbol{b} = \mathbf{0}$, and $\boldsymbol{x}^* = \mathbf{0}$, so the error equals the iterate exactly. Run Jacobi and Gauss–Seidel from the same random initial vector $\boldsymbol{x}^{(0)} \sim \mathcal{N}(0, \mathbf{I})$.

1. Plot the error profile $\boldsymbol{e}^{(k)} = \boldsymbol{x}^{(k)}$ at iterations $k = 0, 1, 3, 10, 20$. Describe how the spectral composition of the error changes. Observe the behavior of the high-frequency components.
2. Decompose $\boldsymbol{e}^{(k)}$ into Fourier mode amplitudes $\hat{e}_j^{(k)} = \langle \boldsymbol{e}^{(k)}, \phi^{(j)} \rangle / \|\phi^{(j)}\|^2$ and plot $|\hat{e}_j^{(k)}|$ vs. j at the same iterations. Which modes decay fast, which persist?
3. The Jacobi iteration matrix has eigenvalues $\mu_j = \cos(j\pi h)$. Use this to predict the amplitude of mode j after k sweeps as a function of the initial amplitude $|\hat{e}_j^{(0)}|$. Does your plot match the prediction?
4. After 20 sweeps, what does the remaining error look like? Argue, using the eigenvalue formula, why no fine-grid iteration alone can efficiently eliminate it, necessitating a coarse-grid correction.

Def 6.16 **(Fourier Modes on a Uniform Grid)** Consider the 1D model problem with the $n \times n$ tridiagonal matrix $\mathbf{A}$ having 2 on the main diagonal and -1 on the off-diagonals (the finite-difference discretization of $-u'' = f$ on $[0,1]$ scaled by h^2). Let $h = 1/(n+1)$. The *Fourier modes* are the vectors

$$\phi_j^{(k)} = \sin(kj\pi h), \quad j = 1, \ldots, n, \quad k = 1, \ldots, n.$$

These form an orthogonal basis for $\mathbb{R}^n$ (by discrete sine orthogonality) and diagonalize $\mathbf{A}$:

$$\mathbf{A}\phi^{(k)} = \lambda_k \, \phi^{(k)}, \qquad \lambda_k = 2 - 2\cos(k\pi h) = 4\sin^2\!\Big(\frac{k\pi h}{2}\Big).$$

Small k yields long-wavelength (smooth) modes, while large k yields short-wavelength (oscillatory) modes.

Thm 6.18 **(Eigenvalues of the Jacobi Iteration Matrix)** For the 1D model problem, the Jacobi iteration matrix $\mathbf{T}_J = \mathbf{I} - \frac{1}{2}\mathbf{A}$ has eigenvalues

$$\mu_k = \cos(k\pi h), \quad k = 1, \ldots, n,$$

with eigenvectors $\phi^{(k)}$. After ν Jacobi sweeps, mode k of the error is scaled by $\mu_k^\nu = \cos^\nu(k\pi h)$.

Proof Since $\mathbf{D} = 2\mathbf{I}$, the Jacobi iteration matrix is $\mathbf{T}_J = \mathbf{I} - \frac{1}{2}\mathbf{A}$. Applying $\mathbf{T}_J$ to the eigenvector $\phi^{(k)}$:

$$\begin{aligned}\mathbf{T}_J\phi^{(k)} &= \left(\mathbf{I} - \frac{1}{2}\mathbf{A}\right)\phi^{(k)} = \phi^{(k)} - \frac{1}{2}\mathbf{A}\phi^{(k)} \\ &= \phi^{(k)} - \frac{1}{2}\lambda_k\phi^{(k)} = \left(1 - \frac{\lambda_k}{2}\right)\phi^{(k)}.\end{aligned}$$

Substituting $\lambda_k = 2 - 2\cos(k\pi h)$ yields

$$1 - \frac{\lambda_k}{2} = 1 - (1 - \cos(k\pi h)) = \cos(k\pi h).$$

Therefore $\mathbf{T}_J\phi^{(k)} = \cos(k\pi h)\phi^{(k)}$. □

Def 6.17 **(Low- and High-Frequency Modes)** With respect to a grid of mesh size $h = 1/(n+1)$, partition the n Fourier modes by frequency:

- *Low-frequency (smooth) modes*: $k = 1, \ldots, \lfloor n/2 \rfloor$, where $k\pi h \in (0, \pi/2)$. These satisfy $\mu_k = \cos(k\pi h) \in (0,1)$, and for small k, $\mu_k \approx 1 - (k\pi h)^2/2 \approx 1$, so these modes decay slowly.
- *High-frequency (oscillatory) modes*: $k = \lfloor n/2 \rfloor + 1, \ldots, n$, where $k\pi h \in (\pi/2, \pi)$.

Remark 6.17 Standard Jacobi ($\omega = 1$) is a poor smoother, because the highest-frequency mode ($k = n$) satisfies $\mu_n = \cos(n\pi h) = \cos(\pi n/(n+1)) \approx -1$, so the iterate oscillates in sign rather than decaying. *Weighted (damped) Jacobi* with $\omega = 2/3$ fixes this by controlling the entire high-frequency band.

Def 6.18 **(Weighted Jacobi Iteration)** The *weighted Jacobi* method introduces a damping parameter $\omega \in (0, 1)$:

$$\boldsymbol{x}^{(k+1)} = \boldsymbol{x}^{(k)} + \omega \mathbf{D}^{-1}\big(\boldsymbol{b} - \mathbf{A}\boldsymbol{x}^{(k)}\big),$$

with iteration matrix $\mathbf{T}_\omega = \mathbf{I} - \omega \mathbf{D}^{-1}\mathbf{A}$ and eigenvalues

$$\mu_k(\omega) = 1 - \omega(1 - \cos(k\pi h)).$$

The standard multigrid choice is $\omega = 2/3$.

Thm 6.19 **(Smoothing Property of Weighted Jacobi)** For the 1D model problem with $\omega = 2/3$, the *smoothing factor*

$$\bar{\mu} = \max_{\lfloor n/2 \rfloor < k \le n} |\mu_k(2/3)| = \frac{1}{3}.$$

That is, every high-frequency component of the error is reduced by at least $2/3$ in a single weighted Jacobi step.

Proof For $k > n/2$, we have $k\pi h \in (\pi/2, \pi)$, so $1 - \cos(k\pi h) \in (1, 2)$. With $\omega = 2/3$:

$$\mu_k(2/3) = 1 - \tfrac{2}{3}(1 - \cos(k\pi h)) \in \Big(1 - \tfrac{4}{3},\ 1 - \tfrac{2}{3}\Big) = \Big(-\tfrac{1}{3},\ \tfrac{1}{3}\Big).$$

Hence $|\mu_k| < 1/3$ for all high-frequency modes. The supremum $1/3$ is approached as $k \to \lfloor n/2 \rfloor^+$ (where $\cos(k\pi h) \to 0^-$). □

Remark 6.18 After ν weighted Jacobi sweeps, every high-frequency error component is reduced by $(1/3)^\nu$. The remaining error is dominated by *smooth* modes, which are nearly constant on the scale of h. This smooth, slowly-varying error is the target of the coarse-grid correction in multigrid.

Example **Eigenvalue spectrum of $\mathbf{T}_{2/3}$ for $n = 15$, $h = 1/16$.**
The eigenvalues $\mu_k = 1 - \frac{2}{3}(1 - \cos(k\pi/16))$ for selected k:

- $k = 1$ (smoothest): $\mu_1 \approx 0.981$ (minimal decay per sweep).
- $k = 4$: $\mu_4 = 1 - \frac{2}{3}(1 - \cos(\pi/4)) \approx 0.804$ (slow decay).
- $k = 8$: $\mu_8 = 1 - \frac{2}{3}(1 - \cos(\pi/2)) = 1/3 \approx 0.333$.
- $k = 12$: $\mu_{12} = 1 - \frac{2}{3}(1 - \cos(3\pi/4)) \approx -0.138$ (rapid decay).
- $k = 15$ (most oscillatory): $\mu_{15} \approx -0.330$ (magnitude bounded by $1/3$).

Modes $k \ge 8$ all satisfy $|\mu_k| \le 1/3$, confirming the smoothing property. Modes $k \le 7$ range from 0.63 to 0.98 and are essentially untouched by the smoother.

6.6.2 The Two-Grid Correction Scheme

A smooth error on a fine grid of mesh size h looks *oscillatory* on a coarser grid of mesh size $2h$. More precisely, the smooth fine-grid modes ($k < n/2$) oscillate at wavelength $> 2h$ and are well-represented on the coarse grid. Once the high-frequency components have been attenuated by the smoother, the remaining error lives in a low-dimensional smooth subspace that a direct solve on a coarse grid of size $n/2$ can eliminate efficiently.

Def 6.19 **(Prolongation Operator)** The *prolongation* (or interpolation) operator $\mathbf{P} : \mathbb{R}^m \to \mathbb{R}^n$ (where $m = (n-1)/2$ for grids satisfying $n = 2m+1$) transfers a coarse-grid vector to the fine grid by *linear interpolation*. For a coarse-grid vector $\boldsymbol{v} \in \mathbb{R}^m$, the fine-grid vector $\mathbf{P}\boldsymbol{v} \in \mathbb{R}^n$ is defined by

$$(\mathbf{P}\boldsymbol{v})_{2j} = v_j, \qquad (\mathbf{P}\boldsymbol{v})_{2j-1} = \tfrac{1}{2}(v_{j-1} + v_j), \quad j = 1, \ldots, m,$$

with boundary conventions $v_0 = v_{m+1} = 0$. In matrix form, $\mathbf{P}$ is $n \times m$ with entries 1 at coarse-grid points and $\frac{1}{2}$ weights at fine-grid midpoints.

Def 6.20 **(Restriction Operator)** The *restriction* operator $\mathbf{R} : \mathbb{R}^n \to \mathbb{R}^m$ transfers a fine-grid vector to the coarse grid. The standard *full-weighting restriction* averages each coarse point with its fine-grid neighbors:

$$(\mathbf{R}\boldsymbol{w})_j = \tfrac{1}{4}w_{2j-1} + \tfrac{1}{2}w_{2j} + \tfrac{1}{4}w_{2j+1}, \quad j = 1, \ldots, m.$$

In matrix form, $\mathbf{R} = \frac{1}{2}\mathbf{P}^T$. This choice ensures that the Galerkin coarse-grid operator $\mathbf{A}_c = \mathbf{RAP}$ is SPD whenever $\mathbf{A}$ is SPD.

Remark 6.19 The relationship $\mathbf{R} = \frac{1}{2}\mathbf{P}^T$ is called the *variational principle* and guarantees that $\mathbf{A}_c = \mathbf{RAP}$ is a consistent, spectrally equivalent discretization on the coarse grid. For the 1D model problem, $\mathbf{A}_c$ equals the natural finite-difference matrix on the $2h$-grid exactly. This special property simplifies analysis but does not hold in general for variable-coefficient or higher-dimensional problems.

Def 6.21 **(Two-Grid Correction Scheme)** Given $\mathbf{A}\boldsymbol{x} = \boldsymbol{b}$ on the fine grid, one step of the *two-grid (TG) method* from an iterate $\boldsymbol{x}^{(0)}$ is:

1. **Pre-smoothing:** Apply $\nu_1 \geq 0$ smoother steps to get $\tilde{\boldsymbol{x}}$.
2. **Residual:** Compute $\boldsymbol{r} = \boldsymbol{b} - \mathbf{A}\tilde{\boldsymbol{x}}$.
3. **Restrict:** Form the coarse residual $\boldsymbol{r}_c = \mathbf{R}\boldsymbol{r} \in \mathbb{R}^m$.
4. **Coarse solve:** Solve $\mathbf{A}_c\boldsymbol{e}_c = \boldsymbol{r}_c$ exactly for $\boldsymbol{e}_c \in \mathbb{R}^m$.
5. **Prolong and correct:** Set $\hat{\boldsymbol{x}} = \tilde{\boldsymbol{x}} + \mathbf{P}\boldsymbol{e}_c$.
6. **Post-smoothing:** Apply $\nu_2 \geq 0$ smoother steps to $\hat{\boldsymbol{x}}$ to get $\boldsymbol{x}^{(1)}$.

The coarse-grid system $\mathbf{A}_c \boldsymbol{e}_c = \boldsymbol{r}_c$ solves the *residual equation*: if $\boldsymbol{x}^* - \tilde{\boldsymbol{x}}$ is smooth and lies in the range of $\mathbf{P}$, this recovers the error exactly.

Thm 6.20 **(Ideal Coarse-Grid Correction)** If the error $\boldsymbol{\epsilon} = \boldsymbol{x}^* - \tilde{\boldsymbol{x}}$ lies in the range of $\mathbf{P}$ (i.e., $\boldsymbol{\epsilon} = \mathbf{P}\boldsymbol{\delta}$ for some $\boldsymbol{\delta} \in \mathbb{R}^m$), then the two-grid correction recovers the exact solution: $\hat{\boldsymbol{x}} = \boldsymbol{x}^*$.

Proof Let $\boldsymbol{\epsilon} = \boldsymbol{x}^* - \tilde{\boldsymbol{x}} = \mathbf{P}\boldsymbol{\delta}$ for some $\boldsymbol{\delta} \in \mathbb{R}^m$. The residual vector is

$$\boldsymbol{r} = \boldsymbol{b} - \mathbf{A}\tilde{\boldsymbol{x}} = \mathbf{A}(\boldsymbol{x}^* - \tilde{\boldsymbol{x}}) = \mathbf{A}\boldsymbol{\epsilon} = \mathbf{A}\mathbf{P}\boldsymbol{\delta}.$$

Applying the restriction operator $\mathbf{R}$:

$$\boldsymbol{r}_c = \mathbf{R}\boldsymbol{r} = \mathbf{R}\mathbf{A}\mathbf{P}\boldsymbol{\delta} = \mathbf{A}_c\boldsymbol{\delta}.$$

Since $\mathbf{A}_c$ is nonsingular, solving the coarse problem $\mathbf{A}_c \boldsymbol{e}_c = \boldsymbol{r}_c$ yields

$$\boldsymbol{e}_c = \mathbf{A}_c^{-1}\boldsymbol{r}_c = \mathbf{A}_c^{-1}\mathbf{A}_c\boldsymbol{\delta} = \boldsymbol{\delta}.$$

Prolongating the coarse correction and updating gives

$$\hat{\boldsymbol{x}} = \tilde{\boldsymbol{x}} + \mathbf{P}\boldsymbol{e}_c = \tilde{\boldsymbol{x}} + \mathbf{P}\boldsymbol{\delta} = \tilde{\boldsymbol{x}} + \boldsymbol{\epsilon} = \boldsymbol{x}^*.$$

Thus the coarse-grid correction eliminates the error exactly. □

Example **Two-grid correction on the 1D model problem ($n = 7$, $h = 1/8$).**
Take $\mathbf{A}$ the 7×7 tridiagonal matrix with $2/-1$, $\boldsymbol{b} = \mathbf{1}$, $\boldsymbol{x}^{(0)} = \mathbf{0}$. After $\nu_1 = 2$ weighted Jacobi sweeps the residual norm is $\|\boldsymbol{r}\| \approx 0.82$, and after 20 Jacobi sweeps it is ≈ 0.32. By contrast, a single two-grid step with $\nu_1 = \nu_2 = 2$ reduces the residual to ≈ 0.08, which is roughly a $10\times$ improvement over pure smoothing at equal cost, because the coarse-grid solve resolves the smooth error components that the smoother leaves behind.

6.6.3 The Multigrid V-Cycle

The two-grid method replaces the fine-grid iteration matrix with one that damps all error components, at the cost of an exact coarse-grid solve. For large problems the coarse grid may itself be large. Multigrid applies the idea recursively. Instead of solving the coarse-grid problem exactly, it applies the two-grid scheme again at the coarser level, and so on, until the coarsest grid is trivially small (a single unknown). This gives the *V-cycle*.

Def 6.22 **(Multigrid Grid Hierarchy)** For $n = 2^L - 1$, define L nested grids with mesh sizes $h_\ell = 2^{-\ell}$, giving $n_\ell = 2^\ell - 1$ interior points at level $\ell = 1, \dots, L$. The finest grid is $\ell = L$ (mesh size h) and the coarsest is $\ell = 1$ (a single point). At each level ℓ we have a system matrix $\mathbf{A}_\ell$ (built via Galerkin: $\mathbf{A}_{\ell-1} = \mathbf{R}_\ell \mathbf{A}_\ell \mathbf{P}_\ell$), restriction $\mathbf{R}_\ell : \mathbb{R}^{n_\ell} \to \mathbb{R}^{n_{\ell-1}}$, and prolongation $\mathbf{P}_\ell : \mathbb{R}^{n_{\ell-1}} \to \mathbb{R}^{n_\ell}$.

Def 6.23 **(Multigrid V-Cycle)** $\texttt{MG}(\ell, \boldsymbol{x}, \boldsymbol{b})$: one multigrid V-cycle at level ℓ, returning an improved approximation to $\mathbf{A}_\ell \boldsymbol{x} = \boldsymbol{b}$:

1. **Base case** ($\ell = 1$): return $\mathbf{A}_1^{-1} \boldsymbol{b}$ (exact solve on a 1×1 system).
2. **Pre-smoothing:** $\boldsymbol{x} \leftarrow S_\ell^{\nu_1}(\boldsymbol{x}, \boldsymbol{b})$ (ν_1 steps of the smoother at level ℓ).
3. **Restrict residual:** $\boldsymbol{r}_{\ell-1} \leftarrow \mathbf{R}_\ell(\boldsymbol{b} - \mathbf{A}_\ell \boldsymbol{x})$.
4. **Coarse-grid correction:** $\boldsymbol{e}_{\ell-1} \leftarrow \texttt{MG}(\ell - 1, \mathbf{0}, \boldsymbol{r}_{\ell-1})$.
5. **Correct:** $\boldsymbol{x} \leftarrow \boldsymbol{x} + \mathbf{P}_\ell \boldsymbol{e}_{\ell-1}$.
6. **Post-smoothing:** $\boldsymbol{x} \leftarrow S_\ell^{\nu_2}(\boldsymbol{x}, \boldsymbol{b})$ (ν_2 steps of the smoother).
7. Return $\boldsymbol{x}$.

Calling `MG` once as the coarse-grid solver in step 4 gives the V-cycle. Calling it twice gives the *W-cycle*, which is more robust on difficult problems at higher cost.

Remark 6.20 A single V-cycle descends from the finest level L to the coarsest level 1 (the "down stroke") applying pre-smoothing and restriction, then ascends back (the "up stroke") applying prolongation and post-smoothing. Typical choices are $\nu_1 = \nu_2 = 1$ or 2. The W-cycle replaces the single recursive call in step 4 with two calls, more aggressively correcting smooth error at the cost of roughly $\log n$ extra work.

Thm 6.21 **(Mesh-Independent Convergence Principle)** Consider a family of SPD discretizations with grid-independent smoothing and approximation constants. If the chosen smoother damps the components not represented on the coarse grid and the transfer operators approximate the remaining smooth components uniformly in h, then the two-grid error-propagation operator satisfies

$$\|\boldsymbol{e}^{(k+1)}\|_{\mathbf{A}} \le \rho \, \|\boldsymbol{e}^{(k)}\|_{\mathbf{A}},$$

where $\rho < 1$ is independent of the mesh size. The same conclusion extends to a V-cycle when the recursive coarse solve preserves these uniform estimates. For the uniform-grid Poisson problem, weighted Jacobi or Gauss–Seidel with standard interpolation and restriction satisfies these hypotheses.

Proof The complete proof depends on the discretization and transfer operators. Its structure combines two estimates:

- **Smoothing property:** After ν smoother steps, the high-frequency components of the error satisfy $\|\mathbf{T}_\omega^\nu \boldsymbol{e}\|_{\mathbf{A}} \leq \eta(\nu)\|\boldsymbol{e}\|_{\mathbf{A}}$ where $\eta(\nu) \to 0$ as $\nu \to \infty$, uniformly in h.
- **Approximation property:** The coarse-grid correction resolves the smooth components: $\|(I - \mathbf{P}\mathbf{A}_c^{-1}\mathbf{R}\mathbf{A})\boldsymbol{e}\|_{\mathbf{A}} \leq Ch^2\|\mathbf{A}\boldsymbol{e}\|$ for a constant C independent of h.

The smoother first removes error that cannot be represented accurately on the coarse grid. The coarse-grid correction then approximates the remaining component. Uniform constants in these two estimates give a contraction factor independent of h. This argument is a proof outline. A numerical value of ρ requires the precise smoother, boundary conditions, and transfer operators. □

Remark 6.21 For classical iterations on the 1D Poisson problem, $\rho(\mathbf{T}_J) = \cos(\pi h) \approx 1 - \pi^2 h^2/2 \to 1$ as $h \to 0$, requiring $O(h^{-2}) = O(n)$ Jacobi iterations. With the V-cycle, the number of iterations to reach a fixed tolerance is bounded by a constant independent of n. The cost per V-cycle is $O(n)$ (geometric series over levels), making the total cost $O(n)$, which is the optimal asymptotic complexity for this class of problems.

Thm 6.22 **($O(n)$ Complexity of the V-Cycle)** For L-level multigrid on a problem with $n = n_L$ unknowns at the finest level, each V-cycle requires $O(n)$ floating-point operations, independent of the number of levels L.

Proof Let W_ℓ denote the work at level ℓ per V-cycle. At each level, the smoother and residual computation cost $O(n_\ell)$, and $n_\ell = n_L/2^{L-\ell}$. The total work is

$$W = \sum_{\ell=1}^{L} O(n_\ell) = O\left(\sum_{\ell=1}^{L} \frac{n}{2^{L-\ell}}\right) = O(n)\sum_{j=0}^{L-1} \frac{1}{2^j} < O(2n) = O(n).$$

The geometric series converges regardless of L, giving $O(n)$ total. □

Example **Multigrid vs. classical iterations on the 2D Poisson problem.**
Discretize $-\Delta u = f$ on the unit square with an $n \times n$ grid ($h = 1/(n+1)$), giving $N = n^2$ unknowns. Approximate iteration counts to reduce the residual by 10^{-8}, and total work in units of N multiplications:

Method	Iterations	Work/iter	Total work
Jacobi	$O(n^2)$	$O(N)$	$O(N^2)$
Gauss-Seidel	$O(n^2)$	$O(N)$	$O(N^2)$
Optimal SOR	$O(n)$	$O(N)$	$O(N^{3/2})$
Multigrid V-cycle	$O(1)$	$O(N)$	$O(N)$

For $n = 100$ ($N = 10^4$): multigrid requires ≈ 10 V-cycles while Gauss-Seidel requires ≈ 4000 iterations. At $n = 1000$ ($N = 10^6$): multigrid still requires ≈ 10 V-cycles, demonstrating mesh-independence, while Gauss-Seidel requires $\approx 400{,}000$ iterations.

Ex 6.10 **Compute. (Smoothing property)** For the $n = 15$ 1D model problem:

1. Compute the eigenvalues μ_k of the standard Jacobi ($\omega = 1$) and weighted Jacobi ($\omega = 2/3$) iteration matrices. Plot $|\mu_k|$ as a function of k on a single figure.
2. Starting from a random initial error $\boldsymbol{e}^{(0)} \in \mathbb{R}^{15}$, apply 3 steps of each method. Plot the initial and final errors, and describe qualitatively how each method reshapes the error.
3. Identify the "worst-case" mode (largest $|\mu_k|$) for each method over the high-frequency range $k > 7$. Why is the standard Jacobi smoother inadequate for multigrid?

Ex 6.11 **Compute. (Two-grid implementation)** Implement the two-grid correction scheme for the 1D model problem with $n = 2^m - 1$ interior points.

1. Write `restrict(r)` using the full-weighting formula and `prolongate(e)` using linear interpolation for transferring between a fine grid and the next coarser grid.
2. Implement `two_grid(A, b, x0, nu1, nu2)` with ν_1 pre-smoothing and ν_2 post-smoothing steps of weighted Jacobi ($\omega = 2/3$) and an exact coarse-grid solve via `np.linalg.solve`.
3. For $m = 6$ ($n = 63$), plot residual norm versus iteration count for: (a) weighted Jacobi alone, (b) repeated two-grid cycling. Use a semi-log scale. Estimate the per-cycle convergence factor ρ for the two-grid method.

Ex 6.12 **Compute. (Multigrid V-cycle)** Extend the two-grid implementation to a full V-cycle.

1. Implement `vcycle(x, b, A_list, level, nu1, nu2)` recursively. At level 1 (a single unknown), solve exactly. Test first on the two-grid case to verify it matches your Part 2 implementation.
2. Run the V-cycle on the 1D Poisson problem with $n = 2^{10} - 1 = 1023$ and $\nu_1 = \nu_2 = 1$. Measure the empirical convergence factor $\rho_k = \|\boldsymbol{r}^{(k)}\|/\|\boldsymbol{r}^{(k-1)}\|$ for $k = 1, \ldots, 15$. Is it approximately constant (mesh-independent)?
3. Repeat for $n = 2^{12} - 1 = 4095$. Compare the convergence factors. Then vary $\nu_1 = \nu_2 \in \{1, 2, 3\}$ and report the convergence factor versus the extra cost per cycle.

Read **Further reading.** Saad and Greenbaum give complementary treatments of Krylov methods and preconditioning (Saad, 2003; Greenbaum, 1997). The original conjugate-gradient and GMRES papers remain readable (Hestenes and Stiefel, 1952; Saad and Schultz, 1986). For multigrid, the tutorial by Briggs, Henson, and McCormick is a natural next step (Briggs et al., 2000).

6.7 Randomized Methods in Linear Algebra

Some quantities are expensive to compute exactly but inexpensive to estimate. The diagonal of $\mathbf{A}^{-1}$ requires n solves, a truncated SVD of an $m \times n$ matrix costs $O(mn \min(m, n))$ flops, and the Cholesky factor of a sparse matrix may be much denser than the matrix. Randomized methods replace each exact calculation by one that acts on a few random vectors or random samples. They accept an error that is controlled in expectation or with high probability, and they reuse one expensive object, such as a factorization or a product with $\mathbf{A}$, for every sample.

We first estimate traces and diagonals from random probe vectors. We then compute low-rank approximations from the range of $\mathbf{A}\mathbf{\Omega}$ for a random matrix $\mathbf{\Omega}$. Finally, we show how random sampling limits fill during Gaussian elimination on graph Laplacians, which produces a preconditioner for the conjugate gradient method.

6.7.1 Trace and Diagonal Estimation

The trace of $\mathbf{A}^{-1}$ and the diagonal of $\mathbf{A}^{-1}$ appear as sums of variances in statistics, as effective resistances in graphs, and as sensitivities of linear models. Computing $\operatorname{diag}(\mathbf{A}^{-1})$ exactly requires solving $\mathbf{A}\boldsymbol{x} = \boldsymbol{e}_i$ for every i. The estimators below use only products $\boldsymbol{z} \mapsto \mathbf{A}\boldsymbol{z}$, or solves with $\mathbf{A}$, applied to a few random vectors $\boldsymbol{z}$.

Thm 6.23 **(Hutchinson's Trace Estimator)** Let $\mathbf{A} \in \mathbb{R}^{n \times n}$ be symmetric, and let $\boldsymbol{z} \in \mathbb{R}^n$ have independent entries with mean zero and variance one. Then $\mathbb{E}[\boldsymbol{z}^T \mathbf{A} \boldsymbol{z}] = \operatorname{tr}(\mathbf{A})$. If the entries of $\boldsymbol{z}$ are ± 1 with equal probability, then

$$\operatorname{Var}(\boldsymbol{z}^T \mathbf{A} \boldsymbol{z}) = 2\Big(\|\mathbf{A}\|_F^2 - \sum_{i=1}^{n} a_{ii}^2\Big).$$

If the entries of $\boldsymbol{z}$ are standard normal, then $\operatorname{Var}(\boldsymbol{z}^T \mathbf{A} \boldsymbol{z}) = 2\|\mathbf{A}\|_F^2$.

Proof Expanding the quadratic form gives $\boldsymbol{z}^T \mathbf{A} \boldsymbol{z} = \sum_{i,j} a_{ij} z_i z_j$. Since $\mathbb{E}[z_i z_j]$ is one for $i = j$ and zero otherwise, the expectation is $\sum_i a_{ii}$.
For ± 1 entries, $z_i^2 = 1$, so $\boldsymbol{z}^T \mathbf{A} \boldsymbol{z} - \operatorname{tr}(\mathbf{A}) = 2 \sum_{i<j} a_{ij} z_i z_j$ by symmetry. The products $z_i z_j$ with $i < j$ have mean zero and variance one, and two different products are uncorrelated because $\mathbb{E}[z_i z_j z_k z_l] = 0$ unless $\{i, j\} = \{k, l\}$. Hence the variance is $4 \sum_{i<j} a_{ij}^2 = 2 \sum_{i \neq j} a_{ij}^2$.
For standard normal entries, write $\mathbf{A} = \mathbf{Q}\mathbf{\Lambda}\mathbf{Q}^T$ by Thm 4.4. The vector $\boldsymbol{g} = \mathbf{Q}^T \boldsymbol{z}$ is again standard normal, and $\boldsymbol{z}^T \mathbf{A} \boldsymbol{z} = \sum_i \lambda_i g_i^2$. Each g_i^2 has variance two and the g_i are independent, so the variance is $2 \sum_i \lambda_i^2 = 2\|\mathbf{A}\|_F^2$. □

Averaging k independent samples divides the variance by k. For a symmetric positive semidefinite matrix, $\|\mathbf{A}\|_F^2 = \sum_i \lambda_i^2 \leq (\sum_i \lambda_i)^2 = \operatorname{tr}(\mathbf{A})^2$, so with Gaussian probes the relative standard deviation of the averaged estimate is at most $\sqrt{2/k}$. The error decreases like $k^{-1/2}$, independently of n, which is the usual Monte Carlo rate. One hundred probes

therefore give a relative standard deviation of at most about fourteen percent, and the estimator is most useful when this accuracy is sufficient or when $\|\mathbf{A}\|_F$ is much smaller than $\operatorname{tr}(\mathbf{A})$ (Hutchinson, 1990).

Prop 6.24 **(Diagonal Estimator)** Let $\boldsymbol{z}_1, \dots, \boldsymbol{z}_k$ be independent vectors with independent ± 1 entries, and let $\odot$ denote the entrywise product. Then

$$\boldsymbol{d}_k = \frac{1}{k} \sum_{l=1}^{k} \boldsymbol{z}_l \odot (\mathbf{A}\boldsymbol{z}_l)$$

satisfies $\mathbb{E}[\boldsymbol{d}_k] = \operatorname{diag}(\mathbf{A})$.

Proof Entry i of $\boldsymbol{z} \odot (\mathbf{A}\boldsymbol{z})$ is $z_i \sum_j a_{ij} z_j = a_{ii} + \sum_{j \neq i} a_{ij} z_i z_j$, because $z_i^2 = 1$. The second sum has mean zero, so each term of the average has mean a_{ii}. □

To estimate $\operatorname{diag}(\mathbf{A}^{-1})$, we replace $\mathbf{A}\boldsymbol{z}_l$ by $\mathbf{A}^{-1}\boldsymbol{z}_l$ (Bekas et al., 2007). One sparse factorization of $\mathbf{A}$ is computed once, and each probe then costs one pair of triangular solves. For $k \ll n$, this replaces n solves by k solves. Bekas, Kokiopoulou, and Saad also use structured probe vectors that are orthogonal on blocks of the matrix, which removes the error from entries within those blocks.

6.7.2 Randomized Low-Rank Approximation

By Thm 4.21, the best rank-k approximation of $\mathbf{A}$ is given by its truncated SVD. Computing the full SVD of a large matrix in order to keep a few singular triplets wastes almost all of the work. If the singular values decay, the dominant left singular vectors span most of the range of $\mathbf{A}$. A product $\mathbf{A}\boldsymbol{\omega}$ with a random vector $\boldsymbol{\omega}$ is a random combination of the columns of $\mathbf{A}$, and it lies mostly in that dominant subspace. A few such products therefore span most of the dominant range.

Def 6.24 **(Randomized SVD)** Let $\mathbf{A} \in \mathbb{R}^{m \times n}$, let k be the target rank, and let $p \geq 2$ be an oversampling parameter.

1. Draw $\boldsymbol{\Omega} \in \mathbb{R}^{n \times (k+p)}$ with independent standard normal entries.
2. Form $\mathbf{Y} = \mathbf{A}\boldsymbol{\Omega}$ and compute its thin QR factorization $\mathbf{Y} = \mathbf{Q}\mathbf{R}$.
3. Form $\mathbf{B} = \mathbf{Q}^T\mathbf{A} \in \mathbb{R}^{(k+p) \times n}$ and compute its SVD $\mathbf{B} = \tilde{\mathbf{U}}\boldsymbol{\Sigma}\mathbf{V}^T$.
4. Return the approximate SVD $\mathbf{A} \approx (\mathbf{Q}\tilde{\mathbf{U}})\boldsymbol{\Sigma}\mathbf{V}^T$, truncated to rank k if required.

The algorithm touches $\mathbf{A}$ only through the two products $\mathbf{A}\boldsymbol{\Omega}$ and $\mathbf{Q}^T\mathbf{A}$. For a dense matrix these cost $O(mn(k+p))$ flops, and for a sparse matrix $O(\operatorname{nnz}(\mathbf{A})(k+p))$. The

QR factorization and the small SVD cost $O((m+n)(k+p)^2)$. The approximation error is $\|\mathbf{A} - \mathbf{Q}\mathbf{Q}^T\mathbf{A}\|$, because $\mathbf{Q}\tilde{\mathbf{U}}\mathbf{\Sigma}\mathbf{V}^T = \mathbf{Q}\mathbf{Q}^T\mathbf{A}$. Halko, Martinsson, and Tropp prove that

$$\mathbb{E}\,\|\mathbf{A} - \mathbf{Q}\mathbf{Q}^T\mathbf{A}\|_F \le \Big(1 + \frac{k}{p-1}\Big)^{1/2}\Big(\sum_{j>k}\sigma_j^2\Big)^{1/2}$$

for Gaussian $\mathbf{\Omega}$ (Halko et al., 2011). The second factor is the error of the truncated SVD, so the randomized method loses at most the first factor in expectation. With $p = k+1$, that factor is $\sqrt{2}$.

Remark 6.22 **(Slowly decaying singular values)** When the singular values decay slowly, the range of $\mathbf{A\Omega}$ contains significant components from the trailing singular vectors. Replacing $\mathbf{Y} = \mathbf{A\Omega}$ by $\mathbf{Y} = (\mathbf{A}\mathbf{A}^T)^q\mathbf{A\Omega}$ multiplies the jth singular direction by σ_j^{2q+1} and suppresses the trailing directions, exactly as in power iteration (Def 4.5). In floating-point arithmetic, the columns are orthonormalized after each multiplication, because the powers otherwise become numerically dependent (Halko et al., 2011).

6.7.3 Random Sampling in Gaussian Elimination

Section 2.4 showed that eliminating a vertex joins all of its neighbors into a clique, and that the resulting fill makes sparse Cholesky expensive on large meshes. For the Laplacian of a weighted graph, the clique has an explicit form. Random sampling can replace it by a sparse graph that is correct on average.

Prop 6.25 **(Elimination in a Graph Laplacian)** Let $\mathbf{L} = \mathbf{D} - \mathbf{W}$ be the Laplacian of a connected graph with nonnegative edge weights w_{ij} and degrees $d_i = \sum_j w_{ij}$. Eliminating vertex v produces a Schur complement that is again a graph Laplacian. Its graph is the graph of $\mathbf{L}$ with v removed, plus an edge of weight $w_{iv}w_{jv}/d_v$ between each pair of neighbors i, j of v.

Proof The Schur complement is $\mathbf{S} = \mathbf{B} - \boldsymbol{r}\boldsymbol{r}^T/d_v$, where $r_i = -w_{iv}$ and $\mathbf{B}$ is $\mathbf{L}$ with row and column v removed. For $i \neq j$, $s_{ij} = -w_{ij} - w_{iv}w_{jv}/d_v$, which is the negative of the stated edge weight. Every row of $\mathbf{L}$ sums to zero, so row i of $\mathbf{B}$ sums to w_{iv}. Row i of $\boldsymbol{r}\boldsymbol{r}^T/d_v$ sums to $w_{iv}\sum_j w_{jv}/d_v = w_{iv}$. Hence every row of $\mathbf{S}$ sums to zero, and $\mathbf{S}$ has nonpositive off-diagonal entries, which characterizes a graph Laplacian. □

Prop 6.26 **(An Unbiased Sparse Replacement for the Clique)** Let vertex v have neighbors $1, \dots, \delta$ with weights $w_i = w_{iv}$ and $d = \sum_i w_i$. For each neighbor i, draw one neighbor j with probability w_j/d and, if $j \neq i$, add an edge between i and j with weight $w_i/2$. The Laplacian of the resulting graph of at most δ edges has expected value equal to the Laplacian of the clique in Prop 6.25.

Proof Let $\mathbf{L}_{ij} = (\boldsymbol{e}_i - \boldsymbol{e}_j)(\boldsymbol{e}_i - \boldsymbol{e}_j)^T$ be the Laplacian of a single unit edge, with $\mathbf{L}_{ii} = \mathbf{0}$. The expected Laplacian of the edge added for neighbor i is $\sum_j (w_j/d)(w_i/2)\mathbf{L}_{ij}$. Summing over i counts each unordered pair twice, so the expectation is $\sum_{i<j}(w_iw_j/d)\mathbf{L}_{ij}$, which is the Laplacian of the clique. □

Eliminating every vertex in this way produces a factor $\tilde{\mathbf{L}}$ whose number of nonzero entries is proportional to the number of edges, rather than the superlinear fill of exact elimination. The factor is too inaccurate to solve with directly, but $\tilde{\mathbf{L}}\tilde{\mathbf{L}}^T$ approximates $\mathbf{L}$ well enough to serve as a preconditioner for the conjugate gradient method of Section 6.4. Kyng and Sachdeva use a refinement of this sampling, eliminate the vertices in random order, and prove that the resulting solver reaches relative accuracy ε in $O(m \log^3 n \log(1/\varepsilon))$ expected time for a graph with m edges (Kyng and Sachdeva, 2016). Practical implementations eliminate vertices in minimum-degree order, which reduces fill further but is not covered by the proof.

6.7.4 Exercises

Ex 6.13

1. **Check.** Verify Prop 6.26 for a vertex with three neighbors of weights 1, 2, and 3 by listing every outcome of the sampling and its probability.

2. **Derive.** Show that Hutchinson's estimator with ± 1 entries is exact for every diagonal matrix. Then show that, for every symmetric matrix, its variance is at most that of the Gaussian estimator, with equality exactly when the diagonal of $\mathbf{A}$ is zero.

3. **Compute.** Let $\mathbf{A}$ be the one-dimensional Poisson matrix with $n = 1000$. Estimate $\operatorname{tr}(\mathbf{A}^{-1})$ with $k = 10, 100, 1000$ probes using one Cholesky factorization. Plot the relative error against k on logarithmic axes and compare the slope with $-1/2$.

4. **Compute.** Construct a 1000×500 matrix with singular values $\sigma_j = 2^{-j}$ and random singular vectors. Apply Def 6.24 with $k = 20$ and $p = 0, 5, 20$, and compare $\|\mathbf{A} - \mathbf{Q}\mathbf{Q}^T\mathbf{A}\|_2$ with σ_{21}. Repeat with $\sigma_j = j^{-1/2}$, with and without one step of the power iteration in the remark.

7 Ordinary Differential Equations

7.1 Numerical Methods for Ordinary Differential Equations

An ordinary differential equation specifies the instantaneous velocity of a continuous state trajectory. A numerical time-stepping method replaces this continuous flow by a sequence of computable discrete states. The core mathematical questions in numerical ODE integration are consistency, zero-stability, global convergence, stiffness, and geometric structure preservation.

7.1.1 Initial Value Problems and Well-Posedness

Def 7.1 **(Initial Value Problem)** An initial value problem (IVP) for a system of ordinary differential equations has the form

$$\boldsymbol{y}'(t) = \boldsymbol{f}(t, \boldsymbol{y}(t)), \qquad \boldsymbol{y}(t_0) = \boldsymbol{y}_0,$$

where $t \in [t_0, T]$ and $\boldsymbol{y}(t) \in \mathbb{R}^m$. The function $\boldsymbol{f} : [t_0, T] \times \mathbb{R}^m \to \mathbb{R}^m$ is the vector field.

Thm 7.1 **(Picard–Lindelöf Existence and Uniqueness)** Let $\boldsymbol{f}(t, \boldsymbol{y})$ be continuous in t and uniformly Lipschitz continuous in $\boldsymbol{y}$ on the domain $[t_0, T] \times \mathbb{R}^m$:

$$\|\boldsymbol{f}(t, \boldsymbol{u}) - \boldsymbol{f}(t, \boldsymbol{v})\| \le L \|\boldsymbol{u} - \boldsymbol{v}\| \qquad \text{for all } \boldsymbol{u}, \boldsymbol{v} \in \mathbb{R}^m,$$

where $L < \infty$ is the Lipschitz constant. Then the IVP has a unique continuous solution $\boldsymbol{y}(t)$ on $[t_0, T]$.

Proof The differential equation $\boldsymbol{y}' = \boldsymbol{f}(t, \boldsymbol{y})$ with initial condition $\boldsymbol{y}(t_0) = \boldsymbol{y}_0$ is equivalent to the Picard integral equation:

$$\boldsymbol{y}(t) = \boldsymbol{y}_0 + \int_{t_0}^{t} \boldsymbol{f}(s, \boldsymbol{y}(s))\, ds.$$

Assume first that $L > 0$. Define the Picard operator $\mathcal{T}$ on the Banach space $C([t_0, T]; \mathbb{R}^m)$ equipped with the weighted norm $\|\boldsymbol{y}\|_* = \max_{t \in [t_0, T]} e^{-2L(t - t_0)} \|\boldsymbol{y}(t)\|$. For any two functions $\boldsymbol{u}, \boldsymbol{v}$:

$$\begin{aligned}
\|(\mathcal{T}\boldsymbol{u})(t) - (\mathcal{T}\boldsymbol{v})(t)\| &\le \int_{t_0}^{t} \|\boldsymbol{f}(s, \boldsymbol{u}(s)) - \boldsymbol{f}(s, \boldsymbol{v}(s))\|\, ds \\
&\le L \int_{t_0}^{t} \|\boldsymbol{u}(s) - \boldsymbol{v}(s)\|\, ds \\
&= L \int_{t_0}^{t} e^{2L(s - t_0)} \left(e^{-2L(s - t_0)} \|\boldsymbol{u}(s) - \boldsymbol{v}(s)\| \right) ds \\
&\le L \|\boldsymbol{u} - \boldsymbol{v}\|_* \int_{t_0}^{t} e^{2L(s - t_0)}\, ds \\
&= \frac{1}{2} \left(e^{2L(t - t_0)} - 1 \right) \|\boldsymbol{u} - \boldsymbol{v}\|_*.
\end{aligned}$$

Multiplying through by $e^{-2L(t - t_0)}$ yields $\|\mathcal{T}\boldsymbol{u} - \mathcal{T}\boldsymbol{v}\|_* \le \tfrac{1}{2} \|\boldsymbol{u} - \boldsymbol{v}\|_*$. Because the operator $\mathcal{T}$ is a strict contraction with modulus $1/2$ on a complete metric space, the Banach Fixed-Point Theorem guarantees the existence of a unique fixed point $\boldsymbol{y}^* = \mathcal{T}\boldsymbol{y}^*$, which is the unique solution to the IVP. If $L = 0$, then $\boldsymbol{f}(t, \boldsymbol{u}) = \boldsymbol{f}(t, \boldsymbol{v})$ for every $\boldsymbol{u}, \boldsymbol{v}$, so $\mathcal{T}$ has contraction modulus zero in the ordinary supremum norm. $\square$

7.1.2 Consistency, Stability, and the Discrete Gronwall Lemma

A numerical time-stepping method is convergent if the computed trajectory approaches the exact trajectory as the step size $h \to 0$. We quantify error at each step and track its accumulation over a finite time interval.

Def 7.2 **(Local Truncation Error and One-Step Defect)** Let $\boldsymbol{y}_{n+1} = \Phi_h(t_n, \boldsymbol{y}_n)$ denote an autonomous or non-autonomous one-step method. The local truncation error is the defect of the exact solution:

$$\boldsymbol{\tau}_{n+1} = \frac{\boldsymbol{y}(t_{n+1}) - \Phi_h(t_n, \boldsymbol{y}(t_n))}{h}.$$

The method is **consistent of order** p if $\|\boldsymbol{\tau}_{n+1}\| = O(h^p)$ as $h \to 0$ for all sufficiently smooth solutions.

Lem 7.2 **(Discrete Gronwall Lemma)** Let $\{e_n\}_{n=0}^{N}$ be a sequence of non-negative real numbers satisfying the linear recurrence

$$e_{n+1} \le (1+\alpha)e_n + \beta, \qquad n = 0, 1, \dots, N-1,$$

where $\alpha, \beta \ge 0$. For $\alpha > 0$,

$$e_n \le e^{n\alpha} e_0 + \beta \frac{e^{n\alpha} - 1}{\alpha}.$$

For $\alpha = 0$, the corresponding bound is $e_n \le e_0 + n\beta$.

Proof Expanding the recurrence by induction gives

$$e_n \le (1+\alpha)^n e_0 + \beta \sum_{k=0}^{n-1} (1+\alpha)^k = (1+\alpha)^n e_0 + \beta \frac{(1+\alpha)^n - 1}{\alpha}.$$

Using the standard inequality $1 + \alpha \le e^{\alpha}$ for all $\alpha \ge 0$, we have $(1+\alpha)^n \le (e^{\alpha})^n = e^{n\alpha}$, which completes the proof. □

Thm 7.3 **(Fundamental Theorem of One-Step Convergence)** Let Φ_h be a consistent one-step method of order p applied to an IVP whose vector field $\boldsymbol{f}$ is Lipschitz continuous with constant L. Suppose the numerical increment function satisfies $\|\Phi_h(t, \boldsymbol{u}) - \Phi_h(t, \boldsymbol{v})\| \le (1 + \Lambda h)\|\boldsymbol{u} - \boldsymbol{v}\|$ for some constant $\Lambda \ge 0$. Then on the fixed interval $[t_0, T]$ with $Nh = T - t_0$, the global error $\boldsymbol{e}_n = \boldsymbol{y}(t_n) - \boldsymbol{y}_n$ satisfies

$$\max_{0 \le n \le N} \|\boldsymbol{e}_n\| \le e^{\Lambda(T-t_0)} \|\boldsymbol{e}_0\| + \frac{e^{\Lambda(T-t_0)} - 1}{\Lambda} \max_{k} \|\boldsymbol{\tau}_k\|,$$

where the quotient is interpreted as $T - t_0$ when $\Lambda = 0$. Consequently, when the numerical and exact solutions use the same initial value, a local error of order $O(h^{p+1})$ yields a global error of order $O(h^p)$.

Proof Express the error at step $n+1$ by subtracting the numerical update from the exact solution:

$$\begin{aligned}\boldsymbol{e}_{n+1} &= \boldsymbol{y}(t_{n+1}) - \boldsymbol{y}_{n+1} \\ &= [\Phi_h(t_n, \boldsymbol{y}(t_n)) + h\boldsymbol{\tau}_{n+1}] - \Phi_h(t_n, \boldsymbol{y}_n) \\ &= [\Phi_h(t_n, \boldsymbol{y}(t_n)) - \Phi_h(t_n, \boldsymbol{y}_n)] + h\boldsymbol{\tau}_{n+1}.\end{aligned}$$

Taking norms and applying the triangle inequality yields

$$\|\boldsymbol{e}_{n+1}\| \le (1 + \Lambda h)\|\boldsymbol{e}_n\| + h \max_k \|\boldsymbol{\tau}_k\|.$$

Applying the Discrete Gronwall Lemma (Lem 7.2) with $\alpha = \Lambda h$ and $\beta = h \max_k \|\boldsymbol{\tau}_k\|$ gives

$$\|\boldsymbol{e}_n\| \le e^{n\Lambda h}\|\boldsymbol{e}_0\| + h \max_k \|\boldsymbol{\tau}_k\| \frac{e^{n\Lambda h} - 1}{\Lambda h} = e^{\Lambda(t_n - t_0)}\|\boldsymbol{e}_0\| + \frac{e^{\Lambda(t_n - t_0)} - 1}{\Lambda} \max_k \|\boldsymbol{\tau}_k\|.$$

Since $t_n - t_0 \le T - t_0$ and $\max_k \|\boldsymbol{\tau}_k\| = O(h^p)$, the global error is bounded by $O(h^p)$ when initial data are exact ($\boldsymbol{e}_0 = \boldsymbol{0}$). □

7.1.3 Euler Methods and Stability Functions

Def 7.3 **(Forward Euler, Backward Euler, and Trapezoidal Methods)** For time step $h = t_{n+1} - t_n$:

1. **Forward Euler (explicit):** $\boldsymbol{y}_{n+1} = \boldsymbol{y}_n + h\boldsymbol{f}(t_n, \boldsymbol{y}_n)$.
2. **Backward Euler (implicit):** $\boldsymbol{y}_{n+1} = \boldsymbol{y}_n + h\boldsymbol{f}(t_{n+1}, \boldsymbol{y}_{n+1})$.
3. **Trapezoidal Rule (implicit Crank–Nicolson):**

$$\boldsymbol{y}_{n+1} = \boldsymbol{y}_n + \frac{h}{2}\left[\boldsymbol{f}(t_n, \boldsymbol{y}_n) + \boldsymbol{f}(t_{n+1}, \boldsymbol{y}_{n+1})\right].$$

Thm 7.4 **(Stability Functions for Linear Test Equations)** Applied to the Dahlquist test equation $y' = \lambda y$ with $z = h\lambda \in \mathbb{C}$, the numerical update satisfies $y_{n+1} = R(z)y_n$. The stability functions are:

$$R_{\mathrm{FE}}(z) = 1 + z, \qquad R_{\mathrm{BE}}(z) = \frac{1}{1 - z}, \qquad R_{\mathrm{TR}}(z) = \frac{1 + z/2}{1 - z/2}.$$

The region of absolute stability is $\mathcal{S} = \{z \in \mathbb{C} : |R(z)| \le 1\}$. Backward Euler and the Trapezoidal Rule are A-stable: $\mathbb{C}^- = \{z \in \mathbb{C} : \mathrm{Re}(z) \le 0\} \subseteq \mathcal{S}$. Backward Euler is additionally L-stable: $\lim_{z \to -\infty} R_{\mathrm{BE}}(z) = 0$.

Proof For Forward Euler: $y_{n+1} = y_n + h\lambda y_n = (1 + z)y_n$. For Backward Euler: $y_{n+1} = y_n + zy_{n+1} \implies (1 - z)y_{n+1} = y_n \implies y_{n+1} = \frac{1}{1-z}y_n$. For Trapezoidal Rule: $y_{n+1} =$

$y_n+\frac{z}{2}(y_n+y_{n+1}) \implies (1-z/2)y_{n+1}=(1+z/2)y_n \implies y_{n+1}=\frac{1+z/2}{1-z/2}y_n$. Let $z=x+iy$ with $x\le 0$. For Backward Euler, $|1-z|^2=(1-x)^2+y^2\ge 1+y^2\ge 1$, so $|R_{\mathrm{BE}}(z)|=\frac{1}{|1-z|}\le 1$. For Trapezoidal Rule,

$$|1+z/2|^2=(1+x/2)^2+(y/2)^2=1+x+\frac{x^2+y^2}{4},$$
$$|1-z/2|^2=(1-x/2)^2+(y/2)^2=1-x+\frac{x^2+y^2}{4}.$$

Because $x\le 0$, we have $1+x\le 1-x$, which implies $|1+z/2|^2\le|1-z/2|^2$ and $|R_{\mathrm{TR}}(z)|\le 1$. Thus both methods are A-stable. Along the negative real axis, $\lim_{x\to-\infty}|R_{\mathrm{BE}}(x)|=\lim_{x\to-\infty}\frac{1}{1-x}=0$, proving L-stability. In contrast, $\lim_{x\to-\infty}R_{\mathrm{TR}}(x)=-1$, so the Trapezoidal Rule is not L-stable. □

7.1.4 Runge–Kutta Methods

Runge–Kutta methods achieve high orders of accuracy without storing historical steps by evaluating intermediate stages within each time step.

Def 7.4 **(Classical Fourth-Order Runge–Kutta Method (RK4))** For an autonomous ODE $\boldsymbol{y}'=\boldsymbol{f}(\boldsymbol{y})$, the classical RK4 method computes four stages:

$$\begin{aligned}\boldsymbol{k}_1&=\boldsymbol{f}(\boldsymbol{y}_n),\\ \boldsymbol{k}_2&=\boldsymbol{f}\left(\boldsymbol{y}_n+\frac{h}{2}\boldsymbol{k}_1\right),\\ \boldsymbol{k}_3&=\boldsymbol{f}\left(\boldsymbol{y}_n+\frac{h}{2}\boldsymbol{k}_2\right),\\ \boldsymbol{k}_4&=\boldsymbol{f}\left(\boldsymbol{y}_n+h\boldsymbol{k}_3\right),\end{aligned}$$

and sets the update

$$\boldsymbol{y}_{n+1}=\boldsymbol{y}_n+\frac{h}{6}\left(\boldsymbol{k}_1+2\boldsymbol{k}_2+2\boldsymbol{k}_3+\boldsymbol{k}_4\right).$$

The one-step defect is $O(h^5)$. With the normalization in Def 7.2, the local truncation error and the global error are both $O(h^4)$.

Thm 7.5 **(RK4 Stability Polynomial)** Applied to the scalar linear test equation $y'=\lambda y$, the stability function of RK4 is

$$R(z)=1+z+\frac{z^2}{2}+\frac{z^3}{6}+\frac{z^4}{24}=\sum_{k=0}^{4}\frac{z^k}{k!},\qquad z=h\lambda.$$

Proof Substituting $f(y) = \lambda y$ into the RK4 stages:

$$\begin{aligned}
k_1 &= \lambda y_n, \\
k_2 &= \lambda\left(y_n + \frac{h}{2}k_1\right) = \lambda\left(1 + \frac{z}{2}\right) y_n, \\
k_3 &= \lambda\left(y_n + \frac{h}{2}k_2\right) = \lambda\left(1 + \frac{z}{2} + \frac{z^2}{4}\right) y_n, \\
k_4 &= \lambda\left(y_n + hk_3\right) = \lambda\left(1 + z + \frac{z^2}{2} + \frac{z^3}{4}\right) y_n.
\end{aligned}$$

Substituting these into the update formula:

$$\begin{aligned}
y_{n+1} &= y_n + \frac{h}{6}\left[\lambda + 2\lambda\left(1 + \frac{z}{2}\right) + 2\lambda\left(1 + \frac{z}{2} + \frac{z^2}{4}\right) + \lambda\left(1 + z + \frac{z^2}{2} + \frac{z^3}{4}\right)\right] y_n \\
&= y_n + \frac{z}{6}\left[1 + (2 + z) + \left(2 + z + \frac{z^2}{2}\right) + \left(1 + z + \frac{z^2}{2} + \frac{z^3}{4}\right)\right] y_n \\
&= y_n + \frac{z}{6}\left[6 + 3z + z^2 + \frac{z^3}{4}\right] y_n \\
&= \left(1 + z + \frac{z^2}{2} + \frac{z^3}{6} + \frac{z^4}{24}\right) y_n.
\end{aligned}$$

□

7.1.5 Symplectic Integrators and Geometric Mechanics

Hamiltonian systems preserve geometric phase-space volume and energy. Standard Runge–Kutta methods dissipate or amplify energy over long integration horizons. Symplectic integrators preserve the underlying canonical symplectic 2-form.

Def 7.5 **(Velocity Verlet Algorithm)** For a separable Hamiltonian $H(\boldsymbol{q}, \boldsymbol{v}) = \frac{1}{2}\|\boldsymbol{v}\|_2^2 + V(\boldsymbol{q})$ with equations of motion $\boldsymbol{q}' = \boldsymbol{v}$ and $\boldsymbol{v}' = \mathbf{F}(\boldsymbol{q}) = -\nabla V(\boldsymbol{q})$, the Velocity Verlet method is:

$$\begin{aligned}
\boldsymbol{v}_{n+1/2} &= \boldsymbol{v}_n + \frac{h}{2}\mathbf{F}(\boldsymbol{q}_n), \\
\boldsymbol{q}_{n+1} &= \boldsymbol{q}_n + h\boldsymbol{v}_{n+1/2}, \\
\boldsymbol{v}_{n+1} &= \boldsymbol{v}_{n+1/2} + \frac{h}{2}\mathbf{F}(\boldsymbol{q}_{n+1}).
\end{aligned}$$

Thm 7.6 **(Symplecticity and Area Preservation of Velocity Verlet)** The one-step Velocity Verlet map $\Psi_h : (\boldsymbol{q}_n, \boldsymbol{v}_n) \mapsto (\boldsymbol{q}_{n+1}, \boldsymbol{v}_{n+1})$ preserves the canonical symplectic structure. In particular, the Jacobian matrix $\mathbf{J} = \frac{\partial(\boldsymbol{q}_{n+1}, \boldsymbol{v}_{n+1})}{\partial(\boldsymbol{q}_n, \boldsymbol{v}_n)}$ satisfies

$$\det(\mathbf{J}) = 1.$$

Equivalently, its Jacobian satisfies $\mathbf{J}^T\mathbf{\Omega}\mathbf{J} = \mathbf{\Omega}$ for the canonical symplectic matrix $\mathbf{\Omega}$. Symplecticity implies $\det(\mathbf{J}) = 1$, so Velocity Verlet also preserves phase-space volume.

Proof Decompose the Velocity Verlet step into the composition of three elementary shears:

$$\Psi_h = \psi_3 \circ \psi_2 \circ \psi_1,$$

where:

$$\begin{aligned}
\psi_1(\boldsymbol{q}, \boldsymbol{v}) &= \left(\boldsymbol{q},\, \boldsymbol{v} + \frac{h}{2}\mathbf{F}(\boldsymbol{q})\right), \\
\psi_2(\boldsymbol{q}, \boldsymbol{v}) &= (\boldsymbol{q} + h\boldsymbol{v},\, \boldsymbol{v}), \\
\psi_3(\boldsymbol{q}, \boldsymbol{v}) &= \left(\boldsymbol{q},\, \boldsymbol{v} + \frac{h}{2}\mathbf{F}(\boldsymbol{q})\right).
\end{aligned}$$

Evaluating the Jacobian of each shear map:

$$\mathbf{J}_1 = \begin{pmatrix} \mathbf{I} & \mathbf{0} \\ \frac{h}{2}\nabla\mathbf{F}(\boldsymbol{q}) & \mathbf{I} \end{pmatrix}, \qquad \mathbf{J}_2 = \begin{pmatrix} \mathbf{I} & h\mathbf{I} \\ \mathbf{0} & \mathbf{I} \end{pmatrix}, \qquad \mathbf{J}_3 = \begin{pmatrix} \mathbf{I} & \mathbf{0} \\ \frac{h}{2}\nabla\mathbf{F}(\boldsymbol{q}_{n+1}) & \mathbf{I} \end{pmatrix}.$$

Because $\mathbf{F} = -\nabla V$, the Hessian block $\mathbf{S} = \frac{h}{2}\nabla\mathbf{F} = -\frac{h}{2}\nabla^2 V$ is symmetric ($\mathbf{S}^T = \mathbf{S}$). With canonical symplectic matrix $\mathbf{\Omega} = \begin{pmatrix} \mathbf{0} & \mathbf{I} \\ -\mathbf{I} & \mathbf{0} \end{pmatrix}$, evaluating the quadratic form for $\mathbf{J}_1$:

$$\begin{aligned}
\mathbf{J}_1^T\mathbf{\Omega}\mathbf{J}_1 &= \begin{pmatrix} \mathbf{I} & \mathbf{S} \\ \mathbf{0} & \mathbf{I} \end{pmatrix}\begin{pmatrix} \mathbf{0} & \mathbf{I} \\ -\mathbf{I} & \mathbf{0} \end{pmatrix}\begin{pmatrix} \mathbf{I} & \mathbf{0} \\ \mathbf{S} & \mathbf{I} \end{pmatrix} \\
&= \begin{pmatrix} -\mathbf{S} & \mathbf{I} \\ -\mathbf{I} & \mathbf{0} \end{pmatrix}\begin{pmatrix} \mathbf{I} & \mathbf{0} \\ \mathbf{S} & \mathbf{I} \end{pmatrix} = \begin{pmatrix} -\mathbf{S}+\mathbf{S} & \mathbf{I} \\ -\mathbf{I} & \mathbf{0} \end{pmatrix} = \mathbf{\Omega}.
\end{aligned}$$

Similarly, for the translation shear $\mathbf{J}_2$:

$$\begin{aligned}
\mathbf{J}_2^T\mathbf{\Omega}\mathbf{J}_2 &= \begin{pmatrix} \mathbf{I} & \mathbf{0} \\ h\mathbf{I} & \mathbf{I} \end{pmatrix}\begin{pmatrix} \mathbf{0} & \mathbf{I} \\ -\mathbf{I} & \mathbf{0} \end{pmatrix}\begin{pmatrix} \mathbf{I} & h\mathbf{I} \\ \mathbf{0} & \mathbf{I} \end{pmatrix} \\
&= \begin{pmatrix} \mathbf{0} & \mathbf{I} \\ -\mathbf{I} & h\mathbf{I} \end{pmatrix}\begin{pmatrix} \mathbf{I} & h\mathbf{I} \\ \mathbf{0} & \mathbf{I} \end{pmatrix} = \begin{pmatrix} \mathbf{0} & \mathbf{I} \\ -\mathbf{I} & -h\mathbf{I}+h\mathbf{I} \end{pmatrix} = \mathbf{\Omega}.
\end{aligned}$$

The third shear $\mathbf{J}_3$ has the same form as $\mathbf{J}_1$ and satisfies $\mathbf{J}_3^T\mathbf{\Omega}\mathbf{J}_3 = \mathbf{\Omega}$. By the chain rule, $\mathbf{J} = \mathbf{J}_3\mathbf{J}_2\mathbf{J}_1$, so

$$\mathbf{J}^T\mathbf{\Omega}\mathbf{J} = \mathbf{J}_1^T\mathbf{J}_2^T\left(\mathbf{J}_3^T\mathbf{\Omega}\mathbf{J}_3\right)\mathbf{J}_2\mathbf{J}_1 = \mathbf{J}_1^T\left(\mathbf{J}_2^T\mathbf{\Omega}\mathbf{J}_2\right)\mathbf{J}_1 = \mathbf{J}_1^T\mathbf{\Omega}\mathbf{J}_1 = \mathbf{\Omega}.$$

Each shear has unit determinant: $\det(\mathbf{J}_1) = \det(\mathbf{J}_2) = \det(\mathbf{J}_3) = 1$. Therefore

$$\det(\mathbf{J}) = \det(\mathbf{J}_3)\det(\mathbf{J}_2)\det(\mathbf{J}_1) = 1 \cdot 1 \cdot 1 = 1.$$

□

7.1.6 Exercises

Ex 7.1

1. **Derive. The discrete Gronwall lemma and Forward Euler convergence.**

   (a) Let $\boldsymbol{y}(t)$ solve $\boldsymbol{y}' = \boldsymbol{f}(t, \boldsymbol{y})$ with $\|\boldsymbol{f}(t, \boldsymbol{u}) - \boldsymbol{f}(t, \boldsymbol{v})\| \le L\|\boldsymbol{u} - \boldsymbol{v}\|$. Use Taylor's theorem to show that the one-step defect of Forward Euler satisfies $\|\boldsymbol{y}(t_{n+1}) - (\boldsymbol{y}(t_n) + h\boldsymbol{f}(t_n, \boldsymbol{y}(t_n)))\| \le \frac{h^2}{2} M_2$, where $M_2 = \max \|\boldsymbol{y}''(t)\|$.

   (b) Derive the error recurrence $\|\boldsymbol{e}_{n+1}\| \le (1 + Lh)\|\boldsymbol{e}_n\| + \frac{h^2}{2} M_2$.

   (c) Apply Lem 7.2 to establish the rigorous global error bound $\|\boldsymbol{e}_n\| \le \frac{M_2}{2L}(e^{L(t_n - t_0)} - 1)h$.

2. **Derive. A-stability and L-stability of implicit methods.**

   (a) Plot the boundary of the Forward Euler stability region $|1 + z| = 1$ in the complex plane. Show that for real $\lambda < 0$, stability requires $h \le 2/|\lambda|$.

   (b) For the implicit Trapezoidal Rule, prove that $|R_{\mathrm{TR}}(z)| = 1$ if and only if $\mathrm{Re}(z) = 0$. Explain why this implies that undamped physical oscillations are integrated without artificial numerical dissipation.

   (c) Apply the Trapezoidal Rule and Backward Euler to the stiff problem $y' = -1000y$, $y(0) = 1$ with $h = 0.1$. Show that Trapezoidal Rule produces oscillatory values $(-99/101)^n \approx (-0.98)^n$, whereas Backward Euler dampens the mode immediately to $(1/101)^n \approx 10^{-2n}$.

3. **Explore. Symplectic geometry of the harmonic oscillator.** Consider the harmonic oscillator $q' = v$, $v' = -q$ with exact energy $E(q, v) = \frac{1}{2}(q^2 + v^2)$.

   (a) Compute the one-step update matrix $\mathbf{M}_{\mathrm{FE}}$ for Forward Euler and show that $\det(\mathbf{M}_{\mathrm{FE}}) = 1 + h^2 > 1$.

   (b) Compute the one-step update matrix $\mathbf{M}_{\mathrm{Verlet}}$ for Velocity Verlet. Confirm that $\det(\mathbf{M}_{\mathrm{Verlet}}) = 1$.

   (c) Show that Velocity Verlet conserves an exact modified shadow Hamiltonian $\tilde{H}(q, v) = \frac{1}{2}(q^2 + v^2) - \frac{h^2}{4} q^2 = H(q, v) + O(h^2)$.

4. **Compute. Adaptive step size control via embedded Runge–Kutta pairs.**

   (a) An embedded pair computes a lower-order approximation $\boldsymbol{y}_{n+1}^{[p]}$ and a higher-order approximation $\boldsymbol{y}_{n+1}^{[p+1]}$ sharing stages. Show that the difference $\boldsymbol{e}_{n+1} = \boldsymbol{y}_{n+1}^{[p+1]} - \boldsymbol{y}_{n+1}^{[p]}$ satisfies $\|\boldsymbol{e}_{n+1}\| = Ch^{p+1} + O(h^{p+2})$.

   (b) Given error tolerance ε and current step size h_{old}, derive the optimal step size formula $h_{\mathrm{new}} = s h_{\mathrm{old}} (\varepsilon / \|\boldsymbol{e}_{n+1}\|)^{1/(p+1)}$ with safety factor $s \in (0, 1)$.

   (c) Explain why adaptive solvers reject a step and repeat with smaller h when $\|\boldsymbol{e}_{n+1}\| > \varepsilon$, and why step growth is capped by $h_{\mathrm{new}} \le 5 h_{\mathrm{old}}$ in production codes.

5. **Explore. Stiffness in chemical cascades and BDF methods.** Consider the singularly perturbed initial value problem $\varepsilon y' = -y + \cos(t)$ with $y(0) = 0$ and $\varepsilon = 10^{-4}$.

   (a) Find the exact solution and identify the fast boundary layer transient $e^{-t/\varepsilon}$ and the slow quasi-steady state $\cos(t) + \varepsilon \sin(t)$.

   (b) Explain why explicit RK4 requires $h \lesssim 2.8 \times 10^{-4}$ throughout the entire simulation, even after the transient has decayed to zero.

   (c) Formulate the BDF2 update $\frac{3\boldsymbol{y}_{n+1} - 4\boldsymbol{y}_n + \boldsymbol{y}_{n-1}}{2h} = \boldsymbol{f}(t_{n+1}, \boldsymbol{y}_{n+1})$ and show that it allows time steps $h = 0.1$ in the smooth regime.

Read **Further reading.** Hairer, Nørsett, and Wanner give a careful account of ODE methods and their analysis (Hairer et al., 1993). Dahlquist's paper is the classical source for the stability barrier for linear multistep methods (Dahlquist, 1963). Quarteroni, Sacco, and Saleri provide a broader numerical-analysis reference (Quarteroni et al., 2007).

8 Applications and Research Bridges

8.1 Principal Component Analysis (PCA)

(Running example: data matrices) Least squares used the data matrix to predict a response. PCA uses the same matrix to find the orthogonal directions along which the observations vary most. The SVD answers both questions.

8.1.1 Definitions and Preprocessing

Def 8.1 **(Data Centering)** Data matrix $\mathbf{X} \in \mathbb{R}^{n \times p}$ (n samples, p features) must be **centered**:

$$X_{ij} \leftarrow X_{ij} - \bar{x}_j, \quad \bar{x}_j = \frac{1}{n} \sum_{i=1}^{n} X_{ij}.$$

Throughout this section, $\mathbf{X}$ is assumed centered.

Def 8.2 **(Sample Covariance Matrix)** The matrix $\mathbf{S} = \frac{1}{n-1} \mathbf{X}^T \mathbf{X} \in \mathbb{R}^{p \times p}$.

- **Diagonal:** S_{jj} is the variance of feature j.
- **Off-diagonal:** S_{ij} is the covariance between features i and j.
- **Property:** $\mathbf{S}$ is symmetric positive semidefinite (SPSD).

Ex 8.1 **Derive.**

1. Show that centering makes each column of $\mathbf{X}$ have mean zero.
2. Prove that the diagonal entries of $\mathbf{S}$ are sample variances.
3. Prove that $\mathbf{S}$ is symmetric positive semidefinite by computing $\boldsymbol{z}^T\mathbf{S}\boldsymbol{z}$.

8.1.2 PCA via SVD

Thm 8.1 **(Principal Directions and Scores)** Let $\mathbf{X} = \mathbf{U}\mathbf{\Sigma}\mathbf{V}^T$ be the SVD of the centered data matrix.

- **Principal Directions (Loadings):** Columns of $\mathbf{V}$ (eigenvectors of $\mathbf{S}$).
- **Principal Components (Scores):** $\mathbf{Z} = \mathbf{X}\mathbf{V} = \mathbf{U}\mathbf{\Sigma}$.
- **Variances:** $\lambda_i = \mathrm{Var}(\boldsymbol{z}_i) = \frac{\sigma_i^2}{n-1}$.

Proof Substituting the SVD $\mathbf{X} = \mathbf{U}\mathbf{\Sigma}\mathbf{V}^T$ into the sample covariance matrix $\mathbf{S}$:

$$\begin{aligned}\mathbf{S} &= \frac{1}{n-1}\mathbf{X}^T\mathbf{X} \\ &= \frac{1}{n-1}\left(\mathbf{V}\mathbf{\Sigma}^T\mathbf{U}^T\right)\left(\mathbf{U}\mathbf{\Sigma}\mathbf{V}^T\right) \\ &= \frac{1}{n-1}\mathbf{V}\mathbf{\Sigma}^T(\mathbf{U}^T\mathbf{U})\mathbf{\Sigma}\mathbf{V}^T \\ &= \mathbf{V}\left(\frac{\mathbf{\Sigma}^2}{n-1}\right)\mathbf{V}^T.\end{aligned}$$

Because $\mathbf{V}$ is orthogonal and $\frac{\mathbf{\Sigma}^2}{n-1}$ is diagonal with non-negative entries $\frac{\sigma_i^2}{n-1}$, this is the spectral decomposition of $\mathbf{S}$. The columns $\boldsymbol{v}_i$ are eigenvectors of $\mathbf{S}$ with eigenvalues $\lambda_i = \sigma_i^2/(n-1)$. The principal component score matrix is

$$\mathbf{Z} = \mathbf{X}\mathbf{V} = \left(\mathbf{U}\mathbf{\Sigma}\mathbf{V}^T\right)\mathbf{V} = \mathbf{U}\mathbf{\Sigma}(\mathbf{V}^T\mathbf{V}) = \mathbf{U}\mathbf{\Sigma}.$$

The sample covariance matrix of the scores is

$$\mathbf{S}_\mathbf{Z} = \frac{1}{n-1}\mathbf{Z}^T\mathbf{Z} = \frac{1}{n-1}(\mathbf{\Sigma}\mathbf{U}^T)(\mathbf{U}\mathbf{\Sigma}) = \frac{\mathbf{\Sigma}^2}{n-1} = \mathrm{diag}(\lambda_1, \ldots, \lambda_p),$$

confirming that distinct score columns $\boldsymbol{z}_i, \boldsymbol{z}_j$ have sample covariance zero for $i \neq j$. □

Remark 8.1 **(PCA stability)** Forming $\mathbf{X}^T\mathbf{X}$ squares the 2-norm condition number: $\kappa_2(\mathbf{X}^T\mathbf{X}) = \kappa_2(\mathbf{X})^2$ when $\mathbf{X}$ has full column rank. Compute the singular vectors directly from an SVD when small singular values or numerical rank matter.

Ex 8.2 **Derive.**

1. Starting from $\mathbf{X} = \mathbf{U}\mathbf{\Sigma}\mathbf{V}^T$, compute $\mathbf{S} = (n-1)^{-1}\mathbf{X}^T\mathbf{X}$.
2. Show that the columns of $\mathbf{V}$ are eigenvectors of $\mathbf{S}$.
3. Show that the corresponding eigenvalues are $\lambda_i = \sigma_i^2/(n-1)$.
4. Compute the score matrix $\mathbf{Z} = \mathbf{X}\mathbf{V}$ and prove that $\mathrm{Cov}(\mathbf{Z})$ is diagonal.
5. Use Ex 4.3 to explain the stability warning above.

8.1.3 Variance and Dimensionality Reduction

Def 8.3 **(Proportion of Variance Explained (PVE))** For the first k components:

$$\mathrm{PVE}_k = \frac{\sum_{i=1}^{k} \sigma_i^2}{\sum_{i=1}^{p} \sigma_i^2}.$$

Thm 8.2 **(Optimal Approximation)** The rank-k matrix

$$\mathbf{X}_k = \sum_{i=1}^{k} \sigma_i \boldsymbol{u}_i \boldsymbol{v}_i^T$$

is the optimal k-dimensional linear approximation of the data in the Frobenius norm, by the Eckart–Young theorem.

Ex 8.3 **Derive.**

1. Use Thm 4.21 to prove Thm 8.2.
2. Show that $\|\mathbf{X} - \mathbf{X}_k\|_F^2 = \sum_{i>k} \sigma_i^2$.
3. Show that $\mathrm{PVE}_k = 1 - \|\mathbf{X} - \mathbf{X}_k\|_F^2 / \|\mathbf{X}\|_F^2$.
4. Explain why a PVE threshold is a modeling choice, not a theorem.

8.1.4 Implementation Details

Remark 8.2 **(Standardization)** If features have different units, PCA will be dominated by large-scale features. Standardize by dividing each centered column by its standard deviation: $X_{ij} \leftarrow (X_{ij} - \bar{x}_j)/s_j$.

Remark 8.3 **(Interpretation)** Principal components are directions of large variance. A large loading identifies a direction of variation in the data matrix. It does not by itself explain why that variation occurs.

Def 8.4 **(Dual PCA for $p \gg n$)** When the number of features greatly exceeds the number of samples, we may diagonalize the $n \times n$ Gram matrix $\mathbf{K} = \mathbf{X}\mathbf{X}^T$ instead of the $p \times p$ covariance matrix. For each nonzero singular value, the feature-space principal direction is recovered from $\boldsymbol{v}_i = \mathbf{X}^T \boldsymbol{u}_i / \sigma_i$. Kernel PCA replaces this linear Gram matrix by a centered nonlinear kernel matrix.

8.1.5 Exercises

Ex 8.4

1. **Compute.** Center $\mathbf{X} = \begin{pmatrix} 2 & 1 \\ -1 & 3 \\ -1 & -4 \end{pmatrix}$ and compute $\mathbf{S}$. Find the PVE for $k = 1$.
2. **Derive.** Prove that PC scores $\boldsymbol{z}_i, \boldsymbol{z}_j$ are uncorrelated for $i \neq j$ using Ex 8.2.
3. **Compute.** Load the Iris dataset. Plot the data in the basis of the first two PCs. Do species cluster?
4. **Check.** Verify that the sum of variances $\sum \lambda_i$ equals the total variance $\mathrm{Tr}(\mathbf{S})$.

Read **Further reading.** Jolliffe is a standard statistical reference for principal component analysis (Jolliffe, 2002). The Golub–Kahan paper gives the bidiagonalization behind modern large-scale SVD calculations (Golub and Kahan, 1965).

8.2 Spectral Graph Theory, Diffusion Maps, and Graph Laplacians

Graph algorithms translate into numerical linear algebra once edge weights and vertex degrees are assembled into matrices. Undirected graphs lead to symmetric positive semidefinite Laplacians whose spectra characterize connectivity and graph cuts. Random walks and diffusion maps generalize these spectra to nonlinear manifold learning, offering multiscale geometric filtering and permutation-invariant representations for physical systems.

8.2.1 Graph Matrices and the Laplacian

Def 8.5 **(Adjacency and Degree Matrices)** For a weighted undirected graph $G = (V, E)$ with n vertices:

- **Adjacency A:** $A_{ij} = w_{ij} \geq 0$ if $(i, j) \in E$, and $A_{ij} = 0$ otherwise. We assume $w_{ij} = w_{ji}$ and $A_{ii} = 0$.
- **Degree D:** Diagonal matrix with entries $D_{ii} = d_i = \sum_{j=1}^{n} w_{ij}$.

Def 8.6 **(Unnormalized Graph Laplacian)** The unnormalized graph Laplacian is

$$\mathbf{L} = \mathbf{D} - \mathbf{A}.$$

For any signal $\boldsymbol{x} \in \mathbb{R}^n$ on the vertices, the action of $\mathbf{L}$ acts as a discrete Laplace operator:

$$(\mathbf{L}\boldsymbol{x})_i = \sum_{j \sim i} w_{ij}(x_i - x_j).$$

The associated quadratic form satisfies

$$\boldsymbol{x}^T \mathbf{L} \boldsymbol{x} = \frac{1}{2} \sum_{i,j=1}^{n} w_{ij}(x_i - x_j)^2 = \sum_{(i,j) \in E} w_{ij}(x_i - x_j)^2.$$

Prop 8.3 **(Laplacian Positive Semidefinite)** For an undirected graph with nonnegative weights, $\mathbf{L}$ is symmetric positive semidefinite, and $\mathbf{L}\mathbf{1} = \mathbf{0}$.

Proof Symmetry follows from $w_{ij} = w_{ji}$, so $\mathbf{A} = \mathbf{A}^T$ and $\mathbf{L} = \mathbf{D} - \mathbf{A} = \mathbf{L}^T$. Expanding the quadratic form algebraically from definitions:

$$\begin{aligned}
\boldsymbol{x}^T \mathbf{L} \boldsymbol{x} = \boldsymbol{x}^T (\mathbf{D} - \mathbf{A}) \boldsymbol{x} &= \sum_{i=1}^{n} d_i x_i^2 - \sum_{i,j=1}^{n} w_{ij} x_i x_j \\
&= \sum_{i=1}^{n} \left(\sum_{j=1}^{n} w_{ij} \right) x_i^2 - \sum_{i,j=1}^{n} w_{ij} x_i x_j \\
&= \frac{1}{2} \sum_{i,j=1}^{n} w_{ij} x_i^2 + \frac{1}{2} \sum_{i,j=1}^{n} w_{ij} x_j^2 - \sum_{i,j=1}^{n} w_{ij} x_i x_j \\
&= \frac{1}{2} \sum_{i,j=1}^{n} w_{ij}(x_i - x_j)^2 = \sum_{(i,j) \in E} w_{ij}(x_i - x_j)^2.
\end{aligned}$$

Because $w_{ij} \geq 0$ and $(x_i - x_j)^2 \geq 0$, each term is non-negative, so $\boldsymbol{x}^T \mathbf{L} \boldsymbol{x} \geq 0$ for all $\boldsymbol{x} \in \mathbb{R}^n$, proving positive semidefiniteness. For $\boldsymbol{x} = \mathbf{1}$, every coordinate satisfies $(\mathbf{L}\mathbf{1})_i = d_i - \sum_{j=1}^{n} w_{ij} = 0$, so $\mathbf{L}\mathbf{1} = \mathbf{0}$. □

Thm 8.4 **(Spectral Properties of the Graph Laplacian)** Let the eigenvalues of $\mathbf{L}$ be ordered as $0 = \lambda_1 \leq \lambda_2 \leq \cdots \leq \lambda_n$.

1. $\lambda_1 = 0$ with eigenvector $\boldsymbol{v}_1 = \mathbf{1}$.
2. The multiplicity of eigenvalue 0 equals the number of connected components of G.
3. The second smallest eigenvalue λ_2 is the algebraic connectivity (Fiedler value). It satisfies $\lambda_2 > 0$ if and only if G is connected.

Proof Proposition 8.3 shows that all eigenvalues of $\mathbf{L}$ are nonnegative and that $\mathbf{1} \in \ker(\mathbf{L})$, so $\lambda_1 = 0$.
It remains to identify the entire nullspace. If $\boldsymbol{x} \in \ker(\mathbf{L})$, then

$$0 = \boldsymbol{x}^T \mathbf{L} \boldsymbol{x} = \sum_{(i,j)\in E} w_{ij}(x_i - x_j)^2.$$

Every term is nonnegative. Hence $x_i = x_j$ across every edge with positive weight, and therefore $\boldsymbol{x}$ is constant on each connected component. Conversely, a vector that is constant on each connected component has zero difference across every edge and belongs to $\ker(\mathbf{L})$. The indicator vectors of the connected components form a basis for this nullspace, so the multiplicity of eigenvalue zero equals the number of components.
In particular, G is connected exactly when $\ker(\mathbf{L}) = \text{span}\{\mathbf{1}\}$. In that case zero is a simple eigenvalue, so $\lambda_2 > 0$. If G is disconnected, the nullspace has dimension at least two and $\lambda_2 = 0$. □

Cor 8.5 **(Rayleigh Characterization of Algebraic Connectivity)** For a connected undirected graph,

$$\lambda_2 = \min_{\boldsymbol{x}\perp\mathbf{1},\, \boldsymbol{x}\neq\mathbf{0}} \frac{\boldsymbol{x}^T\mathbf{L}\boldsymbol{x}}{\boldsymbol{x}^T\boldsymbol{x}} = \min_{\boldsymbol{x}\perp\mathbf{1},\, \boldsymbol{x}\neq\mathbf{0}} \frac{\sum_{(i,j)\in E} w_{ij}(x_i - x_j)^2}{\sum_i x_i^2}.$$

Proof By Thm 8.4, a connected graph has an orthonormal eigenbasis with $\boldsymbol{q}_1 = \mathbf{1}/\sqrt{n}$ and $0 = \lambda_1 < \lambda_2 \leq \cdots \leq \lambda_n$. If $\boldsymbol{x} \perp \mathbf{1}$, then $\boldsymbol{x} = \sum_{k=2}^n c_k \boldsymbol{q}_k$, and

$$\frac{\boldsymbol{x}^T\mathbf{L}\boldsymbol{x}}{\boldsymbol{x}^T\boldsymbol{x}} = \frac{\sum_{k=2}^n \lambda_k c_k^2}{\sum_{k=2}^n c_k^2} \geq \lambda_2.$$

Equality holds for a Fiedler eigenvector $\boldsymbol{q}_2$. The second expression follows from Def 8.6. □

Def 8.7 **(Normalized Laplacians)** Normalized Laplacians account for high-degree vertices:

- **Symmetric Normalized Laplacian:** $\mathbf{L}_{\text{sym}} = \mathbf{D}^{-1/2}\mathbf{L}\mathbf{D}^{-1/2} = \mathbf{I} - \mathbf{D}^{-1/2}\mathbf{A}\mathbf{D}^{-1/2}$. Its spectrum lies in the interval $[0, 2]$.
- **Random Walk Laplacian:** $\mathbf{L}_{\text{rw}} = \mathbf{D}^{-1}\mathbf{L} = \mathbf{I} - \mathbf{D}^{-1}\mathbf{A}$. It is similar to $\mathbf{L}_{\text{sym}}$ and shares its eigenvalues.

8.2.2 Spectral Clustering

Thm 8.6 **(Spectral Relaxation of Graph Cuts)** Partitioning a graph into two disjoint sets A and B to minimize the normalized cut is NP-hard. Relaxing the discrete indicator vector

to a continuous vector subject to an orthogonality constraint yields the Rayleigh quotient problem

$$\min_{\boldsymbol{x}\perp\mathbf{D}\mathbf{1},\,\boldsymbol{x}\neq\mathbf{0}} \frac{\boldsymbol{x}^T\mathbf{L}\boldsymbol{x}}{\boldsymbol{x}^T\mathbf{D}\boldsymbol{x}}.$$

The optimal solution is the Fiedler eigenvector $\boldsymbol{v}_2$ of the generalized eigenvalue problem $\mathbf{L}\boldsymbol{v} = \lambda\mathbf{D}\boldsymbol{v}$, or equivalently the second eigenvector of $\mathbf{L}_{\text{rw}}$. Spectral clustering embeds vertices in $\mathbb{R}^k$ using the first k nontrivial eigenvectors, then partitions the coordinates using k-means.

8.2.3 Diffusion Maps and Nonlinear Manifolds

Remark 8.4 **(From graphs to manifolds)** When data points $\boldsymbol{x}_1, \ldots, \boldsymbol{x}_N \in \mathbb{R}^d$ lie on a low-dimensional manifold embedded in high-dimensional space, ambient Euclidean distances can obscure intrinsic connectivity or create shortcuts across a curved manifold. Diffusion maps define a random walk on a graph constructed over the point cloud and, under suitable sampling and bandwidth limits, approximate diffusion generated by a Laplace–Beltrami operator.

Def 8.8 **(Diffusion Operator and Random Walk)** Given point cloud $\{\boldsymbol{x}_i\}_{i=1}^N \subset \mathbb{R}^d$, construct a pairwise affinity matrix $\mathbf{K} \in \mathbb{R}^{N\times N}$ using a Gaussian kernel with bandwidth $\delta > 0$:

$$K_{ij} = \exp\left(-\frac{\|\boldsymbol{x}_i - \boldsymbol{x}_j\|^2}{2\delta^2}\right).$$

Let $d_i = \sum_{j=1}^N K_{ij}$ denote the row sums, and define $\mathbf{D} = \operatorname{diag}(d_1, \ldots, d_N)$. The row-stochastic diffusion matrix is

$$\mathbf{M} = \mathbf{D}^{-1}\mathbf{K}.$$

The entry $M_{ij} = K_{ij}/d_i$ defines the transition probability from point $\boldsymbol{x}_i$ to point $\boldsymbol{x}_j$ in a single diffusion step.

Thm 8.7 **(Spectral Decomposition of the Diffusion Operator)** The matrix $\mathbf{M} = \mathbf{D}^{-1}\mathbf{K}$ is similar to the real symmetric matrix

$$\mathbf{S} = \mathbf{D}^{-1/2}\mathbf{K}\mathbf{D}^{-1/2} = \mathbf{D}^{1/2}\mathbf{M}\mathbf{D}^{-1/2}.$$

Let $\mathbf{S} = \mathbf{V}\boldsymbol{\Lambda}\mathbf{V}^T$ be the eigendecomposition of $\mathbf{S}$ with orthonormal eigenvectors $\mathbf{V} = [\boldsymbol{v}_0, \boldsymbol{v}_1, \ldots, \boldsymbol{v}_{N-1}]$ and eigenvalues

$$1 = \lambda_0 > \lambda_1 \geq \lambda_2 \geq \cdots \geq \lambda_{N-1} \geq 0.$$

Then $\mathbf{M}$ has real eigenvalues λ_k with right eigenvectors $\boldsymbol{\psi}_k = \mathbf{D}^{-1/2}\boldsymbol{v}_k$ and left eigenvectors $\boldsymbol{\phi}_k = \mathbf{D}^{1/2}\boldsymbol{v}_k$. The right eigenvectors satisfy $\mathbf{M}\boldsymbol{\psi}_k = \lambda_k\boldsymbol{\psi}_k$, with $\boldsymbol{\psi}_0 = \mathbf{1}$.

Proof Because $\mathbf{K}$ is symmetric and strictly positive, $\mathbf{S} = \mathbf{D}^{-1/2}\mathbf{K}\mathbf{D}^{-1/2}$ is real symmetric. By the spectral theorem (Thm 4.4), $\mathbf{S} = \mathbf{V}\mathbf{\Lambda}\mathbf{V}^T$ with orthonormal eigenvectors $\mathbf{V}^T\mathbf{V} = \mathbf{I}$. Rewriting the similarity relation:

$$\mathbf{M} = \mathbf{D}^{-1/2}\mathbf{S}\mathbf{D}^{1/2}.$$

Multiplying by $\boldsymbol{\psi}_k = \mathbf{D}^{-1/2}\boldsymbol{v}_k$:

$$\mathbf{M}\boldsymbol{\psi}_k = \left(\mathbf{D}^{-1/2}\mathbf{S}\mathbf{D}^{1/2}\right)\left(\mathbf{D}^{-1/2}\boldsymbol{v}_k\right) = \mathbf{D}^{-1/2}\mathbf{S}\boldsymbol{v}_k = \lambda_k\mathbf{D}^{-1/2}\boldsymbol{v}_k = \lambda_k\boldsymbol{\psi}_k.$$

Similarly, defining left eigenvectors $\boldsymbol{\phi}_k = \mathbf{D}^{1/2}\boldsymbol{v}_k$:

$$\boldsymbol{\phi}_k^T\mathbf{M} = \left(\mathbf{D}^{1/2}\boldsymbol{v}_k\right)^T\left(\mathbf{D}^{-1/2}\mathbf{S}\mathbf{D}^{1/2}\right) = \boldsymbol{v}_k^T\mathbf{S}\mathbf{D}^{1/2} = \lambda_k\boldsymbol{v}_k^T\mathbf{D}^{1/2} = \lambda_k\boldsymbol{\phi}_k^T.$$

Biorthogonality follows directly:

$$\boldsymbol{\phi}_j^T\boldsymbol{\psi}_k = \left(\mathbf{D}^{1/2}\boldsymbol{v}_j\right)^T\left(\mathbf{D}^{-1/2}\boldsymbol{v}_k\right) = \boldsymbol{v}_j^T\boldsymbol{v}_k = \delta_{jk}.$$

Because $\mathbf{M}$ is row-stochastic, $\mathbf{M}\mathbf{1} = \mathbf{1}$, so $\lambda_0 = 1$ with right eigenvector $\boldsymbol{\psi}_0 = \mathbf{1}$. □

Def 8.9 **(Diffusion Distance and Diffusion Map Coordinates)** The t-step diffusion operator $\mathbf{M}^t$ represents running the random walk forward for time t. The diffusion distance between points $\boldsymbol{x}_i$ and $\boldsymbol{x}_j$ measures the difference between their conditional transition distributions weighted by the stationary measure $\boldsymbol{\pi}$:

$$D_t(\boldsymbol{x}_i, \boldsymbol{x}_j)^2 = \|\mathbf{M}^t(i,:) - \mathbf{M}^t(j,:)\|^2_{\mathbf{D}^{-1}} = \sum_{k=1}^{N-1}\lambda_k^{2t}\left(\psi_k(i) - \psi_k(j)\right)^2.$$

The diffusion map embedding of point $\boldsymbol{x}_i$ into an m-dimensional Euclidean space is

$$\Psi_t(\boldsymbol{x}_i) = \begin{pmatrix}\lambda_1^t\psi_1(i)\\ \lambda_2^t\psi_2(i)\\ \vdots \\ \lambda_m^t\psi_m(i)\end{pmatrix} \in \mathbb{R}^m.$$

Under this embedding, Euclidean distance in $\mathbb{R}^m$ equals the truncated diffusion distance:

$$\|\Psi_t(\boldsymbol{x}_i) - \Psi_t(\boldsymbol{x}_j)\|_2^2 \approx D_t(\boldsymbol{x}_i, \boldsymbol{x}_j)^2.$$

Remark 8.5 **(Diffusion time as a geometric low-pass filter)** The factor λ_k^{2t} in the diffusion distance acts as an exponential low-pass filter on manifold coordinates. Because $\lambda_k < 1$ for all $k \geq 1$, increasing t rapidly suppresses high-frequency modes associated with small eigenvalues, leaving only the dominant macroscopic geometric coordinates.

Remark 8.6 **(Density normalization)** The random walk of Def 8.8 depends on how densely the points sample the manifold as well as on its geometry. Coifman and Lafon separate the two

effects by renormalizing the kernel before forming the random walk (Coifman and Lafon, 2006). For a parameter $\alpha \in [0, 1]$, they replace K_{ij} by $K_{ij}/(d_i^\alpha d_j^\alpha)$ and then compute new row sums and the row-stochastic matrix as before. In the limit of many points and small bandwidth, $\alpha = 0$ gives the classical normalized graph Laplacian, whose limit depends on the sampling density. The choice $\alpha = 1/2$ gives the generator of a Fokker–Planck diffusion, and $\alpha = 1$ recovers the Laplace–Beltrami operator independently of the density. The renormalized kernel is still symmetric, so Thm 8.7 applies with the new row sums.

8.2.4 Permutation Invariance and Graph Spectra

Remark 8.7 **(Combinatorial bottleneck in coordinate representations)** In molecular systems and particle clusters with N identical atoms, the state is specified by atomic positions in $\mathbb{R}^{3N}$. Comparing two configurations $\boldsymbol{x}, \boldsymbol{y} \in \mathbb{R}^{3N}$ directly requires optimizing over all $N!$ atomic index permutations $\sigma \in S_N$, in addition to Euclidean rotations and translations. This combinatorial alignment cost is intractable for moderate N.

Thm 8.8 **(Permutation Invariance of Graph Spectra)** Let $\mathbf{D} \in \mathbb{R}^{N\times N}$ be the pairwise interatomic distance matrix with entries $D_{ij} = \|\mathbf{r}_i - \mathbf{r}_j\|_2$. Under any atom permutation $\sigma \in S_N$, the distance matrix transforms by an orthogonal permutation matrix $\mathbf{P}_\sigma$:

$$\mathbf{D}_\sigma = \mathbf{P}_\sigma \mathbf{D} \mathbf{P}_\sigma^T.$$

Let $\mathbf{L}$ be any graph Laplacian constructed from edge weights $w_{ij} = f(D_{ij})$. Then the permuted Laplacian is

$$\mathbf{L}_\sigma = \mathbf{P}_\sigma \mathbf{L} \mathbf{P}_\sigma^T.$$

Because $\mathbf{P}_\sigma$ is orthogonal, $\mathbf{L}_\sigma$ is orthogonally similar to $\mathbf{L}$. Therefore, the eigenvalue spectrum

$$\mathrm{spec}(\mathbf{L}) = (\lambda_1, \lambda_2, \ldots, \lambda_N)$$

is invariant under all atomic permutations $\sigma \in S_N$ and all rigid Euclidean isometries SE(3).

Proof Because rigid rotations and translations preserve pairwise distances $\|\mathbf{r}_i - \mathbf{r}_j\|_2$, the matrix $\mathbf{D}$ is invariant under SE(3).
Under a permutation $\sigma \in S_N$, vertex i maps to $\sigma(i)$, so the permuted affinity matrix has entries $W_{\sigma(i),\sigma(j)} = W_{ij}$. In matrix notation, $\mathbf{W}_\sigma = \mathbf{P}_\sigma \mathbf{W} \mathbf{P}_\sigma^T$. Similarly, the degree matrix transforms as $\mathbf{D}_\sigma = \mathbf{P}_\sigma \mathbf{D} \mathbf{P}_\sigma^T$. Thus

$$\mathbf{L}_\sigma = \mathbf{D}_\sigma - \mathbf{W}_\sigma = \mathbf{P}_\sigma(\mathbf{D} - \mathbf{W})\mathbf{P}_\sigma^T = \mathbf{P}_\sigma \mathbf{L} \mathbf{P}_\sigma^T.$$

Because permutation matrices are orthogonal ($\mathbf{P}_\sigma^{-1} = \mathbf{P}_\sigma^T$), the characteristic polynomial satisfies

$$\det(\mathbf{L}_\sigma - \lambda \mathbf{I}) = \det(\mathbf{P}_\sigma(\mathbf{L} - \lambda \mathbf{I})\mathbf{P}_\sigma^T) = \det(\mathbf{P}_\sigma)\det(\mathbf{L} - \lambda \mathbf{I})\det(\mathbf{P}_\sigma^T) = \det(\mathbf{L} - \lambda \mathbf{I}).$$

The roots of the characteristic polynomial are identical. Thus the eigenvalue multiset $\mathrm{spec}(\mathbf{L})$ is invariant under any relabeling of identical particles. □

8.2.5 Bandwidth Selection via Affinity Scaling

Remark 8.8 **(Sigmoidal behavior of the affinity sum)** The Gaussian affinity kernel $K_{ij} = \exp(-\|\boldsymbol{x}_i - \boldsymbol{x}_j\|^2/(2\delta^2))$ requires selecting the bandwidth parameter δ. If δ is too small, points are isolated and $K_{ij} \approx 0$ for $i \neq j$. If δ is too large, points are fully connected and $K_{ij} \approx 1$. Define the total affinity sum

$$S(\delta) = \sum_{i=1}^{N}\sum_{j=1}^{N} \exp\left(-\frac{\|\boldsymbol{x}_i - \boldsymbol{x}_j\|^2}{2\delta^2}\right).$$

Plotting $\ln S(\delta)$ against $\ln \delta$ yields a sigmoidal curve:

1. As $\delta \to 0$, $K_{ij} \to 0$ for $i \neq j$, so $S(\delta) \to N$ and $\ln S(\delta) \to \ln N$.
2. As $\delta \to \infty$, $K_{ij} \to 1$ for all i, j, so $S(\delta) \to N^2$ and $\ln S(\delta) \to 2\ln N$.
3. In the intermediate scaling regime, $S(\delta) \propto \delta^d$, where d is the intrinsic manifold dimension. The slope $\frac{d\ln S(\delta)}{d\ln \delta} \approx d$ estimates dimension.

Choosing δ in the linear transition regime ensures that the kernel resolves intrinsic manifold geometry without collapsing into disconnected points or a uniform complete graph.

8.2.6 Exercises

Ex 8.5

1. **Compute. (Path graph Laplacian spectrum)** For a three-node unweighted path graph $1-2-3$, write down $\mathbf{A}$ and $\mathbf{L}$. Compute the eigenvalues and eigenvectors of $\mathbf{L}$ by hand. Verify that $\lambda_1 = 0$ with $\boldsymbol{v}_1 = (1,1,1)^T$, and compute the algebraic connectivity λ_2.
2. **Derive. (Rayleigh quotient for algebraic connectivity)** For the path graph in the preceding exercise, minimize the quotient in Cor 8.5 over vectors of the form $(a, b, -a-b)^T$. Verify that the minimum equals the computed value of λ_2. Then evaluate the numerator on a two-level vector that is constant on each side of a proposed cut and explain how weak cross-cut weights lower its energy.
3. **Derive. (Permutation invariance of graph spectra)** Consider a cluster of four identical atoms in $\mathbb{R}^3$.
   (a) Let the coordinates be arranged in a vector $\boldsymbol{x} \in \mathbb{R}^{12}$. Explain why comparing two configurations $\boldsymbol{x}$ and $\boldsymbol{y}$ requires testing $4! = 24$ permutations.
   (b) Construct the interatomic distance matrix $\mathbf{D}$ and form the graph Laplacian $\mathbf{L}$. Prove that for any permutation $\sigma \in S_4$, $\mathbf{L}_\sigma = \mathbf{P}_\sigma \mathbf{L} \mathbf{P}_\sigma^T$.
   (c) Explain how sorting the eigenvalues $\mathrm{spec}(\mathbf{L}) = (\lambda_1, \lambda_2, \lambda_3, \lambda_4)$ produces a coordinate-free, permutation-invariant fingerprint for molecular structure.

4. **Explore. (Diffusion time as a geometric low-pass filter)** Let a diffusion operator on a data manifold have leading eigenvalues $\lambda_1 = 0.98$ and $\lambda_2 = 0.80$.

   (a) Compute the ratio of modal contributions $(\lambda_2/\lambda_1)^{2t}$ for diffusion times $t = 1$, $t = 5$, and $t = 20$.

   (b) Explain why increasing t contracts local high-frequency fluctuations while preserving macroscopic manifold geometry.

   (c) What happens to the diffusion distance $D_t(\boldsymbol{x}_i, \boldsymbol{x}_j)$ as $t \to \infty$?

5. **Explore. (Sigmoidal bandwidth selection)** Let $N = 100$ data points be sampled uniformly from a circle of radius $R = 1$ in $\mathbb{R}^2$.

   (a) Write down the formula for the affinity sum $S(\delta) = \sum_{i,j} \exp(-\|\boldsymbol{x}_i - \boldsymbol{x}_j\|^2/(2\delta^2))$.

   (b) Evaluate the asymptotic limits of $\ln S(\delta)$ as $\delta \to 0$ and $\delta \to \infty$.

   (c) In the intermediate scaling regime $\delta \in [0.1, 0.5]$, show that the slope $\frac{d \ln S(\delta)}{d \ln \delta} \approx 1$, reflecting the one-dimensional geometry of the circle.

Read **Further reading.** Chung develops spectral graph theory from the graph-Laplacian side (Chung, 1997). Belkin and Niyogi introduce Laplacian eigenmaps, while Coifman and Lafon develop diffusion maps and the density normalization described above (Belkin and Niyogi, 2003; Coifman and Lafon, 2006).

8.3 Matrix Exponentials and Krylov Subspace Actions

(Running example: Markov chains) A probability vector governed by a finite-state generator satisfies $\boldsymbol{p}(t) = e^{t\mathbf{A}}\boldsymbol{p}_0$. The exponential action developed here advances that distribution without forming $e^{t\mathbf{A}}$. The next chapter adds positivity, conservation, and truncation structure.

Linear evolution equations describe continuous physical and probabilistic processes throughout scientific computing. Consider the autonomous initial-value problem

$$\frac{d\boldsymbol{u}}{dt} = \mathbf{A}\boldsymbol{u}(t), \qquad \boldsymbol{u}(0) = \boldsymbol{v}, \tag{2}$$

where $\mathbf{A} \in \mathbb{R}^{n\times n}$ or $\mathbb{C}^{n\times n}$ is an infinitesimal generator or spatial discretization matrix, and $\boldsymbol{v}$ is a prescribed initial state. The analytical solution is given by the matrix exponential action

$$\boldsymbol{u}(t) = e^{t\mathbf{A}}\boldsymbol{v}. \tag{3}$$

When n is large and $\mathbf{A}$ is sparse, computing the full matrix $e^{t\mathbf{A}}$ is computationally prohibitive. We evaluate the vector action $e^{t\mathbf{A}}\boldsymbol{v}$ directly through projection onto Krylov subspaces.

8.3.1 Definition, Properties, and the Sparse Bottleneck

Def 8.10 **(The Matrix Exponential)** For any square matrix $\mathbf{A} \in \mathbb{C}^{n\times n}$, the matrix exponential is defined by the convergent power series

$$e^{\mathbf{A}} = \sum_{k=0}^{\infty} \frac{\mathbf{A}^k}{k!} = \mathbf{I} + \mathbf{A} + \frac{1}{2}\mathbf{A}^2 + \frac{1}{6}\mathbf{A}^3 + \dots$$

For any scalar $t \in \mathbb{R}$, the matrix function $e^{t\mathbf{A}}$ satisfies $\frac{d}{dt}e^{t\mathbf{A}} = \mathbf{A}e^{t\mathbf{A}} = e^{t\mathbf{A}}\mathbf{A}$.

Thm 8.9 **(Algebraic Properties of the Matrix Exponential)** Let $\mathbf{A}, \mathbf{B} \in \mathbb{C}^{n\times n}$.

1. If $\mathbf{A}$ and $\mathbf{B}$ commute ($\mathbf{AB} = \mathbf{BA}$), then $e^{\mathbf{A}+\mathbf{B}} = e^{\mathbf{A}}e^{\mathbf{B}}$. If they do not commute, this equality fails in general.
2. The matrix exponential is always nonsingular, with inverse $(e^{\mathbf{A}})^{-1} = e^{-\mathbf{A}}$.
3. For any nonsingular similarity transform $\mathbf{S}$, $e^{\mathbf{S}\mathbf{A}\mathbf{S}^{-1}} = \mathbf{S}e^{\mathbf{A}}\mathbf{S}^{-1}$. In particular, if $\mathbf{A} = \mathbf{Q}\mathbf{\Lambda}\mathbf{Q}^{-1}$ is diagonalizable, then $e^{\mathbf{A}} = \mathbf{Q}\,\mathrm{diag}(e^{\lambda_1}, \dots, e^{\lambda_n})\mathbf{Q}^{-1}$.
4. The determinant satisfies Jacobi's formula: $\det(e^{\mathbf{A}}) = e^{\mathrm{trace}(\mathbf{A})}$.

Proof Property (1) follows by expanding the Cauchy product of the series for $e^{\mathbf{A}}$ and $e^{\mathbf{B}}$, where commutativity permits the binomial theorem for $(\mathbf{A}+\mathbf{B})^k$. Property (2) is a direct consequence of (1) with $\mathbf{B} = -\mathbf{A}$, since $\mathbf{A}$ commutes with $-\mathbf{A}$ and $e^{\mathbf{0}} = \mathbf{I}$. Property (3) follows from $(\mathbf{S}\mathbf{A}\mathbf{S}^{-1})^k = \mathbf{S}\mathbf{A}^k\mathbf{S}^{-1}$. Property (4) follows from the Schur decomposition $\mathbf{A} = \mathbf{Q}\mathbf{T}\mathbf{Q}^*$: the diagonal entries of $\mathbf{T}$ are the eigenvalues λ_i, the diagonal entries of $e^{\mathbf{T}}$ are e^{λ_i}, and the determinant of a triangular matrix is the product of its diagonal entries: $\det(e^{\mathbf{A}}) = \det(e^{\mathbf{T}}) = \prod_{i=1}^{n} e^{\lambda_i} = e^{\sum \lambda_i} = e^{\mathrm{trace}(\mathbf{A})}$. □

Remark 8.9 **(Dense evaluation vs. the vector action bottleneck)** Classical methods for computing the full $n \times n$ matrix $e^{\mathbf{A}}$ rely on scaling and squaring with rational Padé approximants. Evaluating an $[m/m]$ Padé approximant requires computing matrix powers and solving matrix linear systems, incurring $O(n^3)$ operations. More critically, even when $\mathbf{A}$ is sparse, the exponential $e^{t\mathbf{A}}$ is typically dense. Storing $e^{t\mathbf{A}}$ for $n = 10^6$ requires eight terabytes of memory in double precision. Many evolution calculations require only the action $e^{t\mathbf{A}}\boldsymbol{v}$, which can be approximated without forming the full exponential.

8.3.2 Krylov Subspace Approximation of the Exponential Action

To compute the action $e^{t\mathbf{A}}\boldsymbol{v}$ for large sparse $\mathbf{A}$, we project the evolution onto an expanding Krylov subspace $\mathcal{K}_m(\mathbf{A}, \boldsymbol{v}) = \mathrm{span}\{\boldsymbol{v}, \mathbf{A}\boldsymbol{v}, \dots, \mathbf{A}^{m-1}\boldsymbol{v}\}$.

Thm 8.10 **(Arnoldi Approximation of $e^{t\mathbf{A}}\boldsymbol{v}$)** Let $\mathbf{A} \in \mathbb{C}^{n\times n}$ and let $\boldsymbol{v} \in \mathbb{C}^n$ be a nonzero vector with $\beta = \|\boldsymbol{v}\|_2$. After m steps of the Arnoldi iteration starting from $\boldsymbol{v}_1 = \boldsymbol{v}/\beta$, we obtain the Arnoldi relation

$$\mathbf{A}\mathbf{V}_m = \mathbf{V}_m\mathbf{H}_m + h_{m+1,m}\boldsymbol{v}_{m+1}\boldsymbol{e}_m^T,$$

where $\mathbf{V}_m \in \mathbb{C}^{n\times m}$ has orthonormal columns and $\mathbf{H}_m = \mathbf{V}_m^* \mathbf{A}\mathbf{V}_m \in \mathbb{C}^{m\times m}$ is upper Hessenberg. The Arnoldi approximation to the exponential action is

$$e^{t\mathbf{A}}\boldsymbol{v} \approx \beta \mathbf{V}_m e^{t\mathbf{H}_m}\boldsymbol{e}_1.$$

Proof We approximate the solution of $\boldsymbol{u}'(t) = \mathbf{A}\boldsymbol{u}(t)$ with $\boldsymbol{u}(0) = \boldsymbol{v}$ within the subspace $\mathcal{K}_m(\mathbf{A}, \boldsymbol{v})$ by setting $\boldsymbol{u}_m(t) = \mathbf{V}_m \boldsymbol{z}_m(t)$, where $\boldsymbol{z}_m(t) \in \mathbb{C}^m$. The residual defect of the ODE is $\boldsymbol{r}_m(t) = \boldsymbol{u}_m'(t) - \mathbf{A}\boldsymbol{u}_m(t) = \mathbf{V}_m \boldsymbol{z}_m'(t) - \mathbf{A}\mathbf{V}_m \boldsymbol{z}_m(t)$. Imposing the Galerkin orthogonality condition $\mathbf{V}_m^* \boldsymbol{r}_m(t) = \mathbf{0}$ yields

$$\mathbf{V}_m^* \mathbf{V}_m \boldsymbol{z}_m'(t) - \mathbf{V}_m^* \mathbf{A}\mathbf{V}_m \boldsymbol{z}_m(t) = \mathbf{0} \implies \boldsymbol{z}_m'(t) = \mathbf{H}_m \boldsymbol{z}_m(t).$$

The initial condition $\boldsymbol{u}_m(0) = \boldsymbol{v} = \beta \boldsymbol{v}_1 = \beta \mathbf{V}_m \boldsymbol{e}_1$ requires $\boldsymbol{z}_m(0) = \beta \boldsymbol{e}_1$. Solving the reduced $m \times m$ linear system gives $\boldsymbol{z}_m(t) = \beta e^{t\mathbf{H}_m}\boldsymbol{e}_1$. Multiplying by $\mathbf{V}_m$ recovers $\boldsymbol{u}_m(t) = \beta \mathbf{V}_m e^{t\mathbf{H}_m}\boldsymbol{e}_1$. □

Thm 8.11 **(Residual Representation and A Posteriori Error Bound)** The defect in the differential equation satisfies

$$\boldsymbol{u}_m'(t) - \mathbf{A}\boldsymbol{u}_m(t) = -\beta h_{m+1,m} [\boldsymbol{e}_m^T e^{t\mathbf{H}_m}\boldsymbol{e}_1] \boldsymbol{v}_{m+1}.$$

Integrating the defect via variation of constants yields the exact error identity

$$e^{t\mathbf{A}}\boldsymbol{v} - \beta \mathbf{V}_m e^{t\mathbf{H}_m}\boldsymbol{e}_1 = \beta h_{m+1,m} \int_0^t e^{(t-\tau)\mathbf{A}} \boldsymbol{v}_{m+1} [\boldsymbol{e}_m^T e^{\tau \mathbf{H}_m}\boldsymbol{e}_1]\, d\tau.$$

If the operator $\mathbf{A}$ generates a contractive semigroup satisfying $\|e^{\tau\mathbf{A}}\| \le 1$ for all $\tau \ge 0$, the error is bounded by

$$\|e^{t\mathbf{A}}\boldsymbol{v} - \beta \mathbf{V}_m e^{t\mathbf{H}_m}\boldsymbol{e}_1\| \le \beta |h_{m+1,m}| \int_0^t |\boldsymbol{e}_m^T e^{\tau\mathbf{H}_m}\boldsymbol{e}_1|\, d\tau.$$

Proof Using the Arnoldi relation $\mathbf{A}\mathbf{V}_m = \mathbf{V}_m \mathbf{H}_m + h_{m+1,m}\boldsymbol{v}_{m+1}\boldsymbol{e}_m^T$, the ODE defect is

$$\begin{aligned}
\boldsymbol{u}_m'(t) - \mathbf{A}\boldsymbol{u}_m(t) &= \mathbf{V}_m \boldsymbol{z}_m'(t) - \mathbf{A}\mathbf{V}_m \boldsymbol{z}_m(t) \\
&= \mathbf{V}_m \mathbf{H}_m \boldsymbol{z}_m(t) - \left(\mathbf{V}_m \mathbf{H}_m + h_{m+1,m}\boldsymbol{v}_{m+1}\boldsymbol{e}_m^T\right) \boldsymbol{z}_m(t) \\
&= -h_{m+1,m}\boldsymbol{v}_{m+1}\boldsymbol{e}_m^T \boldsymbol{z}_m(t) \\
&= -\beta h_{m+1,m} [\boldsymbol{e}_m^T e^{t\mathbf{H}_m}\boldsymbol{e}_1] \boldsymbol{v}_{m+1}.
\end{aligned}$$

The approximation error $\boldsymbol{e}(t) = \boldsymbol{u}(t) - \boldsymbol{u}_m(t)$ satisfies $\boldsymbol{e}'(t) = \mathbf{A}\boldsymbol{e}(t) - (\boldsymbol{u}_m'(t) - \mathbf{A}\boldsymbol{u}_m(t))$ with $\boldsymbol{e}(0) = \mathbf{0}$. Integrating via variation of constants:

$$\boldsymbol{e}(t) = \int_0^t e^{(t-\tau)\mathbf{A}} \left[\mathbf{A}\boldsymbol{u}_m(\tau) - \boldsymbol{u}_m'(\tau)\right] d\tau = \beta h_{m+1,m} \int_0^t e^{(t-\tau)\mathbf{A}} \boldsymbol{v}_{m+1} [\boldsymbol{e}_m^T e^{\tau\mathbf{H}_m}\boldsymbol{e}_1]\, d\tau.$$

Taking norms with $\|e^{(t-\tau)\mathbf{A}}\| \le 1$ and $\|\boldsymbol{v}_{m+1}\|_2 = 1$ yields

$$\|\boldsymbol{e}(t)\| \le \beta|h_{m+1,m}| \int_0^t \|e^{(t-\tau)\mathbf{A}}\|_2 \|\boldsymbol{v}_{m+1}\|_2 |\boldsymbol{e}_m^T e^{\tau \mathbf{H}_m} \boldsymbol{e}_1|\, d\tau \le \beta|h_{m+1,m}| \int_0^t |\boldsymbol{e}_m^T e^{\tau \mathbf{H}_m} \boldsymbol{e}_1|\, d\tau.$$

□

Remark 8.10 **(Superlinear convergence of Krylov actions)** Hochbruck and Lubich bound the error of the Krylov approximation in terms of the spectrum and the time step (Hochbruck and Lubich, 1997). For a symmetric negative semidefinite matrix with eigenvalues in $[-4\rho, 0]$, the error in approximating $e^{\tau \mathbf{A}}\boldsymbol{v}$ from $\mathcal{K}_m(\mathbf{A}, \boldsymbol{v})$ is bounded by a constant multiple of $e^{-m^2/(5\rho\tau)}$ when $\sqrt{4\rho\tau} \le m \le 2\rho\tau$. For $m \ge 2\rho\tau$ the bound decays superlinearly in m. The relevant scale is therefore $\rho\tau$, not $\|\mathbf{A}\|_2$ alone, and in the first regime the required dimension grows only like the square root of $\rho\tau$. The same paper gives analogous bounds for non-Hermitian matrices whose numerical range lies in a known region of the left half-plane. The projected approximation also adapts to the spectrum seen from the starting vector rather than to the full matrix norm alone. Grid-independent convergence requires additional spectral or semigroup assumptions and should be checked for the discretization at hand.

8.3.3 The Continuous-Adjoint Sensitivity Formula

In parameter inference, we estimate unknown coefficients in an evolution model from discrete observations. Consider an affine generator family

$$\mathbf{A}(\boldsymbol{\theta}) = \mathbf{A}_0 + \sum_{j=1}^{M_{\text{lib}}} \theta_j \mathbf{A}_j, \tag{4}$$

where $\mathbf{A}_0$ is a baseline operator, the matrices $\mathbf{A}_j$ represent candidate physical mechanisms, and $\boldsymbol{\theta} = (\theta_1, \dots, \theta_{M_{\text{lib}}})^T$ is a vector of unknown rate parameters.

Thm 8.12 **(Adjoint Gradient Formula for Exponential Actions)** Let $\boldsymbol{y} = e^{h\mathbf{A}(\boldsymbol{\theta})}\boldsymbol{v}$ denote the state propagated across an interval of length h. Let $\ell(\boldsymbol{y})$ be a differentiable scalar loss with endpoint gradient $\nabla_{\boldsymbol{y}}\ell$. Then the derivative of the loss with respect to coefficient θ_j is

$$\frac{\partial \ell}{\partial \theta_j} = h\,\mathrm{Re} \int_0^1 \boldsymbol{\lambda}(s)^* \mathbf{A}_j \boldsymbol{f}(s)\, ds,$$

where the forward path $\boldsymbol{f}(s)$ and adjoint path $\boldsymbol{\lambda}(s)$ are defined by

$$\boldsymbol{f}(s) = e^{sh\mathbf{A}}\boldsymbol{v}, \qquad \boldsymbol{\lambda}(s) = e^{(1-s)h\mathbf{A}^*}\nabla_{\boldsymbol{y}}\ell, \qquad 0 \le s \le 1.$$

Proof Let $\boldsymbol{f}(s)$ satisfy the ODE $\boldsymbol{f}'(s) = h\mathbf{A}(\boldsymbol{\theta})\boldsymbol{f}(s)$ with $\boldsymbol{f}(0) = \boldsymbol{v}$ and $\boldsymbol{y} = \boldsymbol{f}(1)$. Differentiating this initial-value problem with respect to θ_j yields the tangent equation

$$\frac{d}{ds}\left(\frac{\partial \boldsymbol{f}}{\partial \theta_j}\right) = h\mathbf{A}\left(\frac{\partial \boldsymbol{f}}{\partial \theta_j}\right) + h\mathbf{A}_j \boldsymbol{f}(s), \qquad \frac{\partial \boldsymbol{f}(0)}{\partial \theta_j} = \mathbf{0}.$$

Applying variation of constants at the endpoint $s = 1$ gives

$$\frac{\partial \boldsymbol{y}}{\partial \theta_j} = h \int_0^1 e^{(1-s)h\mathbf{A}} \mathbf{A}_j \boldsymbol{f}(s)\, ds.$$

Applying the chain rule to the scalar loss $\ell(\boldsymbol{y})$ and taking the real part produces

$$\begin{aligned}\frac{\partial \ell}{\partial \theta_j} &= \mathrm{Re}\left((\nabla_{\boldsymbol{y}} \ell)^* \frac{\partial \boldsymbol{y}}{\partial \theta_j} \right) = h\,\mathrm{Re} \int_0^1 (\nabla_{\boldsymbol{y}} \ell)^* e^{(1-s)h\mathbf{A}} \mathbf{A}_j \boldsymbol{f}(s)\, ds \\ &= h\,\mathrm{Re} \int_0^1 \left(e^{(1-s)h\mathbf{A}^*} \nabla_{\boldsymbol{y}} \ell \right)^* \mathbf{A}_j \boldsymbol{f}(s)\, ds = h\,\mathrm{Re} \int_0^1 \boldsymbol{\lambda}(s)^* \mathbf{A}_j \boldsymbol{f}(s)\, ds.\end{aligned}$$

□

Remark 8.11 **(The adjoint sharing principle)** Evaluating a separate directional Fréchet derivative $L_{\exp}(h\mathbf{A}, h\mathbf{A}_j)$ for each candidate matrix $\mathbf{A}_j$ repeats the forward exponential propagation M_{lib} times. The adjoint identity separates propagation from the candidate directions. We compute the forward path $\boldsymbol{f}$ and adjoint path $\boldsymbol{\lambda}$ once per observation interval, and evaluate their contractions against $\mathbf{A}_j$ via numerical quadrature.

8.3.4 Shared Krylov Spaces and Quadrature

To evaluate the bilinear integral numerically, we approximate both paths using independent forward and adjoint Krylov spaces, and integrate the contraction using Gauss–Legendre quadrature.

Thm 8.13 **(Bilinear Quadrature Evaluation)** Let $\mathcal{K}_{m_f}(\mathbf{A}, \boldsymbol{v})$ have basis $\mathbf{V}_f$ and projected Hessenberg matrix $\mathbf{H}_f$, with $\beta_f = \|\boldsymbol{v}\|_2$. Let $\mathcal{K}_{m_a}(\mathbf{A}^*, \nabla_{\boldsymbol{y}} \ell)$ have basis $\mathbf{V}_a$ and projected matrix $\mathbf{H}_a$, with $\beta_a = \|\nabla_{\boldsymbol{y}} \ell\|_2$. At any Gauss–Legendre node $s_q \in [0, 1]$ with quadrature weight w_q, the path approximations are

$$\boldsymbol{f}^{(q)} = \beta_f \mathbf{V}_f e^{s_q h \mathbf{H}_f} \boldsymbol{e}_1, \qquad \boldsymbol{\lambda}^{(q)} = \beta_a \mathbf{V}_a e^{(1-s_q)h\mathbf{H}_a} \boldsymbol{e}_1.$$

The coefficient gradient evaluates to

$$\frac{\partial \ell}{\partial \theta_j} \approx h \sum_{q=1}^{n_q} w_q\, \mathrm{Re}\left[(\boldsymbol{\lambda}^{(q)})^* \mathbf{A}_j \boldsymbol{f}^{(q)} \right].$$

Remark 8.12 **(One-sided termination in refinement heuristics)** A common stopping diagnostic tests the product of forward and adjoint residuals $\rho_f \rho_a$. If the forward path reaches an invariant subspace, $\rho_f = 0$, causing the product to vanish even when the adjoint path is at depth $m_a = 2$ and completely unresolved. Reliable stopping criteria require monitoring forward and adjoint convergence separately.

8.3.5 Generator Recovery via the Matrix Logarithm

When discrete-time snapshot maps $\mathbf{K}_h \approx e^{h\mathcal{A}}$ are observed rather than continuous trajectories, we invert the exponential map using the principal matrix logarithm.

Def 8.11 **(The Matrix Logarithm)** Let $\mathbf{K} \in \mathbb{C}^{n\times n}$ have no eigenvalues on the closed negative real axis $(-\infty, 0]$. The principal matrix logarithm $\log(\mathbf{K})$ is the unique matrix $\mathbf{A}$ such that $e^{\mathbf{A}} = \mathbf{K}$ and all eigenvalues of $\mathbf{A}$ satisfy $|\operatorname{Im}(\lambda)| < \pi$.

Remark 8.13 **(Projection and finite-time closure bias)** If P denotes the orthogonal projection onto a subspace of observables $\mathcal{F}$, the projected generator obtained from finite-time observations satisfies

$$\widehat{\mathbf{A}}_h = h^{-1} \log\left(P e^{h\mathcal{A}}|_{\mathcal{F}}\right) \neq P\mathcal{A}|_{\mathcal{F}}.$$

Unless $\mathcal{F}$ is invariant under $\mathcal{A}$, projection and exponentiation do not commute. The resulting finite-time closure bias can be expanded in powers of h, but its leading order depends on which terms leave the observable space. Any rate such as $O(h^r)$ therefore requires a separately defined closure depth and corresponding assumptions.

8.3.6 Exercises

Ex 8.6

1. **Compute. Dense vs. Action Complexity:** A sparse matrix $\mathbf{A} \in \mathbb{R}^{10^6\times 10^6}$ has 10 nonzero entries per row. Compute the memory required to store $e^{h\mathbf{A}}$ as a dense array. Compare this with storing $m = 30$ Arnoldi basis vectors.

2. **Compute. Adjoint Sharing:** Let an overcomplete library contain $M_{\text{lib}} = 200$ candidate directions. Calculate the number of sparse matrix–vector products needed to evaluate all 200 coefficient gradients using (a) directional Fréchet derivatives and (b) the adjoint Krylov formula with 5 quadrature nodes.

3. **Check. Logarithm Branch Cuts:** Let an evolution operator have oscillatory eigenvalue $\lambda = -\gamma + i\omega$ with $\omega = 50\pi$. Determine the maximum observation interval h that avoids branch-cut crossing in the principal matrix logarithm.

4. **Explore. Taylor vs. Krylov Convergence:** For the 1D discrete Laplacian $\mathbf{A}$ with $\|\mathbf{A}\|_2 = 4 \times 10^4$ and $t = 10^{-2}$, compare the polynomial degrees suggested by a direct Taylor remainder bound and by Krylov projection. State every estimate you use, since neither degree is determined by $\|\mathbf{A}\|_2$ alone.

5. **Explore. One-Sided Termination:** Construct a 3×3 example where $\boldsymbol{v}$ is an exact eigenvector of $\mathbf{A}$, yielding forward residual $\rho_f = 0$ at step 1, while the adjoint starting vector is not an eigenvector. Explain why product heuristics fail in this setting.

Read **Further reading.** Higham gives the general theory of matrix functions (Higham, 2008). Moler and Van Loan explain why apparently reasonable methods for the matrix exponential can fail, and Al-Mohy and Higham develop a modern action method that avoids forming the exponential (Moler and Van Loan, 2003; Al-Mohy and Higham, 2011).

8.4 Markov Chains, Continuous Generators, and the CME

(Running example: Markov chains) In this setting the entries of the exponential action are transition probabilities and its column sums preserve total mass. Consequently, the probability mass that leaks out of a truncated state space is an observable measure of the truncation error.

Markov chains model random systems where transitions depend only on the current state. Discrete-time systems evolve via column-stochastic matrices $\boldsymbol{x}^{(k+1)} = \mathbf{P}\boldsymbol{x}^{(k)}$. Continuous-time Markov jump processes evolve via infinitesimal generators $\dot{\boldsymbol{p}}(t) = \mathbf{A}\boldsymbol{p}(t)$, leading to the Chemical Master Equation. Large or infinite state spaces require projection onto finite subgenerators with rigorous 1-norm leakage bounds.

8.4.1 Discrete-Time Markov Chains

Def 8.12 **(Probability Vector)** A vector $\boldsymbol{x} \in \mathbb{R}^n$ is a probability vector if

$$x_i \geq 0, \qquad \sum_{i=1}^{n} x_i = 1.$$

The entry x_i represents the probability of occupying state i.

Def 8.13 **(Column-Stochastic Matrix)** A matrix $\mathbf{P} \in \mathbb{R}^{n\times n}$ is column-stochastic if:

1. $P_{ij} \geq 0$ for all i, j.
2. $\sum_{i=1}^{n} P_{ij} = 1$ for each column j.

Remark 8.14 **(Column convention)** Column j of $\mathbf{P}$ contains the transition probabilities from state j to all destination states i. The update rule for a probability distribution is $\boldsymbol{x}^{(k+1)} = \mathbf{P}\boldsymbol{x}^{(k)}$.

Thm 8.14 **(Spectrum of a Stochastic Matrix)** If $\mathbf{P}$ is column-stochastic, then $\lambda = 1$ is an eigenvalue, and every eigenvalue satisfies $|\lambda| \leq 1$.

Proof The column sum condition implies $\mathbf{1}^T\mathbf{P} = \mathbf{1}^T$. Thus $\lambda = 1$ is a left eigenvalue with eigenvector $\mathbf{1}^T$, which guarantees that 1 is an eigenvalue of $\mathbf{P}$.
To bound the spectral radius, let $\mathbf{P}^T\boldsymbol{v} = \lambda\boldsymbol{v}$ with $\boldsymbol{v} \neq \mathbf{0}$. Choose index j such that $|v_j| = \|\boldsymbol{v}\|_\infty = \max_i |v_i|$. Taking the j-th component of the eigenvalue equation yields

$$|\lambda|\,|v_j| = \left|\sum_{i=1}^{n} P_{ij}v_i\right| \leq \sum_{i=1}^{n} P_{ij}|v_i| \leq \left(\sum_{i=1}^{n} P_{ij}\right)|v_j| = |v_j|.$$

Dividing by $|v_j| > 0$ gives $|\lambda| \leq 1$. $\square$

Def 8.14 **(Stationary Distribution)** A probability vector $\boldsymbol{\pi}$ is stationary if $\mathbf{P}\boldsymbol{\pi} = \boldsymbol{\pi}$. It is a right eigenvector of $\mathbf{P}$ associated with $\lambda = 1$, normalized so that $\mathbf{1}^T\boldsymbol{\pi} = 1$.

Thm 8.15 **(Perron–Frobenius for Markov Chains)** If $\mathbf{P}$ is irreducible (every state can be reached from any other state in finitely many steps) and primitive (aperiodic, so $\mathbf{P}^k > 0$ elementwise for some $k \geq 1$), then:

1. $\lambda_1 = 1$ is an algebraically and geometrically simple eigenvalue.
2. $|\lambda_i| < 1$ for all other eigenvalues $\lambda_i \neq 1$.
3. The stationary distribution $\boldsymbol{\pi}$ is strictly positive in every entry.
4. For any initial probability vector $\boldsymbol{x}^{(0)}$, $\lim_{k\to\infty} \mathbf{P}^k\boldsymbol{x}^{(0)} = \boldsymbol{\pi}$.

Def 8.15 **(Spectral Gap and Mixing Time)** For a primitive chain with eigenvalues ordered as $1 = \lambda_1 > |\lambda_2| \geq \cdots \geq |\lambda_n|$, the spectral gap is

$$\delta = 1 - |\lambda_2|.$$

For a diagonalizable matrix, the error $\|\boldsymbol{x}^{(k)} - \boldsymbol{\pi}\|_2$ decays asymptotically as $\mathcal{O}(|\lambda_2|^k)$. The number of steps k_ε required to reach error tolerance ε satisfies

$$k_\varepsilon \approx \frac{\log(1/\varepsilon)}{\delta}.$$

Def 8.16 **(Detailed Balance (Reversibility))** A Markov chain satisfies detailed balance with respect to $\boldsymbol{\pi}$ if

$$P_{ij}\pi_j = P_{ji}\pi_i \qquad \text{for all } i, j.$$

A chain satisfying detailed balance is reversible.

Thm 8.16 **(Real Spectrum of Reversible Chains)** Let $\mathbf{P}$ satisfy detailed balance with stationary distribution $\boldsymbol{\pi} > 0$. Define $\mathbf{D}_{\boldsymbol{\pi}} = \operatorname{diag}(\boldsymbol{\pi})$. Then the matrix

$$\mathbf{S} = \mathbf{D}_{\boldsymbol{\pi}}^{-1/2}\mathbf{P}\mathbf{D}_{\boldsymbol{\pi}}^{1/2}$$

is symmetric. Consequently, all eigenvalues of $\mathbf{P}$ are real, and $\mathbf{P}$ has an orthogonal basis of eigenvectors in the inner product weighted by $\mathbf{D}_{\boldsymbol{\pi}}^{-1}$.

Proof The (i, j) entry of $\mathbf{S}$ is

$$S_{ij} = \pi_i^{-1/2} P_{ij} \pi_j^{1/2} = \frac{P_{ij}\pi_j}{\sqrt{\pi_i \pi_j}}.$$

By detailed balance, $P_{ij}\pi_j = P_{ji}\pi_i$. Substituting this equality gives

$$S_{ij} = \frac{P_{ji}\pi_i}{\sqrt{\pi_i \pi_j}} = \pi_j^{-1/2} P_{ji} \pi_i^{1/2} = S_{ji}.$$

Thus $\mathbf{S} = \mathbf{S}^T$. Because $\mathbf{P}$ is similar to the real symmetric matrix $\mathbf{S}$, its eigenvalues are real. □

8.4.2 Continuous-Time Markov Chains and the CME

Def 8.17 **(Continuous-Time Infinitesimal Generator)** A continuous-time Markov jump process on a discrete state space evolves according to

$$\frac{d}{dt}\boldsymbol{p}(t) = \mathbf{A}\boldsymbol{p}(t),$$

where $\boldsymbol{p}(t)$ is the probability distribution vector and $\mathbf{A}$ is an infinitesimal generator matrix satisfying:

1. $A_{ij} \geq 0$ for all $i \neq j$ (transition rates between distinct states are nonnegative).
2. $\sum_i A_{ij} = 0$ for every column j (column sums vanish).
3. $A_{jj} = -\sum_{i \neq j} A_{ij} \leq 0$ (diagonal entries equal the negative total exit rate from state j).

Def 8.18 **(Chemical Master Equation (CME))** Consider a chemically reacting system with S molecular species and R reaction channels. The discrete state $\boldsymbol{x} \in \mathbb{N}_0^S$ records the copy number of each species. Each reaction channel $k \in \{1, \ldots, R\}$ is characterized by a stoichiometric change vector $\boldsymbol{\nu}_k \in \mathbb{Z}^S$ and a propensity function $\alpha_k(\boldsymbol{x}; \boldsymbol{c}) \geq 0$ parameterized by rate constants $\boldsymbol{c}$.
The Chemical Master Equation governs the probability $p(\boldsymbol{x}, t)$ of occupying state $\boldsymbol{x}$ at time t:

$$\frac{\partial p(\boldsymbol{x}, t)}{\partial t} = \sum_{k=1}^{R} \Big[\alpha_k(\boldsymbol{x} - \boldsymbol{\nu}_k; \boldsymbol{c})\, p(\boldsymbol{x} - \boldsymbol{\nu}_k, t) - \alpha_k(\boldsymbol{x}; \boldsymbol{c})\, p(\boldsymbol{x}, t) \Big].$$

Enumerating the reachable states yields the linear system of ordinary differential equations

$$\dot{\boldsymbol{p}}(t) = \mathbf{A}(\boldsymbol{c})\boldsymbol{p}(t), \qquad \boldsymbol{p}(0) = \boldsymbol{p}_0,$$

whose exact formal solution is the matrix exponential action $\boldsymbol{p}(t) = \exp(t\mathbf{A}(\boldsymbol{c}))\boldsymbol{p}_0$.

8.4.3 Assembling the Generator and Simulating Trajectories

Two computations start from the list of reactions. The first builds the generator $\mathbf{A}(\boldsymbol{c})$ on a finite set of states and evolves the whole distribution. The second samples individual trajectories of the jump process and never enumerates the state space. The generator is needed for the finite state projection below. Sampled trajectories provide the data for the Koopman estimators of Section 8.5, and their histograms give an independent check on computed distributions.

Def 8.19 **(Generator Assembly by Forward Enumeration)** Let $\mathcal{S}$ be a finite set of n states, stored in a hash table that maps each state $\boldsymbol{x}$ to its index $\iota(\boldsymbol{x}) \in \{1, \dots, n\}$. For each $\boldsymbol{x} \in \mathcal{S}$ and each reaction k with $\alpha_k(\boldsymbol{x}; \boldsymbol{c}) > 0$:

1. Compute the destination $\boldsymbol{y} = \boldsymbol{x} + \boldsymbol{\nu}_k$.
2. If $\boldsymbol{y} \in \mathcal{S}$, append the triplet $\big(\iota(\boldsymbol{y}), \iota(\boldsymbol{x}), \alpha_k(\boldsymbol{x}; \boldsymbol{c})\big)$.

Finally, append the diagonal triplet $\big(\iota(\boldsymbol{x}), \iota(\boldsymbol{x}), -\sum_k \alpha_k(\boldsymbol{x}; \boldsymbol{c})\big)$ for each $\boldsymbol{x}$, and convert the triplets to CSR format by Thm 1.26.

The diagonal entry contains the total exit rate, including the rates of reactions whose destination lies outside $\mathcal{S}$. Column $\iota(\boldsymbol{x})$ therefore sums to minus the rate at which probability leaves $\mathcal{S}$ from $\boldsymbol{x}$, which is the leakage measured by the finite state projection bound. Assembly visits each pair of a state and a reaction once and performs one expected constant-time hash lookup for each, so it costs $O(nR)$ operations and produces at most $n(R+1)$ nonzero entries. Searching all pairs of states for possible transitions would cost $O(n^2)$ instead.

Def 8.20 **(Stochastic Simulation Algorithm)** Let the process be in state $\boldsymbol{x}$ at time t. Gillespie's direct method advances it as follows (Gillespie, 1977).

1. Compute the propensities $\alpha_k(\boldsymbol{x}; \boldsymbol{c})$ and their sum $a_0(\boldsymbol{x}) = \sum_{k=1}^{R} \alpha_k(\boldsymbol{x}; \boldsymbol{c})$. If $a_0(\boldsymbol{x}) = 0$, stop.
2. Draw independent u_1, u_2 uniformly from $(0, 1)$.
3. Set $\tau = -\ln(u_1)/a_0(\boldsymbol{x})$, and choose the smallest k with $\sum_{l=1}^{k} \alpha_l(\boldsymbol{x}; \boldsymbol{c}) \geq u_2\, a_0(\boldsymbol{x})$.
4. Replace t by $t + \tau$ and $\boldsymbol{x}$ by $\boldsymbol{x} + \boldsymbol{\nu}_k$.

Thm 8.17 **(Exactness of the Stochastic Simulation Algorithm)** Suppose that from state $\boldsymbol{x}$ each reaction k fires after an independent exponential waiting time with rate $\alpha_k(\boldsymbol{x}; \boldsymbol{c})$, and that the first reaction to fire determines the next state. Then the time to the next reaction is exponential with rate $a_0(\boldsymbol{x})$, the next reaction is k with probability $\alpha_k(\boldsymbol{x}; \boldsymbol{c})/a_0(\boldsymbol{x})$, and these two events are independent. The waiting time and reaction index produced by Def 8.20 have exactly this joint distribution.

Proof Write α_k for $\alpha_k(\boldsymbol{x};\boldsymbol{c})$ and a_0 for $a_0(\boldsymbol{x})$. The probability that reaction k fires first and after time τ is

$$\int_\tau^\infty \alpha_k e^{-\alpha_k s}\prod_{l\neq k} e^{-\alpha_l s}\,ds = \int_\tau^\infty \alpha_k e^{-a_0 s}\,ds = \frac{\alpha_k}{a_0}\,e^{-a_0\tau}.$$

The probability factors into a function of k and a function of τ, which proves the independence and both marginal distributions. In the algorithm, $\Pr(-\ln(u_1)/a_0 > \tau) = \Pr(u_1 < e^{-a_0\tau}) = e^{-a_0\tau}$, so τ is exponential with rate a_0. The reaction index equals k exactly when $u_2 a_0$ falls in an interval of length α_k inside $[0, a_0]$, which has probability α_k/a_0. The two draws are independent. □

Each step costs $O(R)$ operations with a linear search for k, or $O(\log R)$ when the partial sums of the propensities are stored in a binary tree. The number of steps up to time T is about $\int_0^T a_0(\mathbf{X}(t))\,dt$, which is large when some reactions are fast. An expected value $\mathbb{E}[f(\mathbf{X}(T))]$ is estimated by averaging f over N independent trajectories. By the central limit theorem, the error of this average has standard deviation $\sigma_f/\sqrt{N}$, where σ_f is the standard deviation of $f(\mathbf{X}(T))$. Each additional digit of accuracy therefore requires a hundred times more trajectories. The finite state projection below computes the distribution instead, at a cost that grows with the number of retained states rather than with the accuracy.

8.4.4 Finite State Projection (FSP)

Remark 8.15 **(Infinite state bottleneck)** The CME generator $\mathbf{A}(\boldsymbol{c})$ is defined over an infinite or very large discrete state space $\mathbb{N}_0^S$. A finite computation therefore requires truncation, aggregation, or another representation that avoids enumerating the full state space.

Def 8.21 **(Finite State Projection (FSP))** Let the discrete state space be partitioned into a finite retained index set $J \subset \mathbb{N}_0^S$ with $|J| = K$, and an infinite or omitted complement set J^c. Permuting states so that J appears first partitions the full generator:

$$\mathbf{A} = \begin{pmatrix} \mathbf{A}_J & \mathbf{A}_{J,J^c} \\ \mathbf{A}_{J^c,J} & \mathbf{A}_{J^c} \end{pmatrix}.$$

The Finite State Projection of Munsky and Khammash (Munsky and Khammash, 2006) approximates the true marginal probability on J by solving the isolated subgenerator system

$$\dot{\boldsymbol{p}}_J(t) = \mathbf{A}_J \boldsymbol{p}_J(t), \qquad \boldsymbol{p}_J(0) = \boldsymbol{p}_0|_J.$$

Here $\mathbf{A}_J$ has negative diagonal entries and nonnegative off-diagonal entries, but its column sums are strictly nonpositive because transitions into J^c act as probability leaks.

Thm 8.18 **(FSP 1-Norm Leakage Bound)** Let $\boldsymbol{p}(t)$ be the exact probability distribution solving $\dot{\boldsymbol{p}} = \mathbf{A}\boldsymbol{p}$ with initial condition supported on J, so that $\mathbf{1}^T\boldsymbol{p}(t) = 1$. Let $\boldsymbol{p}_J(t) = \exp(t\mathbf{A}_J)\boldsymbol{p}_0|_J$ be the FSP solution. Then:

1. Pointwise underestimation: $\mathbf{0} \leq \boldsymbol{p}_J(t) \leq \boldsymbol{p}|_J(t)$ for all $t \geq 0$.
2. The full-space error of the zero-padded FSP approximation equals the leaked mass:

$$\|\boldsymbol{p}(t) - \widetilde{\boldsymbol{p}}_J(t)\|_1 = 1 - \mathbf{1}^T \boldsymbol{p}_J(t),$$

where $\widetilde{\boldsymbol{p}}_J(t)$ is $\boldsymbol{p}_J(t)$ padded with zeros on J^c. The retained-state error satisfies

$$\|\boldsymbol{p}|_J(t) - \boldsymbol{p}_J(t)\|_1 = \mathbf{1}^T \boldsymbol{p}|_J(t) - \mathbf{1}^T \boldsymbol{p}_J(t) \leq 1 - \mathbf{1}^T \boldsymbol{p}_J(t).$$

Proof The exact retained dynamics satisfy the non-autonomous subgenerator ODE

$$\frac{d}{dt}\boldsymbol{p}|_J(t) = \mathbf{A}_J \boldsymbol{p}|_J(t) + \mathbf{A}_{J,J^c} \boldsymbol{p}|_{J^c}(t), \qquad \boldsymbol{p}|_J(0) = \boldsymbol{p}_0|_J.$$

The FSP approximation satisfies $\frac{d}{dt}\boldsymbol{p}_J(t) = \mathbf{A}_J \boldsymbol{p}_J(t)$ with the same initial condition $\boldsymbol{p}_J(0) = \boldsymbol{p}_0|_J$. Subtracting the two equations gives the defect ODE for the difference $\boldsymbol{\delta}_J(t) = \boldsymbol{p}|_J(t) - \boldsymbol{p}_J(t)$:

$$\frac{d}{dt}\boldsymbol{\delta}_J(t) = \mathbf{A}_J \boldsymbol{\delta}_J(t) + \mathbf{A}_{J,J^c} \boldsymbol{p}|_{J^c}(t), \qquad \boldsymbol{\delta}_J(0) = \mathbf{0}.$$

By Duhamel's formula (variation of constants):

$$\boldsymbol{\delta}_J(t) = \boldsymbol{p}|_J(t) - \boldsymbol{p}_J(t) = \int_0^t e^{(t-s)\mathbf{A}_J} \mathbf{A}_{J,J^c} \boldsymbol{p}|_{J^c}(s)\, ds.$$

Because $\mathbf{A}_J$ has non-negative off-diagonal entries, it is a Metzler matrix, and its matrix exponential $e^{\tau \mathbf{A}_J}$ is entrywise non-negative for all $\tau \geq 0$. Furthermore, the coupling matrix $\mathbf{A}_{J,J^c} \geq \mathbf{0}$ and exact probabilities $\boldsymbol{p}|_{J^c}(s) \geq \mathbf{0}$ are non-negative. Consequently, the integrand is entrywise non-negative, proving that

$$\boldsymbol{p}|_J(t) - \boldsymbol{p}_J(t) \geq \mathbf{0} \implies \mathbf{0} \leq \boldsymbol{p}_J(t) \leq \boldsymbol{p}|_J(t) \quad \text{for all } t \geq 0.$$

Because $\boldsymbol{p}_J(t) \leq \boldsymbol{p}|_J(t)$, the marginal 1-norm difference on J simplifies without absolute value signs:

$$\|\boldsymbol{p}|_J(t) - \boldsymbol{p}_J(t)\|_1 = \sum_{i \in J} \left(p_i(t) - [\boldsymbol{p}_J(t)]_i\right) = \mathbf{1}^T \boldsymbol{p}|_J(t) - \mathbf{1}^T \boldsymbol{p}_J(t).$$

The total probability across the full space satisfies $\mathbf{1}^T \boldsymbol{p}|_J(t) + \mathbf{1}^T \boldsymbol{p}|_{J^c}(t) = 1$. The omitted probability mass on J^c is exactly $1 - \mathbf{1}^T \boldsymbol{p}|_J(t)$. Adding the unmodeled mass on J^c gives

$$\begin{aligned}
\|\boldsymbol{p}(t) - \widetilde{\boldsymbol{p}}_J(t)\|_1 &= \|\boldsymbol{p}|_J(t) - \boldsymbol{p}_J(t)\|_1 + \|\boldsymbol{p}|_{J^c}(t)\|_1 \\
&= \left(\mathbf{1}^T \boldsymbol{p}|_J(t) - \mathbf{1}^T \boldsymbol{p}_J(t)\right) + \left(1 - \mathbf{1}^T \boldsymbol{p}|_J(t)\right) \\
&= 1 - \mathbf{1}^T \boldsymbol{p}_J(t).
\end{aligned}$$

Since total variation distance is one half of the 1-norm distance, the zero-padded approximation has total variation error $(1 - \mathbf{1}^T \boldsymbol{p}_J(t))/2$. A sink-state comparison uses a different state space and must be stated separately. □

8.4.5 Truncation Strategy: Probability Flux versus Probability Mass

Remark 8.16 **(Failure of instantaneous mass truncation)** A naive heuristic for expanding or pruning states in the FSP retains states where the instantaneous probability $p_i(t)$ exceeds a threshold ε. In multiscale or stiff reaction networks, this heuristic can discard a low-mass intermediate that carries substantial probability flux and thereby produce an order-one error.

Def 8.22 **(Probability Flux)** For a transition from state j to state i with rate $A_{ij} > 0$, the instantaneous probability flux at time t is

$$\Phi_{ij}(t) = A_{ij}p_j(t).$$

The cumulative probability flux entering state i over the time interval $[0, T]$ is

$$F_i(T) = \int_0^T \sum_{j \neq i} A_{ij}p_j(s)\, ds.$$

Remark 8.17 **(Transient intermediate states)** Consider a reaction cascade $S_1 \xrightarrow{k_1} S_2 \xrightarrow{k_2} S_3$ with $k_2 \gg k_1$. State S_2 has a rapid exit rate, so its instantaneous probability is tiny: $p_2(t) \approx \frac{k_1}{k_2}p_1(t) \ll 1$. Pruning based on instantaneous mass $p_2(t) < \varepsilon$ deletes S_2 from the state space. However, every reaction producing S_3 must pass through S_2. Deleting S_2 severs the probability flow entirely, causing an $\mathcal{O}(1)$ error in $p_3(T)$.
Robust adaptive FSP algorithms monitor cumulative probability flux rather than instantaneous probability mass. A state may have negligible instantaneous mass yet carry substantial probability flux across the network.

8.4.6 Exercises

Ex 8.7

1. **Compute. (Simulation of a birth–death process)** Consider the reactions $\emptyset \to X$ with propensity k_1 and $X \to \emptyset$ with propensity $k_2 x$, started from $x = 0$. Implement Def 8.20, and compare the histogram of $X(T)$ over 10^4 trajectories with the Poisson distribution of mean $(k_1/k_2)(1-e^{-k_2 T})$. Then assemble the generator on the states $0, \ldots, n_{\max}$ by Def 8.19, compute $e^{T\mathbf{A}}\boldsymbol{e}_1$, and compare all three distributions.

2. **Compute. (Discrete spectrum and mixing)** Under the column-stochastic convention, let $\mathbf{P} = \begin{pmatrix} 0.8 & 0.1 \\ 0.2 & 0.9 \end{pmatrix}$. Find its stationary distribution $\boldsymbol{\pi}$, its second eigenvalue, and the spectral gap δ. Starting from $\boldsymbol{x}^{(0)} = \boldsymbol{e}_1$, derive the exact formula for $\|\mathbf{P}^k\boldsymbol{x}^{(0)} - \boldsymbol{\pi}\|_2$. Compare the first k for which this error is at most 10^{-3} with the estimate $\log(10^3)/\delta$, and explain why the latter is only an asymptotic rule.

3. **Compute. (Detailed balance)** Consider the column-stochastic matrix

$$\mathbf{P} = \begin{pmatrix} 0.6 & 0.2 & 0 \\ 0.4 & 0.5 & 0.1 \\ 0 & 0.3 & 0.9 \end{pmatrix}.$$

Find its stationary distribution $\boldsymbol{\pi}$, verify detailed balance on each edge, and compute $\mathbf{S} = \mathbf{D}_{\boldsymbol{\pi}}^{-1/2}\mathbf{P}\mathbf{D}_{\boldsymbol{\pi}}^{1/2}$. Verify directly that $\mathbf{S}$ is symmetric and that the eigenvalues of $\mathbf{P}$ are real.

4. **Derive. (FSP 1-norm leakage bound derivation)** Consider a CTMC with generator $\mathbf{A}$ partitioned into retained set J and complement J^c.

   (a) Prove that $\mathbf{1}^T\mathbf{A}_J \leq \mathbf{0}^T$ elementwise, with strict inequality for at least one entry if transitions from J to J^c exist.

   (b) Show that $\frac{d}{dt}\big(\mathbf{1}^T\boldsymbol{p}_J(t)\big) \leq 0$, confirming that total modeled probability mass decreases monotonically over time.

   (c) Let $\varepsilon = 1 - \mathbf{1}^T\boldsymbol{p}_J(T)$. Verify from Thm 8.18 that the zero-padded FSP error is exactly ε in the 1-norm. Explain what additional identification of omitted states would be needed before comparing an absorbing sink with the exact distribution.

5. **Explore. (Probability mass versus cumulative flux in stiff cascades)** Consider the three-state continuous-time Markov chain

$$X_1 \xrightarrow{k_1} X_2 \xrightarrow{k_2} X_3$$

with $k_1 = 1$, $k_2 = 10^4$, and initial condition $\boldsymbol{p}(0) = (1, 0, 0)^T$.

   (a) Solve the system analytically for $p_1(t), p_2(t), p_3(t)$.

   (b) Show that $\max_{t\geq 0} p_2(t) \approx 10^{-4}$. If a threshold $\varepsilon = 10^{-3}$ is used to prune states by instantaneous mass, state X_2 is discarded at all times.

   (c) Compute the cumulative flux $\int_0^T k_1 p_1(s)\, ds$ entering X_2 for $T = 3$. Show that over 95% of the total system probability passes through X_2 despite its negligible instantaneous mass.

   (d) Explain why adaptive truncation based on cumulative probability flux preserves state X_2 while mass-based thresholding disconnects the state space.

6. **Derive. (Birth–death process and infinite generator truncation)** For a simple birth–death process with birth rate λ and death rate μx, write down the tridiagonal subgenerator $\mathbf{A}_J$ for copy numbers $x \in \{0, 1, \ldots, N\}$. Formulate the exit rate from boundary state N and derive the leakage rate $\frac{d}{dt}\mathbf{1}^T\boldsymbol{p}_J(t)$ as a function of $p_N(t)$.

Read **Further reading.** The finite-state projection method and its error certificate originate with Munsky and Khammash (Munsky and Khammash, 2006). Higham's treatment of matrix functions is useful background for the exponential calculations used here (Higham, 2008).

8.5 Koopman Theory, EDMD, and Continuous Generators

(Running example: Markov chains) The chemical master equation evolves probability densities, while the Koopman generator evolves observables. They are adjoint descriptions of the same Markov dynamics. This duality connects the matrix exponential, sensitivity formulas, polynomial closure, and data-driven operator recovery.

Koopman operator theory shifts the analysis of nonlinear dynamical systems from finite-dimensional nonlinear state updates to linear transformations on an infinite-dimensional space of observables. Dynamic Mode Decomposition (DMD) and Extended DMD (EDMD) compute finite-dimensional approximations of this operator from snapshot data. In continuous-time Markov jump processes and chemical kinetics, the Koopman generator is the formal adjoint of the Chemical Master Equation, exhibiting exact polynomial closure for unimolecular systems. Recovering continuous rate matrices from discrete Koopman operators requires the matrix logarithm, which suffers from modal noise explosion on fast decaying dynamics.

8.5.1 Koopman Operator Theory

Def 8.23 **(Nonlinear Dynamics and Observables)** Consider a discrete-time dynamical system

$$\boldsymbol{x}^{(k+1)} = \mathbf{F}(\boldsymbol{x}^{(k)}), \qquad \boldsymbol{x} \in \mathcal{M} \subseteq \mathbb{R}^n.$$

An observable is a scalar function $g : \mathcal{M} \to \mathbb{R}$ belonging to a function space $\mathcal{F}$ (such as $L^2(\mathcal{M})$). The Koopman operator $\mathcal{K} : \mathcal{F} \to \mathcal{F}$ is defined by

$$(\mathcal{K}g)(\boldsymbol{x}) = g(\mathbf{F}(\boldsymbol{x})).$$

The operator $\mathcal{K}$ is linear on $\mathcal{F}$, even when the underlying map $\mathbf{F}$ is nonlinear.

Prop 8.19 **(Linearity of the Koopman Operator)** The Koopman operator $\mathcal{K}$ satisfies

$$\mathcal{K}(\alpha g_1 + \beta g_2) = \alpha \mathcal{K} g_1 + \beta \mathcal{K} g_2$$

for all scalars $\alpha, \beta \in \mathbb{R}$ and observables $g_1, g_2 \in \mathcal{F}$.

Proof Evaluating the composition at any state $\boldsymbol{x} \in \mathcal{M}$ gives

$$\begin{aligned}
[\mathcal{K}(\alpha g_1 + \beta g_2)](\boldsymbol{x}) &= (\alpha g_1 + \beta g_2)(\mathbf{F}(\boldsymbol{x})) \\
&= \alpha g_1(\mathbf{F}(\boldsymbol{x})) + \beta g_2(\mathbf{F}(\boldsymbol{x})) \\
&= \alpha [\mathcal{K} g_1](\boldsymbol{x}) + \beta [\mathcal{K} g_2](\boldsymbol{x}) \\
&= [\alpha \mathcal{K} g_1 + \beta \mathcal{K} g_2](\boldsymbol{x}).
\end{aligned}$$

Because this equality holds for all $\boldsymbol{x} \in \mathcal{M}$, the operators are identical. □

Def 8.24 **(Koopman Eigenfunctions and Eigenvalues)** A function $\varphi \in \mathcal{F}$ is a Koopman eigenfunction with eigenvalue $\mu \in \mathbb{C}$ if

$$\mathcal{K}\varphi = \mu\varphi.$$

Along system trajectories, Koopman eigenfunctions evolve linearly:

$$\varphi(\boldsymbol{x}^{(k)}) = \mu^k \varphi(\boldsymbol{x}^{(0)}).$$

A sufficiently rich collection of Koopman eigenfunctions can provide coordinates in which part of the nonlinear dynamics evolves linearly. Such a collection need not exist globally, and finite dictionaries generally give only a projected approximation.

8.5.2 Dynamic Mode Decomposition (DMD) and EDMD

Def 8.25 **(Dynamic Mode Decomposition (DMD))** Given snapshot matrices $\mathbf{X}, \mathbf{Y} \in \mathbb{R}^{n\times M}$ where $\mathbf{Y}$ advances $\mathbf{X}$ by time step h:

$$\mathbf{X} = [\boldsymbol{x}_1, \boldsymbol{x}_2, \ldots, \boldsymbol{x}_M], \qquad \mathbf{Y} = [\boldsymbol{y}_1, \boldsymbol{y}_2, \ldots, \boldsymbol{y}_M], \qquad \boldsymbol{y}_m = \mathbf{F}(\boldsymbol{x}_m),$$

DMD seeks the best-fit linear map $\mathbf{A} \approx \mathbf{Y}\mathbf{X}^\dagger$ in the Frobenius norm.
To compute the reduced model:

1. Compute the rank-r truncated SVD $\mathbf{X} \approx \mathbf{U}_r\mathbf{\Sigma}_r\mathbf{V}_r^T$.
2. Project onto the POD subspace: $\widetilde{\mathbf{A}} = \mathbf{U}_r^T\mathbf{Y}\mathbf{V}_r\mathbf{\Sigma}_r^{-1} \in \mathbb{R}^{r\times r}$.
3. Compute eigendecomposition $\widetilde{\mathbf{A}}\boldsymbol{w}_i = \mu_i\boldsymbol{w}_i$.
4. The DMD modes are $\boldsymbol{\phi}_i = \mathbf{Y}\mathbf{V}_r\mathbf{\Sigma}_r^{-1}\boldsymbol{w}_i$.

Prop 8.20 **(DMD as a Least-Squares Fit)** The minimum-Frobenius-norm solution of

$$\min_{\mathbf{A}} \|\mathbf{Y} - \mathbf{A}\mathbf{X}\|_F$$

is $\mathbf{A}_* = \mathbf{Y}\mathbf{X}^\dagger$. If $\mathbf{X} = \mathbf{U}_r\mathbf{\Sigma}_r\mathbf{V}_r^T$ is a rank-r SVD, the representation of this map on $\mathcal{R}(\mathbf{U}_r)$ is

$$\widetilde{\mathbf{A}} = \mathbf{U}_r^T\mathbf{A}_*\mathbf{U}_r = \mathbf{U}_r^T\mathbf{Y}\mathbf{V}_r\mathbf{\Sigma}_r^{-1}.$$

Proof The rows of $\mathbf{A}\mathbf{X}$ are independent least-squares fits to the corresponding rows of $\mathbf{Y}$. The pseudoinverse therefore gives the minimum-norm minimizer $\mathbf{A}_* = \mathbf{Y}\mathbf{X}^\dagger$. Substituting $\mathbf{X}^\dagger = \mathbf{V}_r\mathbf{\Sigma}_r^{-1}\mathbf{U}_r^T$ and projecting on both sides by $\mathbf{U}_r$ gives

$$\mathbf{U}_r^T\mathbf{A}_*\mathbf{U}_r = \mathbf{U}_r^T\mathbf{Y}\mathbf{V}_r\mathbf{\Sigma}_r^{-1}\mathbf{U}_r^T\mathbf{U}_r = \mathbf{U}_r^T\mathbf{Y}\mathbf{V}_r\mathbf{\Sigma}_r^{-1}.$$

□

Def 8.26 **(Extended Dynamic Mode Decomposition (EDMD))** Extended DMD (Williams et al., 2015) generalizes DMD by lifting state data through a dictionary of nonlinear observables $\boldsymbol{\psi}(\boldsymbol{x}) = (\psi_1(\boldsymbol{x}), \dots, \psi_K(\boldsymbol{x}))^T \in \mathbb{R}^K$.
Form data matrices

$$\boldsymbol{\Psi}_X = [\boldsymbol{\psi}(\boldsymbol{x}_1), \dots, \boldsymbol{\psi}(\boldsymbol{x}_M)] \in \mathbb{R}^{K\times M}, \qquad \boldsymbol{\Psi}_Y = [\boldsymbol{\psi}(\boldsymbol{y}_1), \dots, \boldsymbol{\psi}(\boldsymbol{y}_M)] \in \mathbb{R}^{K\times M}.$$

The finite-dimensional representation $\widehat{\mathbf{K}}_h \in \mathbb{R}^{K\times K}$ of the Koopman operator minimizes $\|\boldsymbol{\Psi}_Y - \mathbf{K}\boldsymbol{\Psi}_X\|_F^2$:

$$\widehat{\mathbf{K}}_h = \boldsymbol{\Psi}_Y \boldsymbol{\Psi}_X^{\dagger} = (\boldsymbol{\Psi}_Y \boldsymbol{\Psi}_X^T)(\boldsymbol{\Psi}_X \boldsymbol{\Psi}_X^T)^{\dagger}.$$

Left eigenvectors of $\widehat{\mathbf{K}}_h$ yield approximations to Koopman eigenfunctions.

Remark 8.18 **(EDMD as a Galerkin method)** When the dictionary consists of the coordinate functions, $\boldsymbol{\psi}(\boldsymbol{x}) = \boldsymbol{x}$, the data matrices reduce to $\mathbf{X}$ and $\mathbf{Y}$, and EDMD reduces to the least-squares map of Prop 8.20. For a richer dictionary, the matrices $\boldsymbol{\Psi}_Y \boldsymbol{\Psi}_X^T/M$ and $\boldsymbol{\Psi}_X \boldsymbol{\Psi}_X^T/M$ are sample averages of inner products between dictionary functions. As the number of samples drawn from a measure μ grows, they converge to the corresponding inner-product matrices in $L^2(\mu)$, and $\widehat{\mathbf{K}}_h$ converges to the Galerkin projection of the Koopman operator onto the span of the dictionary (Williams et al., 2015). Enlarging the dictionary therefore raises a separate approximation question, namely how well that span represents the Koopman eigenfunctions of interest.

8.5.3 CME Koopman Generator and Polynomial Closure

Remark 8.19 **(Koopman generator for continuous Markov jump processes)** For a continuous-time Markov process with master equation $\dot{\boldsymbol{p}}(t) = \mathbf{A}(\boldsymbol{c})\boldsymbol{p}(t)$, the expectation of an observable $g(\boldsymbol{x})$ evolves according to $\frac{d}{dt}\mathbb{E}[g(\boldsymbol{x}(t))] = \frac{d}{dt}\sum_{\boldsymbol{x}} g(\boldsymbol{x})p(\boldsymbol{x},t) = \sum_{\boldsymbol{x}} g(\boldsymbol{x})[\mathbf{A}(\boldsymbol{c})\boldsymbol{p}(t)]_{\boldsymbol{x}} = \sum_{\boldsymbol{x}}[\mathbf{A}(\boldsymbol{c})^T g](\boldsymbol{x})p(\boldsymbol{x},t)$. The Koopman infinitesimal generator $\mathcal{A}$ is the formal adjoint of the master operator: $\mathcal{A} = \mathbf{A}(\boldsymbol{c})^T$.

Def 8.27 **(CME Koopman Generator)** For a chemical reaction network with stoichiometry $\boldsymbol{\nu}_k$ and propensities $\alpha_k(\boldsymbol{x};\boldsymbol{c}) = c_k h_k(\boldsymbol{x})$, the Koopman generator acts on observable g by

$$(\mathcal{A}g)(\boldsymbol{x}) = \sum_{k=1}^{R} \alpha_k(\boldsymbol{x};\boldsymbol{c})\Big(g(\boldsymbol{x}+\boldsymbol{\nu}_k) - g(\boldsymbol{x})\Big).$$

Thm 8.21 **(Polynomial Closure in Chemical Kinetics)** Under mass-action kinetics, propensity functions are falling factorial polynomials:

$$h_k(\boldsymbol{x}) = \prod_{s=1}^{S} x_s^{(\nu_{ks}^-)}, \qquad x_s^{(m)} = x_s(x_s-1)\cdots(x_s-m+1).$$

Let $\mathcal{P}_d$ denote the space of polynomials in $\boldsymbol{x}$ of degree at most d.

1. **Unimolecular networks:** If all reactions have reactant order at most 1 (reactions of form $\emptyset \to X_i$ or $X_i \to X_j$), then $\mathcal{A}\mathcal{P}_d \subseteq \mathcal{P}_d$ for all $d \geq 0$. The subspace $\mathcal{P}_d$ is invariant under $\mathcal{A}$, yielding exact finite-dimensional closure.

2. **Bimolecular networks:** For reactions of reactant order at most 2, $\mathcal{A}\mathcal{P}_d \subseteq \mathcal{P}_{d+1}$. A quadratic propensity can produce degree-$(d+1)$ terms, so the moment equations generally fail to close at degree d unless additional cancellations, conservation laws, or invariant subspaces are present.

Proof Let $g \in \mathcal{P}_d$ be a polynomial of degree d. For each stoichiometric change vector $\boldsymbol{\nu}_k$, the discrete difference operator expands via Taylor's formula on polynomials:

$$g(\boldsymbol{x} + \boldsymbol{\nu}_k) - g(\boldsymbol{x}) = \sum_{j=1}^{d} \frac{1}{j!} (\boldsymbol{\nu}_k \cdot \nabla)^j g(\boldsymbol{x}).$$

Because differentiation reduces polynomial degree by at least one, each term in the sum has degree at most $d-1$, so $g(\boldsymbol{x} + \boldsymbol{\nu}_k) - g(\boldsymbol{x}) \in \mathcal{P}_{d-1}$. Applying the Koopman generator (Def 8.27):

$$(\mathcal{A}g)(\boldsymbol{x}) = \sum_{k=1}^{R} \alpha_k(\boldsymbol{x}; \boldsymbol{c}) \big(g(\boldsymbol{x} + \boldsymbol{\nu}_k) - g(\boldsymbol{x})\big).$$

For unimolecular reactions, each propensity $\alpha_k(\boldsymbol{x}; \boldsymbol{c})$ has reactant order at most 1, so $\alpha_k \in \mathcal{P}_1$. The product of a degree-1 polynomial and a degree-$(d-1)$ polynomial has degree at most $1 + (d-1) = d$, proving $\mathcal{A}\mathcal{P}_d \subseteq \mathcal{P}_d$. For bimolecular reactions, reactant order is at most 2, so $\alpha_k \in \mathcal{P}_2$. The product has degree at most $2 + (d-1) = d+1$, giving $\mathcal{A}\mathcal{P}_d \subseteq \mathcal{P}_{d+1}$. □

8.5.4 Continuous Rate Recovery and Logarithmic Noise Explosion

Remark 8.20 **(Discrete-to-continuous inversion)** Given an EDMD estimate of the discrete Koopman operator $\widehat{\mathbf{K}}_h \approx \exp(h\mathcal{A})$ over a sampling interval h, the continuous generator is recovered via the principal matrix logarithm:

$$\widehat{\mathbf{A}} = \frac{1}{h} \operatorname{logm}(\widehat{\mathbf{K}}_h).$$

If $\widehat{\mathbf{K}}_h$ is real and has no eigenvalues on the closed negative real axis $(-\infty, 0]$, then its principal matrix logarithm exists and is real. A matrix with eigenvalues on this branch cut may possess other logarithms, but it does not possess the principal logarithm used here.

Thm 8.22 **(Logarithmic Noise Explosion on Fast Decay Modes)** Let $\lambda_i < 0$ be a continuous eigenvalue of the generator $\mathcal{A}$, corresponding to a discrete Koopman eigenvalue $\mu_i = \exp(h\lambda_i) \in (0, 1)$. The relation between discrete and continuous eigenvalues is

$$\lambda_i = \frac{1}{h} \ln \mu_i.$$

The sensitivity of the continuous eigenvalue with respect to perturbations in the discrete eigenvalue is

$$\left|\frac{d\lambda_i}{d\mu_i}\right| = \frac{1}{h\mu_i} = \frac{1}{h\exp(h\lambda_i)}.$$

For fast decaying modes where $-\lambda_i \gg 1$, $\mu_i \to 0$ exponentially fast. Consequently, the noise amplification factor diverges:

$$\lim_{-\lambda_i\to\infty}\left|\frac{d\lambda_i}{d\mu_i}\right| = \infty.$$

A tiny statistical fluctuation $\Delta\mu_i$ in data-driven EDMD induces an exponentially magnified error $\Delta\lambda_i$ in the recovered rate.

Proof Differentiating $\lambda_i = \frac{1}{h}\ln\mu_i$ directly with respect to μ_i yields $\frac{d\lambda_i}{d\mu_i} = \frac{1}{h\mu_i}$. Substituting $\mu_i = \exp(h\lambda_i)$ produces $\frac{1}{h}\exp(-h\lambda_i)$. Because $h > 0$ and $\lambda_i < 0$, $-\lambda_i > 0$, so $\exp(-h\lambda_i) \to \infty$ as $-\lambda_i \to \infty$. □

8.5.5 Exercises

Ex 8.8

1. **Check. (Koopman linearity)** Consider the one-dimensional system $x_{k+1} = x_k^3$. Let $g_1(x) = x$ and $g_2(x) = x^3$.
   (a) Compute $(\mathcal{K}g_1)(x)$ and $(\mathcal{K}g_2)(x)$.
   (b) Verify that while $x \mapsto x^3$ is nonlinear, $\mathcal{K}(\alpha g_1 + \beta g_2) = \alpha\mathcal{K}g_1 + \beta\mathcal{K}g_2$.

2. **Compute. (DMD residual)** Compute $\mathbf{A}_* = \mathbf{Y}\mathbf{X}^\dagger$ for a rank-one snapshot matrix and verify directly that the residual $\mathbf{Y} - \mathbf{A}_*\mathbf{X}$ is orthogonal to the row space of $\mathbf{X}$, as predicted by Prop 8.20.

3. **Derive. (Polynomial closure in unimolecular reaction networks)** Consider the reversible isomerization network $A \underset{c_2}{\overset{c_1}{\rightleftharpoons}} B$. Let x_1, x_2 denote the copy numbers of species A and B.
   (a) Write down the propensities $\alpha_1(\boldsymbol{x}) = c_1x_1$ and $\alpha_2(\boldsymbol{x}) = c_2x_2$, and the stoichiometry vectors $\boldsymbol{\nu}_1 = (-1, 1)^T$ and $\boldsymbol{\nu}_2 = (1, -1)^T$.
   (b) Apply the Koopman generator $\mathcal{A}$ to the linear observable $g(\boldsymbol{x}) = x_1$. Show that $(\mathcal{A}g)(\boldsymbol{x}) = -c_1x_1 + c_2x_2$, which is strictly linear.
   (c) Apply $\mathcal{A}$ to the second-order observable $g(\boldsymbol{x}) = x_1^2$. Show that $(\mathcal{A}g)(\boldsymbol{x})$ contains only polynomials of degree at most 2.
   (d) Explain why the subspace of polynomials of degree at most d is invariant under $\mathcal{A}$ for any unimolecular network.

4. **Derive. (Bimolecular moment hierarchy)** Consider the dimerization reaction $2A \xrightarrow{c} B$. The stoichiometry is $\boldsymbol{\nu} = (-2, 1)^T$ and the propensity is $\alpha(x) = cx(x-1)/2$.

   (a) Compute $(\mathcal{A}g)(x)$ for $g(x) = x$. Show that the result is quadratic in x.

   (b) Explain why computing the time derivative of the mean $\frac{d}{dt}\mathbb{E}[x]$ requires knowledge of the second moment $\mathbb{E}[x^2]$.

5. **Explore. (Logarithmic noise explosion on fast decay modes)** Suppose a continuous dynamical system has decay rates $\lambda_1 = -1$ and $\lambda_2 = -50$. Data snapshots are collected at sampling interval $h = 0.1$.

   (a) Compute the true discrete eigenvalues $\mu_1 = \exp(h\lambda_1)$ and $\mu_2 = \exp(h\lambda_2)$.

   (b) Suppose statistical noise perturbs the discrete estimates by $\Delta\mu = 10^{-3}$. Compute the resulting perturbation $\Delta\lambda_1 \approx \frac{\Delta\mu}{h\mu_1}$.

   (c) Compute the resulting perturbation $\Delta\lambda_2 \approx \frac{\Delta\mu}{h\mu_2}$.

   (d) Explain why recovering continuous generators from discrete snapshots requires regularized regression rather than unconstrained matrix logarithms.

Read **Further reading.** Koopman's original paper gives the operator viewpoint for Hamiltonian systems (Koopman, 1931). Schmid introduces dynamic mode decomposition in its fluid-mechanics form, and Williams, Kevrekidis, and Rowley connect a dictionary-based extension to the Koopman operator (Schmid, 2010; Williams et al., 2015).

9 Appendix: Notation and Software

9.1 Mathematical Notation and Proof Techniques

This section establishes the logical, set-theoretic, and matrix notation used throughout these notes, together with the standard proof structures.

9.1.1 Logic and Set Notation

Def 9.1 **(Logical Quantifiers)**

- **Universal ($\forall$):** "For all."
- **Existential ($\exists$):** "There exists."
- **Example:** "$\forall \varepsilon > 0,\ \exists \delta > 0$" reads as "for every positive ε there is a positive δ."

Def 9.2 **(Implication and Equivalence)**

- **Implication ($P \Rightarrow Q$):** P implies Q.
- **Equivalence ($P \Leftrightarrow Q$):** P if and only if Q (iff).
- **Contrapositive:** $\neg Q \Rightarrow \neg P$. Logically equivalent to $P \Rightarrow Q$.

Def 9.3 **(Set Notation)**

- $x \in S$: x is an element of S.
- $S \subseteq T$: S is a subset of T.
- $S \cup T$ (union), $S \cap T$ (intersection), and $S \setminus T$ (set difference).
- $\{x \in S : P(x)\}$: Set builder notation.
- **Number Sets:** $\mathbb{Z}$ (integers), $\mathbb{R}$ (reals), $\mathbb{C}$ (complex), $\mathbb{R}^n$ (vectors), $\mathbb{R}^{m \times n}$ (matrices).

Ex 9.1 **Check.**

1. Write "every invertible matrix has a nonzero determinant" using quantifiers and implication.
2. State the contrapositive of: "if $\mathbf{A}$ is invertible, then $\mathbf{A}\boldsymbol{x} = \mathbf{0} \Rightarrow \boldsymbol{x} = \mathbf{0}$."
3. Let $S = \{1, 2, 3, 4\}$ and $T = \{3, 4, 5, 6\}$. Compute $S \cup T, S \cap T, S \setminus T$.

9.1.2 Proof Techniques

Def 9.4 **(Direct Proof)** Assume P is true and derive Q through a chain of valid logical steps.

Def 9.5 **(Proof by Contradiction)** Assume $\neg P$ and derive a logical contradiction $(R \wedge \neg R)$. Proves P must hold.

Def 9.6 **(Proof by Induction)** To prove $P(n)$ holds for all integers $n \geq n_0$:

1. **Base case:** Verify $P(n_0)$ directly.
2. **Inductive step:** Assume $P(k)$ and prove $P(k+1)$.

Remark 9.1 **(Numerical proofs)** Often validate that an approximation is stable or that an error term converges. Frequently involves showing $|f(x) - \hat{f}(x)| \leq \varepsilon$ for small ε.

Ex 9.2 **Derive.**

1. Prove: if $\mathbf{A}$ and $\mathbf{B}$ are symmetric, then $\mathbf{A} + \mathbf{B}$ is symmetric.
2. Prove by contradiction: if $\mathbf{A}\boldsymbol{x} = \mathbf{0} \Rightarrow \boldsymbol{x} = \mathbf{0}$, then $\mathbf{A}$ has no zero eigenvalue.
3. By induction, prove $\sum_{k=1}^{n} k = \frac{n(n+1)}{2}$.

9.1.3 Notation Conventions

Remark 9.2 **(Scalars, Vectors, and Matrices)**

- **Scalars:** Lowercase italic (a, b, x).
- **Vectors:** Lowercase bold $(\boldsymbol{x}, \boldsymbol{y}, \boldsymbol{a})$. Default is column vector.
- **Matrices:** Uppercase bold $(\mathbf{A}, \mathbf{B}, \mathbf{Q})$.
- **Indexing:** 1-based (unlike Python). Here x_i is the i-th entry, and a_{ij} is the (i, j) entry.

Remark 9.3 **(Transpose and Inverse)**

- $\mathbf{A}^T$: Transpose.
- $\mathbf{A}^{-1}$: Matrix inverse (square, nonsingular matrices).
- $\mathbf{A}^{-T}$: Transpose of the inverse $(\mathbf{A}^{-1})^T$.
- $\mathbf{A}^k$: k-th power. $\mathbf{A}^0 = \mathbf{I}$.

Remark 9.4 **(Sums and Products)**

$$\sum_{i=1}^{n} a_i = a_1 + \cdots + a_n, \qquad \prod_{i=1}^{n} a_i = a_1 \dots a_n.$$

Remark 9.5 **(Functions)**

- $f : \mathbb{R}^n \to \mathbb{R}^m$: Maps n-vectors to m-vectors.
- $f \circ g$: Function composition $(f \circ g)(\boldsymbol{x}) = f(g(\boldsymbol{x}))$.

Remark 9.6 **(Asymptotic Notation)** Scaling of algorithms is described using Big-O notation ($O(n^2), O(n^3)$). Details in the Computational Complexity section.

9.2 Python and NumPy Primer

This section summarizes the NumPy array conventions, linear-algebra operations, and indexing idioms used throughout the computational examples in these notes.

9.2.1 Arrays and Basic Operations

Def 9.7 **(NumPy Arrays)** The `ndarray` is the fundamental structure.

- **Creation:** `np.array([1, 2])`, `np.zeros((m, n))`, `np.random.randn(m, n)`.
- **Indexing:** 0-indexed. `A[i, j]` for entries, `A[i, :]` for rows, and `A[:, j]` for columns.

Remark 9.7 **(Shapes and rank-1 arrays)** `A.shape` returns `(m, n)`. A 1D array has shape `(n,)`, which is **not** equivalent to a column vector `(n, 1)`. These "rank-1 arrays" are a common source of broadcasting bugs. Use `x[:, None]` or `x.reshape(-1, 1)` to convert to 2D.

Ex 9.3 **Compute.**

1. Create 4×4 identity matrix. Verify `shape` and `dtype`.
2. Extract row 2 and column 3 from a random 3×3 $\mathbf{A}$.
3. Compare `A @ x` for `x.shape == (3,)` and `x.shape == (3,1)`. Record the output shape in each case, then explain why the two representations behave differently under broadcasting.

9.2.2 Linear Algebra Operations

The core routines in `numpy.linalg` provide access to standard BLAS and LAPACK implementations:

Function	Computes
`A @ B`	Matrix product $\mathbf{AB}$
`np.linalg.solve(A, b)`	Solve $\mathbf{A}\boldsymbol{x} = \boldsymbol{b}$ ($O(n^3)$)
`np.linalg.inv(A)`	Matrix inverse (prefer `solve` for linear systems)
`np.linalg.norm(x)`	Euclidean norm $\|\boldsymbol{x}\|_2$
`np.linalg.eig(A)`	Eigenvalues and eigenvectors
`np.linalg.svd(A)`	Singular Value Decomposition
`np.linalg.qr(A)`	QR Factorization
`np.linalg.cond(A)`	Condition number $\kappa_2(\mathbf{A})$

Remark 9.8 **(Vectorization and performance)** Explicit `for` loops in Python are slow. Push heavy lifting into NumPy/BLAS routines by **vectorizing** operations. For example, use `A @ B` instead of triple-nested loops.

Remark 9.9 **(Linear solve versus inversion)** `np.linalg.solve(A, b)` is faster and more stable than `np.linalg.inv(A) @ b`. Explicit inversion is almost never desirable in scientific computation.

Remark 9.10 **(Computed equality)** Use `np.allclose(x, y)` or `np.isclose(x, y)` for computed floating-point quantities. Exact equality is appropriate for integers, booleans, and deliberately exact structural checks.

Remark 9.11 **(Sparse matrices)** Large discretized PDE and graph problems should usually use `scipy.sparse`, not dense NumPy arrays. Use `scipy.sparse.linalg` for sparse solves, eigenvalue computations, CG, and GMRES.

Ex 9.4 **Compute.**

1. Solve a 2×2 system and verify the residual $\|\mathbf{A}\boldsymbol{x} - \boldsymbol{b}\|$.
2. Profile `solve` vs `inv @ b` for $n = 1000$.
3. Compare `np.linalg.eig` and `np.linalg.eigh` (optimized for symmetric matrices).

9.2.3 Key Idioms

- **Element-wise:** `*` and `**` are entry-wise. Matrix powers use `np.linalg.matrix_power`.
- **Broadcasting:** Automatic dimension expansion. `A * v` scales columns of $\mathbf{A}$ by entries of v.

- **Aliases, Views, Copies:** `B = A` makes a second name for the same array. Slices are usually views. Use `B = A.copy()` for independent data.
- **Sparse construction:** use `scipy.sparse.diags` for banded matrices and `scipy.sparse.csr_m` for efficient arithmetic.

Ex 9.5 **Compute.**

1. Let $\mathbf{A} = \begin{pmatrix} 1 & 2 \\ 3 & 4 \end{pmatrix}$. Compare `A * A` and `A @ A`.
2. Demonstrate the "view pitfall" by modifying a slice of a matrix.
3. Use broadcasting to perform diagonal scaling $\mathbf{AD}$ without constructing $\mathbf{D} = \mathrm{diag}(d)$.

Read **Further reading.** Heath gives a broad introduction to scientific computing at roughly the level of these notes (Heath, 2018). For the effects of floating-point arithmetic, Higham is the standard detailed reference (Higham, 2002). Meyer provides an accessible bridge from undergraduate linear algebra to matrix computation (Meyer, 2000).

References

Awad H. Al-Mohy and Nicholas J. Higham. Computing the action of the matrix exponential, with an application to exponential integrators. *SIAM Journal on Scientific Computing*, 33(2): 488–511, 2011. doi: 10.1137/100788860.

Walter E. Arnoldi. The principle of minimized iterations in the solution of the matrix eigenvalue problem. *Quarterly of Applied Mathematics*, 9(1):17–29, 1951. doi: 10.1090/qam/42792.

Costas Bekas, Effrosyni Kokiopoulou, and Yousef Saad. An estimator for the diagonal of a matrix. *Applied Numerical Mathematics*, 57(11–12):1214–1229, 2007.

Mikhail Belkin and Partha Niyogi. Laplacian eigenmaps for dimensionality reduction and data representation. *Neural Computation*, 15(6):1373–1396, 2003. doi: 10.1162/089976603321780317.

William L. Briggs, Van Emden Henson, and Steve F. McCormick. *A Multigrid Tutorial*. Society for Industrial and Applied Mathematics, Philadelphia, 2 edition, 2000. doi: 10.1137/1.9780898719505.

Jule G. Charney, Ragnar Fjørtoft, and John von Neumann. Numerical integration of the barotropic vorticity equation. *Tellus*, 2(4):237–254, 1950. doi: 10.3402/tellusa.v2i4.8607.

Fan R. K. Chung. *Spectral Graph Theory*, volume 92 of *CBMS Regional Conference Series in Mathematics*. American Mathematical Society, Providence, 1997.

C. W. Clenshaw and A. R. Curtis. A method for numerical integration on an automatic computer. *Numerische Mathematik*, 2:197–205, 1960. doi: 10.1007/BF01386223.

Ronald R. Coifman and Stéphane Lafon. Diffusion maps. *Applied and Computational Harmonic Analysis*, 21(1):5–30, 2006. doi: 10.1016/j.acha.2006.04.006.

James W. Cooley and John W. Tukey. An algorithm for the machine calculation of complex Fourier series. *Mathematics of Computation*, 19(90):297–301, 1965. doi: 10.1090/S0025-5718-1965-0178586-1.

Germund G. Dahlquist. A special stability problem for linear multistep methods. *BIT Numerical Mathematics*, 3:27–43, 1963. doi: 10.1007/BF01963532.

Timothy A. Davis. *Direct Methods for Sparse Linear Systems*. Society for Industrial and Applied Mathematics, Philadelphia, 2006. doi: 10.1137/1.9780898718881.

Carl de Boor. *A Practical Guide to Splines*. Springer, New York, revised edition, 2001. doi: 10.1007/978-1-4612-6333-3.

James W. Demmel. *Applied Numerical Linear Algebra*. Society for Industrial and Applied Mathematics, Philadelphia, 1997. doi: 10.1137/1.9781611971446.

Alan George. Nested dissection of a regular finite element mesh. *SIAM Journal on Numerical Analysis*, 10(2):345–363, 1973.

Daniel T. Gillespie. Exact stochastic simulation of coupled chemical reactions. *The Journal of Physical Chemistry*, 81(25):2340–2361, 1977.

Gene H. Golub and William Kahan. Calculating the singular values and pseudo-inverse of a matrix. *Journal of the Society for Industrial and Applied Mathematics, Series B: Numerical Analysis*, 2(2):205–224, 1965. doi: 10.1137/0702016.

Gene H. Golub and Charles F. Van Loan. *Matrix Computations*. Johns Hopkins University Press, Baltimore, 4 edition, 2013.

Anne Greenbaum. *Iterative Methods for Solving Linear Systems*. Society for Industrial and Applied Mathematics, Philadelphia, 1997. doi: 10.1137/1.9781611970937.

Ernst Hairer, Syvert P. Nørsett, and Gerhard Wanner. *Solving Ordinary Differential Equations I: Nonstiff Problems*. Springer, Berlin, 2 edition, 1993. doi: 10.1007/978-3-540-78862-1.

Nathan Halko, Per-Gunnar Martinsson, and Joel A. Tropp. Finding structure with randomness: Probabilistic algorithms for constructing approximate matrix decompositions. *SIAM Review*, 53(2):217–288, 2011.

Michael T. Heath. *Scientific Computing: An Introductory Survey*. Society for Industrial and Applied Mathematics, Philadelphia, 2 edition, 2018. doi: 10.1137/1.9781611975581.

Magnus R. Hestenes and Eduard Stiefel. Methods of conjugate gradients for solving linear systems. *Journal of Research of the National Bureau of Standards*, 49(6):409–436, 1952. doi: 10.6028/jres.049.044.

Nicholas J. Higham. *Accuracy and Stability of Numerical Algorithms*. Society for Industrial and Applied Mathematics, Philadelphia, 2 edition, 2002. doi: 10.1137/1.9780898718027.

Nicholas J. Higham. *Functions of Matrices: Theory and Computation*. Society for Industrial and Applied Mathematics, Philadelphia, 2008. doi: 10.1137/1.9780898717778.

Marlis Hochbruck and Christian Lubich. On Krylov subspace approximations to the matrix exponential operator. *SIAM Journal on Numerical Analysis*, 34(5):1911–1925, 1997. doi: 10.1137/S0036142995280572.

Roger A. Horn and Charles R. Johnson. *Matrix Analysis*. Cambridge University Press, Cambridge, 2 edition, 2012. doi: 10.1017/CBO9781139020411.

Alston S. Householder. Unitary triangularization of a nonsymmetric matrix. *Journal of the ACM*, 5(4):339–342, 1958. doi: 10.1145/320941.320947.

M. F. Hutchinson. A stochastic estimator of the trace of the influence matrix for Laplacian smoothing splines. *Communications in Statistics – Simulation and Computation*, 19(2):433–450, 1990.

Ian T. Jolliffe. *Principal Component Analysis*. Springer, New York, 2 edition, 2002. doi: 10.1007/b98835.

Bernard O. Koopman. Hamiltonian systems and transformation in Hilbert space. *Proceedings of the National Academy of Sciences*, 17(5):315–318, 1931. doi: 10.1073/pnas.17.5.315.

Rasmus Kyng and Sushant Sachdeva. Approximate Gaussian elimination for Laplacians: Fast, sparse, and simple. In *Proceedings of the 57th Annual IEEE Symposium on Foundations of Computer Science*, pages 573–582, 2016.

Cornelius Lanczos. An iteration method for the solution of the eigenvalue problem of linear differential and integral operators. *Journal of Research of the National Bureau of Standards*, 45(4):255–282, 1950. doi: 10.6028/jres.045.026.

Randall J. LeVeque. *Finite Difference Methods for Ordinary and Partial Differential Equations*. Society for Industrial and Applied Mathematics, Philadelphia, 2007. doi: 10.1137/1.9780898717839.

Carl D. Meyer. *Matrix Analysis and Applied Linear Algebra*. Society for Industrial and Applied Mathematics, Philadelphia, 2000. doi: 10.1137/1.9780898719512.

David A. Mindell. *Digital Apollo: Human and Machine in Spaceflight*. MIT Press, Cambridge, Massachusetts, 2008.

Cleve Moler and Charles F. Van Loan. Nineteen dubious ways to compute the exponential of a matrix, twenty-five years later. *SIAM Review*, 45(1):3–49, 2003. doi: 10.1137/S00361445024180.

Brian Munsky and Mustafa Khammash. The finite state projection algorithm for the solution of the chemical master equation. *The Journal of Chemical Physics*, 124(4):044104, 2006. doi: 10.1063/1.2145882.

Christopher C. Paige. Error analysis of the Lanczos algorithm for tridiagonalizing a symmetric matrix. *IMA Journal of Applied Mathematics*, 18(3):341–349, 1976. doi: 10.1093/imamat/18.3.341.

Alfio Quarteroni, Riccardo Sacco, and Fausto Saleri. *Numerical Mathematics*. Springer, Berlin, 2 edition, 2007. doi: 10.1007/b98885.

Lewis Fry Richardson. *Weather Prediction by Numerical Process*. Cambridge University Press, Cambridge, 1922. doi: 10.1017/CBO9780511618291.

J. L. Rigal and J. Gaches. On the compatibility of a given solution with the data of a linear system. *Journal of the ACM*, 14(3):543–548, 1967.

Yousef Saad. *Iterative Methods for Sparse Linear Systems*. Society for Industrial and Applied Mathematics, Philadelphia, 2 edition, 2003. doi: 10.1137/1.9780898718003.

Yousef Saad and Martin H. Schultz. GMRES: A generalized minimal residual algorithm for solving nonsymmetric linear systems. *SIAM Journal on Scientific and Statistical Computing*, 7(3):856–869, 1986. doi: 10.1137/0907058.

Peter J. Schmid. Dynamic mode decomposition of numerical and experimental data. *Journal of Fluid Mechanics*, 656:5–28, 2010. doi: 10.1017/S0022112010001217.

Jack Sherman and Winifred J. Morrison. Adjustment of an inverse matrix corresponding to a change in one element of a given matrix. *The Annals of Mathematical Statistics*, 21(1):124–127, 1950. doi: 10.1214/aoms/1177729893.

Josef Stoer and Roland Bulirsch. *Introduction to Numerical Analysis*. Springer, New York, 3 edition, 2002. doi: 10.1007/978-0-387-21738-3.

Gilbert Strang. *Computational Science and Engineering*. Wellesley-Cambridge Press, Wellesley, Massachusetts, 2007.

Lloyd N. Trefethen. *Approximation Theory and Approximation Practice*. Society for Industrial and Applied Mathematics, Philadelphia, 2013.

Lloyd N. Trefethen and David Bau. *Numerical Linear Algebra*. Society for Industrial and Applied Mathematics, Philadelphia, 1997. doi: 10.1137/1.9781611977165.

Lloyd N. Trefethen and Robert S. Schreiber. Average-case stability of Gaussian elimination. *SIAM Journal on Matrix Analysis and Applications*, 11(3):335–360, 1990.

J. A. C. Weideman and Lloyd N. Trefethen. Parabolic and hyperbolic contours for computing the Bromwich integral. *Mathematics of Computation*, 76(259):1341–1356, 2007.

Matthew O. Williams, Ioannis G. Kevrekidis, and Clarence W. Rowley. A data-driven approximation of the Koopman operator: Extending dynamic mode decomposition. *Journal of Nonlinear Science*, 25(6):1307–1346, 2015. doi: 10.1007/s00332-015-9258-5.

Max A. Woodbury. Inverting modified matrices. Technical Report Memorandum Report 42, Statistical Research Group, Princeton University, 1950.

Mihalis Yannakakis. Computing the minimum fill-in is NP-complete. *SIAM Journal on Algebraic and Discrete Methods*, 2(1):77–79, 1981.